

Εθνικο Μετσοβιο Πολυτεχνειο
Σχολη Ηλεκτρολογων Μηχανικων και Μηχανικων Υπολογιστων
Τομεας Τεχνολογιας Πληροφορικης και Υπολογιστων
Εργαστηριο Συστηματων Τεχνητης Νοημοσυνης και Μαθησης

Architectures of Large Language Model-Based Terminal Agents

Diploma Thesis

του

Χαράλαμπου Ιωάννη Σφυρή

Επιβλέπων: Αθανάσιος Βουλόδημος
Αναπληρωτής Καθηγητής Ε.Μ.Π.

Εργαστηριο Συστηματων Τεχνητης Νοημοσυνης και Μαθησης
Αθήνα, Οκτώβριος 2026



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τομέας Τεχνολογίας Πληροφορικής και Υπολογιστών
Εργαστήριο Συστημάτων Τεχνητής Νοημοσύνης και Μάθησης

Architectures of Large Language Model-Based Terminal Agents

Diploma Thesis

του

Χαράλαμπου Ιωάννη Σφυρή

Επιβλέπων: Αθανάσιος Βουλόδημος
Αναπληρωτής Καθηγητής Ε.Μ.Π.

Εγκρίθηκε από την τριμελή εξεταστική επιτροπή την 21η Οκτωβρίου, 2026.

.....	.....	.....
Αθανάσιος Βουλόδημος	Γεώργιος Στάμου	Χρυσούλα Ζέρβα
Αναπληρωτής Καθηγητής Ε.Μ.Π.	Καθηγητής Ε.Μ.Π.	Επίκουρη Καθηγήτρια Ε.Μ.Π.

Αθήνα, Οκτώβριος 2026

.....

Χαραλαμπος Ιωαννης Σφυρης
Διπλωματούχος Ηλεκτρολόγος Μηχανικός
και Μηχανικός Ηλεκτρονικών Υπολογιστών Ε.Μ.Π.

Copyright © – All rights reserved Χαράλαμπος Ιωάννης Σφυρής, 2026.
Με επιφύλαξη παντός δικαιώματος.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Εθνικού Μετσόβιου Πολυτεχνείου.

Περίληψη

Τα μεγάλα γλωσσικά μοντέλα λειτουργούν ολοένα και περισσότερο ως *πράκτορες*: εκτελούν εντολές φλοιού, διαβάζουν και γράφουν αρχεία, τρέχουν κώδικα και επαναλαμβάνουν μέχρι να πετύχουν έναν στόχο. Ένα μεγάλο σώμα εργασιών προτείνει *αρχιτεκτονικές υποστήριξης* γύρω από το βασικό μοντέλο (σχεδιασμό, αυτο-επαλήθευση, συνεργασία πολλαπλών πρακτόρων) και αναφέρει βελτιωμένη επιτυχία, σχεδόν πάντα όμως σε ένα μόνο, συνήθως κορυφαίο, μοντέλο. Καμία εργασία δεν μεταβάλλει *ταυτόχρονα* την αρχιτεκτονική και την ικανότητα του μοντέλου υπό ελεγχόμενο προϋπολογισμό: *εξαρτάται η αξία μιας αρχιτεκτονικής πράκτορα από την ικανότητα του μοντέλου που περιβάλλει;*

Εκτελούμε ένα ελεγχόμενο πλέγμα **14 αρχιτεκτονικών** σε **7 ανοικτά και κλειστά μοντέλα** ευρέος φάσματος ικανότητας, στις 64 εργασίες του TerminalBench με την ένδειξη "easy", υπό ενιαίο προϋπολογισμό 60 βημάτων που **μοιράζεται** μεταξύ των υπο-πρακτόρων αντί να προστίθεται. Ένα ξεχωριστό σκέλος *ετερογενούς επίβλεψης* βάζει ένα ισχυρό μοντέλο να επιβλέπει έναν ασθενέστερο, ώστε να απομονωθεί αν η ικανότητα μπορεί να εγχυθεί στο επίπεδο του συντονισμού. Συνολικά 7424 εκτελέσεις μοντέλου–εργασίας.

Το κεντρικό αποτέλεσμα είναι ότι η αξία μιας αρχιτεκτονικής εξαρτάται από την ικανότητα του μοντέλου, αρκετά ισχυρά ώστε να αντιστρέφει την κατεύθυνση της επίδρασης. Το πλήθος των αρχιτεκτονικών που ξεπερνούν τη γραμμή αναφοράς του ίδιου του μοντέλου μειώνεται από 13 από 13 στα ασθενέστερα μοντέλα σε 0 από 13 στον ισχυρότερο ανοικτό κωδικοποιητή ($\rho = -0.886$, $p = 0.033$ · η κατάταξη αντέχει την αφαίρεση της γραμμής αναφοράς και έλεγχο split-half). Η ίδια αρχιτεκτονική που σώζει ένα αδύναμο μοντέλο επιβαρύνει ένα ισχυρό, οπότε καμία δεν είναι η καλύτερη για όλα, και για τον ισχυρότερο ανοικτό κωδικοποιητή η καλύτερη επιλογή είναι ο απλός πράκτορας. Ο άξονας είναι η ικανότητα στον κώδικα και όχι το μέγεθος: το λιγότερο ωφελημένο μοντέλο είναι ένας κωδικοποιητής 32B, όχι το μεγαλύτερο σε παραμέτρους.

Τα ίχνη ανά εκτέλεση δείχνουν γιατί. Τα μοντέλα αποτυγχάνουν συχνότερα υποβάλλοντας λάθος λύση νωρίς παρά εξαντλώντας τον προϋπολογισμό, καθένα με τον δικό του τρόπο: τα ασθενέστερα παραιτούνται σε λίγα βήματα, άλλα ξοδεύουν όλο τον προϋπολογισμό επαναλαμβάνοντας την ίδια εντολή. Μια αρχιτεκτονική βοηθά όταν προσφέρει κάτι που λείπει από το μοντέλο, όπως έναν έλεγχο πριν την υποβολή, και βλάπτει όταν εκτοπίζει κάτι που ήδη κάνει καλά ή ζητά τεχνούργημα που δεν μπορεί να παραγάγει. Το ίδιο ισχύει ανά είδος εργασίας: προσθέτει λύσεις όπου υπάρχει περιθώριο και τις χάνει σε όσες το μοντέλο ήδη λύνει, οπότε το ερώτημα που μετράει είναι ποια αρχιτεκτονική ταιριάζει σε ποιο μοντέλο και εργασία.

Το σκέλος της επίβλεψης δείχνει ότι η ικανότητα μεταφέρεται στο επίπεδο του συντονισμού μέσω ορισμένων ρόλων: ένας επιβλέπων που *κατευθύνει* (δρομολογεί, σχεδιάζει, καθοδηγεί) ωφελεί κάθε εργάτη (+4, +5, +8), ενώ ο ίδιος που *κρίνει* την έξοδο δεν τη μεταφέρει το ίδιο και είναι αρνητικός στον ισχυρότερο (−6), κάνοντάς τον κυρίως σταθερότερο σε όσα ήδη μπορεί. Επειδή ο προϋπολογισμός μοιράζεται αντί να προστίθεται, τίποτα από αυτά δεν οφείλεται σε μεγαλύτερο προϋπολογισμό ενεργειών. Η αρχιτεκτονική υποστήριξης πρέπει να επιλέγεται βάσει του μετρημένου προφίλ ικανότητας και αποτυχίας ενός μοντέλου και του περιθωρίου μιας εργασίας, όχι να υιοθετείται ως προεπιλογή.

Λέξεις Κλειδιά: Πράκτορες Μεγάλων Γλωσσικών Μοντέλων, Αρχιτεκτονικές Πρακτόρων, Συστήματα Πολλαπλών Πρακτόρων, ReAct, TerminalBench, Ικανότητα Μοντέλου, Μοντέλα Ανοικτών Βαρών, Εμπειρική Αξιολόγηση.

Abstract

Large language models increasingly operate as *agents*: they issue shell commands, read and write files, run code, and iterate toward a goal in a real environment. A large body of work proposes *scaffolding* around the base model (planning, self-verification, multi-agent debate, orchestration) and reports improved success. But most of that work evaluates on a single model, usually a frontier one, and the studies that do control for compute vary one factor at a time. None varies architecture *and* model capability together under a controlled budget, which is what the central question needs: *does the value of an agent architecture depend on the capability of the model it wraps?*

We run a controlled grid of **14 architectures** (single-agent ablations, pre-execution context augmentation, post-execution verification, multi-agent collaboration) across **7 open and closed models** spanning a wide capability range, on the 64 "easy"-labelled tasks of TerminalBench, under a uniform **carved** 60-step budget shared among sub-agents rather than added to them. A separate *heterogeneous supervision* track has a strong model supervise a weaker worker, isolating whether capability can be injected at the coordination layer. The study comprises 7424 model–task runs.

The central result is that an architecture's value depends on the capability of the model it wraps, and the dependence is strong enough to reverse the direction of the effect. The number of architectures that beat a model's own baseline falls from 13 of 13 on one of the weakest models to 0 of 13 on the strongest open coder ($\rho = -0.886$, $p = 0.033$ across the open models). The ordering holds when the baseline is removed from the measure ($\rho = -0.829$) and under a split-half check ($\rho = -0.62$). The same scaffold that rescues a weak model burdens a strong one, an inversion that no single-model study can observe, so no architecture is best for every model, and for the strongest open coder the best choice is the plain baseline. The axis is coding capability rather than parameter count: the least-helped model is a 32B coder, not the largest model by parameters.

The per-run traces show why. Models most often fail by submitting a wrong answer early rather than by running out of budget, and the way they fail differs from model to model: the weakest abandon a task within a few steps, while others spend the whole budget repeating the same command. An architecture helps when it supplies something the model is missing, such as a reason to keep working or a check before submitting, and it hurts when it displaces something the model already does well or asks for an artifact the model cannot produce. The same pattern appears across task types. Scaffolding adds solves where a model still has room and loses them on tasks the model already solves, so the question worth asking is which architecture fits a given model and task.

A smaller, exploratory track asks whether capability can be injected at the coordination layer, with a strong model supervising a weaker worker. It can, through some roles and not others. A supervisor that *directs* the worker, by routing, planning, or guiding, helps every worker (+4, +5, +8 under orchestration). The same supervisor *judging* the worker's output does not transfer capability the same way and is net negative on the strongest worker (-6); its main effect is to make a worker steadier on what it can already do.

Because the step budget is shared among sub-agents rather than added to them, none of this comes from giving scaffolded runs a larger action budget. Scaffolding should be chosen against a model's measured capability and failure profile and against a task's headroom, not adopted as a default improvement.

Keywords: LLM agents, agent architectures, multi-agent systems, ReAct, scaffolding, TerminalBench, model capability, open-weight models, empirical evaluation.

Ευχαριστίες

Ολοκληρώνοντας την παρούσα διπλωματική εργασία, νιώθω την ανάγκη να ευχαριστήσω όσους στάθηκαν δίπλα μου. Ευχαριστώ θερμά τον επιβλέποντά μου, Αθανάσιο Βουλόδημο, για την ευκαιρία να εκπονήσω τη διπλωματική μου στο Εργαστήριο Συστημάτων Τεχνητής Νοημοσύνης και Μάθησης και για την εμπιστοσύνη και την ελευθερία που μου έδωσε να διαμορφώσω την έρευνά μου. Ευχαριστώ επίσης τη Μαρία Λυμπεραίου, τον Γιώργο Φιλανδριανό και την Αγγελική Δημητρίου για τη στενή συνεργασία και την καθοδήγησή τους σε όλη αυτή την πορεία. Τέλος, ευχαριστώ την οικογένειά μου και τους φίλους μου για την αμέριστη στήριξή τους.

Χαράλαμπος Ιωάννης Σφυρής

Contents

Contents 11

List of Figures 15

List of Tables 16

1 Εκτεταμένη Περίληψη στα Ελληνικά 21
- 1.1 Εισαγωγή και Κίνητρο . . . 21
  - 1.1.1 Ένα παράδειγμα του προβλήματος . . . 21
  - 1.1.2 Το κενό που καλύπτει η εργασία . . . 22
  - 1.1.3 Ερευνητικά ερωτήματα . . . 22
- 1.2 Ο Πράκτορας και οι Αρχιτεκτονικές . . . 23
  - 1.2.1 Το κοινό υπόβαθρο . . . 23
  - 1.2.2 Ο προϋπολογισμός: μοιράζεται, δεν προστίθεται . . . 23
  - 1.2.3 Οι δεκατέσσερις αρχιτεκτονικές . . . 23
  - 1.2.4 Το σκέλος ετερογενούς επίβλεψης . . . 23
- 1.3 Μεθοδολογία . . . 24
  - 1.3.1 Το πειραματικό πλέγμα . . . 24
  - 1.3.2 Το σύνολο εργασιών, και γιατί μόνο η εύκολη ζώνη . . . 25
  - 1.3.3 Πόσες εργασίες φέρουν πραγματικά πληροφορία . . . 25
  - 1.3.4 Τι μπορεί και τι δεν μπορεί να τεκμηριώσει ο σχεδιασμός . . . 25
- 1.4 Αποτελέσματα . . . 26
  - 1.4.1 Ο προϋπολογισμός τηρήθηκε, και τα επιπλέον βήματα δεν αποδίδουν περισσότερες λύσεις . . . 26
  - 1.4.2 Η βαθμίδα ικανότητας . . . 26
  - 1.4.3 Η ικανότητα δεν είναι το μέγεθος . . . 27
  - 1.4.4 Τι ακριβώς είναι η ζημιά στα ισχυρά μοντέλα . . . 27
  - 1.4.5 Πώς αποτυγχάνουν τα μοντέλα . . . 28
  - 1.4.6 Ποια αρχιτεκτονική για ποιο μοντέλο . . . 28
  - 1.4.7 Τι κάνει κάθε οικογένεια . . . 29
  - 1.4.8 Το δυαδικό αποτέλεσμα κρύβει το μεγαλύτερο μέρος της κίνησης . . . 30
  - 1.4.9 Τι απαντά η αρχιτεκτονική: το περιθώριο, όχι το θέμα . . . 30
  - 1.4.10 Συμπληρωματικότητα, και ένας κανόνας επιλογής . . . 31
- 1.5 Ετερογενής Επίβλεψη . . . 31
- 1.6 Συζήτηση . . . 32
  - 1.6.1 Μία αρχή για ολόκληρο το πλέγμα . . . 32
  - 1.6.2 Γιατί ο μέσος όρος παραπλανά . . . 33
  - 1.6.3 Η ανάλυση αποτυχιών δεν παράγει αρχιτεκτονική . . . 33
- 1.7 Απειλές στην Εγκυρότητα . . . 33
- 1.8 Συμπεράσματα . . . 34
  - 1.8.1 Πρακτικές συστάσεις . . . 34
  - 1.8.2 Συνεισφορές . . . 34
  - 1.8.3 Περιορισμοί . . . 35
  - 1.8.4 Μελλοντική εργασία . . . 35

2 Introduction 37
- 2.1 Motivation . . . 37

2.2 Problem statement . . . 37
2.3 A worked example of the problem . . . 38
2.4 Research questions . . . 38
2.5 Summary of findings . . . 39
2.6 Contributions . . . 40
2.7 Thesis outline . . . 40

3 Background and Related Work **43**
3.1 Foundations: language models as agents . . . 43
3.1.1 From text generation to environment interaction . . . 43
3.1.2 The ReAct loop . . . 43
3.1.3 Tool use and structured output . . . 44
3.1.4 Context as a physical constraint . . . 44
3.1.5 Reasoning models and test-time compute . . . 44
3.2 Agentic benchmarks . . . 45
3.2.1 TerminalBench . . . 45
3.2.2 The wider landscape . . . 45
3.2.3 What benchmarks report, and what they omit . . . 45
3.3 Agent architectures . . . 46
3.3.1 Planning and decomposition . . . 46
3.3.2 Structured exploration and grounding . . . 46
3.3.3 Runtime observation and intervention . . . 46
3.3.4 Verification, critique, and reflection . . . 47
3.3.5 Multi-agent collaboration . . . 47
3.4 The methodology of agent evaluation . . . 47
3.4.1 Separating the model from the scaffold . . . 47
3.4.2 Why multi-agent systems fail . . . 48
3.4.3 Evaluation cost, subset selection, and variance . . . 48
3.5 The gap this thesis addresses . . . 48
3.5.1 Position against the closest work . . . 49

4 The Agent, Its Architectures, and the Supervision Track **51**
4.1 The shared ReAct executor . . . 51
4.1.1 What a step is . . . 52
4.1.2 Tool interface . . . 52
4.1.3 Memory and context management . . . 52
4.1.4 No forced intervention . . . 53
4.2 Safety infrastructure . . . 53
4.2.1 What the harness is not allowed to do . . . 53
4.3 The budget, and why it is carved rather than added . . . 54
4.3.1 Why steps rather than wall-clock or tokens . . . 54
4.3.2 Tokens as a monitored quantity . . . 56
4.3.3 The “it just did more work” objection . . . 56
4.3.4 Residual caveats . . . 56
4.4 The failure modes the architectures target . . . 56
4.5 Summary of the fourteen . . . 57
4.5.1 What the fourteen leave out . . . 57
4.6 G1 — single-agent ablations . . . 59
4.7 G2 — pre-execution context augmentation . . . 59
4.8 G3 — post-execution verification . . . 61
4.9 G4 — multi-agent collaboration . . . 62
4.10 Fairness across all fourteen . . . 63
4.10.1 The alternative we rejected . . . 63
4.11 The heterogeneous supervision track . . . 64
4.11.1 Why separate the roles . . . 64
4.11.2 Design . . . 65
4.11.3 Interpreting the heterogeneous cells . . . 66

5 Experimental Methodology **67**
5.1 The experimental grid . . . 67

5.2 Models . . . 67
5.2.1 What is not in the lineup, and why . . . 67
5.2.2 Serving and quantization . . . 68
5.2.3 Sampling . . . 68
5.3 Dataset . . . 69
5.3.1 Why only the easy band, and what that costs . . . 69
5.3.2 The boundary probe . . . 70
5.3.3 Effective discriminative power . . . 70
5.4 Coverage . . . 70
5.5 Single-run validity . . . 70
5.6 Reading failures from the trace . . . 71
5.7 Statistical treatment . . . 72
5.8 Budget uniformity . . . 73
5.9 What the study consumed . . . 73
5.10 Task metadata . . . 73

6 Results **75**
6.1 The grid . . . 75
6.1.1 What the average hides . . . 76
6.1.2 Budget accounting: did the design hold? . . . 77
6.1.3 What counts as evidence . . . 78
6.2 Capability, not scale, conditions the effect . . . 78
6.2.1 The axis: capability, not parameter count . . . 78
6.2.2 The gradient across all seven models . . . 79
6.2.3 The same result without averaging . . . 81
6.2.4 Why each model responds as it does . . . 82
6.2.5 Three objections . . . 84
6.2.6 The answer to RQ1 . . . 86
6.3 How models fail, and what each architecture repairs . . . 87
6.3.1 How runs end, and how each model fails . . . 87
6.3.2 Inside the dominant failure: four shapes, and they track capability . . . 88
6.3.3 How the architectures fail . . . 90
6.3.4 Did the architectures repair what they were built for? . . . 91
6.3.5 The answer to RQ2 . . . 94
6.4 What makes an architecture help . . . 94
6.4.1 G1: the single-agent ablations . . . 94
6.4.2 G2 context augmentation: it works only when it runs . . . 96
6.4.3 G3 verification: the closest thing to a robust improver . . . 100
6.4.4 G4 multi-agent: coordination cost . . . 104
6.4.5 The answer to RQ3 . . . 106
6.5 What makes a task respond to architecture . . . 107
6.5.1 The category grid . . . 108
6.5.2 Did each family help the category its design implies? . . . 108
6.5.3 Difficulty, read and measured . . . 110
6.5.4 Task shape predicts the family . . . 111
6.5.5 The answer to RQ4 . . . 113
6.6 Model cooperation: heterogeneous supervision . . . 114
6.6.1 Supervision helps, and the architecture decides how much . . . 114
6.6.2 Orchestration: the cost was the routing, not the coordination . . . 115
6.6.3 Context augmentation under a strong supervisor . . . 116
6.6.4 Judging: a stronger judge approves better and rescues less . . . 119
6.6.5 The hardest test: does supervision add capability? . . . 121
6.6.6 The answer to RQ5 . . . 121
6.7 Synthesis . . . 122
6.7.1 Complementarity and the oracle bound . . . 122
6.7.2 A selection rule . . . 123
6.7.3 Cost–benefit . . . 124
6.7.4 There is no default architecture . . . 125

7 Discussion **127**

7.1 One account of the whole grid . . . 127
7.1.1 The deficit must exist . . . 127
7.1.2 The form must be usable . . . 128
7.1.3 The unpaid cost is the budget . . . 128
7.2 Why the aggregate view misleads . . . 128
7.3 Why each family behaves as it does . . . 129
7.3.1 Pre-execution decomposition . . . 129
7.3.2 Post-execution verification . . . 129
7.3.3 Multi-agent collaboration . . . 129
7.4 Injecting capability at the coordination layer . . . 130
7.5 Design guidance . . . 130
7.6 Why failure analysis does not yield architecture . . . 131

8 Threats to Validity **133**
8.1 Statistical conclusion validity . . . 133
8.2 Internal validity . . . 133
8.3 Construct validity . . . 134
8.4 External validity . . . 135

9 Conclusion and Future Work **137**
9.1 Summary . . . 137
9.2 Contributions revisited . . . 137
9.3 Limitations . . . 138
9.4 Future work . . . 138
9.4.1 Interventions the results argue for . . . 139

A Appendices **141**
A.1 The frozen analysis dataset . . . 141
A.1.1 Schema . . . 141
A.1.2 Two decisions that affect every figure . . . 142
A.1.3 Build and validation . . . 142
A.2 Per-architecture implementation notes . . . 142
A.3 Harness engineering . . . 143
A.4 Reading the subtest measure . . . 144
A.5 Task-shape membership . . . 145
A.6 Full per-model results . . . 146

B Bibliography **155**

List of Figures

1.4.1 Η αλληλεπίδραση μοντέλου×αρχιτεκτονικής. Κάθε κελί είναι η μεταβολή σε λυμένες εργασίες σε σχέση με τη γραμμή αναφοράς του ίδιου μοντέλου (σε παρένθεση δίπλα στο όνομα)· το μπλε είναι κέρδος, το κόκκινο ζημιά, η ένταση το μέγεθος. Οι γραμμές πάνε από το ασθενέστερο μοντέλο (πάνω) στο ισχυρότερο (κάτω): το πάνω μέρος είναι κυρίως μπλε και το κάτω κυρίως κόκκινο, ενώ ο μέσος όρος κάθε στήλης είναι σχεδόν μηδενικός: η μηδενική κύρια επίδραση και η μεγάλη αλληλεπίδραση σε μία εικόνα. 26

1.4.2 Σύνθεση των λανθασμένων υποβολών κάθε μοντέλου, με το ασθενέστερο πάνω. Διαβάζοντας προς τα κάτω διαβάζουμε αυξανόμενη ικανότητα: η ζώνη «παραιτήθηκε νωρίς» υποχωρεί και μεγαλώνει το «παρά τρίχα». 29

4.1.1 The shared ReAct executor. Every architecture in this thesis wraps, precedes, interleaves with, or multiplies this loop; none of them replaces it. 51

4.7.1 The three pre-execution topologies. Shaded boxes cannot modify the environment. Each sub-agent's steps are subtracted from the executor's 60, so the three differ in *how much executor budget they trade for context*, not in how much total work is permitted. 60

4.8.1 The four post-execution variants, ordered by *how much of the first attempt the second look inherits*. All four target the same failure mode and all four spend only the budget the phase-1 agent left behind; they differ on one axis. Section 6.4.3 finds that axis (context continuity) to be what separates them. 61

4.9.1 The two multi-agent topologies. G4-Cooperation and G4-Debate share this diagram and differ by a single instruction: whether B continues A's work or challenges it, which is what makes their difference attributable to *critique* rather than to multi-agency. In G4-Orchestrator the strategist is the sole information bridge (dashed returns), and its routing steps are free while the specialists share the same 60. 63

4.10.1 Carved versus additive budgets, drawn to scale, with G2-Planner as the worked example. The sub-agent caps are flat across models: the Explorer may use at most ten steps and the Planner at most five, so in this heaviest pre-execution case the executor is left 45. What the phases actually consume varies, but the total does not: under the carved design a scaffold's sub-agents are paid for out of the executor's allocation, and no run in the grid exceeded 60 reasoning opportunities. Under the additive alternative the scaffold would receive more total agency than the control, and any improvement it showed would be confounded with the extra budget. The cost of the carved choice is stated in Section 4.10: it cannot separate an architecture that is badly conceived from one that is merely expensive. 64

4.11.1 The heterogeneous supervision split. Routing is per *role*, not per run: every component that acts in the environment uses the worker model, every component that only reasons about the work uses the supervisor. Everything below the dashed line (model, budget, tools, prompt, task set) is identical to the corresponding homogeneous cell, so the only variable is which model answers the supervisory call. 65

6.1.1 The model×architecture interaction. Each cell is the change in tasks solved relative to that model's own baseline (parenthesised after the model name); blue is a gain, red a loss, saturation is magnitude, and the signed number is exact. Rows run from the weakest model (top) to the strongest (bottom). Cell deltas span −7 to +6 tasks, yet every column mean is within a few points of zero (Table 6.2): this is the null main effect and the large interaction of Table 6.1 in a single view. 76

6.2.1 The capability gradient. Each point is one model: its baseline solve count against how many of the 13 non-baseline architectures beat that baseline. The relationship is negative and steep. 80
6.3.1 Failure-mode composition of each model's wrong submissions, weakest model on top. Reading down is reading rising capability: the bailed band gives way to near-miss. 89
6.3.2 Failure-mode composition by family, over each family's wrong submissions. Reading top to bottom is reading increasing intervention: the bailed and looped bands shrink and the near-miss band grows, but no family closes the gap to a solve. 91
6.3.3 Change in each architecture's stuck-loop rate against the baseline, under a repeat policy byte-identical across all configurations, so any difference is the architecture. Left of zero is fewer loops. Only the monitor — the one design built against stagnation — and the temperature ablation reduce looping, and only barely; the rest raise it, the gate most. Cooperation and debate are unmeasured: their repeat counter does not survive the two-agent hand-off. 93
6.4.1 How each model answers the G3-Gate's "are you sure?", which holds a submission at most twice. *Re-submits at once* is finalising with no further edits; *one* and *two rounds* went back to edit before submitting again. Models that re-verify are doubting their own work; those that re-submit unchanged are confident or cannot see the fault. 103
6.4.2 Share of the G4-Orchestrator Strategist's routing calls sent to each specialist, per model; the three shares sum to 100. Models are ordered by baseline capability. 106
6.6.1 Routing share across the three specialists, each worker routing for itself (grey) versus Gemini as Strategist (blue); the three shares sum to 100. 116

List of Tables

1.1 Οι 14 αρχιτεκτονικές κατά οικογένεια και μηχανισμό. Η G1 δεν προσθέτει πράκτορες · η G2 παρεμβαίνει πριν ή κατά την εκτέλεση · η G3 μετά την υποβολή · η G4 αντικαθιστά τον μοναδικό πράκτορα με περισσότερους. 24

1.2 Η βαθμίδα ικανότητας. Καθώς ανεβαίνει η γραμμή αναφοράς ενός μοντέλου, μειώνεται το πλήθος των αρχιτεκτονικών που την ξεπερνούν. Η τελευταία στήλη είναι η ένωση των εργασιών που λύνονται από *οποιαδήποτε* αρχιτεκτονική, ένα άνω φράγμα για επιλογή ανά εργασία. 27

1.3 Τι προβλέπει την απόκριση κάθε μοντέλου στην αρχιτεκτονική. Καμία από τις τρεις μεσαίες στήλες δεν είναι ποσοστό επιτυχίας: είναι πόσα βήματα κάνει το μοντέλο πριν αποτύχει (μέσος όρος στις αποτυχημένες εκτελέσεις αναφοράς), ποια είναι η κυρίαρχη αποτυχία του, και αν ο μηχανισμός της αρχιτεκτονικής λειτουργεί πάνω του, δηλαδή το ποσοστό των εκτελέσεων του G2-Planner στις οποίες παρήχθη έγκυρο σχέδιο. Οι δύο τελευταίες στήλες μετριούνται σε διαφορετικά κελιά του πλέγματος: η ικανότητα στη γραμμή αναφοράς, η συμμόρφωση εκεί όπου απαιτείται δομημένη έξοδος. 29

3.1 Where each gap statement is addressed. 49

3.2 The six closest works, by what each holds fixed and what it varies. The last row is this thesis. 49

4.1 How the 60-step budget is shared within each architecture family. No configuration gets more environment-acting steps than the baseline; read-only supervisory calls that only observe or route (the monitor's checks, the strategist's routing) are a separate layer and are not counted against the cap. 54

4.2 The recurring baseline failure modes, in order of observed frequency across the baseline runs of all 7 models. The first is the design target of the verification family, the second of the loop-breaking architectures; weak grounding is a frequent *cause* of the first rather than a separate way for a run to end. 57

4.3 The 14 architectures: mechanism, targeted failure mode, and canonical source. 58

4.4 Role assignment in the heterogeneous track. The worker always executes; the supervisor never touches the environment. 65

5.1 The 7 models, ordered by measured baseline capability rather than by parameter count. Baseline is G1-Base solves out of 64. 67

5.2 Served model artifacts: exact checkpoint, serving path, and context window, for reproducibility. All open models are AWQ 4-bit under vLLM; the hosted model is served over an OpenAI-compatible API. 69

5.3 What each task category concretely involves — the deliverable and setting, read from the task statements rather than judged. This orients the benchmark's topic labels used throughout the task-type analysis (Section 6.5); it is description, not a judgement of difficulty or required skill. 74

6.1 The master grid: tasks solved out of 64 by each model under each architecture. Green marks an improvement over that model's own baseline (bold), red a regression. Models are ordered by baseline capability. 75

6.2 Mean effect of each architecture across all seven models. No architecture moves the aggregate by more than a few points: the main effect is near zero while the per-model effects of Table 6.1 are large. 76

6.3 Resource consumption of each architecture relative to the same model's own baseline, averaged over the seven models. The 60-step *cap* is identical for every configuration; what varies is how much of it is consumed. Values above 1 mean the architecture used more of the budget it already had, not that it was given more. 77
6.4 Mean steps consumed by solved versus failed runs, pooled over all 14 architectures. In every model, runs that succeed finish in roughly half the steps of runs that fail. 78
6.5 The capability gradient. As a model's baseline rises, the number of architectures that beat it falls, and the best and worst available architectures move down with it. 79
6.6 Paired percentile bootstrap intervals (10,000 resamples over *tasks*) on the difference in solves against each model's own baseline, for the architecture that helps it most and the one that harms it most. 81
6.7 Every model–architecture–task comparison against that model's own baseline. A *win* is a task the baseline failed and the architecture solved, a *loss* the reverse, and *net* is wins minus losses; *moved* is the share of that model's or family's comparisons that were either. The remaining comparisons left the outcome unchanged and carry no information about a difference. 82
6.8 What predicts a model's response to architecture. None of the three middle columns is a solve rate: they are how long the model persists before failing, what its dominant failure is, and whether the scaffold's own machinery functions on it (share of G2-Planner runs producing a schema-valid plan). Models are ordered by baseline capability. 82
6.9 Command chaining per model, from baseline runs: total `bash` calls over the 64 baseline runs and the share of steps that pack several commands into one without inspecting the intermediate output. The rate does not track baseline capability; Devstral-24B is the outlier. 84
6.10 How each model's *failed* baseline runs ended, as a share of that model's failures, with the number of failed runs and the median steps before failing. The *other* column is mostly context overflow, a harness-epoch artifact rather than a model property (Section 8.2). . . 87
6.11 What a wrong submission actually was, read mechanically from each model's failed baseline runs that ended in one (Section 5.6): *bailed* if it quit within ten steps below the near-miss bar, *looped* if it spent itself repeating one command, *off-track* if a full-length attempt still passed under half, *near-miss* if it passed at least half the graders' checks. Ordered by baseline capability. 88
6.12 Runs containing a stuck action loop — five or more identical consecutive shell commands — among each model's *baseline* runs. One configuration for every model, so the spread is a property of the model; looping runs almost never recover, and the order is not the capability order. 89
6.13 How runs end, per architecture family, as a share of that family's *failed* runs, with the bare baseline as the reference (the same failures-only basis as Table 6.10). *Ran out* is the step and wall-clock budgets; *other* is mostly context overflow, a harness-epoch artifact (Section 8.2); *new failure modes* are the cycle cap and coordination failure, which only a multi-agent scheme can reach. 90
6.14 The four sub-modes of Table 6.11, now read over each family's wrong submissions rather than each model's. Cooperation and debate are omitted: their repeat counter does not survive the two-agent hand-off, so looped cannot be measured (Figure 6.3.3). 90
6.15 Did each architecture suppress the failure it was built for? Each row takes the coarse target the design chapter assigned it (Table 4.3), maps it to the mechanical mode that is its sharpest measurable form (a wrong submission → bailed, loops → looped), and reads that mode's incidence among the architecture's runs against the baseline rate. Ordered by family. The Δ cell is shaded by outcome: green reduced its target by at least four points, amber by less, red increased it. Cooperation and debate carry no measurable counter — the repeat counter does not survive their two-agent hand-off — so their loop target cannot be scored. 92
6.16 For every model–task pair that flips from failure to solve, the sub-mode of the baseline failure it started from, against how often that sub-mode occurs among all baseline failures. The two columns track each other: no sub-mode is rescued out of proportion to how often it appears — except context exhaustion, rescued least. 93
6.17 Whether the G2 sub-agent produced a *usable* artifact. Percentages are of runs in which the sub-agent ran at all. Models are ordered by baseline capability, which is visibly not the ordering of this table. 96

6.18 G2-Planner decomposition, over valid plans only. *Hard → easy*: mean subtask count on the hardest tasks (at most one baseline solver) against the easiest (at least five); a gap means harder tasks are split into more parts. Δ columns compare solves against baseline and Explorer on the valid-plan tasks only. *Busiest share*: fraction of executor steps taken by the most active sub-executor when a plan has two or more subtasks. The two smallest Qwen coders, which rarely emit a valid plan, are omitted. . . . 98

6.19 How the G2-Monitor behaves per model. *Base steps* is the mean steps the bare baseline takes before failing, which caps how many of the ten-step check-ins the Monitor gets (a model that stops at 13 steps allows at most one). *Injections* is the median guidance injections per run; *obeyed* is the share after which the executor's next command differs from the one before it — the rest reissue it verbatim. Δ vs base is the change in solves against the baseline. . . . 99

6.20 Whether the verifier gets to run, and what the Judge decides. *Submitted* and *engaged* pool the three verifier variants; the gap between them is submissions the budget swallowed before the verifier could act. *Approve/needs-fix* are the Judge's verdicts only (the retry variants issue none). . . . 100

6.21 How often the two dialogue architectures reach consensus before exhausting the budget. A run *converges* when both agents submit the same answer before the eight-turn cap; at the cap they never agreed. *Median turns* is over all runs. Cooperation and Debate side by side isolate what the adversarial stance does to convergence — which, as the columns show, is almost nothing. . . . 104

6.22 The architecture that helps each model most, and its gain over that model's own baseline. Models are ordered by baseline capability. . . . 107

6.23 Solve-rate change over baseline, in percentage points, by task category and architecture (pooled over the seven models). Categories are TerminalBench's own labels; cells are small (1–13 tasks × 7 models) and no significance is claimed. . . . 108

6.24 Solve-rate change over baseline by architecture family and category (percentage points, pooled over the seven models). *Target* is the family the design implies for that category (Section 4.5), assigned from the category's description alone; *Best* is the observed best family (bold in the row), shaded green when it matches the target and red when it does not. *None* marks a category where no family improves on baseline, so there is no winner to match. . . . 109

6.25 Architectural effect binned by two difficulty measures readable before running anything: the benchmark's expert-minute estimate and the length of its reference solution. Neither orders the effect. The Δ column is the mean solve-rate gain of the thirteen architectures over baseline within the bin. Tasks lacking the relevant metadata are omitted, so the bins total 50 and 58 of the 64 tasks respectively. . . . 110

6.26 Architectural effect by task headroom, in solves. Tasks are grouped by how many of the 7 models already solve them at baseline; each band offers tasks × 7 solves. *Net* is the mean, over the thirteen architectures, of solves gained minus solves lost across the whole band; *Worst* and *Best* are the least and most any single architecture nets. . . . 111

6.27 The 64 tasks partitioned by which models ever solve them, across both tracks. For the six open-weight models the last two tiers are indistinguishable: what separates them is entirely Gemini-3.1-FL. . . . 111

6.28 Architectural response by *task shape* rather than topic. *Pre-exec* is G1–G2 and *Post-exec* G3–G4; Δ is each family's mean subtest gain over the shape's baseline, and *Best* names the single strongest architecture with its gain. . . . 112

6.29 Mean subtest gain over baseline by architecture family, per category (categories with at least three tasks). *Pre-execution* is G1–G2 and *post-execution* G3–G4; *positive* counts how many of each family's members beat baseline. . . . 113

6.30 Effect of replacing the supervisory role with Gemini: each cell is the supervised solve count minus the worker's own homogeneous count for that architecture. Workers are ordered by baseline capability, with their solo baseline solves in parentheses. . . . 114

6.31 G4-Orchestrator with the Strategist role given to Gemini, against the same worker strategising for itself. *Cycle-cap* is the share of runs that exhaust the routing budget without submitting. . . . 115

6.32 The same task, `sql-injection-attack`, under Qwen-32B's own strategist and under a Gemini strategist. Both must locate the vulnerable query in `vulnerable_login.py` before an exploit can be written. Qwen-32B solves this task under none of its fourteen own architectures; under a Gemini strategist it does. 116
6.33 Plans for `jsonl-aggregator`: Llama-70B's own against Gemini's, both planning for Llama-70B. Gemini names the glob, the fields, the two rankings and the rounding rule; the worker restates the title. 117
6.34 Plans for `vimscript-vim-quine`: Llama-70B's own against Gemini's. Gemini's second subtask is an empirical verification loop; the worker's criterion ("works as expected") cannot be checked. 118
6.35 Three levels of architectural choice. Selecting the single best architecture for a model is worth between +0 and +6 tasks and collapses at the top of the capability range; selecting the right architecture *per task* raises the bound by a further 6 to 14 tasks, though against single-run noise this excess is real only on the models that still have headroom. 122
6.36 Cost per solved task, pooled over all models (traced runs only, so a solve count can differ by one from Table 6.1), sorted by token efficiency. Steps are capped identically but tokens and time are not, so this is the axis on which architectures genuinely differ in cost. The comparison is against G1-Base, which is neither the cheapest nor the dearest. 124
6.37 Which architecture is best in each (model, task-category) cell, over the 70 cells with at least three tasks. Most cells have no winner, and no architecture wins many of those that do. 125

A.1 Column groups in `runs.csv`. 141
A.2 Baseline performance measured two ways. Every model completes far more of the graders' checks than its pass/fail rate suggests; the gap is work done that the binary metric discards. 144
A.3 The task list behind each shape of Table 6.28, and the unclassified remainder. The five shapes hold the 25 tasks in the top block; the 39 left out are either never solved in the homogeneous grid (no shape can be read from a task that does not move) or turn on domain knowledge or a mixture of shapes rather than one clean shape. 145
A.4 Qwen2.5-Coder-7B: every task under every architecture. • solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4). 147
A.5 Llama-3.1-70B: every task under every architecture. • solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4). . . 148
A.6 Qwen2.5-Coder-14B: every task under every architecture. • solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4). 149
A.7 Devstral-24B: every task under every architecture. • solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4). . . 150
A.8 Qwen2.5-72B: every task under every architecture. • solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4). . . 151
A.9 Qwen2.5-Coder-32B: every task under every architecture. • solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4). 152
A.10 Gemini-3.1-FL: every task under every architecture. • solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4). . . 153

Chapter 1

Εκτεταμένη Περίληψη στα Ελληνικά

1.1 Εισαγωγή και Κίνητρο

Τα μεγάλα γλωσσικά μοντέλα λειτουργούν ολοένα και περισσότερο ως *πράκτορες*: αντί να παράγουν μία απάντηση, εκτελούν εντολές φλοιού, διαβάζουν και γράφουν αρχεία, τρέχουν προγράμματα, παρατηρούν τα αποτελέσματα και επαναλαμβάνουν μέχρι να πετύχουν έναν στόχο μέσα σε ένα πραγματικό περιβάλλον. Η μετατόπιση αυτή έχει γεννήσει ένα εκτεταμένο σώμα εργασιών γύρω από τις *αρχιτεκτονικές υποστήριξης* (scaffolding): μηχανισμούς που τοποθετούνται γύρω από το βασικό μοντέλο ώστε να το κάνουν καλύτερο πράκτορα, όπως ρητό σχεδιασμό και αποσύνθεση εργασιών, δομημένη εξερεύνηση του περιβάλλοντος πριν τη δράση, εκ των υστέρων αυτο-επαλήθευση, κριτική και αναστοχασμό, και συνεργασία πολλαπλών εξειδικευμένων πρακτόρων.

Δύο ιδιότητες της βιβλιογραφίας αυτής περιορίζουν το τι μπορεί να συμπεράνει κανείς. Πρώτον, η αξιολόγηση γίνεται συνήθως σε *ένα* μόνο μοντέλο, τις περισσότερες φορές κορυφαίο. Δεύτερον, αναφέρονται *συνολικά* ποσοστά επιτυχίας, συχνά χωρίς να κρατείται σταθερός ο υπολογιστικός προϋπολογισμός μεταξύ του πράκτορα αναφοράς και του επαυξημένου συστήματος: μια αρχιτεκτονική που προσθέτει έναν σχεδιαστή και έναν επαληθευτή προσθέτει και τις κλήσεις μοντέλου που αυτοί κάνουν.

Καμία από τις δύο δεν ισχύει καθολικά. Υπάρχουν εργασίες που εξισώνουν ρητά τον προϋπολογισμό σκέψης, και εργασίες που συγκρίνουν δεκάδες μοντέλα. Αυτό που δεν έχει γίνει είναι ο συνδυασμός τους: να μεταβληθούν *ταυτόχρονα* η αρχιτεκτονική και η ικανότητα του μοντέλου, με ταυτόσημη υλοποίηση και διατηρούμενο προϋπολογισμό. Αυτό το κενό κρύβει ένα ερώτημα με πρακτική σημασία: όταν το μοντέλο *δεν* είναι κορυφαίο, ένα κβαντισμένο μοντέλο ανοικτών βαρών 7B–70B σε μία προσιτή κάρτα γραφικών, βοηθά ακόμα η ίδια αρχιτεκτονική; Ή μήπως η αξία της μεταβάλλεται ανάλογα με την ικανότητα του μοντέλου που περιβάλλει; Το ερώτημα δεν είναι λογιστικό. Μια αρχιτεκτονική που προσθέτει ένα δεύτερο πέρασμα συλλογιστικής μπορεί να αποδώσει μόνο αν το πρώτο πέρασμα του μοντέλου είναι *αναθεωρήσιμο*: αν το μοντέλο λύνει ήδη την εργασία με την πρώτη προσπάθεια, το δεύτερο πέρασμα είναι καθαρό κόστος, και αν αποτυγχάνει με τρόπο από τον οποίο δεν μπορεί να ανακάμψει, το δεύτερο πέρασμα δεν έχει τι να σώσει. Ο συλλογισμός αυτός προβλέπει ότι η αξία της αρχιτεκτονικής εξαρτάται από την ικανότητα, πρόβλεψη που δεν έχει ελεγχθεί υπό ελεγχόμενες συνθήκες.

1.1.1 Ένα παράδειγμα του προβλήματος

Δύο εργασίες καθιστούν τη δυσκολία συγκεκριμένη. Και οι δύο δέχονται την *ίδια* φάση εξερεύνησης, έναν πράκτορα μόνο-ανάγνωσης που επιθεωρεί το περιβάλλον και αναφέρει τι βρήκε πριν δράσει ο εκτελεστής, και όμως τα ίχνη δείχνουν τη φάση αυτή να κάνει αντίθετα πράγματα στις δύο.

Η `log-summary` ζητά να μετρηθούν οι γραμμές `ERROR`, `WARNING` και `INFO` σε τρία μικρά αρχεία καταγραφής και να γραφτούν τα σύνολα σε CSV. Στη γραμμή αναφοράς το Devstral-24B ξοδεύει και τα 60 βήματα χωρίς ποτέ να γράψει το αρχείο. Στο πέμπτο βήμα εκδίδει τη σωστή εντολή και το τερματικό επιστρέφει τους σωστούς εννέα αριθμούς, ανακατεμένους και χωρίς ετικέτες: τρεις μετρήσεις ανά αρχείο, χωρίς τίποτα να δηλώνει σε ποια σοβαρότητα ανήκει η καθεμία. Ο πράκτορας επανεκδίδει την ίδια εντολή **55 φορές** και εξαντλεί το όριο των βημάτων.

Με τη φάση εξερεύνησης το ίδιο μοντέλο τελειώνει σε επτά βήματα. Η σύνοψη του εξερευνητή μεταφέρει ολόκληρο το περιεχόμενο και των τριών αρχείων. Ο εκτελεστής τρέχει και αυτός τις δικές του εντολές, λαμβάνει την ίδια αναξιοποίητη έξοδο, και στο πέμπτο βήμα γράφει `ERROR,4 WARNING,3 INFO,8`, τιμές που προκύπτουν από το κείμενο της σύνοψης και από τίποτε άλλο που είδε. Αυτό που προσέφερε η αρχιτεκτονική δεν ήταν στρατηγική αλλά *αναγνώσιμο κανάλι* για πληροφορία που το τερματικό απέτυχε να παραδώσει.

Η `fix-git` ζητά την ανάκτηση δουλειάς που δεν βρίσκεται σε κανένα κλαδί και τη συγχώνευσή της. Στη γραμμή αναφοράς το Qwen-32B περνά κάθε έλεγχο σε 34 βήματα: στο δωδέκατο τρέχει `git reflog`, εντοπίζει μια μη αναφερόμενη υποβολή, την ανακτά και επιλύει τη σύγκρουση.

Με την ίδια φάση εξερεύνησης αποτυγχάνει σε κάθε έλεγχο. Ο εξερευνητής επιθεωρεί είκοσι ένα αρχεία, όλα τα PDF και τις μικρογραφίες του αποθετηρίου, και επιστρέφει σύνοψη με κενό πεδίο απαιτήσεων και παρατηρήσεις για ερευνητικές εργασίες. Η απάντηση δεν βρίσκεται σε αρχείο αλλά στο ημερολόγιο αναφορών, που ένας επιθεωρητής αρχείων δεν μπορεί να δει. Ο εκτελεστής ψάχνει στο ιστορικό που ήδη προσεγγίζει, βρίσκει αλλαγές που είναι ήδη συγχωνευμένες, και υποβάλλει ότι δεν χρειάζεται καμία ενέργεια. Δεν τρέχει ποτέ `git reflog`.

Η αρχιτεκτονική δεν έγινε καλύτερη ή χειρότερη ανάμεσα στις δύο εργασίες. Αυτό που άλλαξε είναι το αν όσα προσέφερε ήταν αυτά που έλειπαν, και ο μηχανισμός φαίνεται στις ίδιες τις εκτελέσεις αντί να συμπεραίνεται από το αποτέλεσμα.

1.1.2 Το κενό που καλύπτει η εργασία

Η βιβλιογραφία τεκμηριώνει τρία πράγματα: ότι η υποστήριξη πρακτόρων παίρνει έναν μικρό αριθμό αναγνωρίσιμων μορφών, ότι η αξιολόγησή της γίνεται συμβατικά σε ένα μοντέλο και χωρίς έλεγχο του υπολογιστικού κόστους, και ότι ο διαχωρισμός μεταξύ ικανότητας μοντέλου και συνεισφοράς αρχιτεκτονικής είναι υπαρκτός. Αυτό που δεν περιέχει είναι μια ελεγχόμενη μέτρηση της *αλληλεπίδρασης* των δύο. Τρεις προτάσεις οριοθετούν τη συνεισφορά, καθεμία διατυπωμένη έναντι ονομαστικών εργασιών ώστε να είναι ελέγξιμη:

1. Καμία δημοσιευμένη εργασία δεν εκτελεί ελεγχόμενη μελέτη αρχιτεκτονικής × ικανότητας μοντέλου σε εργασίες πράκτορα τερματικού, με *ταυτόσημη* υλοποίηση υποστήριξης και *διατηρούμενο* υπολογιστικό προϋπολογισμό.
2. Καμία εργασία δεν απομονώνει τον άξονα «επεξεργασία πριν την εκτέλεση» έναντι «επαλήθευση μετά την εκτέλεση» σε πολλαπλές οικογένειες ανοικτών μοντέλων υπό κοινό προϋπολογισμό. Οι δύο οικογένειες προτείνονται τυπικά χωριστά και αξιολογούνται σε διαφορετικά μοντέλα.
3. Καμία εργασία δεν αξιολογεί την *ετερογενή επίβλεψη*, δηλαδή ένα ισχυρό μοντέλο σε εποπτικό ρόλο ενώ ένα ασθενέστερο εκτελεί, ως ελεγχόμενη μεταβλητή δίπλα στις ομοιογενείς διαμορφώσεις με τις οποίες πρέπει να συγκριθεί.

1.1.3 Ερευνητικά ερωτήματα

Πέντε ερωτήματα οργανώνουν τη μελέτη. Δεν είναι ανεξάρτητα: το πρώτο θεμελιώνει ένα αποτέλεσμα, τα τρία επόμενα το εξηγούν σε τρία διαφορετικά επίπεδα, και το πέμπτο ρωτά αν το όριο που αυτό αποκαλύπτει μπορεί να ξεπεραστεί από έξω.

- **EE1 — Εξάρτηση από την ικανότητα.** Βελτιώνει η αρχιτεκτονική υποστήριξης την επίδοση *ομοιόμορφα*, ή η επίδρασή της, μαζί με το *πρόσημό* της, εξαρτάται από την ικανότητα του υποκείμενου μοντέλου; Και υποκαθίσταται αυτή η ικανότητα από το μέγεθος του μοντέλου, ώστε η κλίμακα να μπορεί να χρησιμεύσει ως άξονας στη θέση της;
- **EE2 — Δομή των αποτυχιών.** Πώς αποτυγχάνουν στην πράξη οι εκτελέσεις, πώς διαφέρουν αυτές οι αποτυχίες ανά μοντέλο, και τι διορθώνει ή εισάγει κάθε αρχιτεκτονική; Μια αρχιτεκτονική μπορεί να βοηθήσει μόνο όπου συμβαίνει η αποτυχία που αντιμετωπίζει.
- **EE3 — Μηχανισμός αρχιτεκτονικής.** Τι κάνει μια αρχιτεκτονική να βοηθά αντί να βλάπτει, ποια ιδιότητα του σχεδιασμού καθορίζει αν βελτιώνει μια εκτέλεση; Και αποτελεί κάποια αρχιτεκτονική γενική βελτίωση, ή η αποτελεσματική επιλογή εξαρτάται από το μοντέλο;
- **EE4 — Ιδιότητες της εργασίας.** Ποιες ιδιότητες μιας εργασίας προβλέπουν *αν* μια αρχιτεκτονική θα τη βοηθήσει, και *ποια* οικογένεια βοηθά περισσότερο; Και τι κοστίζει η απόκτηση κάθε τέτοιας ιδιότητας: διαβάζεται από την εκφώνηση, ή πρέπει πρώτα να εκτελεστεί η εργασία;

- **EE5 — Ετερογενής επίβλεψη.** Μπορεί ένας ισχυρός επιβλέπων που καθοδηγεί έναν ασθενέστερο εργάτη να ξεπεράσει τον ομοιογενή ασθενή πράκτορα, και μέσω ποιων ρόλων; Δηλαδή: μπορεί η ικανότητα να εγχυθεί στο επίπεδο του συντονισμού αντί της εκτέλεσης;

1.2 Ο Πράκτορας και οι Αρχιτεκτονικές

1.2.1 Το κοινό υπόβαθρο

Και οι 14 διαμορφώσεις χτίζονται πάνω στον ίδιο εκτελεστή ReAct: έναν βρόχο που εναλλάσσει συλλογιστική και δράση μέσα σε ένα απομονωμένο περιβάλλον Docker. Ο πράκτορας διαθέτει έξι εργαλεία: εκτέλεση εντολής φλοιού (`bash`), ανάγνωση αρχείου (`read_file`), εγγραφή αρχείου (`write_file`), επεξεργασία αρχείου (`patch_file`), υποβολή (`submit`), και ένα εργαλείο `think` που καταναλώνει βήμα χωρίς να αγγίζει το περιβάλλον. Ένα *βήμα* ορίζεται ως μία κλήση μοντέλου που καταλήγει σε κλήση εργαλείου, ώστε η μονάδα του προϋπολογισμού να είναι η ευκαιρία δράσης και όχι ο χρόνος ή τα tokens.

Το γεγονός ότι όλες οι αρχιτεκτονικές μοιράζονται τον ίδιο κώδικα εκτελεστή έχει μεθοδολογική σημασία: όταν η ίδια αρχιτεκτονική βοηθά ένα μοντέλο και βλάπτει ένα άλλο, η διαφορά δεν μπορεί να αποδοθεί σε διαφορετική υλοποίηση.

1.2.2 Ο προϋπολογισμός: μοιράζεται, δεν προστίθεται

Κάθε διαμόρφωση λειτουργεί υπό τον ίδιο πρωτεύοντα προϋπολογισμό: **60 βήματα συλλογιστικής**, με δίχτυ ασφαλείας 1800 δευτερολέπτων που προορίζεται να μη δεσμεύει.

Η κρίσιμη ιδιότητα είναι ότι μια αρχιτεκτονική πολλαπλών πρακτόρων *δεν* λαμβάνει επιπλέον βήματα: ο προϋπολογισμός των 60 βημάτων **μοιράζεται** μεταξύ των συνιστωσών της, ώστε το συνολικό πλήθος βημάτων να διατηρείται στα $\leq$60 ανεξαρτήτως πλήθους πρακτόρων. Κάθε αρχιτεκτονική λαμβάνει επομένως τον *ίδιο αριθμό ευκαιριών δράσης* με τη γραμμή αναφοράς, και η σύγκριση απομονώνει το *πώς οργανώνεται* η εργασία και όχι το *πόση* επιτρέπεται.

Το μερίδιο κάθε βοηθητικής φάσης είναι φραγμένο εκ των προτέρων, ώστε ο διαμοιρασμός να μη μετατρέπεται σε στέρηση του εκτελεστή: ο εξερευνητής δεν ξεπερνά τα δέκα βήματα και ο σχεδιαστής τα πέντε, οπότε στη βαρύτερη περίπτωση, τον G2-Planner, στον εκτελεστή μένουν σαράντα πέντε από τα εξήντα. Σε καμία εκτέλεση του πλέγματος το σύνολο δεν ξεπέρασε τα εξήντα.

Ως μονάδα προϋπολογισμού επιλέγεται το βήμα και όχι ο χρόνος ή τα tokens, επειδή είναι η μόνη ανεξάρτητη από το υλικό: ο χρόνος συγχέεται με την ταχύτητα εξυπηρέτησης, ενώ ένα όριο tokens θα τιμωρούσε συστηματικά τα μοντέλα με μεγάλο παράθυρο συμφραζομένων.

1.2.3 Οι δεκατέσσερις αρχιτεκτονικές

Οι αρχιτεκτονικές οργανώνονται σε τέσσερις οικογένειες κατά το *πότε* παρεμβαίνει ο μηχανισμός σε σχέση με την εκτέλεση. Καθεμία υλοποιεί ένα δημοσιευμένο σχέδιο και στοχεύει μια συγκεκριμένη αποτυχία που εντοπίστηκε σε ανάλυση ιχνών των εκτελέσεων αναφοράς.

Οι παραλείψεις είναι σκόπιμες. Σχέδια που πολλαπλασιάζουν τις κλήσεις μοντέλου κατά έναν παράγοντα διακλάδωσης, όπως η αναζήτηση σε δέντρο σκέψης, δεν χωρούν σε μοιρασμένο προϋπολογισμό. Σχέδια των οποίων η προϋπόθεση δεν ισχύει εδώ, όπως ο σχεδιασμός με ανάκτηση από σώμα λυμένων προβλημάτων, αποκλείστηκαν επειδή δεν υπάρχει τέτοιο σώμα. Και η πλήρης ιεραρχική αποσύνθεση με τέσσερις ή περισσότερους ρόλους θα έδινε σε κάθε ειδικό περίπου δώδεκα βήματα, οπότε ένα κακό αποτέλεσμα δεν θα ξεχώριζε την κακή αρχιτεκτονική από την υποχρηματοδοτημένη.

1.2.4 Το σκέλος ετερογενούς επίβλεψης

Και οι δεκατέσσερις αρχιτεκτονικές είναι *ομοιογενείς*: κάθε συνιστώσα τους, εκτελεστής, εξερευνητής, σχεδιαστής, επόπτης, κριτής, στρατηγός, είναι το ίδιο υποκείμενο μοντέλο. Το σκέλος της ετερογενούς επίβλεψης σπάει αυτή την παραδοχή, και υπάρχει για να διακρίνει δύο υποθέσεις που τα ομοιογενή δεδομένα δεν ξεχωρίζουν.

Κατά την πρώτη, το όφελος προέρχεται από τη *δομή*: το να υπάρχει σχέδιο είναι καλύτερο από το να μην υπάρχει, ανεξάρτητα από το ποιος το παράγει. Κατά τη δεύτερη, το όφελος προέρχεται από την *ποιότητα* της εποπτικής εξόδου, οπότε ένα ασθενές μοντέλο που επιβλέπει τον εαυτό του κερδίζει λίγα επειδή παράγει ασθενές σχέδιο.

Table 1.1: Οι 14 αρχιτεκτονικές κατά οικογένεια και μηχανισμό. Η G1 δεν προσθέτει πράκτορες · η G2 παρεμβαίνει πριν ή κατά την εκτέλεση · η G3 μετά την υποβολή · η G4 αντικαθιστά τον μοναδικό πράκτορα με περισσότερους.

Αρχιτεκτονική	**Οικογ.**	**Μηχανισμός**
G1-Base	G1	Απλός εκτελεστής ReAct, ελάχιστο prompt, πλήρη 60 βήματα. Η διαμόρφωση ελέγχου.
G1-Prompt	G1	Η βάση συν τρεις οδηγίες: επιθεώρησε τα δεδομένα, γράψε σε αρχεία, επαλήθευσε πριν υποβάλεις.
G1-Temp	G1	Η βάση με θερμοκρασία δειγματοληψίας 0.2 αντί για greedy.
G2-Explorer	G2	Εξερευνητής μόνο-ανάγνωσης επιθεωρεί τον κατάλογο και παράγει δομημένη σύνοψη δεδομένων.
G2-Planner	G2	Εξερευνητής → σχεδιαστής που αποσυνθέτει σε 1–4 υποεργασίες → εκτελεστής ανά υποεργασία.
G2-Analyst	G2	Εξερευνητής και σχεδιαστής ενοποιημένοι σε ένα πέρασμα μόνο-ανάγνωσης που παράγει σύνοψη δεδομένων και θεμελιωμένο σχέδιο μαζί.
G2-Monitor	G2	Κάθε 10 βήματα ένας επόπτης χωρίς εργαλεία βλέπει το ιστορικό και, αν η πορεία έχει ξεφύγει, εισάγει καθοδήγηση.
G3-Instruction	G3	Στην υποβολή, νέος πράκτορας επαληθεύει από την εκφώνηση και την κατάσταση του περιβάλλοντος.
G3-Reflexion	G3	Στην υποβολή, γράφεται αναστοχασμός επί της πρώτης φάσης · νέος πράκτορας επαληθεύει με αυτόν.
G3-Judge	G3	Στην υποβολή, κριτής εγκρίνει ή εισάγει διορθωτική ανατροφοδότηση σε κλώνο της αρχικής μνήμης.
G3-Gate	G3	Η υποβολή αναχαιτίζεται N φορές · ο πράκτορας πρέπει να αυτο-επαληθευτεί πριν περάσει.
G4-Cooperation	G4	Δύο ισότιμοι πράκτορες εναλλάσσονται ανά 7 βήματα στο ίδιο τερματικό.
G4-Debate	G4	Ίδια εναλλαγή, αλλά ο δεύτερος ασκεί κριτική στην προσέγγιση του πρώτου.
G4-Orchestrator	G4	Στρατηγός χωρίς εργαλεία δρομολογεί σε εξειδικευμένους υποπράκτορες που μοιράζονται τα 60 βήματα.

Ο διαχωρισμός των ρόλων τις διακρίνει. Η δρομολόγηση των κλήσεων γίνεται ανά *ρόλο*: κάθε συνιστώσα που δρα στο περιβάλλον χρησιμοποιεί το μοντέλο-εργάτη, κάθε συνιστώσα που απλώς συλλογίζεται *για* την εργασία χρησιμοποιεί το μοντέλο-επιβλέποντα. Έξι από τις δεκατέσσερις αρχιτεκτονικές έχουν εποπτικό ρόλο διαχωρίσιμο με αυτόν τον τρόπο: G2-Explorer, G2-Planner, G2-Analyst, G2-Monitor, G3-Judge και G4-Orchestrator. Επιβλέπων είναι πάντα το φιλοξενούμενο μοντέλο και εργάτες τα τρία μοντέλα που καλύπτουν το εύρος ικανότητας του ανοικτού πλέγματος, δίνοντας $3 \times 6 = 18$ ετερογενή κελιά.

Το μοντέλο-εργάτης, ο προϋπολογισμός του, τα εργαλεία του, το prompt του και το σύνολο εργασιών είναι ταυτόσημα με το αντίστοιχο ομοιογενές κελί. Η *μόνη* μεταβολή είναι ποιο μοντέλο απαντά στην εποπτική κλήση, οπότε η σύγκριση απομονώνει την εποπτική ικανότητα με όλα τα άλλα σταθερά.

1.3 Μεθοδολογία

1.3.1 Το πειραματικό πλέγμα

Η μελέτη είναι ένα πλήρες παραγοντικό σχέδιο 7 μοντέλων × 14 αρχιτεκτονικών πάνω σε 64 εργασίες, συν το σκέλος ετερογενούς επίβλεψης 3 εργατών × 6 αρχιτεκτονικών. Συνολικά περιλαμβάνει 116 κελιά μοντέλου–αρχιτεκτονικής και 7424 γραμμές μοντέλου–εργασίας, από τις οποίες εκτελέστηκαν 7424 (100.0%). Κάθε κελί τρέχει σε κάθε εργασία: κανένα αποτέλεσμα δεν συμπεραίνεται και καμία πολιτική συμπλήρωσης δεν χρειάζεται.

Τα επτά μοντέλα καλύπτουν το φάσμα ικανότητας σε δύο διαστάσεις: μια *ενδο-οικογενειακή* κλίμακα (Qwen2.5-Coder στα 7B, 14B, 32B και το γενικό μοντέλο 72B), η οποία κρατά σταθερή τη συνταγή εκπαίδευσης, και μια *δια-οικογενειακή* διάσταση που προσθέτει έναν κωδικοποιητή της Mistral, ένα γενικό μοντέλο της Meta και ένα φιλοξενούμενο μοντέλο αναφοράς.

Και τα έξι ανοικτά μοντέλα εξυπηρετούνται κβαντισμένα στα 4 bit. Η κβάντιση είναι *ομοιόμορφος περιορισμός* και όχι επιλογή ανά μοντέλο: τα μεγαλύτερα μοντέλα δεν χωρούν στο διαθέσιμο υλικό σε πλήρη ακρίβεια, οπότε η εναλλακτική δεν θα ήταν να μην κβαντιστεί κανένα αλλά να τρέξει μικρότερο σύνολο. Η συνέπεια είναι ότι η μελέτη μετρά *αναπτύξιμα κβαντισμένα τεχνουργήματα* και όχι βάρη πλήρους ακρίβειας,

οπότε τα απόλυτα ποσοστά δεν συγκρίνονται με δημοσιευμένα νούμερα.

1.3.2 Το σύνολο εργασιών, και γιατί μόνο η εύκολη ζώνη

Το κανονικό σύνολο εργασιών είναι οι 64 εργασίες του TerminalBench που φέρουν την *δική του* ένδειξη «easy»: ολόκληρη η ζώνη, χωρίς καμία αφαίρεση. Η χρήση εξωτερικού, αναπαραγώγιμου κριτηρίου, «τρέχουμε όλες τις εύκολες εργασίες» αντί για «τρέχουμε όσες επιλέξαμε», αποκλείει την προφανέστερη ένσταση περί μεροληψίας επιλογής.

Ο περιορισμός σε μία ζώνη δυσκολίας είναι η σημαντικότερη απόφαση οριοθέτησης της μελέτης. Το επιχείρημα υπέρ του είναι ότι η επίδραση μιας αρχιτεκτονικής παρατηρείται μόνο σε εργασίες των οποίων το αποτέλεσμα *μπορεί* να αλλάξει: μια εργασία που δεν λύνει κανένα μοντέλο και μια που τη λύνουν όλα επιστρέφουν σταθερά ανεξαρτήτως αρχιτεκτονικής.

Το επιχείρημα αυτό ήταν αρχικά συμπερασματικό. Είναι πλέον μετρημένο. Εκτελέστηκε ένα *προκαθορισμένο* πείραμα ορίου: 4 μοντέλα που καλύπτουν το φάσμα ικανότητας, το καθένα μόνο σε διαμόρφωση αναφοράς, πάνω σε τυχαίο δείγμα 20 εργασιών *medium* και 20 *hard*, επιλεγμένων αναπαραγώγιμα από τις ετικέτες του ίδιου του benchmark, 160 εκτελέσεις συνολικά. Το δείγμα ορίστηκε εκ των προτέρων ώστε ένα αρνητικό αποτέλεσμα να μην μπορεί να εξηγηθεί εκ των υστέρων, και κρατείται εκτός του κύριου συνόλου ώστε να μην μπορεί ποτέ να εισέλθει σε ποσοστό επιτυχίας του πλέγματος.

Το σήμα αραιώνει απότομα μόλις εγκαταλειφθεί η εύκολη ζώνη. Από 80 προσπάθειες medium πετυχαίνουν 8 (10.0%) · από 80 προσπάθειες hard πετυχαίνει 1 (1.2%). Ακόμα και το ισχυρότερο μοντέλο λύνει 5 από τις 20 medium και 0 από τις hard.

1.3.3 Πόσες εργασίες φέρουν πραγματικά πληροφορία

Δεν φέρουν όλες οι 64 εργασίες πληροφορία για την αρχιτεκτονική. Μετρημένο πάνω στο ολοκληρωμένο πλέγμα, **42 από τις 64 εργασίες λύνονται από το πολύ ένα** από τα 7 μοντέλα στη γραμμή αναφοράς, ενώ 4 λύνονται από έξι ή επτά. Μόνο 18 βρίσκονται στη μεσαία ζώνη (δύο έως πέντε μοντέλα τις λύνουν). Αυτό δεν σημαίνει ότι οι υπόλοιπες είναι αδρανείς: 50 από τις 64 μετακινούνται τουλάχιστον μία φορά υπό κάποιο μοντέλο και κάποια αρχιτεκτονική, και μόνο 14 δεν κινούνται ποτέ. Σημαίνει ότι η *πυκνότητα* των μετακινήσεων είναι πολύ ανομοιόμορφη: κάθε αμφισβητούμενη εργασία παράγει 22 μετακινήσεις έναντι 5 μιας εργασίας στο δάπεδο, οπότε το 59% όλων των μετακινήσεων προέρχεται από λίγο πάνω από το ένα τέταρτο του σώματος. Αυτό καθορίζει τι μπορεί να σημαίνει ένας μέσος όρος κατηγορίας.

Ο λογαριασμός αλλάζει ουσιωδώς αν η μέτρηση γίνει στους επιμέρους ελέγχους των βαθμολογητών αντί στο δυαδικό πέρασε-απέτυχε: 58 από τις 64 εργασίες μετακινούνται υπό κάποια αρχιτεκτονική. Η φαινομενική ακινησία της ζώνης δαπέδου στο δυαδικό μέτρο είναι σε σημαντικό βαθμό ιδιότητα της μετρικής και όχι των εκτελέσεων.

1.3.4 Τι μπορεί και τι δεν μπορεί να τεκμηριώσει ο σχεδιασμός

Κάθε διαμόρφωση εκτελείται *μία* φορά ανά εργασία σε θερμοκρασία 0 · το εύρος προτιμήθηκε έναντι της επανάληψης. Η επιλογή ακολουθεί την πρακτική του πεδίου, όπου οι πίνακες κατάταξης αγωνιστικών benchmark, συμπεριλαμβανομένου του TerminalBench, αναφέρουν μονές εκτελέσεις.

Εφαρμόζοντας το στατιστικό πρωτόκολλο, ακριβές τεστ McNemar σε ζευγαρωτά δυαδικά αποτελέσματα με διόρθωση Holm εντός μοντέλου και ζευγαρωτό bootstrap για τα διαστήματα εμπιστοσύνης, **κανένα από τα 91 μεμονωμένα κελιά δεν φτάνει σε στατιστική σημαντικότητα**.

Αυτό είναι εγγενές όριο ενός σχεδιασμού με μία εκτέλεση ανά κελί, και καθορίζει τη μορφή κάθε ισχυρισμού: κανένα συμπέρασμα δεν στηρίζεται σε μεμονωμένο κελί. Αυτό που τα δεδομένα *μπορούν* να στηρίξουν είναι δηλώσεις για τη *σχέση* μεταξύ ικανότητας και οφέλους, οι οποίες συγκεντρώνουν πληροφορία από πολλά κελιά, και αυτές είναι ισχυρές. Η πιο ανθεκτική μορφή του κεντρικού ισχυρισμού δεν απαιτεί καμία παραδοχή διάταξης: στα δύο ασθενέστερα μοντέλα 24 από 26 κελιά βελτιώνουν τη γραμμή αναφοράς, ενώ στα υπόλοιπα πέντε η κατεύθυνση αντιστρέφεται (42 από 65 επιδεινώνουν), με διαφορά μέσης επίδρασης +3.51 εργασίες (null με επανατοποθέτηση, $p \approx 0.033$).

Η συνεισφορά της εργασίας είναι μηχανιστική, όχι στατιστική. Αυτό που προσφέρει πέρα από έναν πίνακα αποτελεσμάτων είναι η εξήγηση του *γιατί* συγκεκριμένες αρχιτεκτονικές μετακίνησαν συγκεκριμένες εργασίες σε συγκεκριμένα μοντέλα, διαβασμένη από τις ίδιες τις εκτελέσεις.

1.4 Αποτελέσματα

Τα αποτελέσματα παρουσιάζονται στη σειρά των ερευνητικών ερωτημάτων. Προηγείται όμως ένας έλεγχος του ίδιου του σχεδιασμού, γιατί αν δεν περάσει, καμία σύγκριση που ακολουθεί δεν είναι έγκυρη.

1.4.1 Ο προϋπολογισμός τηρήθηκε, και τα επιπλέον βήματα δεν αποδίδουν περισσότερες λύσεις

Το όριο των 60 βημάτων δεν ξεπεράστηκε ποτέ σε καμία από τις 7404 εκτελέσεις με ίχνος, αλλά η *κατανάλωση* διαφέρει σημαντικά μεταξύ αρχιτεκτονικών, επειδή η γραμμή αναφοράς δεν εξαντλεί τον προϋπολογισμό της. Γεννάται έτσι μια λεπτότερη ένσταση: αν η αρχιτεκτονική κάνει τον πράκτορα να δουλέψει περισσότερο, μήπως το όφελος είναι απλώς η επιπλέον δουλειά;

Τα δεδομένα την απαντούν αποφασιστικά. Οι εκτελέσεις που *πετυχαίνουν* καταναλώνουν περίπου τα μισά βήματα από εκείνες που αποτυγχάνουν, και το μοτίβο ισχύει σε *όλα* τα μοντέλα χωρίς εξαίρεση. Εντός κάθε μοντέλου η κατανάλωση βημάτων *δεν* προβλέπει το όφελος, ενώ οι λιτότερες αρχιτεκτονικές είναι από τις αποδοτικότερες. Τα επιπλέον βήματα είναι *σύμπτωμα* δυσκολίας, όχι *μηχανισμός* επιτυχίας.

1.4.2 Η βαθμίδα ικανότητας

Η ενότητα αυτή απαντά στο **EE1**. Το κεντρικό αποτέλεσμα: το πλήθος των αρχιτεκτονικών που ξεπερνούν τη γραμμή αναφοράς του ίδιου του μοντέλου *μειώνεται* καθώς η γραμμή αυτή *ανεβαίνει*. Σε ένα από τα ασθενέστερα μοντέλα βελτιώνουν και οι 13 από τις 13 μη-βασικές αρχιτεκτονικές · στον ισχυρότερο ανοικτό κωδικοποιητή, **0**. Τα δύο μεγέθη που συσχετίζονται είναι, ανά μοντέλο, η *ικανότητά* του (πόσες εργασίες λύνει χωρίς καμία αρχιτεκτονική) και το *όφελος* που αντλεί από αυτές. Στα έξι τοπικά εξυπηρετούμενα μοντέλα η πρώτη ποσότητα ανεβαίνει από 4 σε 19 ενώ η δεύτερη πέφτει, και η συσχέτιση κατάταξης δίνει $\rho = -0.886$ ($p = 0.033$, $n = 6$): τιμή κοντά στο -1, δηλαδή σχεδόν τέλεια *αντίστροφη* σχέση. **Όσο ικανότερο το μοντέλο, τόσο λιγότερες αρχιτεκτονικές το ωφελούν.** Το ίδιο νούμερο προκύπτει είτε το όφελος μετρηθεί ως μέση επίδραση είτε ως πλήθος αρχιτεκτονικών που ξεπερνούν τη γραμμή αναφοράς. Με το φιλοξενούμενο μοντέλο η συσχέτιση εξασθενεί σε $\rho = -0.679$ ($p = 0.110$), και αναφέρονται και οι δύο τιμές.

Η ίδια αρχιτεκτονική αλλάζει επομένως *πρόσημο* κατά μήκος του φάσματος. Ο G2-Monitor, καλύτερη αρχιτεκτονική για το Llama-70B με $+5$, είναι -5 στο Qwen-32B. Η αντιστροφή αυτή είναι ακριβώς ό,τι δεν μπορεί να παρατηρήσει μια μελέτη ενός μοντέλου. Το Σχήμα 1.4.1 δείχνει ολόκληρο το πλέγμα με μια ματιά, και ο Πίνακας 1.2 συνοψίζει τη βαθμίδα ανά μοντέλο.

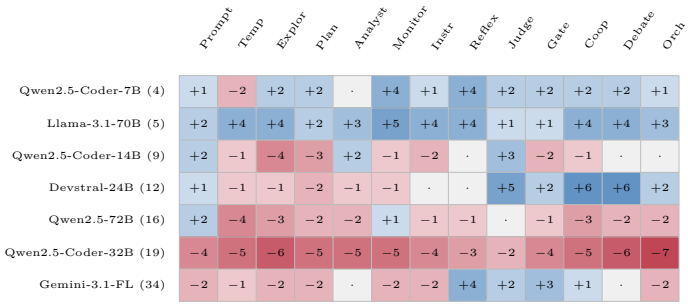


Figure 1.4.1: Η αλληλεπίδραση μοντέλου×αρχιτεκτονικής. Κάθε κελί είναι η μεταβολή σε λυμένες εργασίες σε σχέση με τη γραμμή αναφοράς του ίδιου μοντέλου (σε παρένθεση δίπλα στο όνομα) · το μπλε είναι κέρδος, το κόκκινο ζημιά, η ένταση το μέγεθος. Οι γραμμές πάνε από το ασθενέστερο μοντέλο (πάνω) στο ισχυρότερο (κάτω): το πάνω μέρος είναι κυρίως μπλε και το κάτω κυρίως κόκκινο, ενώ ο μέσος όρος κάθε στήλης είναι σχεδόν μηδενικός: η μηδενική κύρια επίδραση και η μεγάλη αλληλεπίδραση σε μία εικόνα.

Το ίδιο αποτέλεσμα σε διακριτά γεγονότα

Και οι δύο διατυπώσεις της βαθμίδας είναι μέσοι όροι, οπότε δικαιολογείται η ένσταση ότι ο μέσος όρος κρύβει δομή αντί να την αποκαλύπτει. Στην ατομική σύγκριση (ένα μοντέλο, μία αρχιτεκτονική, μία εργασία, έναντι της γραμμής αναφοράς του ίδιου μοντέλου) υπάρχουν τέσσερα δυνατά αποτελέσματα, ακριβώς τα

Table 1.2: Η βαθμίδα ικανότητας. Καθώς ανεβαίνει η γραμμή αναφοράς ενός μοντέλου, μειώνεται το πλήθος των αρχιτεκτονικών που την ξεπερνούν. Η τελευταία στήλη είναι η ένωση των εργασιών που λύνονται από *οποιαδήποτε* αρχιτεκτονική, ένα άνω φράγμα για επιλογή ανά εργασία.

Μοντέλο	Γρ. αναφοράς	# αρχ. > αναφ.	Καλύτερη	Ένωση
Qwen2.5-Coder-7B	4/64	11/13	Monitor	14/64
Llama-3.1-70B	5/64	13/13	Monitor	20/64
Qwen2.5-Coder-14B	9/64	3/13	Judge	21/64
Devstral-24B	12/64	6/13	Coop	32/64
Qwen2.5-72B	16/64	2/13	Prompt	28/64
Qwen2.5-Coder-32B	19/64	0/13	Judge	31/64
Gemini-3.1-FL	34/64	4/13	Reflex	49/64

τέσσερα κελιά που διαβάζει ένα ζευγαρωτό τεστ: *νίκη* (η αναφορά απέτυχε, η αρχιτεκτονική έλυσε), *ήττα* (το αντίστροφο), ή καμία μεταβολή.

Σε ολόκληρο το πλέγμα υπάρχουν 333 νίκες έναντι 344 ηττών, καθαρό αποτέλεσμα -11 εργασίες, και **5147 από τις 5824 συγκρίσεις δεν αλλάζουν απολύτως τίποτα**. Η σχεδόν εξουδετέρωση επιβιώνει επομένως σε ένα μέτρο που δεν υπολογίζει κανέναν μέσο όρο, και η κατάταξη των οικογενειών παραμένει η ίδια: η G3 έχει το πιο θετικό ισοζύγιο και η G4 ήπια θετικό, ενώ η G1 και η G2 είναι αρνητικές, με την G2 περισσότερο.

Η ατομική οπτική απαντά όμως και ένα ερώτημα που οι μέσοι όροι δεν μπορούν καν να θέσουν: όταν μια αρχιτεκτονική μετατρέπει μια εργασία, είναι αυτό ιδιότητα του ζεύγους (αρχιτεκτονική, εργασία), ή ανήκει και το μοντέλο στην περιγραφή; Παίρνοντας κάθε ζεύγος που η αρχιτεκτονική αποδεδειγμένα *μπορεί* να κερδίσει, και ως επιλέξιμα τα μοντέλα των οποίων η γραμμή αναφοράς απέτυχε, η κατανομή είναι έντονα ασύμμετρη: **το 74% των ζευγών μετατρέπει ακριβώς ένα μοντέλο** και το 80% λιγότερα από τα μισά.

Δύο στατιστικά ερμηνεύουν αυτή την κατανομή, και λένε διαφορετικά πράγματα. Οι νίκες είναι **2.9× πιο συγκεντρωμένες από ό,τι θα έδινε η τύχη**, άρα υπάρχει γνήσια συνιστώσα που ανήκει στο ζεύγος (αρχιτεκτονική, εργασία). Ταυτόχρονα, δύο μοντέλα που αποτυγχάνουν την ίδια εργασία σώζονται από επικαλυπτόμενα σύνολα αρχιτεκτονικών με διάμεσο $\kappa = 0.12$, συμφωνία υπαρκτή αλλά ασθενής.

Το συμπέρασμα τοποθετεί το αποτέλεσμα με ακρίβεια: **η αρχιτεκτονική ωφέλεια έχει συνιστώσα σε επίπεδο εργασίας, και είναι ασθενής**. Η πρόταση «η αρχιτεκτονική Α βοηθά στην εργασία Τ» είναι υπερασπίσιμη τάση αλλά φτωχή πρόβλεψη· η πρόταση που στέκει κατονομάζει τρία πράγματα: αρχιτεκτονική, εργασία *και* μοντέλο.

Ο μέσος όρος είναι η λάθος σύνοψη

Το ίδιο εύρημα εξηγεί γιατί ο μέσος όρος παραπλανά: κατά μέσο όρο σε όλα τα μοντέλα, καμία αρχιτεκτονική δεν μετακινεί το συνολικό ποσοστό πάνω από λίγες μονάδες. Αυτό δεν σημαίνει ότι η αρχιτεκτονική είναι ασθενής παρέμβαση· οι μεγάλες επιδράσεις ανά μοντέλο έχουν αντίθετα πρόσημα και αλληλοαναιρούνται, οπότε ο μέσος όρος κρύβει ακριβώς την αλληλεπίδραση που είναι το πραγματικό αποτέλεσμα.

1.4.3 Η ικανότητα δεν είναι το μέγεθος

Ο άξονας κατά μήκος του οποίου διαβάζεται κάθε αποτέλεσμα *δεν* μπορεί να αντικατασταθεί από το πλήθος παραμέτρων. Εντός της οικογένειας Qwen, το μοντέλο 32B *κωδικοποιητή* λύνει 19 εργασίες ενώ το *γενικό* 72B λύνει 16. Μεταξύ οικογενειών, το γενικό Llama 70B λύνει 5: κάτω από τον κωδικοποιητή 14B και μόλις πάνω από τον 7B. Ένα μοντέλο 70B και ένα 7B χωρίζονται από *μία* εργασία. Η κβάντιση δεν εξηγεί την κατάταξη, κάτι που ελέγχεται χωριστά σε τρεις ανεξάρτητες βάσεις.

1.4.4 Τι ακριβώς είναι η ζημιά στα ισχυρά μοντέλα

Η βαθμίδα λέει ότι τα ισχυρά μοντέλα χάνουν έδαφος υπό αρχιτεκτονική, χωρίς να λέει γιατί. Τρεις μηχανισμοί είναι συμβατοί με αυτό: τα συμφραζόμενα του scaffold *αραιώνουν* τις παρατηρήσεις του εκτελεστή, τα βήματά του *αφαιρούνται* από την εκτέλεση, ή η καθοδήγησή του *υποκαθιστά* μια προσέγγιση που δούλευε. Οι τρεις κάνουν διαφορετικές προβλέψεις, οπότε διαχωρίζονται στα δεδομένα.

Ο πληθυσμός είναι κάθε περίπτωση όπου ένα από τα τρία ισχυρότερα μοντέλα λύνει μια εργασία στη γραμμή αναφοράς και την αποτυγχάνει υπό κάποια αρχιτεκτονική: 211 συμβάντα υποχώρησης, καθένα ελεγχόμενο ζεύγος με μία μόνο μεταβλητή αλλαγμένη. Η υπερχείλιση συμφραζομένων ευθύνεται για 10% και η εξάντληση προϋπολογισμού για 20%. Η κυρίαρχη περίπτωση, στο **70%**, είναι μια εκτέλεση που ολοκλήρωσε, είχε κάτι να παραδώσει, και παρέδωσε κάτι χειρότερο από αυτό που παρήγαγε η γραμμή αναφοράς υπό ταυτόσημες συνθήκες.

Το επιχείρημα του προϋπολογισμού αποτυγχάνει και σε δεύτερο, οξύτερο έλεγχο. Αν η αρχιτεκτονική έβλαπτε στερώντας βήματα από την εκτέλεση, οι χαμένες εκτελέσεις θα ήταν *συντομότερες*. Είναι μακρύτερες: στα 211 ζεύγη η διάμεσος της χαμένης εκτέλεσης είναι 28 βήματα έναντι 18 της εκτέλεσης που έλυσε την ίδια εργασία στο ίδιο μοντέλο, και το 63% δουλεύει περισσότερο από όσο χρειάστηκε η γραμμή αναφοράς. Οι εκτελέσεις με scaffold δεν κόβονται στη μέση · δουλεύουν περισσότερο και καταλήγουν κάπου χειρότερα.

Η κατανομή της ζημιάς δείχνει τι είδους υποκατάσταση συμβαίνει, αφού πρώτα αφαιρεθεί μια σύγχυση. Από τις 143 λανθασμένες υποβολές με αναλυτική βαθμολόγηση, το 39% δεν περνά κανέναν έλεγχο · ο αριθμός όμως δεν διαβάζεται ως έχει, γιατί μια εργασία με *έναν* έλεγχο μηδενίζεται αυτομάτως μόλις αποτύχει, και το 55% των μηδενικών υποχωρήσεων βρίσκεται σε τέτοιες εργασίες. Περιοριζόμενοι στις 112 υποχωρήσεις σε εργασίες με τουλάχιστον δύο ελέγχους, όπου η βαθμολογία μπορεί να ξεχωρίσει την παρά τρίχα αποτυχία από την κατάρρευση, **το 22% εξακολουθεί να μην περνά τίποτα**: ένα μοντέλο που έλυνε την εργασία και τώρα αποτυγχάνει σε κάθε ανεξάρτητο έλεγχο δεν σπρώχτηκε ελαφρά εκτός πορείας. Πέρα από αυτό δεν αποδίδουμε το φαινόμενο σε συγκεκριμένες αρχιτεκτονικές: τα κατά αρχιτεκτονική πλήθη σε αυτό το υποσύνολο κυμαίνονται από τρία έως δεκατέσσερα συμβάντα, πολύ λίγα για τέτοιον ισχυρισμό.

1.4.5 Πώς αποτυγχάνουν τα μοντέλα

Η εξήγηση αρχίζει από τις αποτυχίες, επειδή μια αρχιτεκτονική μπορεί να επισκευάσει μόνο μια αποτυχία που όντως συμβαίνει: αυτό είναι το **EE2**.

Σε *κάθε* μοντέλο, η μεγαλύτερη κατηγορία αποτυχίας είναι μια εκτέλεση που τερμάτισε επειδή ο πράκτορας *αποφάσισε ότι τελείωσε* και υπέβαλε λανθασμένη απάντηση. Η εξάντληση του προϋπολογισμού, η αποτυχία που θα αντιμετώπιζε το «δώσε στον πράκτορα περισσότερα βήματα», είναι παντού μειοψηφία. Αυτό εξηγεί γιατί η εκ των υστέρων επαλήθευση είναι η αρχιτεκτονικά καλύτερα στοχευμένη οικογένεια: είναι η μόνη που παρεμβαίνει τη στιγμή που συμβαίνει η κυρίαρχη αποτυχία.

Κάτω από αυτό το κοινό μοτίβο, τα μοντέλα αποτυγχάνουν με χαρακτηριστικά διαφορετικούς τρόπους. Το Llama-70B εγκαταλείπει νωρίς και με βεβαιότητα, υποβάλλοντας λάθος απάντηση μετά από 13 βήματα κατά μέσο όρο. Το Devstral-24B βρίσκεται στο αντίθετο άκρο: καταναλώνει κατά μέσο όρο 54 βήματα και εξαντλεί τον προϋπολογισμό του. Ακόμη και μεταξύ των ασθενών μοντέλων η αποτυχία διχάζεται, ανάμεσα στην πρόωρη παραίτηση και στο αδιάκοπο grind. Οι διαφορές αυτές μετρώνται *ανεξάρτητα* από τα ποσοστά επιτυχίας, γι' αυτό η αντιστοιχία τους με τις αρχιτεκτονικές που βοηθούν κάθε μοντέλο αποτελεί τεκμήριο και όχι επανάληψη. Το Σχήμα 1.4.2 αναλύει τις λανθασμένες υποβολές κάθε μοντέλου σε τέσσερα είδη, και το μοτίβο κινείται με την ικανότητα: τα ασθενή μοντέλα παραιτούνται νωρίς, τα ισχυρότερα αποτυγχάνουν παρά τρίχα.

Μία αποτυχία, τέλος, ανήκει αποκλειστικά στις αρχιτεκτονικές και δεν μπορεί να εμφανιστεί σε έναν πράκτορα αναφοράς: το scaffold επικοινωνεί μέσω ενός δομημένου τεχνουργήματος, και ένα μοντέλο που δεν μπορεί να το παραγάγει δεν έχει εκτελέσει την αρχιτεκτονική που του εφαρμόστηκε.

Ο βρόχος που κρύβει η κατηγορία τερματισμού

Πίσω από τις κατηγορίες τερματισμού υπάρχει μια συμπεριφορά που δεν καταγράφεται σε καμία από αυτές. Σε **27.9% των εκτελέσεων** ο πράκτορας εκδίδει την ίδια ακριβώς εντολή πέντε ή περισσότερες φορές στη σειρά, και 89% από αυτές αποτυγχάνουν. Το φαινόμενο είναι ιδιότητα του μοντέλου και δεν ακολουθεί τη σειρά ικανότητας: η συχνότητά του κυμαίνεται από κάτω από 10% έως περίπου δύο τρίτα των εκτελέσεων ανά μοντέλο. Πάνω από τις μισές τέτοιες εκτελέσεις καταγράφονται ως «υπέβαλε», όχι ως «εξάντλησε τον προϋπολογισμό», οπότε η στασιμότητα κρύβεται μέσα στην κυρίαρχη κατηγορία αποτυχίας αντί να φαίνεται ως δική της.

1.4.6 Ποια αρχιτεκτονική για ποιο μοντέλο

Το **EE3** ρωτά τι κάνει μια αρχιτεκτονική να βοηθά, και αν η αποτελεσματική επιλογή εξαρτάται από το μοντέλο. Εξαρτάται: η αρχιτεκτονική που βοηθά ένα μοντέλο είναι εκείνη που ταιριάζει στον τρόπο με τον

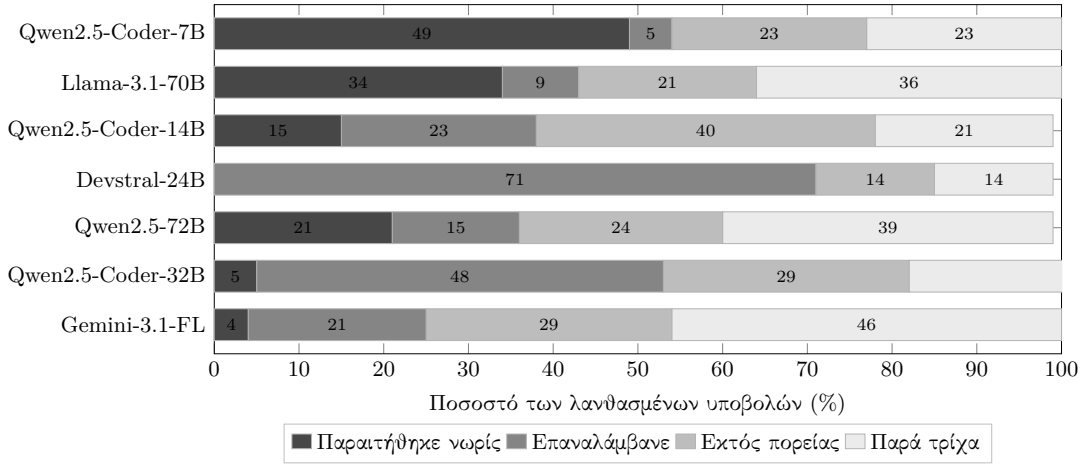


Figure 1.4.2: Σύνθεση των λανθασμένων υποβολών κάθε μοντέλου, με το ασθενέστερο πάνω. Διαβάζοντας προς τα κάτω διαβάζουμε αυξανόμενη ικανότητα: η ζώνη «παραιτήθηκε νωρίς» υποχωρεί και μεγαλώνει το «παρά τρίχα».

οποίο αυτό αποτυγχάνει, και καμία δεν είναι η καλύτερη για περισσότερα από δύο από τα επτά. Το πρακτικό πόρισμα είναι ότι τρεις μετρήσεις, καμία από τις οποίες δεν είναι ποσοστό επιτυχίας, προβλέπουν την απόκριση κάθε μοντέλου χωρίς πλήρη σάρωση αρχιτεκτονικών: *πόσο νωρίς σταματά*, *ποια είναι η κυρίαρχη αποτυχία του*, και *αν ο μηχανισμός της αρχιτεκτονικής λειτουργεί πάνω του*.

Η τρίτη είναι η λιγότερο αναμενόμενη. Η ικανότητα παραγωγής έγκυρης δομημένης εξόδου αποδεικνύεται *ξεχωριστός άξονας* από την ικανότητα επίλυσης: το Qwen-14B λύνει σχεδόν διπλάσιες εργασίες από το Llama-70B και παράγει έγκυρα σχέδια το ένα εικοστό των φορών. Όπου η συμμόρφωση αποτυγχάνει, μια καταγεγραμμένη υποχώρηση μετρά μια αρχιτεκτονική που *δεν έτρεξε ποτέ*, όχι μία που έτρεξε και απέτυχε. Ο Πίνακας 1.3 συγκεντρώνει τις τρεις μετρήσεις για κάθε μοντέλο, μαζί με την αρχιτεκτονική που τελικά του ταιριάζει.

Table 1.3: Τι προβλέπει την απόκριση κάθε μοντέλου στην αρχιτεκτονική. Καμία από τις τρεις μεσαίες στήλες δεν είναι ποσοστό επιτυχίας: είναι πόσα βήματα κάνει το μοντέλο πριν αποτύχει (μέσος όρος στις αποτυχημένες εκτελέσεις αναφοράς), ποια είναι η κυρίαρχη αποτυχία του, και αν ο μηχανισμός της αρχιτεκτονικής λειτουργεί πάνω του, δηλαδή το ποσοστό των εκτελέσεων του G2-Planner στις οποίες παρήχθη έγκυρο σχέδιο. Οι δύο τελευταίες στήλες μετριούνται σε διαφορετικά κελιά του πλέγματος: η ικανότητα στη γραμμή αναφοράς, η συμμόρφωση εκεί όπου απαιτείται δομημένη έξοδος.

Μοντέλο	Αναφ.	Βήματα	Κυρίαρχη αποτυχία	Έγκυρα σχέδια	Βοηθούν
Qwen2.5-Coder-7B	4	17	65% υποβολή, 32% ctx	**0%**	11/13
Llama-3.1-70B	5	13	**100% υποβολή**	95%	**13/13**
Qwen2.5-Coder-14B	9	40	85% υποβολή	**5%**	3/13
Devstral-24B	12	54	81% υποβ., 15% budget	98%	6/13
Qwen2.5-72B	16	29	69% υποβολή	97%	2/13
Qwen2.5-Coder-32B	19	41	47% υποβ., 33% ctx	59%	**0/13**
Gemini-3.1-FL	34	42	93% υποβολή	84%	4/13

Οι μηχανισμοί ταιριάζουν με τις αποτυχίες: ο G2-Monitor βοηθά το μοντέλο που σταματά νωρίς (+5) και βλάπτει εκείνο που καταναλώνει σχεδόν όλα του τα βήματα (-1) · η επαλήθευση βοηθά το μοντέλο του οποίου το μόνο απομένον πρόβλημα είναι μια λανθασμένη ολοκληρωμένη απάντηση.

1.4.7 Τι κάνει κάθε οικογένεια

Οι δεκατέσσερις ετυμηγορίες, διαβασμένες μαζί, δίνουν τέσσερα διακριτά προφίλ.

Η **G1** θέτει ένα κατώφλι που η πολυπλοκότητα οφείλει να ξεπεράσει. Τρεις προτάσεις οδηγιών βελτιώνουν πέντε από τα επτά μοντέλα χωρίς καμία επιπλέον κλήση μοντέλου και χωρίς κατανάλωση προϋπολογισμού, και για ένα μοντέλο είναι η καλύτερη αρχιτεκτονική του πλέγματος. Διαβασμένο ανά κελί μοντέλου–κατηγορίας, το G1-Prompt κερδίζει περισσότερα κελιά από κάθε άλλη αρχιτεκτονική.

Η **G2** είναι η ασθενέστερη οικογένεια, και τρεις χωριστοί μηχανισμοί συνεισφέρουν. Δεν *τρέχει* σε μοντέλα που δεν παράγουν τα τεχνουργήματά της. *Σχεδιάζει χωρίς πληροφορία* σε εργασίες των οποίων η δομή αποκαλύπτεται μόνο μέσω εκτέλεσης. Και *ξοδεύει προϋπολογισμό σε δομή που το μοντέλο δεν χρειαζόταν*, που είναι η συνήθης περίπτωση στα ικανά μοντέλα. Υπάρχει και τέταρτος, ζήτημα εφαρμοσιμότητας: η οικογένεια προϋποθέτει ότι υπάρχουν δεδομένα στο περιβάλλον για να θεμελιωθεί η ανάλυση, και όπου δεν υπάρχουν, η φάση θεμελίωσης σπαταλά τα βήματά της και στέλνει και τον εκτελεστή να ψάχνει.

Η **G3** είναι η ανθεκτικότερη οικογένεια, για δομικό λόγο: είναι *υπό συνθήκη*. Καταναλώνει προϋπολογισμό μόνο όταν ο πράκτορας υποβάλλει, οπότε όταν η πρώτη προσπάθεια είναι ήδη σωστή κοστίζει σχεδόν τίποτα, και στοχεύει την κυρίαρχη αποτυχία. Δύο ευρήματα το περιορίζουν: η επαλήθευση *περιορίζεται συχνά από τον προϋπολογισμό*, καθώς μια αναχαιτισμένη υποβολή αφήνει στον επαληθευτή περιθώριο να δράσει μόλις στο 63% των περιπτώσεων, και είναι *ευαίσθητη στη βαθμονόμηση* με τρόπο που δεν ισχύει για τις άλλες.

Η **G4** εισάγει αποτυχίες που δεν μπορούν να συμβούν αλλού, όπως η αποτυχία συντονισμού και η εξάντληση κύκλων. Είναι ταυτόχρονα η οικογένεια που προχωρά περισσότερο τις δύσκολες εργασίες στους επιμέρους ελέγχους, και ο G4-Orchestrator είναι η καλύτερη αρχιτεκτονική στη μία κατηγορία της οποίας οι εργασίες είναι γνήσια πολυσταδιακές.

1.4.8 Το δυαδικό αποτέλεσμα κρύβει το μεγαλύτερο μέρος της κίνησης

Κάθε εργασία βαθμολογείται από ανεξάρτητους ελέγχους και θεωρείται λυμένη μόνο αν περάσουν *όλοι*. Το κατώφλι αυτό απορρίπτει πολλή πληροφορία: στη μελέτη, **περισσότερες εκτελέσεις περνούν *κάποιους* ελέγχους ενώ αποτυγχάνουν συνολικά, από όσες πετυχαίνουν εντελώς**.

Στις 32 εργασίες των οποίων το δυαδικό αποτέλεσμα παραμένει ουσιαστικά ακίνητο (λιγότερο από δύο λύσεις διαφορά μεταξύ της καλύτερης και της χειρότερης αρχιτεκτονικής), οι επιμέρους έλεγχοι δείχνουν κίνηση, αλλά μικρή και ως επί το πλείστον αρνητική. Μόνο 3 από τις δεκατρείς αρχιτεκτονικές είναι θετικές: ο G4-Debate στις +3.0 μονάδες, ο G3-Judge στις +1.1 και ο G3-Instruction στις +0.1. Η οικογένεια πριν την εκτέλεση είναι αρνητική σε ολόκληρο το εύρος (-0.5 έως -3.0), όπως και οι δύο φθηνές παραλλαγές της G1.

Δύο συμπεράσματα προκύπτουν. Το στενό είναι ότι οι επιμέρους έλεγχοι όντως καταγράφουν δουλειά που το δυαδικό μέτρο πετά: οι δύο σχεδιάσεις που βασίζονται στην *κριτική μιας ολοκληρωμένης προσπάθειας* είναι εκείνες που προχωρούν αυτές τις εργασίες, ο ίδιος μηχανισμός που τα δυαδικά αποτελέσματα αποδίδουν στην G3, ορατός σε εργασίες που δεν περνούν ποτέ το κατώφλι. Το ευρύτερο είναι ότι το μέγεθος της επίδρασης είναι μικρό: η καλύτερη αρχιτεκτονική της μελέτης προχωρά τους ελέγχους κατά περίπου τρεις μονάδες, και οι περισσότερες τους οπισθοχωρούν. Η ανάλυση στους επιμέρους ελέγχους σώζει το *πρόσημο* της ιστορίας της G3 σε αυτές τις εργασίες, όχι το μέγεθός της.

1.4.9 Τι απαντά η αρχιτεκτονική: το περιθώριο, όχι το θέμα

Το **EE4** ρωτά ποιες ιδιότητες μιας εργασίας προβλέπουν *αν* μια αρχιτεκτονική θα τη βοηθήσει και *ποια* οικογένεια βοηθά περισσότερο. Η απάντηση δεν είναι θεματική. Ομαδοποιώντας τις εργασίες κατά το *πόσα μοντέλα τις λύνουν ήδη*, το πρόσημο του δυαδικού αποτελέσματος καθορίζεται στα άκρα από τη ζώνη: κάθε αρχιτεκτονική χάνει λύσεις στις ήδη λυμένες εργασίες (-12 έως +0) και κάθε αρχιτεκτονική κερδίζει στις εργασίες-δάπεδο (+2 έως +7), ενώ οι μεσαίες ζώνες δεν αποφαίνονται.

Στους επιμέρους ελέγχους η ίδια εικόνα στηρίζεται ασθενέστερα, και η διαφορά αξίζει να δηλωθεί. Και οι δύο οικογένειες βρίσκονται στα χειρότερά τους όπου το περιθώριο είναι μικρότερο και είναι θετικές μόνο στο δάπεδο (+2.1 και +0.8 μονάδες), οπότε το περιθώριο *φράσσει* την επίδραση χωρίς να τη διατάσσει: η οικογένεια μετά την εκτέλεση διαγράφει +0.0, +0.8, -0.8, +2.1 στις τέσσερις ζώνες, πέφτοντας στην προτελευταία αντί να ανεβαίνει μονότονα. Η μεγαλύτερη μεμονωμένη κίνηση είναι κόστος και όχι όφελος: η οικογένεια πριν την εκτέλεση χάνει -4.4 μονάδες στις 10 εργασίες που τα περισσότερα μοντέλα λύνουν ήδη.

Μια αντίπαλη υπόθεση, ότι το *μήκος* της εργασίας προβλέπει την απόκριση, ελέγχθηκε και απορρίφθηκε: οι σύντομες εργασίες βοηθιούνται *περισσότερο*, όχι λιγότερο. Η σύγχυση προκύπτει επειδή οι σύντομες και οι ήδη λυμένες εργασίες συσχετίζονται σε αυτό το σύνολο. Το ίδιο ισχύει για τη δηλωμένη δυσκολία, η οποία δεν διαχωρίζει τις περιπτώσεις.

Μια συστηματική ανάλυση και των δεκατριών κατηγοριών εργασιών οδηγεί σε μια ταξινομία *σχημάτων*: όπου η δυσκολία είναι *περιβαλλοντική*, δηλαδή τι είναι εγκατεστημένο, κερδίζει η εκ των υστέρων επαλήθευση· όπου η εργασία είναι *προδιαγεγραμμένος μετασχηματισμός* δεδομένων που ήδη υπάρχουν, κερδίζουν οι

αρχιτεκτονικές πριν την εκτέλεση· όπου η εργασία απαιτεί *πρόταση, έλεγχο και αναθεώρηση*, οι αρχιτεκτονικές πριν την εκτέλεση είναι *επιβλαβείς*, διότι η πρώτη παρατήρηση ακυρώνει το σχέδιο. Η ταξινομία αυτή διαβάστηκε από εκτελέσεις των οποίων τα αποτελέσματα ήταν ήδη γνωστά και προσφέρεται ως υπόθεση με ταυτοποιημένο μηχανισμό.

Η προφανής ένσταση (ότι τα σχήματα θα έπρεπε να προκύψουν αλγοριθμικά αντί χειροκίνητα) δοκιμάστηκε και αναφέρεται. Ομαδοποιώντας τις εργασίες με βάση το διάνυσμα απόκρισής τους στις 13 αρχιτεκτονικές προκύπτει δομή που *φαίνεται* ερμηνεύσιμη, αλλά **δεν αναπαράγεται**: δείκτης Rand 0.41 μεταξύ τυχαίων αρχικοποιήσεων και 0.40 όταν αφαιρεθεί ένα μοντέλο. Η μόνη ευσταθής διαμέριση ανακτά απλώς το πρόσημο της απόκρισης, που το περιθώριο ήδη δίνει. Το εμπόδιο είναι ο θόρυβος και όχι η μέθοδος: κάθε συντεταγμένη είναι διαφορά δύο μεμονωμένων εκτελέσεων. Επαναλήψεις στα αμφισβητούμενα κελιά θα καθιστούσαν το ερώτημα απαντήσιμο, και αυτός είναι ένας δεύτερος, ανεξάρτητος λόγος για επανάληψη.

1.4.10 Συμπληρωματικότητα, και ένας κανόνας επιλογής

Καμία αρχιτεκτονική δεν κυριαρχεί, αλλά δεν είναι και περιττές: λύνουν *διαφορετικά* υποσύνολα. Η επιλογή της καλύτερης *μεμονωμένης* αρχιτεκτονικής κερδίζει από +0 έως +6 εργασίες και μειώνεται απότομα καθώς ανεβαίνει η ικανότητα. Αν αντ' αυτού επιλεγεί η αρχιτεκτονική που ταιριάζει σε *κάθε* εργασία, το περιθώριο που ανακτάται είναι πραγματικό στα ασθενέστερα μοντέλα (+3.2 εργασίες πάνω από τον θόρυβο μιας μεμονωμένης εκτέλεσης) και εντός θορύβου στο ισχυρότερο.

Οι δύο δηλώσεις δεν συγκρούονται: είναι το ίδιο εύρημα από δύο οπτικές. Ο μέσος όρος καταστρέφει το σήμα ακριβώς επειδή τα πλεονεκτήματα των αρχιτεκτονικών είναι ειδικά ανά εργασία και αλληλοαναιρούνται. Το ερώτημα «βοηθά αυτή η αρχιτεκτονική;» είναι σχεδόν αναπάντητο: εξαρτάται από την εργασία, καθώς κερδίζει όπου το μοντέλο έχει ακόμη περιθώριο και χάνει στις ήδη λυμένες, ώστε στο σύνολο τα δύο αλληλοαναιρούνται. Το ερώτημα «ποια αρχιτεκτονική για αυτή την εργασία;» αποδίδει εκεί όπου το μοντέλο έχει ακόμη περιθώριο.

Τα προφίλ των μοντέλων συντίθενται σε έναν κανόνα επιλογής που χρειάζεται μόνο μετρήσεις διαθέσιμες από *μία* εκτέλεση αναφοράς:

> Αν το ποσοστό έγκυρων τεχνουργημάτων του μοντέλου είναι κάτω από 40%, χρησιμοποίησε μόνο G1: το scaffold δεν θα τρέξει. Αλλιώς, αν σταματά νωρίς, σχεδόν οτιδήποτε βοηθά. Αλλιώς, αν καταναλώνει σχεδόν όλα του τα βήματα, χρησιμοποίησε G4 και αναδιάρθρωσε την προσπάθεια αντί να την επεκτείνεις. Αλλιώς χρησιμοποίησε G3 και επαλήθευσε την ολοκληρωμένη προσπάθεια.

Ο κανόνας ονομάζει τη σωστή οικογένεια για **6 από τα 7** μοντέλα, με μια επιφύλαξη που μετράει περισσότερο από τον ίδιο τον αριθμό: σε 3 από τις επτά περιπτώσεις η «καλύτερη οικογένεια» κρίνεται με διαφορά **μικρότερη της μισής εργασίας**, που με μία εκτέλεση ανά διαμόρφωση βρίσκεται μέσα στον θόρυβο. Ο αριθμός πρέπει επομένως να διαβάζεται ως «ο κανόνας δείχνει τη σωστή περιοχή» και όχι ως ποσοστό ακρίβειας.

Η μία καθαρή αστοχία είναι διδακτική. Στο Qwen-7B ο κανόνας λέει G1, επειδή το μοντέλο παράγει έγκυρο σχέδιο μόλις στο 0% των εκτελέσεων. Η οικογένεια με τον υψηλότερο μέσο όρο είναι όμως η G3, ένα τέταρτο της εργασίας πάνω από μια G2 που στηρίζεται εξ ολοκλήρου στον G2-Monitor, *τη μόνη αρχιτεκτονική της G2 που δεν εκπέμπει καθόλου δομημένο τεχνούργημα*. Ο κανόνας προβλέπει ορθά ότι τα scaffolds με τεχνούργημα αποτυγχάνουν σε αυτό το μοντέλο, και σφάλλει μόνο επειδή αντιμετωπίζει την οικογένεια ως ενιαίο μπλοκ· η διαφορά G3–G2 είναι έτσι κι αλλιώς εντός της μισής εργασίας.

Είναι κανόνας παραγόμενος από αυτά τα επτά μοντέλα και αξιολογημένος στα ίδια επτά, οπότε προσφέρεται ως διαδικασία απόφασης με δηλωμένο μηχανισμό και όχι ως επικυρωμένος προβλεπτής. Η αξία του είναι ότι και οι τρεις είσοδοί του προκύπτουν από **μία** εκτέλεση αναφοράς.

1.5 Ετερογενής Επίβλεψη

Το σκέλος αυτό απαντά στο **EE5**: αν η υποστήριξη βοηθά προσφέροντας ό,τι λείπει από ένα μοντέλο, μπορεί αυτό να προέλθει από ένα *διαφορετικό, ισχυρότερο* μοντέλο; Αντικαθιστούμε μόνο τον εποπτικό ρόλο με ένα ισχυρό μοντέλο, ενώ κάθε δράση στο περιβάλλον εκτελείται ακόμα από τον εργάτη.

Δύο αρχιτεκτονικές κινούνται σε *αντίθετες* κατευθύνσεις καθώς ο εργάτης ισχυροποιείται: ο G4-Orchestrator ανεβαίνει (+4, +5, +8) ενώ ο G3-Judge πέφτει (+3, −3, −6).

Ο ισχυρός στρατηγός πετυχαίνει περισσότερα με λιγότερα. Σε κάθε εργάτη, ο G4-Orchestrator με επιβλέποντα παράγει περισσότερες λύσεις με λιγότερους κύκλους, λιγότερα βήματα και λιγότερα tokens. Ο αδύναμος στρατηγός που καθοδηγεί τους δικούς του ειδικούς εξαντλεί το όριο κύκλων σε 26% των εκτελέσεων, έναντι 15% υπό ικανό στρατηγό. Αυτό αναπλαισιώνει το «κόστος συντονισμού»: το κόστος του G4-Orchestrator δεν βρίσκεται στον συντονισμό αυτόν καθαυτόν αλλά στην ποιότητα των αποφάσεων δρομολόγησης, και με ικανότερο στρατηγό μειώνεται αντίστοιχα.

Ένας ισχυρότερος κριτής δεν μεταφέρεται όπως οι κατευθυντικοί ρόλοι. Στο ισχυρότερο μοντέλο ο κριτής-Gemini *εγκρίνει* τουλάχιστον εξίσου καλά με τον ίδιο τον εργάτη, αλλά οι *απορρίψεις* του σώζουν σαφώς λιγότερο, και έτσι το ισοζύγιο βγαίνει αρνητικό. Δεν είναι ότι η κριτική του είναι πολύ αφηρημένη: διαβάζοντας τα ίδια τα κείμενα, οι κριτικές του είναι σαφώς ακριβέστερες και πιο συγκεκριμένες. Το αν οι επιπλέον απορρίψεις του βρίσκονται πάνω από την ανοχή του βαθμολογητή, ή είναι ακριβείς αλλά πέρα από τη δυνατότητα του εργάτη να τις επισκευάσει, δεν μπορεί να διαχωριστεί, γιατί το benchmark βαθμολογεί μόνο την τελική κατάσταση και όχι την υποβολή που είδε ο κριτής. Έτσι, για μια πύλη, εκείνο που κρίνει το αποτέλεσμα δεν είναι η ωμή ικανότητα του επιβλέποντα αλλά η ευθυγράμμιση του πήχη του με το κριτήριο.

Η γενίκευση επιβεβαιώνεται σε ολόκληρο το σκέλος. Μετρώντας τις εργασίες που ο εργάτης δεν λύνει ποτέ μόνος του, οι κατευθυντικοί ρόλοι τις σώζουν σε 2.6% των περιπτώσεων και οι αξιολογικοί σε 0.0%. Σε μία μόνο περίπτωση σε ολόκληρο το πλέγμα μια επιβλεπόμενη διαμόρφωση έλυσε εργασία που καμία άλλη δεν έλυσε, και αυτό που προσέφερε ο επιβλέπων ήταν μία παρατήρηση για το περιβάλλον που ο εργάτης *είχε ήδη τυπώσει και δεν διάβασε*. Η ικανότητα συντονισμού και η προσοχή στο περιβάλλον δεν είναι ο ίδιος πόρος, και ο δεύτερος είναι φθηνότερος στην παροχή.

> *Σε κατευθυντικό ρόλο, ένας ισχυρότερος επιβλέπων είναι καλύτερος, γιατί προσθέτει καλύτερη πληροφορία. Σε αξιολογικό ρόλο, ένας ισχυρότερος δεν είναι κατ' ανάγκη καλύτερη πύλη: η αξία ενός κατωφλιού εξαρτάται από την ευθυγράμμισή του με το κριτήριο που βαθμολογείται, όχι από την ικανότητα πίσω του.*

Τα οικονομικά ακολουθούν τον ίδιο κανόνα. Ανά λυμένη εργασία η ετερογένεια είναι φθηνότερη σε tokens και για τους τρεις εργάτες (-18%, -18% και -12%), παρότι προσθέτει δεύτερο πράκτορα: το όφελός της είναι η σπαταλημένη δουλειά που αποτρέπει, οπότε ο παρονομαστής πέφτει ταχύτερα από όσο ανεβαίνει η κατανάλωση. Ο χρόνος ρολογιού δεν αναφέρεται, επειδή οι εκτελέσεις καλύπτουν περισσότερες από μία διαμορφώσεις υλικού και τα δευτερόλεπτα συγχέουν την αρχιτεκτονική με το μηχάνημα που την εξυπηρέτησε.

1.6 Συζήτηση

1.6.1 Μία αρχή για ολόκληρο το πλέγμα

Κάθε αποτέλεσμα της μελέτης συμβιβάζεται με μία αρχή: *μια αρχιτεκτονική αποδίδει όταν προσφέρει κάτι που λείπει από το μοντέλο, σε μορφή που το μοντέλο μπορεί να χρησιμοποιήσει, και κοστίζει τον προϋπολογισμό που καταναλώνει όταν δεν το κάνει.* Καθεμία από τις τρεις ρήτρες είναι ξεχωριστά μετρήσιμη και αποτυγχάνει με διαφορετικό, παρατηρήσιμο τρόπο.

Το έλλειμμα πρέπει να υπάρχει. Η αξία που προσθέτει μια αρχιτεκτονική φράσσεται από την απόσταση μεταξύ όσων πετυχαίνει το μοντέλο στην πρώτη του προσπάθεια και όσων θα μπορούσε να πετύχει με αναθεώρηση. Όπου η απόσταση αυτή είναι μηδέν, η αρχιτεκτονική έχει περιθώριο μόνο για ζημιά. Διατυπωμένη θετικά, η αρχή γίνεται πιο αιχμηρή: *η υποστήριξη και η ικανότητα είναι μερικά υποκατάστατα.* Το scaffold εκτελεί *εξωτερικά*, σε ξεχωριστό πράκτορα ή ξεχωριστή φάση, την εξερεύνηση, τον σχεδιασμό και τον αυτο-έλεγχο που ένα ισχυρότερο μοντέλο έχει εσωτερικεύσει. Σε αυτή την ανάγνωση, η αντίστροφη σχέση με την ικανότητα δεν είναι εμπειρική περιέργεια προς εκ των υστέρων εξήγηση αλλά συνέπεια του τι *είναι* η παρέμβαση.

Η μορφή πρέπει να είναι αξιοποιήσιμη. Αυτή είναι η ρήτρα που η βιβλιογραφία δεν προβλέπει. Μια αρχιτεκτονική επικοινωνεί μέσω ενός τεχνουργήματος, και αν το μοντέλο δεν μπορεί να το παραγάγει ή να δράσει βάσει αυτού, η αρχιτεκτονική δεν τρέχει. Η συμμόρφωση σε δομημένη έξοδο διαφέρει κατά έναν παράγοντα είκοσι μεταξύ των μοντέλων χωρίς να ακολουθεί την ικανότητα, γεγονός που αναπλαισιώνει σημαντικό μέρος των αποτελεσμάτων της G2 ως scaffolds που ποτέ δεν εκτελέστηκαν.

Το απλήρωτο κόστος είναι ο προϋπολογισμός. Επειδή τα βήματα μοιράζονται και δεν προστίθενται, κάθε βήμα που δαπανάται στη μηχανική της αρχιτεκτονικής είναι βήμα που δεν δαπανάται στην εκτέλεση. Σε μοντέλο με πραγματικό έλλειμμα η ανταλλαγή αξίζει · σε μοντέλο χωρίς έλλειμμα είναι καθαρή απώλεια.

1.6.2 Γιατί ο μέσος όρος παραπλανά

Η μηδενική κύρια επίδραση έχει άμεση μεθοδολογική συνέπεια. Μια μελέτη που αξιολογεί ένα scaffold σε ένα μοντέλο και αναφέρει βελτίωση έχει μετρήσει *ένα κελί μιας αλληλεπίδρασης* και το ανέφερε ως κύρια επίδραση. Ανάλογα με το ποιο μοντέλο επέλεξε, η ίδια αρχιτεκτονική θα στήριζε το αντίθετο συμπέρασμα, και με βάση αυτά τα δεδομένα η επιλογή του μοντέλου μετράει περισσότερο από την επιλογή της αρχιτεκτονικής. Ένας αρχιτεκτονικός ισχυρισμός οφείλει επομένως να συνοδεύεται από το προφίλ ικανότητας των μοντέλων στα οποία μετρήθηκε.

1.6.3 Η ανάλυση αποτυχιών δεν παράγει αρχιτεκτονική

Οι δεκατέσσερις αρχιτεκτονικές δεν επιλέχθηκαν αυθαίρετα: καθεμία επιλέχθηκε επειδή η ανάλυση ιχνών των εκτελέσεων αναφοράς εντόπισε μια συγκεκριμένη, επαναλαμβανόμενη αποτυχία και η αρχιτεκτονική την αντιμετώπιζε άμεσα. Ο πράκτορας που δεν επιθεωρούσε ποτέ τα αρχεία εισόδου γέννησε τον εξερευνητή· εκείνος που επανέκδιδε είκοσι φορές την ίδια σπασμένη εντολή γέννησε τον επόπτη· εκείνος που υπέβαλλε δουλειά αποτυγχάνουσα σε προφανή έλεγχο γέννησε τον κριτή.

Αυτό καθιστά το πλέγμα ελεγχόμενο τεστ της ίδιας της επαγωγής, και η επαγωγή δεν επιβεβαιώνεται. Οι διαγνώσεις ήταν σωστές: τα αποτελέσματα επιβεβαιώνουν κάθε μία ως πραγματική και συχνή συμπεριφορά. Αυτό που δεν ισχύει είναι το βήμα από το «να γιατί αποτυγχάνει» στο «άρα διόρθωσε αυτό». Μια παρέμβαση που αφαιρεί έναν τρόπο αποτυχίας δεν βελτιώνει με αυτόν τον τρόπο το αποτέλεσμα, επειδή πληρώνεται από τον ίδιο προϋπολογισμό που χρειαζόταν ο εκτελεστής και επειδή εισάγει δικούς της τρόπους αποτυχίας: η σύνοψη του εξερευνητή εκτοπίζει την παρατήρηση του εκτελεστή, η συμβουλή του επόπτη αγνοείται, ο κριτής απορρίπτει δουλειά που ήταν ήδη σωστή. Το βήμα σπάει σιωπηλά, ακριβώς επειδή η διάγνωση στην οποία στηρίζεται είναι ορθή.

1.7 Απειλές στην Εγκυρότητα

Στατιστική εγκυρότητα. Καμία διαμόρφωση δεν επαναλαμβάνεται, οπότε τα επιμέρους δέλτα κελιών βρίσκονται εντός θορύβου. Η greedy αποκωδικοποίηση αφαιρεί την ελεγχόμενη συνιστώσα της διακύμανσης, αλλά η εξυπηρέτηση σε παρτίδες δεν είναι ντετερμινιστική σε επίπεδο bit: οι αναγωγές κινητής υποδιαστολής εξαρτώνται από τον τρόπο ομαδοποίησης των αιτημάτων και μπορούν να αλλάξουν ένα οριακό $\arg\max$. Σε μία απάντηση αυτό είναι αόρατο· σε βρόχο πράκτορα ένα διαφορετικό token νωρίς παράγει διαφορετική εντολή και από εκεί διαφορετική τροχιά.

Τι παρατηρεί πραγματικά ο πράκτορας. Η παρατήρηση που επιστρέφεται μετά από κάθε εντολή δεν είναι η τυπική έξοδος της εντολής αλλά η ορατή οθόνη του τερματικού. Ο δείκτης έναρξης που χρησιμοποιεί ο περιτυλιγμένος κώδικας είναι γνήσιο πρόθεμα του δείκτη λήξης, οπότε οι δύο αναζητήσεις εντοπίζουν την ίδια θέση και η εξαγωγή επιστρέφει ολόκληρο το παράθυρο. Το μοντέλο λαμβάνει επομένως την οθόνη όπως θα την έβλεπε άνθρωπος: αντίγραφα εντολών, προτροπές φλοιού και ό,τι έχει αφήσει πίσω της προηγούμενη εντολή. Η πληροφορία συνήθως υπάρχει μέσα στον θόρυβο και χάνεται μόνο όταν μια μακρόχρονη εντολή γεμίσει το παράθυρο.

Πρόκειται για ελάττωμα της υλοποίησης που γράφτηκε για αυτή τη μελέτη και όχι του benchmark. Η συνέπειά του για τις συγκρίσεις είναι περιορισμένη: η ρουτίνα εξαγωγής είναι ταυτόσημη και στις οκτώ υλοποιήσεις συστημάτων, οπότε κάθε αρχιτεκτονική και κάθε μοντέλο αντιμετώπισαν το ίδιο κανάλι παρατήρησης και καμία αντίθεση της εργασίας δεν συγχέεται από αυτό. Αυτό που αλλάζει είναι το *κατασκεύασμα*: η μελέτη μετρά πράκτορες που λειτουργούν πάνω σε θορυβώδη όψη τερματικού, κάτι που αποτελεί έναν από τους λόγους για τους οποίους τα απόλυτα ποσοστά δεν πρέπει να διαβάζονται έναντι δημοσιευμένων νούμερων άλλων πλαισίων.

Ονομαζόμενο έναντι μετρούμενου κατασκευάσματος. Μια εργασία ονομάζει μια ικανότητα· το πλαίσιο εκτέλεσης αποφασίζει ποια ικανότητα ασκείται στην πράξη. Το σαφέστερο παράδειγμα είναι μια εργασία που το benchmark πλαισιώνει ως οπτική συλλογιστική παρέχοντας μια εικόνα. Επειδή το πλαίσιο δεν διαθέτει κανάλι εικόνας, κανένα μοντέλο δεν βλέπει την εικόνα και όλα πρέπει να γράψουν κώδικα υπολογιστικής όρασης, οπότε η εργασία όπως εκτελείται μετρά αντίληψη-μέσω-κώδικα.

Ποιότητα υλοποίησης των αρχιτεκτονικών. Οι αρχιτεκτονικές είναι *δικές μας* υλοποιήσεις δημοσιευμένων σχεδίων. Μια κακή υλοποίηση θα υποτιμούσε το δυναμικό της οικογένειάς της. Το γεγονός ότι ταυτόσημος κώδικας βελτιώνει τα ασθενή μοντέλα και επιβαρύνει τα ισχυρά περιορίζει την ένσταση χωρίς

να την εξαλείφει. Την αποδυναμώνει περαιτέρω εξωτερική ένδειξη: ανεξάρτητες ομάδες που μετρούν την ίδια σχέση με δικές τους υλοποιήσεις, υπό εξισωμένους προϋπολογισμούς και κατά μήκος εύρους ικανότητας, αναφέρουν την ίδια κατεύθυνση [1], [2].

Μια διόρθωση που χωρίζει το πλέγμα στον χρόνο. Η σοβαρότερη εσωτερική απειλή εντοπίστηκε αργά. Ο μηχανισμός που περικόπτει παλιά μηνύματα όταν γεμίζει το παράθυρο συμφραζομένων ήταν αναξιόπιστος στις πρώτες εκτελέσεις: και οι 138 εκτελέσεις που τερμάτισαν σε υπερχείλιση συμφραζομένων έτρεξαν πριν από μια διόρθωση τον Μάιο του 2026, και μετά από αυτήν η κατηγορία δεν εμφανίζεται ούτε μία φορά. Ο θιγόμενος πληθυσμός είναι 585 εκτελέσεις (9.3%), συγκεντρωμένες σε λίγα κελιά, τέσσερα εκ των οποίων είναι γραμμές αναφοράς. Μια τεχνητά χαμηλή γραμμή αναφοράς διογκώνει το μετρούμενο κέρδος · η κατεύθυνση αυτή ευνοεί τον ισχυρισμό μόνο στο ένα από τα τέσσερα (το ασθενές μοντέλο) και στρέφεται *εναντίον* του στα άλλα τρία, όπου το εύρημα είναι ότι η αρχιτεκτονική βλάπτει. Ακόμη και ένας ακραίος έλεγχος ευαισθησίας, που υποθέτει ότι κάθε μία από τις υπερχειλισμένες εκτελέσεις θα είχε λύσει την εργασία της, ανεβάζει απότομα τις θιγόμενες γραμμές αναφοράς και αναδιατάσσει τον άξονα ικανότητας, όμως η συσχέτιση κατάταξης που προκύπτει, $\rho = -0.886$, παραμένει έντονα αρνητική, οπότε η βαθμίδα δεν είναι τεχνούργημα του σφάλματος.

Εξωτερική εγκυρότητα. Το πλέγμα χρησιμοποιεί μόνο την εύκολη ζώνη. Το πείραμα ορίου τεκμηριώνει τον περιορισμό από τη μία πλευρά, αλλά αφήνει ανέλεγκτη την ίδια την πρόβλεψη του μηχανισμού. Επιπλέον, ο επιβλέπων στο ετερογενές σκέλος είναι φιλοξενούμενος ενώ οι εργάτες είναι τοπικοί και κβαντισμένοι, οπότε η εποπτική ικανότητα συγχέεται με τη στοίβα εξυπηρέτησης.

1.8 Συμπεράσματα

1.8.1 Πρακτικές συστάσεις

Οι ρήτρες της ενότητας 1.6 αποκτούν την ακόλουθη επιχειρησιακή μορφή.

- **Μέτρηση του μοντέλου πριν από την επιλογή αρχιτεκτονικής.** Το ποσοστό επιτυχίας της γραμμής αναφοράς προβλέπει το όφελος · το πλήθος παραμέτρων δεν λειτουργεί ως υποκατάστατο.
- **Επιβεβαίωση ότι η αρχιτεκτονική εκτελέστηκε, πριν από την ερμηνεία αρνητικού αποτελέσματος.** Σε μοντέλο που παράγει σπανίως έγκυρα σχέδια, τα αποτελέσματα του σχεδιαστή μετρούν συμμόρφωση σε σχήμα και όχι σχεδιασμό.
- **Προτεραιότητα στους μηχανισμούς υπό συνθήκη.** Η επαλήθευση ενεργοποιείται μόνο κατά την υποβολή, οπότε δεν κοστίζει σχεδόν τίποτα όταν η πρώτη προσπάθεια είναι ήδη σωστή.
- **Ορισμός του κατωφλιού σύγκρισης από την G1 και όχι από τον γυμνό πράκτορα.** Τρεις προτάσεις οδηγιών βελτιώνουν πέντε από τα επτά μοντέλα χωρίς κατανάλωση προϋπολογισμού, και για ένα από αυτά αποτελούν την καλύτερη αρχιτεκτονική του πλέγματος.
- **Αντιστοίχιση του μηχανισμού με τον τρόπο αποτυχίας του συγκεκριμένου μοντέλου.** Η επέκταση μιας εκτέλεσης ωφελεί το μοντέλο που σταματά νωρίς και δεν προσφέρει τίποτα σε εκείνο που ήδη καταναλώνει σχεδόν όλα του τα βήματα.
- **Επιλογή ανά εργασία και όχι ανά μοντέλο.** Το κέρδος της πρώτης είναι πολλαπλάσιο εκείνου της δεύτερης, εκεί όπου το μοντέλο έχει ακόμη περιθώριο (πέρα από τον θόρυβο στα ασθενέστερα, εντός θορύβου στο ισχυρότερο).

1.8.2 Συνεισφορές

1. **Ένα ελεγχόμενο αποτέλεσμα εξαρτώμενο από την ικανότητα.** Σε 7 μοντέλα και 14 αρχιτεκτονικές υπό σταθερό, μοιρασμένο προϋπολογισμό, το πλήθος των αρχιτεκτονικών που ξεπερνούν τη γραμμή αναφοράς μειώνεται απότομα καθώς αυτή ανεβαίνει, και η ίδια αρχιτεκτονική αλλάζει πρόσημο κατά μήκος του φάσματος · κατά μέσο όρο η κύρια επίδραση είναι σχεδόν μηδενική, η ποσοτική μορφή του ισχυρισμού ότι καμία αρχιτεκτονική δεν αποτελεί καθολική βελτίωση.
2. **Συμπληρωματικότητα**: διαφορετικές αρχιτεκτονικές λύνουν διαφορετικές εργασίες, οπότε το σύνολο που θα μπορούσε να φτάσει ένα μοντέλο επιλέγοντας ανά εργασία την αρχιτεκτονική που ταιριάζει είναι μεγαλύτερο από ό,τι δίνει οποιαδήποτε μεμονωμένη. Το επιπλέον περιθώριο είναι πραγματικό στα ασθενέστερα μοντέλα και εξαφανίζεται στο ισχυρότερο, ακολουθώντας το ίδιο μοτίβο ικανότητας: οι αρχιτεκτονικές ανακτούν εργασίες μόνο όπου το μοντέλο έχει ακόμη περιθώριο.

3. **Μηχανιστική ερμηνεία με δύο χωριστά μετρήσιμες συνθήκες**: μια αρχιτεκτονική ωφελεί ένα μοντέλο μόνο όπου υπάρχει απόσταση ανάμεσα στην πρώτη του προσπάθεια και σε όσα θα μπορούσε να φτάσει μια αναθεώρηση, και η απαίτηση το τεχνούργημα της αρχιτεκτονικής να είναι σε μορφή που το μοντέλο μπορεί να παραγάγει και να αξιοποιήσει. Και οι δύο στηρίζονται σε συμπεριφορικά ίχνη ανά εκτέλεση.
4. **Μια προκαταρκτική ευρετική επιλογής ανά μοντέλο** (παρατήρηση, όχι επικυρωμένος κανόνας) από μετρήσεις που παρέχει μία μόνο εκτέλεση αναφοράς, που δείχνει τη σωστή οικογένεια για 6 από τα 7 μοντέλα, με 3 από αυτές τις επιλογές εντός θορύβου.
5. **Μελέτη ετερογενούς επίβλεψης, και μια διάκριση ρόλων**: η ικανότητα μεταφέρεται μέσω του επιπέδου συντονισμού, αλλά μόνο μέσω ρόλων που *προσθέτουν* πληροφορία· οι ρόλοι που *φιλτράρουν* πρέπει να βαθμονομηθούν ως προς το κριτήριο που βαθμολογείται.
6. **Αναπαραγώγιμο υπόβαθρο ανάλυσης**: ολόκληρη η μελέτη παγώνει σε πίνακα 7424 γραμμών, από τον οποίο παράγεται μηχανικά κάθε ποσοτικός ισχυρισμός της εργασίας.

1.8.3 Περιορισμοί

Οι αρχιτεκτονικές είναι δικές μας υλοποιήσεις δημοσιευμένων σχεδίων· ο ισχυρισμός είναι στενός και ελεγχόμενος. Το εύρος δυσκολίας των εργασιών είναι περιορισμένο: το πλέγμα αρχιτεκτονικών χρησιμοποιεί μόνο τις εύκολες εργασίες. Το προκαθορισμένο πείραμα ορίου (160 εκτελέσεις αναφοράς σε τυχαίες medium και hard εργασίες) επιβεβαιώνει τον περιορισμό από τη μία πλευρά, καθώς το ποσοστό επιτυχίας πέφτει στο 10.0% και 1.2% αντίστοιχα, αλλά αφήνει ανέλεγκτη την ίδια την πρόβλεψη του μηχανισμού: αν η ερμηνεία μέσω του περιθωρίου αναθεώρησης είναι σωστή, τα ισχυρά μοντέλα σε δυσκολότερες εργασίες θα έπρεπε να ανακτούν έλλειμμα και να ωφελούνται ξανά, μετατρέποντας τη μονότονη βαθμίδα σε καμπύλη με εσωτερικό βέλτιστο. Οι μονές εκτελέσεις σημαίνουν ότι κανένα μεμονωμένο κελί δεν είναι στατιστικά διακριτό. Και η ταξινομία σχημάτων εργασιών είναι εκ των υστέρων, οπότε προσφέρεται ως υπόθεση με ταυτοποιημένο μηχανισμό και όχι ως επικυρωμένος προβλεπτής.

1.8.4 Μελλοντική εργασία

Η πολυτιμότερη επέκταση είναι μια ζώνη δυσκολίας ρυθμισμένη ώστε να αφήνει σε ένα ικανό μοντέλο περιθώριο αναθεώρησης, εκτελεσμένη υπό το *πλήρες* πλέγμα αρχιτεκτονικών και όχι μόνο σε διαμόρφωση αναφοράς. Αυτό αποτελεί άμεσο και διαψεύσιμο έλεγχο του μηχανισμού. Δεύτερη, φθηνότερη κατεύθυνση είναι η επανάληψη σε στοχευμένο υποσύνολο κελιών, τη γραμμή αναφοράς κάθε μοντέλου και την καλύτερη και χειρότερη αρχιτεκτονική του, ώστε το επιχείρημα που στηρίζεται σε κατεύθυνση να αποκτήσει διαστήματα εμπιστοσύνης με κλάσμα του κόστους του πλήρους πλέγματος.

Δύο παρεμβάσεις προκύπτουν από συγκεκριμένα αρνητικά αποτελέσματα. Το σαφέστερο είναι ότι η *ανίχνευση πετυχαίνει και η ανάκαμψη αποτυγχάνει*: ο επόπτης εντοπίζει σωστά τη στασιμότητα και ο εκτελεστής συνεχίζει ούτως ή άλλως. Κάθε παρέμβαση που δοκιμάστηκε είναι *συμβουλευτική*, προσθέτει λέξεις σε συμφραζόμενα που το μοντέλο είναι ελεύθερο να αγνοήσει. Η αδοκίμαστη εναλλακτική είναι *εξαναγκαστική*: άρνηση εκτέλεσης εντολής που επαναλαμβάνει πρόσφατη, ή υποχρεωτική απαρίθμηση εναλλακτικών πριν επιτραπεί νέα προσπάθεια. Ο κίνδυνος πρέπει να μετρηθεί αντί να υποτεθεί, καθώς σε εργασίες όπου η επιμονή είναι η σωστή στρατηγική ένα τέτοιο μπλοκάρισμα στρέφεται εναντίον μιας προσέγγισης που δούλευε. Η δεύτερη παρέμβαση αφορά το κενό μεταξύ σκέψης και δράσης: ένα scaffold θα μπορούσε να απαιτεί από κάθε βήμα συλλογιστικής να καταλήγει σε δηλωμένη επόμενη ενέργεια και στη συνέχεια να ελέγχει την ενέργεια που όντως έγινε έναντι της δήλωσης, μετατρέποντας μια ανεπιβεβαίωτη πρόθεση σε επαληθεύσιμη.

Chapter 2

Introduction

2.1 Motivation

Large language models increasingly act as *agents*. Rather than producing a single response, they issue shell commands, read and write files, execute programs, observe the results, and iterate toward a goal inside a real environment.[1] This shift from text generation to environment interaction has produced a large and fast-moving body of work on *agent scaffolding*: architectural machinery placed around the base model to make it a better agent. Proposed scaffolds include explicit task decomposition and planning, structured exploration of the environment before acting, post-hoc self-verification, critique and reflection, and collaboration among multiple specialised agents.

Two properties of this work limit what can be concluded from it. It is usually evaluated on a single model, most often a frontier one, and it reports aggregate success rates, often without holding the compute budget fixed between the single-agent baseline and the augmented system: an architecture that adds a planner and a verifier also adds the model calls those components make.

Neither property is universal, and the exceptions matter enough to name. Some studies equalise the thinking budget explicitly and find the multi-agent advantage disappears [3]; others compare scaffolds across dozens of models, though to establish ranking fidelity rather than per-architecture effects [4]. What has not been done is to vary architecture and model capability *together*, under an identical implementation and a conserved budget, which Section 3.5 states as this thesis's first gap.

Both properties conceal a question that matters to anyone deploying agents outside a frontier laboratory, a population that is not marginal. A prominent position paper argues that small models, not large ones, are the natural substrate for agentic systems, on the grounds that agent workloads are repetitive, specialised and cost-sensitive [5]. If that is right, the behaviour of scaffolding on models of this size is the operative case rather than the degenerate one. When the model is not frontier, a quantized 7B–70B open-weight model served on a single accessible GPU, does the same scaffolding still help, or does its value change with the capability of the model it wraps? The question is not just bookkeeping. A scaffold that adds a second reasoning pass can only pay off if the model's first pass is *revisable*: if the model already solves the task on its first attempt the second pass is overhead, and if it fails in a way it cannot recover from the second pass cannot help either. That reasoning predicts architectural value should be conditional on capability rather than uniform, a prediction not yet tested under controlled conditions.

2.2 Problem statement

This thesis measures **how agent architecture interacts with base-model capability** on terminal tasks. We evaluate 14 architectures across 7 models on a fixed, externally defined task set, holding the reasoning-step budget constant and *carving* it across sub-agents so that no configuration is granted extra work. The design deliberately fixes everything except the two variables of interest: the architecture, and the capability of the model executing it.

Three properties distinguish the study from the literature it builds on:

[1] All code and data for this study, including the frozen run-level dataset from which every number in this thesis is generated, are openly available at github.com/harrissfiris/terminal-agent-architectures.

- **The capability axis is populated, not assumed.** Seven models spanning roughly an order of magnitude in effective agentic capability run the *identical* scaffold code. Four of them belong to one family at four sizes, which separates scale from training recipe.
- **The budget is conserved.** A multi-agent configuration receives the same 60 reasoning steps as the plain baseline, divided among its components. This forecloses the standard objection that multi-agent gains merely reflect additional compute.
- **The analysis is mechanistic, not only aggregate.** Every run records its termination reason, step and token consumption, tool-call statistics, and architecture-specific instrumentation (whether a plan validated, whether a verifier engaged, how many coordination cycles were consumed). Claims about *why* an architecture helps or hurts are grounded in these traces.

2.3 A worked example of the problem

Two tasks make the difficulty concrete before any results are reported. Both receive the identical G2-Explorer configuration, a read-only agent that inspects the environment and reports what it finds before the executor acts, and yet the traces show that phase doing opposite things across the two.

`log-summary` asks an agent to count `ERROR`, `WARNING` and `INFO` lines across three small log files and write the totals to a CSV. At baseline Devstral-24B spends its entire 60-step budget and never writes the file. Its fifth step issues the correct command, `grep -c` over the three severities, and the terminal returns the correct nine numbers, but interleaved and unlabelled, three counts per file with nothing to say which severity each belongs to. The agent reissues that same command **fifty-five times** until the step budget runs out.

Under G2-Explorer the same model finishes in seven steps. The explorer's summary carries the full contents of all three files, fifteen lines in total. The executor still runs its own `grep` calls and still receives the same unusable output; on step five it writes `ERROR,4 WARNING,3 INFO,8`, which are the counts derivable from the summary text and from nothing else it saw. What the architecture supplied here was a *legible channel* for information the terminal was failing to deliver, not a better strategy.

`fix-git` asks an agent to recover work that is no longer on any branch and merge it into master. At baseline Qwen-32B solves it completely in 34 steps: it checks the branches, fails to reach a remote, and on step twelve runs `git reflog`, which surfaces an unreferenced commit titled "Move to Stanford". It checks that commit out, branches, merges, resolves the resulting conflict by hand, and passes every grader check.

Under the same G2-Explorer it fails, passing none of them. The explorer inspects twenty-one files (every PDF and thumbnail in the repository) and returns a summary whose requirements field is empty and whose observations concern research papers. The answer to this task is not in a file; it is in the reflog, which a file inspector cannot see. The executor, handed an inventory of documents, searches the history it can already reach, finds changes that are *already* merged, and submits that no action is needed. It never runs `git reflog`.

The architecture did not become better or worse between the two tasks. What changed is whether what it supplied was what the model was missing, and the mechanism is visible in the runs rather than inferred from the outcome. This thesis sets out to say, systematically and with evidence, when that is the case.

2.4 Research questions

Five questions organise the study. They are not independent: the first establishes a result, the next three explain it at three different levels, and the fifth asks whether the limit it exposes can be crossed from outside. Chapter 6 takes them in this order for that reason.

- **RQ1 — Capability conditioning.** Does agent scaffolding improve agentic coding *uniformly*, or does its effect (including its *sign*) depend on the capability of the underlying model? And is that capability substitutable by model size, so that scale could serve as the axis in its place?
- **RQ2 — Failure structure.** How do runs actually fail, how do those failures differ by model, and what does each architecture repair or introduce?

- **RQ3 — Architecture mechanism.** What makes an architecture help rather than hurt, which property of a design decides whether it improves a run? And is any single architecture a general improvement, or does the effective choice depend on the model?
- **RQ4 — Task properties.** Which properties of a task predict *whether* an architecture will help it, and *which* family helps most? And what does each such property cost to obtain: can it be read from the task statement, or must the task be run first?
- **RQ5 — Heterogeneous supervision.** Can a strong supervisor driving a weak worker outperform the homogeneous weak agent, and through which roles? That is: can capability be injected at the coordination layer rather than the execution layer?

2.5 Summary of findings

The study's principal results, stated here so that the chapters that follow can be read with the conclusions in view. Findings **F1**–**F5** answer the five questions above in order; **F6**–**F7** are properties of the grid as a whole that no single question asks about. Chapter 6 is organised on the same numbering, and Section 2.6 records which finding each contribution rests on.

F1 (RQ1). Architecture is not a universal improvement, its sign depends on the model, and the relevant property is not size. The number of architectures beating a model's own baseline falls from 13 of 13 on one of the weakest models to 0 of 13 on the strongest open coder. The same scaffold that rescues one model burdens another; averaged over the models these opposite-sign effects cancel to a near-zero main effect, so the aggregate hides the interaction rather than contradicting it. That axis is coding capability, not parameter count: the least-helped model is the strongest open coder (a 32B), not the largest by parameters (a 72B), while one of the most-helped is a 70B whose own baseline is among the lowest. Counting discrete conversions rather than averages sharpens this: architectural wins do cluster by task, at 2.9 times the rate chance would produce, but two models failing the same task are rescued by overlapping sets of architectures at only $\kappa = 0.12$. The task-level component is real and weak, so "this architecture helps on this task" is a tendency rather than a prediction unless the model is named too.

F2 (RQ2). The dominant failure is not running out of budget, and models fail in characteristically different ways. Across every model the largest failure category is a run that stopped because the agent decided it was finished and submitted something wrong, which is what makes post-execution verification the best-targeted family. The profiles differ sharply underneath that: one model abandons tasks after 13 steps, another grinds its whole budget away to 54, and the weak models themselves split between quitting too early and never quitting at all. A separate failure mode belongs only to architectures: a model can be strong enough to solve a task and still unable to emit the structured artifact the scaffold needs, in which case a reported regression measures a scaffold that never ran.

F3 (RQ3). The architecture that helps a model is the one matched to how that model fails, and no architecture is best for more than two of the seven. Mid-run redirection helps the model that stops early and hurts the one that grinds, from identical code. Five different architectures are best across the seven models, spanning all four families, and for the strongest open coder the correct choice is the plain baseline.

F4 (RQ4). Architecture responds to a task's headroom and shape, not its topic. At the extremes the sign of the effect is pinned by how many models already solve a task: every architecture loses solves on already-solved tasks and every architecture gains on tasks nothing solves, while the contested middle settles nothing. Task difficulty and task length both fail to separate the cases. Pass/fail also hides real work: more runs pass some of their grader checks while failing overall than pass outright. What it hides is smaller than that framing suggests, though. On tasks whose pass/fail outcome never changes, only the two critique-based designs advance the graders' checks at all, by +3.0 and +1.1 points, while every pre-execution architecture moves them backwards.

F5 (RQ5). Capability can be injected at the coordination layer, but only through certain roles. A strong model supervising a weak one improves outcomes when its role is to *direct* (routing, planning, guiding) and degrades them when its role is to *judge*: a stronger judge approves at least as well, but its rejections rescue *less* rather than more, so raw capability does not transfer through an evaluative role the way it does through a directive one.

F6. Selecting the architecture per task is worth more than selecting it per model, on the models that still have room to gain. Choosing the single best architecture for a model is worth

between +0 and +6 tasks and collapses as capability rises. A per-task oracle raises the bound to 6 to 14 tasks, but pooling many single runs inflates any union, so the bound is read against a noise null: the excess over it is real on the weaker models (+3.2 tasks) and within noise on the strongest (-0.2). The per-task view recovers real headroom where a model still misses tasks, not a capability-independent bound.

F7. A single baseline run already points at the per-model choice (a preliminary heuristic). Because the architecture that helps is the one matched to how a model fails, three measurements a single baseline run supplies (how early the model stops, what it fails at, and whether it can emit a valid artifact) name the right family for 6 of 7 models, without an architectural sweep. It is a practical rule rather than a validated one: it is read from the same seven models it scores, and 3 of those calls turn on margins under half a task.

2.6 Contributions

1. **A capability-conditional result on agent architecture.** *[F1; gap 1]* Across 7 models and 14 architectures under a fixed, carved budget, the number of architectures that beat a model's own baseline falls steeply as that baseline rises ($\rho = -0.886$ across the open models, $p = 0.033$), from 13/13 on one of the weakest models to 0/13 on the strongest open coder. The same architecture changes sign across the capability range, and averaged over the models no architecture moves the aggregate solve rate by more than a few points: a null main effect with a large interaction, which is the quantitative form of the claim that *no architecture is a universal improvement.*

2. **A complementarity result.** *[F6]* Different architectures solve different tasks, so the set a model could reach by choosing the architecture that fits each task is larger than what any single architecture gives it. That extra room is real on the weaker models and fades on the strongest, following the same capability pattern as the rest of the study: architectures recover tasks only where a model still has room.

3. **A mechanistic account with two separately measurable conditions.** *[F2, F4; gap 2]* An architecture helps a model only where there is distance between the model's first attempt and what a revision could reach, so its value tracks how much room the model leaves for a second look. A second condition the literature does not anticipate is that the architecture's artifact must be in a form the model can produce and act on. Both are supported by per-run evidence: model-specific failure signatures, verifier engagement rates, and a format-compliance effect that varies by a factor of twenty across models without tracking capability, under which complex architectures degrade small models by breaking their structured output rather than their reasoning.

4. **A preliminary per-model selection heuristic, offered as an observation rather than a validated rule.** *[F7]* Because the architecture that helps is the one matched to how the model fails, how early a model stops, what it fails at, and whether it emits a valid artifact point at the right family for 6 of 7 models without an architectural sweep. It is read from the same seven models it scores, off a single noisy baseline, and 3 of the calls turn on margins under half a task, so it is a direction worth validating rather than a result.

5. **A heterogeneous supervision study, and a role distinction.** *[F5; gap 3]* A separate track in which a strong supervisor drives a weak worker isolates whether capability transfers through the coordination layer. It does, but only through roles that *add* information; in a role that *gates* output a stronger supervisor does not transfer the same way: its rejections rescue less rather than more, and on the strongest worker the trade turns net negative.

6. **A reproducible analysis substrate.** *[infrastructure]* The entire study is frozen into a per-run tabular dataset covering 7424 model–task rows, from which every number in this thesis is generated mechanically.

2.7 Thesis outline

Chapter 3 positions the work against the agent, multi-agent, verification, and benchmark literature, and states the gap precisely. Chapter 4 describes the agent system in three parts: the shared ReAct substrate on which every configuration is built together with the budget model that makes them comparable, then the 14 architectures themselves, each with its mechanism, the failure mode it targets and its canonical

source (Section 4.4 onward), and finally the heterogeneous supervision track (Section 4.11). Chapter 5 gives the experimental methodology: models, dataset, coverage, single-run validity, statistical treatment, and the objective task metadata used throughout. Chapter 6 is the empirical core, taking the five research questions in the order stated above. Chapter 7 develops the account of when an architecture helps, that it pays only where a model leaves room for a second look, and derives design guidance. Chapter 8 states threats to validity, and Chapter 9 concludes.

Chapter 3

Background and Related Work

This chapter establishes the technical vocabulary the thesis uses and positions the study against four bodies of work: the foundations of language-model agents, the benchmarks that evaluate them, the architectures this thesis instantiates, and the methodological literature on how agent evaluation should be conducted. It closes by stating the gap in falsifiable terms. The review covers work published up to and including August 2026; the field moves quickly, and work appearing after that date was not searched systematically.

3.1 Foundations: language models as agents

3.1.1 From text generation to environment interaction

The transformer architecture [6], the demonstration that scale alone yields general few-shot capability [7], the scaling laws that made capability a predictable function of compute [8], the instruction tuning that made models follow a task description at all [9], and the frontier systems those lines produced [10] all concern a model producing one output. A language model used for text generation is evaluated on that single output. Used as an *agent* it is evaluated on a trajectory: it proposes an action, the environment responds, and the response conditions everything that follows. Four properties distinguish the second setting and shape every design decision in this thesis.

Partial observability. The model does not begin with the information it needs. A terminal agent cannot see which packages are installed, what a data file contains, or whether a service is running until it looks. Much of what appears as reasoning failure is a model acting on assumptions it never checked, a point Section 6.5 returns to with evidence.

Irreversibility. Actions change the world. A deleted file is gone; a corrupted repository stays corrupted. Unlike sampling several candidate answers and picking one, an agent's mistakes accumulate into the state its later actions must work from.

Error compounding. Because each observation conditions the next action, an early wrong turn does not cost one step but redirects the remainder of the trajectory. This is why agentic failures are frequently long rather than short, a point that becomes quantitative in Section 6.1.2, where failed runs consume roughly twice the steps of successful ones.

A budget. Every agent operates under a finite allowance of steps, tokens or time. The allowance is not incidental: it is what makes the trade-off between deliberation and execution real. An architecture that spends part of the budget on planning necessarily leaves less for acting, which is the observation Section 4.3 builds the study's fairness model on.

3.1.2 The ReAct loop

The *ReAct* paradigm [11] interleaves reasoning and acting in a single loop: the model emits a thought, then an action, observes the result, and continues. Its significance is less the specific format than the demonstration that reasoning traces and environment actions belong in the same context, since the model's own prior reasoning is available when it interprets the next observation.

The loop has since become a research programme in its own right: several surveys map the resulting design space of reasoning, planning and tool use [12], [13], [14], and one line recasts it in the vocabulary of cognitive architectures, separating memory, action space and decision procedure [15]. Agents built on it have been demonstrated in open-ended environments [16] and in social simulation [17].

ReAct is the substrate for every configuration in this thesis (Chapter 4). All 14 architectures wrap, precede, interleave with, or multiply this loop; none replaces it. That uniformity is what allows differences between configurations to be attributed to the architecture rather than to the underlying agent.

Subsequent work extends the loop along three axes that map directly onto the architecture families evaluated here. *Reflection* adds a verbal self-critique between attempts, so that a failed trajectory informs the next one [18]. *Action representation* replaces JSON tool calls with executable code, consolidating the action space [19]. *Search* explores several reasoning paths rather than committing to one, either as a tree over thoughts [20], as a graph permitting merges [21], or with explicit value estimates over trajectories [22]. The last of these is not evaluated in this thesis: search multiplies the compute budget by the branching factor, which is the confound the carved budget of Section 4.3 exists to remove.

3.1.3 Tool use and structured output

An agent acts through tools. The capability can be trained into the model [23] or elicited by prompting, and the action space itself is a design choice, program synthesis in place of natural-language reasoning [24], or executable code in place of JSON calls [19]. Whatever the choice, a tool call must be expressed in a form the harness can parse. Two mechanisms exist. *Native function calling* has the model emit tool calls through a dedicated API channel, with the provider enforcing the schema. *Prompt-mode calling* describes the schema in the system prompt and requires the model to emit conforming JSON in its ordinary output.

This thesis uses prompt-mode calling for every model, including the hosted one. The reason is fairness: native function calling is unavailable or unreliable on the quantized open-weight models in the study, so using it where available would confound the architecture under test with the serving stack. The cost of that choice is that *format compliance becomes a variable*. This is not a peculiarity of our setting: model behaviour is acutely sensitive to surface features of the prompt, with accuracy varying by tens of points across semantically equivalent formats [25], and instruction-following is measured as its own axis precisely because it does not follow from task capability [26]. Section 6.4.2 shows it to be a consequential one here: a capable model can fail to participate in an architecture simply because it cannot reliably emit the artifact the architecture requires.

3.1.4 Context as a physical constraint

Every model has a finite context window, and an agent's conversation grows monotonically: each step appends an action and an observation, and observations from a terminal can be large. When the conversation approaches the window, something must be discarded. What survives matters beyond mere capacity: models attend unevenly across a long context, retrieving reliably from its beginning and end while degrading in the middle [27], and an entire line of work treats context as a managed resource with an explicit memory hierarchy [28].

This thesis treats the context window as a *per-model physical capacity* rather than an experimental variable: it is a property of the deployed artifact, like quantization. The consequence is that models in this study differ in window size by almost an order of magnitude, and Section 6.2.4 shows the effect is not the simple one might expect: the model with the highest context-exhaustion rate is not the one with the smallest window, and a model consuming 75% more tokens in the same window overflows less often, because its trimming behaves differently.

3.1.5 Reasoning models and test-time compute

A more recent class of models is trained to produce extended internal reasoning before answering, trading inference compute for accuracy [29]. The underlying finding is that inference-time compute is a substitutable resource: allocated well it can outperform additional parameters [30], and even naive allocation (sampling repeatedly and keeping whatever passes a verifier) lifts coverage log-linearly over four orders of magnitude [31], with self-consistency the classical instance [32]. Such models are deliberately excluded from this study. An internal reasoning trace is itself a form of scaffolding (deliberation before action) so including reasoning models would confound the axis under test: a gain could be attributed either to the architecture or to reasoning the model performs regardless. Holding this constant keeps the comparison

clean, at the cost of not speaking to how architecture interacts with built-in deliberation. Chapter 9 identifies this as future work.

3.2 Agentic benchmarks

3.2.1 TerminalBench

TerminalBench [33] evaluates agents on real terminal tasks. Each task ships a containerised environment, a natural-language instruction, and a suite of executable tests written by the task author. The agent is given shell access and must bring the environment to a state the tests accept. A task counts as solved only when *all* of its tests pass.

Four properties make it the right benchmark for this study. The environment is **real**, not simulated: commands genuinely execute and their effects persist. Grading is **executable** rather than model-judged, so success is not itself an LLM judgement. Tasks carry an **author-supplied difficulty label**, which supplies an external, reproducible selection criterion: this thesis runs *all* tasks labelled easy, not a self-selected subset, which forecloses the most obvious selection-bias objection. And the **tool surface is minimal**: raw shell rather than a curated IDE toolkit, so the agent's competence is not confounded with the quality of bespoke tooling.

Reported results on the benchmark are sobering. Frontier agents score below 65% on the harder task set, and the benchmark's own analysis identifies substantial headroom. Crucially for this thesis, that evaluation varies the *agent product*, a model and its scaffold together, as shipped , and not the architecture as a controlled variable. That is the axis this thesis supplies.

3.2.2 The wider landscape

Adjacent benchmarks locate the setting. *InterCode* [34] established interactive coding as an evaluation paradigm, code written against execution feedback rather than emitted in one shot, and its bash environment is the closest predecessor to the setting used here. *SWE-bench* [35] evaluates repository-level issue resolution: a longer horizon over a codebase rather than a terminal session, and the setting that motivated much modern agent scaffolding, including the agent–computer interface work [36] whose budget conventions Chapter 4 compares against. *TheAgentCompany* [37] extends the horizon further into simulated professional work, where the strongest agent completes 30% of tasks autonomously. *IDE-Bench* [38] moves the same question into an IDE-native interface with structured search and editing tools. Further afield, *AgentBench* [39] evaluates agents across eight environments, *OSWorld* [40] in a full desktop, *GAIA* [41] on assistant tasks requiring tool use, *MLAgentBench* [42] on machine-learning experimentation, and τ-bench [43] on tool-agent-user interaction, the last introducing a `pass^k` metric explicitly because agents prove inconsistent across repeated trials, a concern Section 5.5 inherits. Alongside the benchmarks sit the agent *products* they are used to rank, general-purpose platforms such as OpenHands [44] and task-specialised pipelines such as AutoCodeRover [45], which vary scaffold and model together as an engineering choice rather than as an experimental variable.

One recent benchmark is worth separating out because its finding is directly about the axis this thesis tests. *LongCLI-Bench* [46] studies long-horizon agentic programming in command-line interfaces and reports that *self-correction offers marginal gains* while plan injection and interactive guidance yield substantially larger improvements. That is the pre-execution/post-execution contrast, observed in the same environment class, and it points the opposite way to this thesis's result, which is exactly why the comparison is developed in Section 3.5 rather than left implicit.

Two contrasts matter for interpreting this thesis's results. Benchmarks providing rich, purpose-built tooling (structured code search, semantic file editing) report higher absolute performance than raw-shell settings, so the numbers here should not be read as a ceiling on what agents can do with better tools. And benchmarks evaluating *static* code generation, the lineage running from HumanEval [47] to contamination-resistant successors that refresh their problems [48], measure a different capability entirely, since the model emits one answer with no environment interaction: the failure modes catalogued in Section 6.3 (premature submission, budget exhaustion, context overflow) cannot occur there.

3.2.3 What benchmarks report, and what they omit

Agentic benchmarks conventionally report a single number: the fraction of tasks solved, usually from one run per configuration, with two consequences this thesis addresses explicitly.

First, **binary grading discards partial progress**. A task with fourteen tests scores identically whether the agent passed none or thirteen. Section 6.5 shows this is not a minor loss of resolution: more runs in this study pass *some* of their tests while failing overall than pass outright, and architectures that appear inert on solve rates advance those runs substantially. The methodological literature has made the general form of this argument before: discontinuous metrics can manufacture apparent discontinuities in capability [49] that smooth metrics do not show, and the recommendation to report a continuous measure alongside the binary one is adopted here.

Second, **single-run evaluation is the field norm**, and this thesis follows it in order to purchase breadth. That norm has been questioned on its own terms: evaluation that ignores non-determinism can mislead, though the same work finds greedy decoding to outperform sampling on most tasks [50], which is the setting used here. Holistic evaluation frameworks make the complementary argument that a single scalar is never a sufficient report [51]. Section 5.5 sets out what single-run costs and what it therefore permits the study to claim.

3.3 Agent architectures

This section reviews the designs the thesis instantiates. Each is presented with its original claim and its evaluation context, so that Section 4.5 can describe *our* implementation and Chapter 6 can report where the original claim reproduces across a capability range.

One observation applies to the whole section and motivates the study. These designs were, almost without exception, proposed and validated on a single strong model, and reported as improvements over a single-agent baseline without holding the compute budget fixed. Neither practice is illegitimate (both are what one does when introducing a method) but together they leave open whether the reported gains are properties of the architecture, of the model it was tuned on, or of the additional model calls it makes.

3.3.1 Planning and decomposition

The oldest idea in agent scaffolding is that a complex task should be decomposed before it is attempted. The idea predates agents: *least-to-most* prompting solves a problem by first reducing it to a sequence of simpler sub-problems [52], and *plan-and-solve* separates devising a plan from carrying it out [53]. Moving into agents, *MetaGPT* [54] assigns explicit software-engineering roles, with a planner producing a specification that executors implement, and reports that structured role-based decomposition outperforms unstructured multi-agent chat. *ADaPT* [55] makes decomposition recursive and *as-needed*: a task is attempted directly, and only decomposed further when the attempt fails. That conditionality is a direct precursor to a finding in Chapter 6: unconditional decomposition spends budget on tasks that did not need it.

Our G2-Planner and G2-Analyst architectures instantiate this axis (Section 4.7), with the budget for planning *carved* from the executor's allowance rather than added to it.

3.3.2 Structured exploration and grounding

If partial observability is the defining property of the setting, then establishing what the environment contains before acting is an obvious remedy. A weaker but older form of the same idea has the model interrogate itself for the information it is missing before answering [56]. *Agentless* [57] makes the strongest form of the argument: a fixed pipeline of localise, repair, validate (with no agentic loop at all) outperforms several agent frameworks on repository-level tasks, suggesting that much of what scaffolding provides is disciplined information gathering rather than autonomy. Our G2-Explorer architecture isolates this component: a read-only phase whose only output is a structured summary of what the environment contains.

3.3.3 Runtime observation and intervention

A third position is that intervention should happen *during* execution rather than before or after it. *CAMEL* [58] introduces inception prompting, in which one agent shapes another's behaviour through injected framing rather than instruction, the mechanism our G2-Monitor adopts, injecting guidance as the executor's own reasoning step rather than as an external message. Section 6.6.3 shows that this delivery detail is not cosmetic: whether guidance is phrased in the second or third person separates the interventions that work from those that do not.

3.3.4 Verification, critique, and reflection

Post-hoc verification asks whether completed work is correct. *Reflexion* [18] has the agent write a verbal self-critique after a failed attempt and retain it for the next, converting failure into usable context; *Self-Refine* [59] iterates critique and revision within a single attempt; and *self-debugging* [60] narrows this to code, having the model explain its own program and read execution results.

The literature is, however, genuinely divided on whether *intrinsic* self-correction works at all. A prominent negative result finds that without external feedback LLMs largely fail to correct their own reasoning, and sometimes degrade after trying [61]. The reconciling position is that the feedback must come from outside the model: *CRITIC* [62] shows self-correction succeeding when critiques are grounded in tool output, and process supervision improves results by judging intermediate steps rather than final answers [63], [64]. This thesis's G3 family sits precisely on that fault line: its verifier has environment access but no test suite (Section 4.8), and Section 6.4.3 reports which side of the divide the results fall on. *LLM-as-judge* [65] uses a separate model to evaluate work, and (relevantly for this thesis) documents systematic biases in such judgements, including verbosity and self-preference; a subsequent survey catalogues these and the attempts to mitigate them [66]. Section 6.6.4 adds a further one specific to in-the-loop use: a stronger judge's rejections rescue *less* often rather than more, so raw judging capability does not transfer the way it does in a directive role.

Our G3 family instantiates this axis as a single submit-intercept framework with four variants (Section 4.8) that differ *only* in how the second look is structured, which is what lets the comparison isolate the contribution of context continuity rather than of verification in general.

3.3.5 Multi-agent collaboration

The final family replaces the single agent with several. *AutoGen* [67] provides general multi-agent conversation with programmable termination and routing; *ChatDev* [68] organises agents around a waterfall development process; *multi-agent debate* [69] has agents argue toward a better answer; and *mixture-of-agents* [70] aggregates several models' proposals. Orchestrator designs [71] add a router that dispatches to specialists at runtime, which our G4-Orchestrator follows.

AgentVerse [72] studies emergent behaviour in such groups, and a communication-centric survey organises the family by what the agents actually exchange [73].

A sceptical counter-literature has grown alongside it, and this thesis belongs to that side of the debate. Simply instantiating more agents and voting improves results, which suggests that some of the reported benefit is sampling rather than collaboration [74]. Under *equalised* thinking budgets, single agents match or beat multi-agent systems on multi-hop reasoning, the authors attributing prior gains to unaccounted computation [3]. And the advantage of multi-agent systems has been observed to shrink as the underlying model improves [1], which is the capability gradient of this thesis, arrived at independently on a different task family.

This is the family where the compute-control question is sharpest, since n agents make approximately n times the model calls unless something prevents it. Section 4.3 describes what this thesis does about that, and Section 6.1.2 verifies it against what the runs actually consumed.

3.4 The methodology of agent evaluation

This is the literature closest to the present study, because it concerns how agent performance should be *measured* rather than how it should be improved.

3.4.1 Separating the model from the scaffold

The most direct antecedent applies item-response theory to aggregated leaderboard data [75], decomposing observed agent performance into a model component and a scaffold component and using the decomposition to predict success on unseen model–scaffold combinations. Its contribution is to establish that the separation is real and statistically identifiable across many benchmarks.

The relationship to this thesis is complementary rather than competitive. That work asks whether per-task success *can be predicted* across combinations, and answers it with a statistical model fitted to observational data spanning many benchmarks. This thesis asks *when and why* an architecture helps, and answers it with a controlled grid on a single benchmark in which everything except the architecture

and the model is held fixed. The former establishes that the question is well posed; the latter supplies the mechanism for one setting. Neither substitutes for the other: a predictive decomposition cannot say *why* a scaffold fails on a given model, and a mechanism established on one benchmark does not generalise without further evidence.

3.4.2 Why multi-agent systems fail

A closely related line catalogues multi-agent failures directly [76], annotating more than 1,600 traces across seven frameworks into a taxonomy of fourteen failure modes spanning system-design issues, inter-agent misalignment, and failures of task verification. The methodological point (that multi-agent failures are structured and recurrent rather than idiosyncratic) is one this thesis reproduces independently: the failure classes derived in Section 6.3 arise from the same three regions.

Two of this thesis's findings can be read as instances of that taxonomy with a controlled comparison attached. Coordination failure appears here as a measurable termination class that *only* multi-agent configurations can exhibit (Section 6.3.3), and verification appears as a role where a stronger verifier fails to transfer, since its rejections rescue less rather than more (Section 6.6.4). The difference in approach is that the prior work is cross-framework on varied tasks, whereas this study is cross-architecture and cross-model on a fixed task set, which is what allows the failure rate to be attributed to the architecture rather than to the framework or the task.

3.4.3 Evaluation cost, subset selection, and variance

A third line asks how much evaluation is enough. Work on efficient agent benchmarking [4] finds that absolute score prediction degrades under scaffold-driven distribution shift while *rank-order* prediction remains stable, and that restricting evaluation to tasks of intermediate historical pass rate preserves ranking fidelity while cutting the number of tasks by 44–70%, a finding motivated by the observation that tasks everything solves and tasks nothing solves carry no information.

This thesis inherits both the constraint and the remedy. Restricting to the benchmark's easy-labelled tasks is a difficulty filter of the same kind, applied for the same reason: these constrained models fail almost entirely on harder tasks, so the architectural signal lives in the easier band. And Section 6.5.3 arrives at the discriminating-band observation directly from the data: the headroom distribution is lopsided, with most of the 64 tasks sitting at or near the floor and only a narrow contested band moving under any architecture, which bounds what any aggregate over the full set can show.

3.5 The gap this thesis addresses

The literature above establishes three things: that agent scaffolding takes a small number of recognisable forms, that its evaluation is conventionally single-model and compute-uncontrolled, and that the separation between model capability and scaffold contribution is real enough to be worth measuring. What it does not contain is a controlled measurement of the interaction between them.

Three statements delimit the contribution. Each is falsifiable: a matching prior paper would require revising the corresponding claim. Each is also stated *against named work* rather than against the literature in the abstract, so that a reader can check it.

1. **No prior published work performs a controlled architecture × model-capability ablation on terminal-agent tasks** with an *identical* scaffold implementation and a *conserved* compute budget across configurations. The nearest neighbours each vary a different thing: the psychometric decomposition [75] separates model from scaffold statistically across *many* benchmarks rather than experimentally within one; efficient benchmarking [4] varies scaffolds and models at scale but to establish *ranking fidelity under subset selection*, not per-cell effects; MAST [76] varies frameworks and annotates failures without a controlled architecture contrast; and Terminal-Bench 2.0 [77] reports frontier-agent numbers on the benchmark without varying architecture or open-model family systematically.

2. **No prior work isolates the pre-execution-decomposition versus post-execution-verification axis across multiple open model families under a common budget.** The two families are typically proposed separately and evaluated on different models, so their relative merits have not been measured on common ground. The closest result is LongCLI-Bench [46], discussed

below, which reports the contrast in the same environment class but for a single model and a single configuration of each.

3. **No prior work evaluates heterogeneous supervision** (a strong model occupying a supervisory role while a weaker model executes) as a controlled variable alongside the homogeneous configurations it must be compared against. This is the axis on which Section 4.11 and Section 6.6 make the study's most original contribution.

Table 3.1 records where each of these gaps is addressed.

Table 3.1: Where each gap statement is addressed.

Gap	Addressed by	Where
(1)	The 7 × 14 grid under a carved budget, with the budget verified against measured consumption	§6.1, §6.1.2
(2)	The pre-execution versus post-execution family split, by task structure	§6.5.4
(3)	Three workers × six supervisory roles, paired against their homogeneous counterparts	§4.11, §6.6

These three are claims about what the *literature* lacks, and they are deliberately fewer than the five research questions of Section 2.4. A gap statement says what has not been measured; a research question says what this study asks in order to measure it. Gaps (1) and (2) are answered by RQ1 and RQ4 respectively and gap (3) by RQ5, while RQ2 and RQ3 (how models fail, and which architecture suits which model) claim no novelty against prior work and exist to *explain* the result the first gap produces. They are the reason this thesis reports a mechanism rather than a table.

3.5.1 Position against the closest work

Table 3.2 places the study against the six works whose questions overlap most. The pattern is that most neighbours hold fixed what this thesis varies, and vary what this thesis holds fixed, which is why the results are complementary rather than competing. The one exception, the concurrent [2], varies architecture as well, and is treated separately below.

Table 3.2: The six closest works, by what each holds fixed and what it varies. The last row is this thesis.

Work	Question	Method	Scaffold
Scaling agent systems [2]	How does a multi-agent system's gain *scale* with coordination, capability and task?	5 canonical topologies, 260 configurations, a fitted predictive model	varied
Agent psychometrics [75]	Can per-task success be *predicted* for unseen model–scaffold pairs?	IRT on aggregated leaderboards, many benchmarks	observed
Efficient benchmarking [4]	How few tasks preserve the *ranking*?	33 scaffolds × 70+ models, subset selection	varied
MAST [76]	Why do multi-agent systems *fail*?	1,600+ annotated traces, 7 frameworks, 14 failure modes	varied
Terminal-Bench 2.0 [77]	How hard are realistic terminal tasks?	89 tasks, frontier agents, leaderboard	fixed
LongCLI-Bench [46]	Does self-correction or guidance help long-horizon CLI work?	one model, one configuration per intervention	two points
This thesis	*When and why* does an architecture help or hurt?	7 models × 14 architectures × 64 tasks, one implementation, carved budget	**the variable**

Three of these deserve more than a table row.

The closest question, reached by the opposite method. The concurrent [2] is the nearest work by question: it asks when scaling a multi-agent system helps, varies five canonical coordination topologies across many models and tasks, and reports that *architecture–task alignment* decides the outcome, with the measured effect of a given topology running from strongly positive to strongly negative depending on

whether the coordination matches the task. That is the same shape as this thesis's central result, that an architecture's sign is contingent on the model and the task rather than fixed, reached independently, which is reassuring for both. The two differ in what they vary and what they produce. That work varies *coordination topology* among multiple agents and fits a predictive scaling model; this thesis varies the *single-agent scaffolding spectrum* (prompting, reflection, verification, and a heterogeneous supervision track) on open-weight models under a carved budget, and reads the contingency off per-run traces rather than a fitted curve. One supplies a law that predicts *how much*; this supplies the mechanism for *why*, in a setting the other does not cover.

The predictive decomposition, and what it cannot supply. The psychometric decomposition [75] is the work closest in *concept*: it also treats agent performance as separable into a model term and a scaffold term, and it demonstrates that the separation is statistically identifiable. The difference is what the separation is for. A fitted decomposition answers *how much* of an observed score to attribute to the scaffold, across combinations nobody has run; it cannot say *why* a particular scaffold fails on a particular model, because the failure never enters the model as anything but a residual. This thesis takes the opposite trade: one benchmark, one implementation, no predictive generality, and, in exchange, the traces that show a monitor's advice being ignored, a plan failing to validate, or a judge rejecting correct work. The two results are the same claim approached from opposite ends, and the fair reading is that the statistical work establishes the question is well posed while this work supplies the mechanism for one setting.

A finding that points the other way, and why. LongCLI-Bench [46] evaluates long-horizon agentic programming in command-line interfaces (the closest environment to the one used here) and reports that *self-correction offers marginal gains* while injected plans and interactive guidance help substantially. Read naively, that is the reverse of this thesis's family split, in which post-execution verification is the most robust family and pre-execution planning the weakest. The tension is stated plainly rather than omitted, and three differences plausibly account for it, each of which the present design makes visible.

First, **the plans differ in origin**. The guidance in that setting is *human-injected*; the plans here are generated by the same model that will execute them, and Section 6.4.2 shows that on several models they do not even validate against the schema. A plan supplied by a competent external source and a plan the model writes for itself are not the same intervention, and the heterogeneous supervision track of Section 4.11 is an attempt to interpolate between them.

Second, **the horizons differ**. Long-horizon work gives a plan more to coordinate and gives planning overhead more room to amortise; a task median of 14 steps (Section 4.3) gives it very little. This is a scope condition of this study rather than a disagreement, and it makes the same prediction as the headroom account: planning should recover value as tasks lengthen.

Third, **the budgets differ**. Injected guidance costs the agent nothing from its own allocation, whereas here every planning step is subtracted from execution (Section 4.10). Under an additive budget the pre-execution family would be measured on what its plans are worth; under a carved one it is measured on whether they are worth the steps they cost. Both are legitimate questions, and the answers differ.

None of these dissolves the disagreement, and it should not be dissolved by argument. It is stated here as the strongest available external check on the study's central claim, and Chapter 9 names harder and longer tasks as the extension that would settle it.

Chapter 4

The Agent, Its Architectures, and the Supervision Track

Every configuration evaluated in this thesis is a variant of one agent implementation. This chapter describes the whole system in the order that makes the comparison interpretable: first what all configurations *share*, then what *varies* between them, and finally the one track that varies something else entirely.

Section 4.1–4.3 give the shared substrate: the executor loop, the tool interface, the memory model, the safety infrastructure, and the budget model that makes architectures with different numbers of components comparable at all. Sections 4.4–4.10 define the 14 architectures themselves, each with the mechanism as implemented, the failure mode it targets, and how its share of the budget is carved. Section 4.11 introduces the heterogeneous supervision track, in which the roles are filled by *different* models.

The purpose of the whole chapter is to establish that differences observed in Chapter 6 are attributable to architecture rather than to incidental engineering. Everything in the first three sections is identical across all fourteen configurations and all seven models.

4.1 The shared ReAct executor

Every configuration is built on one ReAct executor. The model runs a Thought → Action → Observation loop against the task's Docker container, which is driven through a persistent `tmux` session: the model emits a tool call, the tool executes in the container, and the resulting observation is appended to the conversation and fed back. The episode ends when the agent calls `submit`, or when the budget is exhausted; on the final step the agent is explicitly told it must submit. Figure 4.1.1 shows the loop that every one of the 14 architectures wraps, precedes, interleaves with, or multiplies.

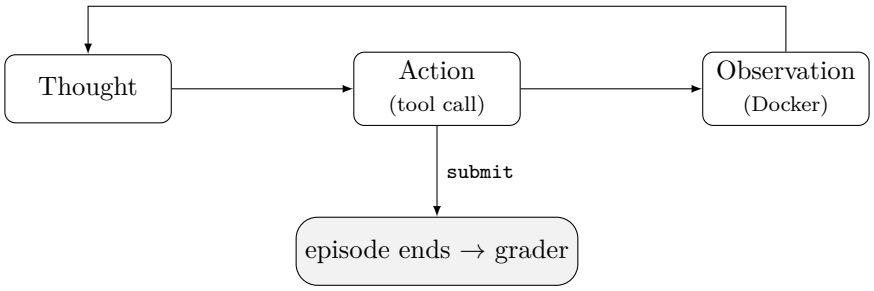


Figure 4.1.1: The shared ReAct executor. Every architecture in this thesis wraps, precedes, interleaves with, or multiplies this loop; none of them replaces it.

The persistent session matters more than it might appear. Because state survives between steps (the working directory, environment variables, background processes, files written), the agent operates on a stateful environment rather than a sequence of independent commands. This is what makes the failure

mode observed in Section 6.5 possible, where an agent concludes it has finished a task that requires a service to still be running.

4.1.1 What a step is

Because the step is this study's budget unit, its definition needs to be precise.

One step is one model call plus the tool execution it requests. On each step the model receives the complete state: the system prompt, the task instruction, and the full conversation so far including every previous action and its observation, and emits exactly one tool call. The harness executes it, appends the result, and the next step begins. The model therefore decides *once* per step, with everything it has learned available to it.

This has two consequences, the second easy to misread.

A step is a decision point, not a shell command. A `bash` step submits a command *string*, which may itself contain several commands joined by `&&`, semicolons, or pipes. In this corpus **32% of `bash` steps contain more than one command**, and an agent routinely writes `apt-get update && apt-get install -y python3-opencv` as a single step. The 60-step budget is therefore *not* a limit of 60 shell commands; a run may execute several hundred. What it limits is how many times the agent stops, looks at what happened, and decides again.

That distinction is exactly what makes the step the right unit for this study. Chaining commands is a form of *confidence*: the agent commits to a sequence without inspecting intermediate output. Pausing to look is what costs a step. Budgeting by steps therefore prices deliberation rather than activity, which is the quantity an architecture comparison should hold constant, and it is why an architecture that inserts a planning phase genuinely gives up executor decisions in exchange.

4.1.2 Tool interface

The tool set is deliberately small: `bash`, `read_file`, `write_file`, `patch_file`, `think`, and `submit`. Three design decisions in this list are worth justifying, because each affects what the study can measure.

Tool calls use prompt-mode schemas. The schema is described in the system prompt and the model emits conforming JSON in its ordinary output, rather than through a provider's native function-calling channel. This is forced by fairness: native calling is unavailable or unreliable on the AWQ-quantized open models in the study, so using it where available would confound the architecture under test with the serving stack. Every model in the grid uses the same interface, including the hosted one.

The cost is that emitting a well-formed tool call becomes part of what is being measured. Section 6.4.2 shows this is consequential: structured-output compliance varies across models by a factor of twenty and does not track task capability.

`think` is a real tool. The agent can reason without acting, and that reasoning enters the conversation as an ordinary step. This exists so that deliberation is *visible and budgeted* rather than hidden, and it is the mechanism by which G2-Monitor delivers its guidance (Section 4.7): injected text appears as the executor's own `think` output rather than as an external message. Section 6.6.3 shows that this delivery detail determines whether the guidance works.

Writing is separated from patching. `write_file` replaces a file wholesale; `patch_file` applies a targeted edit. Keeping them distinct means the trace records whether an agent rewrote or revised, which is a behavioural signal the analysis uses.

4.1.3 Memory and context management

An agent's conversation grows monotonically, and terminal observations can be large: a directory listing or a compiler log easily runs to thousands of tokens. Since every model has a finite context window, the harness must decide what to discard.

Token accounting is *calibrated rather than estimated*: the harness derives a characters-per-token ratio from the server's own reported usage instead of assuming a fixed ratio, so the estimate tracks the actual tokenizer of the model being served. When a request would exceed the window, the harness trims the oldest history and retries; the number of messages discarded is recorded per run.

That recording turns out to matter. Section 6.2.4 reports that Devstral-24B consumes 75% more tokens than Qwen-32B in the same 28k window and discards 3.4 times as many messages, so it retains far less of its own history. The context window is a physical capacity, but how a model's traces interact with the trimming mechanism shapes how much of its history survives. Overflow *terminations* themselves are a separate matter, confined to a pre-fix harness epoch (Section 8.2) rather than a stable per-model trait.

Context capacity is deliberately **not** equalised across models. It is a property of the deployed artifact, like quantization, and equalising it downward would handicap models for a capability they genuinely possess. The consequence (that models differ in window size by almost an order of magnitude) is stated as a threat in Chapter 8.

4.1.4 No forced intervention

The harness never force-terminates a run. Repeated identical actions and unparseable outputs trigger *nudges* and explicit correction messages, but never a hard kill: the natural budget is the sole terminator.

This is a deliberate fairness decision, and it was a revision of an earlier design that did hard-kill after repeated failures. Three arguments motivated the change. A hard kill is an *arbitrary intervention* that could cut off a model which repeats an action before recovering. It is *redundant*, because the step and wall-clock budgets already terminate infinite loops. And most importantly for this study, it would *erase the evidence*: if loops are terminated early, the failure-mode analysis of Section 6.3 cannot observe them, and the distinction between a model that abandons tasks at 13 steps and one that grinds to 54 (the distinction that predicts which architecture helps each) would be invisible.

The cost of the choice is efficiency: a looping run burns its full budget rather than being cut short. That is accepted, because the run is being measured rather than deployed.

4.2 Safety infrastructure

The agent executes arbitrary shell commands in a container, and a small guard layer intercepts commands before execution. Its three actions are recorded per run as counters: *blocked* (refused outright), *warned* (allowed with a caution appended), and *rewritten* (transformed into a safer equivalent).

Two properties of this layer matter for the validity of the study. First, it is **identical across all configurations and models**, so it cannot differentially advantage any architecture. Second, its activity is **reported as data rather than applied silently**: the counters are part of the frozen dataset, so a reader can verify that guard interventions are not doing hidden work. In practice they are rare and evenly distributed.

4.2.1 What the harness is not allowed to do

A study that builds its own harness must say what that harness is forbidden from doing, because the easiest way to inflate an agent's score is to let the scaffolding solve part of the task. Four constraints were enforced throughout and are stated so a reader can check them against the released code.

1. **No task-specific code path anywhere.** The harness contains no branch keyed on a task identifier, no hint text, and no hardcoded answer. Every task is processed by the same code.
2. **Interventions are content-free.** The guard layer above and the anti-repetition nudges carry no information about the task: they can tell an agent that it is repeating itself, never what it should do instead. An intervention that supplied task content would be solving rather than scaffolding.
3. **Environment normalisation is uniform.** Fixes such as permitting `pip` to install into the system environment, or making `apt-get` non-interactive, apply to every task and every configuration alike. They remove friction from the container, not from the problem.
4. **No deterministic component solves anything.** No configuration contains a rule-based step that can produce task content or pass a grader check; everything that advances a task is the model's own output. What the architectures vary is how that output is organised.

These are stated because they are checkable, not because they are surprising. The architectures differ in how the model's own effort is organised; none of them differ in how much of the task the harness does for it, because the answer is always none.

A related piece of infrastructure is the *robust command transport*. Commands that are fragile under shell quoting (multi-line scripts, very long commands, heavy escaping) are transmitted base64-encoded and decoded inside the container rather than passed through the shell directly, which preserves working directory and environment. Simple commands take the ordinary path. This exists because an early version of the study lost runs to transport corruption rather than to model error, which is a harness artifact rather than a finding.

4.3 The budget, and why it is carved rather than added

Every configuration operates under the same primary budget: **60 reasoning steps**, with an **1800 s** wall-clock safety net intended to be non-binding. A step is one executor decision, one model call plus, usually, one tool execution.

The critical property is that a multi-agent architecture receives no extra steps. The 60-step budget is **carved** across its components, so total model reasoning steps stay at $\leq$60 regardless of how many agents the architecture contains. Table 4.1 gives the split for each family.

Table 4.1: How the 60-step budget is shared within each architecture family. No configuration gets more environment-acting steps than the baseline; read-only supervisory calls that only observe or route (the monitor's checks, the strategist's routing) are a separate layer and are not counted against the cap.

Group	Architecture	Allocation
G1	baseline, prompt, temp	one executor, the full 60
G2	explorer	executor receives 60 − explorer(5–10)
G2	planner	executor receives 60 − explorer − planner(5)
G2	analyst	executor receives 60 − analyst(10)
G2	monitor	one executor, the full 60; the monitor's calls are interleaved, not charged
G3	all four variants	phase-1 executor and verifier *share* the 60; the verifier engages inside the budget, never on top of it
G4	cooperation, debate	two agents alternate 7-step turns within the 60
G4	orchestra	specialists share the 60; the strategist's routing calls perform no environment work and are not charged

Every architecture therefore gives its executor the *same budget of environment actions* as the plain baseline; the one addition in some architectures is read-only supervisory calls (the Monitor's observations, the orchestra's routing) that take no action and sit in a separate layer. The comparison isolates *how the work is organised*, not *how much work is allowed*.

4.3.1 Why steps rather than wall-clock or tokens

The study's central comparison is across models of different capability, which requires a hardware-independent budget unit. Otherwise a model is rewarded or penalised for properties unrelated to its reasoning.

Wall-clock is confounded by inference speed. A model served on slower hardware, or a larger model, or a different quantization, produces fewer steps per second without being less capable. Budgeting by time would conflate capability with serving throughput. Wall-clock is therefore retained only as a safety net to terminate genuine hangs.

Tokens are hardware-independent but not comparable across models, because context windows differ by almost an order of magnitude in this study. A large-context model retains more history and therefore spends more tokens per step for reasons of physical capacity rather than effort; a token cap would systematically penalise it.

The reasoning step is both hardware-independent and interpretable. It counts decision points (how many opportunities to act the agent received) which is the quantity an architecture comparison should hold constant.

Why sixty, and was it enough?

A budget must be large enough not to truncate work the agent would have completed, and small enough that the experiment is affordable and that failures terminate. Sixty was chosen before the final grid was run; the completed data allows the choice to be assessed rather than merely asserted.

The distribution of steps used by *solved* runs is strongly concentrated at the low end:

Solved runs ($n = 1{,}611$)	Steps used
Median	14
75th percentile	25
90th percentile	44
95th percentile	55
99th percentile	60

Half of all solutions are reached within 14 steps and nine in ten within 44. Against this, failed runs use a median of 50 steps, and 30% of them are still going when the budget expires. This is the *solve-fast-or-fail-slowly* pattern that justifies a cap at all: past a certain point, additional steps overwhelmingly extend unproductive runs rather than complete productive ones.

Was 60 enough? Mostly, but not entirely, and the answer is more interesting than a clean yes. Only **6.8% of solved runs required more than 50 steps**, so a 50-step cap would have cost 110 of 1,611 solutions. But 58 runs (3.6% of all solves) succeeded at *exactly* step 60, which means they finished on the last decision available to them. A budget under which some runs succeed on the final step is a budget that is truncating a tail, and a larger one would have recovered a few more solutions.

The relevant question for this study is not whether 60 is optimal but whether it *biases the comparison*, and it does not: the cap is identical for every architecture and every model, and the truncated tail is small and distributed across configurations. What it does mean is that all reported solve rates are mild underestimates in absolute terms, a point restated in Chapter 8, since it bears on the G3 family in particular, whose verification phase competes for the same tail of the budget.

Sixty in the context of published practice

Published agent evaluations use step limits spanning an order of magnitude, and the spread is explained by what each study is optimising for rather than by disagreement about how many steps an agent needs.

Setting	Limit	Optimising for
SWE-agent [36], cost-conservative guidance	50 turns	cost control
This study	**60 steps**	**grid feasibility**
Some TerminalBench harness configurations	110 steps	single-run performance
TerminalBench default harness	200 turns	single-run performance
mini-SWE-agent [78] default	250 steps	single-run performance

Two observations reconcile these. First, the high limits belong to evaluations whose primary stopping signal is *cost* rather than steps: SWE-agent's documentation is explicit that "without limiting cost, the average cost will also converge to infinity, as the agent will never stop iterating", and sets the step limit high precisely because cost is doing the real work. A study that must run thousands of configurations cannot adopt that design, because an open-ended budget makes the grid unaffordable and, more importantly, makes architectures incomparable: an architecture that induces longer runs would consume more compute by construction.

Second, and decisively, **the limits are far above what successful trajectories actually use**. SWE-agent with GPT-4 reports a median of 12 turns on solved instances with 75% completing within 18, under a cost budget rather than a step cap (the 250-step figure above is mini-SWE-agent's default, not SWE-agent's). Our own distribution is close (median 14, 75th percentile 25) despite a different benchmark, different models and a different harness. The convergence is reassuring: it suggests roughly a dozen decisions is what solving a task of this kind takes, and that both studies' caps are generous relative to it. Sixty was therefore a defensible choice rather than an optimal one, and defensible in the direction that matters for this study: generous enough not to truncate the work being compared, tight enough that failures terminate and the grid stays affordable.

4.3.2 Tokens as a monitored quantity

Tokens are deliberately *not* a budget. Making them one would reintroduce the context-window confound above. They are instead recorded and reported as an analysis dimension: tokens per solved task is the honest cost axis (Section 6.7.3), and the share of tokens spent on coordination rather than execution quantifies architectural overhead. In short, *steps bound the experiment; tokens illuminate it.*

4.3.3 The "it just did more work" objection

The standard criticism of multi-agent results is that reported gains reflect additional model calls rather than better organisation. It is not a niche objection: the test-time-compute literature has made spending itself an axis of comparison, showing that accuracy on many tasks rises with the number of samples or agents drawn [31], [70], [74], and that allocating inference compute well can substitute for parameters outright [30], so that any method reporting a win without fixing its budget may be reporting the budget. The convention that followed (plot accuracy against tokens or calls rather than quoting a raw win rate) is the standard this design is built to meet in advance. The carved budget forecloses the naive form of the objection by construction: the configurations cannot win by attempting more, because they are not permitted to.

A subtler version survives the design and must be settled empirically. If an architecture causes the agent to consume more of a budget it already had, and Section 6.1.2 shows they do, from 0.62× to 2.91× the baseline's step usage, is the benefit simply the extra work? That question is answered with data rather than design in Section 6.1.2, and the answer is no: step consumption does not predict benefit within any model, the leanest architectures are among the most effective, and successful runs are systematically shorter than failed ones.

4.3.4 Residual caveats

Two properties of the budget are not equalised, and one is, exactly.

The **wall-clock net is non-binding in practice**: measured per-step time is a few seconds across all models and serving stacks, so a full 60 steps completes well inside 1800 s, and **0.5% of traced runs end on the clock**. The few that do are environment-setup tasks where a *single* command runs for minutes inside local Docker: the trigger is the task, not the model's token speed, though in practice these endings fall mostly on Devstral-24B (and to a lesser degree Qwen-72B), the models that push such tasks furthest before stopping.

Context windows are not equalised, for the reasons given in Section 4.1.3.

The step ceiling holds without exception. No run in the grid consumed more than 60 steps, so no configuration was ever permitted more agency than the baseline; the measurement, and what became of the tighter caps used in earlier phases, are given in Section 5.8. Everything to this point is held constant. The 14 architectures defined in the sections that follow are what varies, and they are configuration variants of the substrate just described rather than fourteen independent implementations, which is what allows the comparison to attribute differences to the architecture rather than to incidental engineering.

They are grouped by the question each answers: **G1** isolates single-agent factors, **G2** adds pre-execution context, **G3** adds post-execution verification, and **G4** adds multi-agent collaboration. Each is presented with three things: the mechanism as implemented, the failure mode it targets, and how its share of the 60-step budget is carved out of the total fixed in Section 4.3. Section 4.4 defines the failure modes first, since the architectures are best understood as responses to them.

4.4 The failure modes the architectures target

The failure taxonomy is derived empirically from the baseline runs of all 7 models, not postulated in advance. Two findings from that derivation shape the whole design space. The dominant failure is not running out of budget but submitting a wrong answer. And models fail in characteristically different ways, a difference that predicts how much any architecture can help them.

Read from those early baseline runs, the recurring failures fall into a few coarse kinds, listed in Table 4.2 by frequency. This is the design-time view, and it is kept coarse on purpose: it is what one can see in a handful of runs, and it is enough to say what each architecture is built against. The dominant kind, a wrong submission, is uninformative at this resolution; Section 6.3.2 returns to it with the full results and

decomposes it into four mechanical shapes read directly from the traces. The taxonomy here and the measured one there are the same phenomena seen at two resolutions: the coarse, motivating view first, the precise, measured view once the experiments are in.

Table 4.2: The recurring baseline failure modes, in order of observed frequency across the baseline runs of all 7 models. The first is the design target of the verification family, the second of the loop-breaking architectures; weak grounding is a frequent *cause* of the first rather than a separate way for a run to end.

Failure mode	**What it looks like**
Wrong submission	The agent stops and hands in an answer that fails the tests. The single largest category, in every model, and the one that granting more steps would not touch.
Loops / stagnation	High-effort cycles that never converge; the budget is consumed without progress.
Context exhaustion	The token window is reached before convergence, concentrated in small-context models. Later found confined to a pre-fix harness epoch (Section 8.2).
Weak grounding	The agent acts before inspecting the input, a frequent *cause* of a wrong submission rather than a distinct way for a run to end. Motivates the pre-execution family.

Two rarer endings complete the picture and motivate no architecture of their own: wall-clock exhaustion, at 0.5% of runs and concentrated on the slow-grinding models (chiefly Devstral-24B, with a smaller share on Qwen-72B), and (for the multi-agent family alone) a failure to coordinate, which cannot occur without a second agent. A third design concern, tasks structurally too large for a single flat loop, is a property of the task rather than a failure signature of a run; it motivates the decomposition and routing architectures (G2-Planner, G4-Orchestrator), whose behavioural symptom, when they fail, is stagnation.

The classification is applied uniformly: the harness emits nine distinct termination strings, mapped once, in the dataset build rather than per analysis, onto behavioural classes, so no two sections of this thesis can classify a run differently. Section 6.3.1 reports the measured distribution per model, and it is that distribution, rather than the taxonomy itself, that predicts which architecture helps which model.

4.5 Summary of the fourteen

Table 4.3 states each architecture's mechanism as implemented, the failure mode it targets, and the published work it derives from. The sections after it give each family in full; Appendix A.2 records the exact structured artifact each sub-agent must emit and the fallback taken when it does not.

From failure modes to task-kinds. The design targets failure modes, not TerminalBench's task categories: the architectures are general patterns, not tuned to this benchmark. But a failure mode is provoked by a recognisable kind of task, so the "Targets" column above implies, for each family, the task-kind it should suit. Grounding designs (G2) should help where the answer must be read from data or the environment before acting; verification designs (G3, and the relay of G4) where a task has an exact deliverable a run can declare done prematurely; loop-breakers (G2-Monitor, the dialogue pair) where search stalls or repeats; and decomposition and routing (G2-Planner, G4-Orchestrator) where the task is multi-stage. This is an a-priori expectation, fixed here by the design. Section 6.5.2 tests whether each family in fact helped the task-kinds it implies, exactly as Section 6.3.4 tests whether each repaired the failure it targets.

4.5.1 What the fourteen leave out

Fourteen architectures is a large grid but not an exhaustive one, and the omissions were deliberate. Three classes of design were considered and excluded.

Designs that duplicate an included mechanism. Tree-of-Thought [20] and its graph-structured successor [21] search over branching candidate continuations, which multiplies the number of model calls by the branching factor, so the one thing a carved budget cannot accommodate, and in any case a mechanism G4-Debate already probes in its cheapest form. Parallel sampling with majority voting is likewise G4-Debate minus the critique step, and its interesting property (does disagreement help, or merely repetition?) is what G4-Debate isolates directly. Self-refinement [59] is not excluded but *split*: G3-Instruction, G3-Reflexion and G3-Judge are three points on its axis, differing in how much of the first attempt the second look inherits.

Table 4.3: The 14 architectures: mechanism, targeted failure mode, and canonical source.

Architecture	**Group**	**Mechanism (as implemented)**	**Targets**	**Lineage**
G1-BASE	G1	Plain ReAct executor, minimal prompt, full 60 steps. The control.	—	[11]
G1-PROMPT	G1	Baseline plus three prompt directives: inspect data, write to files, verify before submit.	grounding, submission	[79]
G1-TEMP	G1	Baseline at sampling temperature 0.2 instead of greedy.	loops	—
G2-EXPLORER	G2	Read-only Explorer inspects the task directory first and emits a structured Data Summary injected into the executor.	grounding	[11]
G2-PLANNER	G2	Explorer → think-only Planner decomposes into 1–4 subtasks → per-subtask executor.	loops (complexity)	[54]
G2-ANALYST	G2	Explorer and Planner merged into one read-only pass emitting a grounded plan.	grounding, submission	[68]
G2-MONITOR	G2	Every 10 steps a tool-less Monitor sees the full history and, if off-track, injects guidance as the executor's own `think` step.	loops	[58]
G3-INSTRUCTION	G3	On submit, a fresh agent re-verifies from instruction and environment state only.	submission	[59]
G3-REFLEXION	G3	On submit, an LLM writes a reflexion on phase 1; a fresh agent then verifies with it.	submission	[18]
G3-JUDGE	G3	On submit, a Judge approves or, on NEEDS-FIX, injects feedback into a clone of the original memory, which resumes.	submission	[65]
G3-GATE	G3	Submit is intercepted N times; the agent must self-verify before the final submit passes.	submission	—
G4-COOPERATION	G4	Two equal agents relay, 7 steps per turn, on the same terminal.	loops	[58], [70]
G4-DEBATE	G4	Same round-robin, but agent B critiques A's approach and tries a different strategy.	loops	[69], [80]
G4-ORCHESTRATOR	G4	A tool-less strategist routes to persistent specialist sub-agents; routing steps are free, specialists share 60.	loops (complexity)	[67]

Designs whose premise does not hold in this setting. Retrieval-augmented planning [81] retrieves similar solved problems to guide the plan; there is no corpus of solved TerminalBench trajectories to retrieve from, and constructing one from our own runs would leak outcomes into the treatment. Long-horizon memory architectures [28] assume episodes long enough for memory to pay off; a terminal task lasting 60 steps within one container has effectively no cross-episode horizon, and persistent note-taking is in any case already available to every agent through `write_file`.

Designs the budget cannot accommodate. Full hierarchical decomposition with four or more specialist executor roles is a natural extension of G4-Orchestrator, but under a carved 60-step budget each specialist would receive on the order of a dozen steps, and a poor result would be uninterpretable, since it would not distinguish a bad architecture from an under-resourced one. G4-Orchestrator with three specialists is already near that boundary, which Section 6.5.2 treats as a live concern rather than a settled one.

What remains is a grid organised by *when* the intervention happens relative to execution (before, during, after, or instead of a single agent) which is the distinction the results turn out to reward. That organising principle was chosen in advance and is defended in Section 6.5.4, where the pre/post split proves to be the axis along which architectures actually separate.

4.6 G1 — single-agent ablations

G1 isolates individual factors on the bare executor without adding agents. Its role is to establish how much can be bought without architecture, so the G2–G4 results are read against a fair floor rather than a deliberately weak control. This matters: Section 6.2.4 finds G1-Prompt to be the best architecture for one of the seven models, and the most frequent winner of all thirteen once the grid is read at the level of model–category cells.

G1-Base — the control. The plain ReAct executor of Section 4.1 [11], with a minimal, non-prescriptive system prompt and the full 60-step budget. The prompt describes the tools and the task and says nothing about method. This deliberate minimalism is what makes it a fair reference: a baseline that already contained good advice would understate every architecture measured against it.

G1-Prompt — the cheap floor. Instruction-level steering in the lineage of chain-of-thought prompting [79]: the identical loop with three directives added to the system prompt, to *inspect the input data before writing code*, *write solutions to files rather than fragile inline commands*, and *verify before submitting*. No structural change, no extra model call, no budget consumed, just a few lines of text.

Each directive targets one of the failure modes catalogued in Section 4.4: acting without grounding, losing work to shell quoting, and a premature wrong submission. The architecture measures how much of the benefit attributed to elaborate scaffolding is available from instruction alone. The extreme instance is `log-summary`, a task whose demand its three directives happen to describe exactly: G1-Prompt reaches 7/7 models where the far more elaborate G2-Analyst falls to 2/7.

G1-Temp — the stochasticity ablation. The baseline in every respect except sampling temperature 0.2 rather than greedy decoding. It tests a specific hypothesis: that some failures are *deterministic dead ends*, where a model re-derives the same wrong action because nothing in its state has changed, and that a small amount of randomness suffices to escape them.

The architecture is included because it is the cheapest possible intervention against loops, and because it provides a control for a confound the other architectures share. Every G2–G4 architecture changes the executor's context, and a changed context changes the sampled trajectory. G1-Temp changes the trajectory *without* changing the context, which isolates that component. Section 6.2.4 reports the one result that makes this worthwhile: on Llama-70B it gains +4 *without lengthening the run at all*, which no information-adding architecture can explain.

4.7 G2 — pre-execution context augmentation

G2 adds a read-only phase, or a runtime observer, that feeds structured context to the executor. All G2 budgets are carved: the sub-agent's steps are subtracted from the executor's 60. Figure 4.7.1 shows the three topologies side by side.

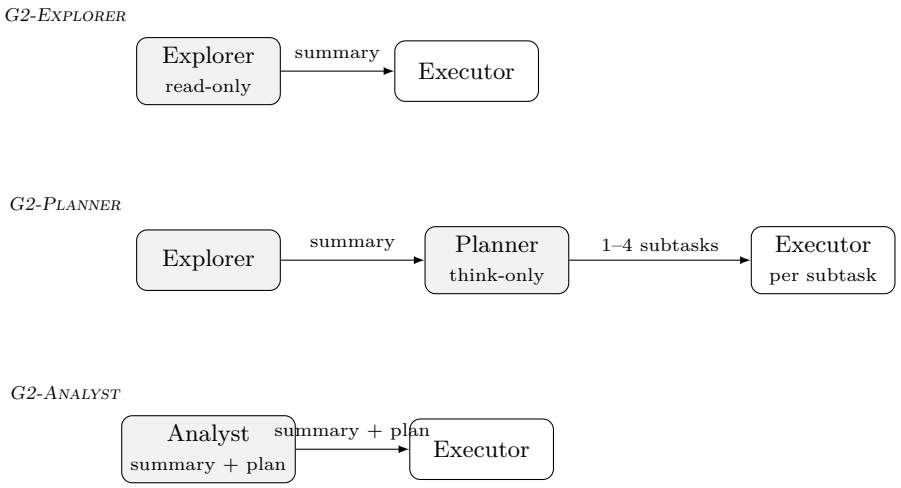


Figure 4.7.1: The three pre-execution topologies. Shaded boxes cannot modify the environment. Each sub-agent's steps are subtracted from the executor's 60, so the three differ in *how much executor budget they trade for context*, not in how much total work is permitted.

G2-Explorer — grounding as a guarantee. The observation-before-action discipline that ReAct [11] asks of a single agent, promoted into a separate phase. A read-only Explorer runs first, restricted to `bash`, `read_file`, and `think`, with the restriction enforced at two levels: only those tool schemas are exposed, and the executor additionally rejects any non-whitelisted command. The double enforcement exists because a single-level restriction is only a prompt request, and the architecture's claim is that grounding becomes an architectural guarantee rather than something the model may choose to do.

The Explorer inspects the task directory and emits a structured Data Summary in JSON, which is injected as context into the main executor. Explorer failure is deliberately *non-fatal*: a malformed summary leaves the executor running, merely ungrounded.

One implementation constraint bounds what this architecture can achieve, and it belongs here rather than as a later surprise. To stop the Explorer from spending the executor's budget on its own reading, its prompt forbids reading any file longer than 80 lines in full; longer files are sampled as the first 30 lines and the last 15. The cap protects the budget, but on a file of any real size **the Explorer never sees the middle**. For an architecture whose purpose is data grounding that is a self-imposed ceiling: a schema change, an anomalous row, or a format shift past line 30 is invisible to it, and the executor inherits a summary that is confidently incomplete. Where G2-Explorer underperforms in Chapter 6, this is one mechanism among the others, and it is a fault of ours, not of the model. An adaptive reading strategy, sampling by structure rather than by line count, is the obvious remedy and is left to Chapter 9. The same decision makes the architecture safe to deploy and makes Section 6.4.2 measurable, since a model that never produces a valid summary degrades gracefully toward the baseline rather than crashing.

G2-Planner — explicit decomposition. The planner–executor separation of MetaGPT [54] and the as-needed decomposition of ADaPT [55], reduced to their common core: Explorer, then a Planner holding only the `think` tool. The Planner cannot run commands, write files, or submit; it can only reason. It decomposes the task into 1–4 sequential subtasks as a JSON plan, each with a description and a success criterion, after which a per-subtask executor loop works through them.

Two design choices bound the risk. The subtask count is capped at four, since a plan with more parts spends more of the executor's budget describing work than doing it. And an invalid plan falls back to single-subtask mode, exactly G2-Explorer behaviour, so the architecture cannot regress below its predecessor by construction. Section 6.6.3 uses the recorded plan artifact to ask what distinguishes a good plan from a poor one.

G2-Analyst — grounding and planning in one pass. Closest to the designer role of ChatDev [68], where one agent produces the specification the others build against. The Analyst merges Explorer and Planner into a single read-only phase producing both a Data Summary and a data-grounded execution plan, on the motivation that a plan written by the agent that inspected the files is grounded by

construction, where a plan written from a summary is grounded only if the summary was good. It asks the sub-agent for more in one structured artifact than any other G2 design.

The architecture therefore packs the most into a single structured pass of any G2 design, and Section 6.6.3 reports that this is also its weakness: it is the one role where a stronger supervisor helps nothing at all, because demanding two artifacts in one structured pass makes schema failure more likely.

G2-Monitor — intervention during execution. The executor runs normally, but every 10 steps a tool-less Monitor is invoked with the *full* cumulative action history. If the run is on track it returns a sentinel and nothing is injected, so the mechanism costs nothing when it is not needed.

When the run is off track, the delivery mechanism is the architecture's distinguishing feature. Guidance is injected as the executor's own `think` call and tool result rather than a user message, inception prompting in the sense of CAMEL [58], so the executor meets the advice as its own prior reasoning rather than an instruction from outside. The Monitor also tracks whether earlier advice was followed and escalates its authority when it is repeatedly ignored.

Section 6.6.3 shows this delivery detail is not cosmetic. Comparing effective against ineffective monitors, the difference is not what they observe but how they address the agent: the ineffective ones almost always write about "the agent" in the third person, barely ever using "you", while the effective ones almost always address it directly.

Whether these sub-agents *function* on a given model is a prerequisite for interpreting any G2 result, and Section 6.4.2 shows it cannot be assumed: the rate at which a model produces a schema-valid plan ranges from 0% to 98% across the seven, and does not track task capability. Where it fails, a recorded G2 regression measures an architecture that never ran as designed rather than one that ran and underperformed.

4.8 G3 — post-execution verification

The G3 family shares one submit-intercept codebase with four variants. A phase-1 agent runs with the budget; when it calls `submit`, a gate intercepts and hands control to variant-specific verification, which uses the *remaining* budget.

One constraint shapes the entire family: the grader's test suite is not present in the container during the run, since the harness copies tests in afterwards. All verification is therefore agent-driven, never test-driven. The four variants target the same failure mode, a wrong submission, and form a progression along one dimension, *how much of the first attempt the second look inherits*, which is what lets the comparison isolate the value of context continuity. Figure 4.8.1 orders the four variants along it.

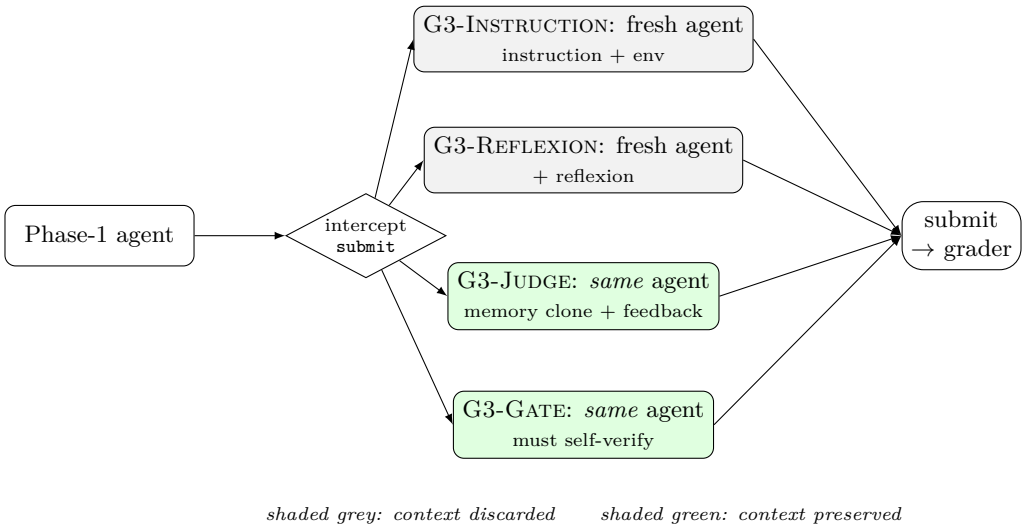


Figure 4.8.1: The four post-execution variants, ordered by *how much of the first attempt the second look inherits*. All four target the same failure mode and all four spend only the budget the phase-1 agent left behind; they differ on one axis. Section 6.4.3 finds that axis (context continuity) to be what separates them.

G3-Instruction — the clean slate. The iterative-refinement loop of Self-Refine [59] stripped of its memory: a fresh agent receives only the task instruction and the current environment state, with no phase-1 trace, and must re-derive the situation from what it can observe. The hypothesis is that a second attempt unbiased by the first trajectory sees the gap the first missed, the first attempt's reasoning being what led it astray.

G3-Reflexion — the clean slate plus a summary of what went wrong. A direct implementation of Reflexion [18]. An LLM first writes a reflexion on what phase 1 did and where it likely fell short; a fresh agent then receives the instruction, that reflexion, and the environment state. This adds the diagnosis of the first attempt without its full trace, testing whether a compressed account of the failure is more useful than either the raw trace or nothing.

G3-Judge — continuity with targeted feedback. An LLM-as-judge [65] placed in the loop rather than used for offline evaluation. A Judge evaluates the work; on approval the submit passes, and on NEEDS-FIX the judge's feedback is injected into a clone of the original agent's memory and that same agent continues, not a fresh instance. This is the only G3 variant preserving full context continuity: the agent that resumes knows what it already tried.

One implementation detail affects interpretation: the verdict parser errs toward NEEDS-FIX and never converts a rejection into an approval. The architecture is therefore biased toward *more* verification, which means its measured effect is a lower bound on what a neutral parser would yield, and, as Section 6.6.4 shows, that bias is sometimes the source of its benefit.

G3-Gate — forced self-verification, no second agent. The one G3 variant with no direct antecedent in the literature, the minimal ablation that separates obligation from information. The submit is intercepted N times, and before the final submit passes the agent must explicitly verify its own work. There is no new model instance and no external judge: the same agent, with its full context, must check before it is permitted to finish.

This is the only G3 variant that adds no information at all, only an obligation. That makes it the cleanest test of whether premature submission is a failure of knowledge or of discipline, and Section 6.5.4 finds that on tasks demanding an exact output file it modestly outperforms the variants that add a second opinion on partial credit, a margin that shrinks once a couple of tasks are set aside, and that reverses in binary solves, where the judge leads.

4.9 G4 — multi-agent collaboration

G4 places multiple executor-class agents on the *same* Docker terminal. They share a real filesystem rather than only exchanging messages, so one agent's edits are directly visible to another. Budgets are carved so that total specialist work stays at 60 steps. Figure 4.9.1 gives the two topologies.

G4-COOPERATION and G4-DEBATE are identical except for one instruction, which makes their difference interpretable: whatever separates them is the effect of critique rather than of multi-agency.

G4-Cooperation — relay. Role-playing collaboration in the manner of CAMEL [58] and Mixture-of-Agents [70], on a shared terminal rather than a shared transcript. Two equal agents take turns on the same task, 7 steps per turn, each handing off with a compressed summary of what it did. Agent B continues A's work. The hypothesis is minimal: that fresh context alone breaks stagnation, since an agent inheriting a summary rather than a full trace is not anchored to the approach that was failing.

G4-Debate — adversarial relay. Multi-agent debate [69], [80], whose premise is that a single model left alone converges on its first framing. The same round-robin, but agent B is instructed to critique A's approach and then try a deliberately different strategy, testing whether forced disagreement improves strategy selection over a plain handoff.

It is also the most format-demanding architecture in the study, its prompt conveying both the task and an adversarial stance while still eliciting well-formed tool calls. Section 6.4.2 shows this cost is real on models with weak structured-output compliance, and Section 6.5.4 shows the compensating benefit, since it is one of the two strongest architectures on tasks that require proposing and revising candidates.

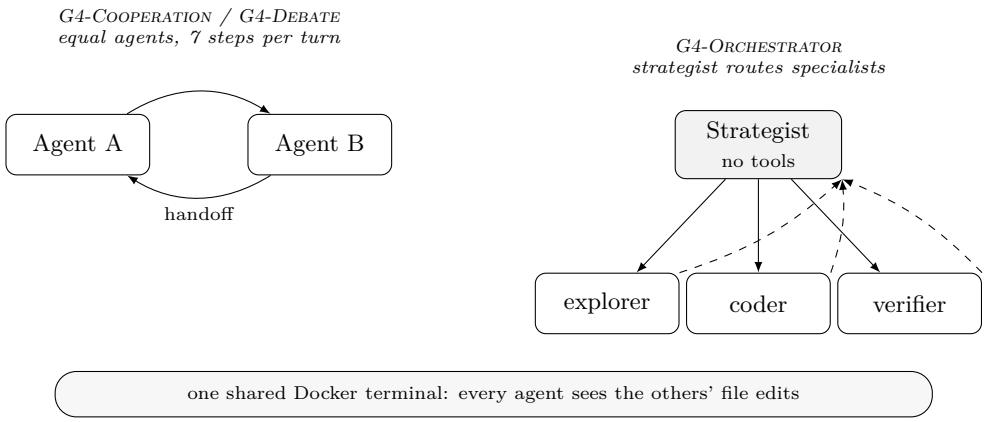


Figure 4.9.1: The two multi-agent topologies. G4-Cooperation and G4-Debate share this diagram and differ by a single instruction: whether B continues A's work or challenges it, which is what makes their difference attributable to *critique* rather than to multi-agency. In G4-Orchestrator the strategist is the sole information bridge (dashed returns), and its routing steps are free while the specialists share the same 60.

G4-Orchestrator — runtime routing to persistent specialists. Dynamic routing in the manner of AutoGen's group chat [67] and the manager role of ChatDev [68], following the orchestrator pattern described by Austin [71]. A tool-less, stateless Strategist decides at each cycle which specialist to deploy, `explorer` (read-only), `coder` (write and patch), or `verifier` (test, fix, submit), and writes it a detailed instruction.

Three design decisions distinguish it from the dialogue architectures. The specialists are **persistent**: each keeps its own memory across re-invocations, so the coder accumulates knowledge of the code it has written while the explorer accumulates knowledge of the environment. The strategist is **stateless**, receiving all relevant state explicitly on each routing call, which prevents it from developing its own drifting picture of the task. And the strategist is **tool-less**: it cannot act on the environment, only decide who does.

The budget is carved as one free routing step plus up to 6 specialist steps per cycle, totalling 60 specialist steps, with a cap of 20 cycles as a safety net. The routing step is free because it performs no work in the environment, a decision defended in Section 4.3; a side effect, noted in Section 6.6.6, is that the strategist's own token cost goes unrecorded.

Because the strategist's decisions are recorded as structured routing calls, this is the one architecture whose policy can be read directly from the traces, which Section 6.6.2 exploits.

4.10 Fairness across all fourteen

The augmentation always comes *out of* the same 60-step budget, never on top: explorer and planner steps are subtracted from the executor's; G3 verification uses the remaining post-submit budget; G4 turns and orchestra cycles are carved from the same 60 rather than added to it. The dialogue schedule partitions the budget into fixed turns, so G4-Debate (eight turns of seven steps) tops out at 56 while G4-Cooperation and the orchestra reach the full 60. No architecture's executor receives *more* environment actions than the baseline (the read-only supervisory calls some add are a separate layer), so the comparison isolates how the work is organised rather than how much work is permitted. Figure 4.10.1 draws the carved and additive alternatives to scale, with G2-Planner as the worked example.

4.10.1 The alternative we rejected

Carving is not the only defensible choice, and the alternative has a real argument behind it. One could equalise *computational cost* instead, fixing total tokens and wall-clock and letting each architecture spend as many steps as it likes inside that envelope. The case for it is that a step is not a constant unit of work: a planner step emits a short JSON plan and touches nothing, while an executor step runs a command, waits for a container, and ingests potentially thousands of tokens of output. Charging both at one step overcharges the planner. Under a token-equalised design the scaffolded architectures would run perhaps

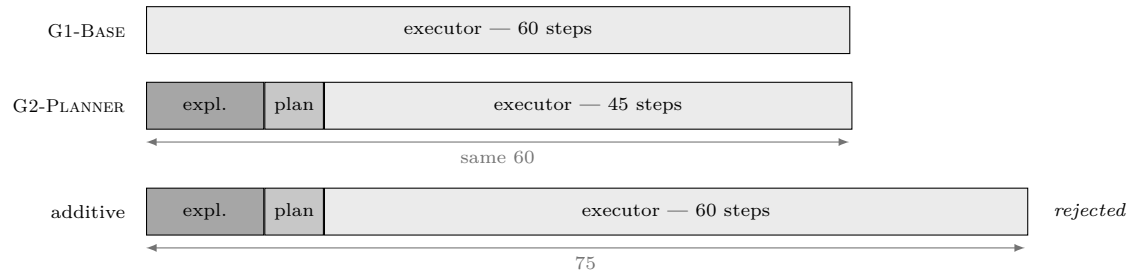


Figure 4.10.1: Carved versus additive budgets, drawn to scale, with G2-Planner as the worked example. The sub-agent caps are flat across models: the Explorer may use at most ten steps and the Planner at most five, so in this heaviest pre-execution case the executor is left 45. What the phases actually consume varies, but the total does not: under the carved design a scaffold's sub-agents are paid for out of the executor's allocation, and no run in the grid exceeded 60 reasoning opportunities. Under the additive alternative the scaffold would receive more total agency than the control, and any improvement it showed would be confounded with the extra budget. The cost of the carved choice is stated in Section 4.10: it cannot separate an architecture that is badly conceived from one that is merely expensive.

60–70 steps for the same money, and the comparison would be about cost-effectiveness rather than about organisation.

We rejected it for three reasons, in increasing order of importance.

First, it makes the treatment endogenous. If each architecture chooses its own step count to fill a token budget, the step count becomes an outcome of the architecture's behaviour rather than a constant of the design, and a regression can no longer be attributed cleanly, since a scaffold that happened to run 68 steps and a scaffold that happened to run 52 are not two settings of one variable.

Second, the token envelope is not actually neutral either. Token consumption is dominated by observation size, which is a property of the *task* and of how the agent chooses to inspect it, so equalising tokens silently penalises architectures that read files carefully, which is the behaviour G2 exists to induce.

Third and decisively, the carved design is the only one that answers this thesis's question. The question is not "does scaffolding buy more than it costs?" but "does organising the same amount of agency differently make it more effective?" A scaffold given extra steps that then outperforms the baseline has demonstrated that more steps help, which we already knew. A scaffold that must pay for its planner out of its executor's budget, and still wins, has demonstrated something about structure. This is what makes the negative results in Chapter 6 interpretable: when a G2 architecture underperforms, the finding is that its planning was not worth the executor steps it consumed, a statement about the exchange rate between deliberation and action, which is the quantity of interest.

The cost of the choice is stated plainly: this design cannot distinguish an architecture that is badly conceived from one that is merely expensive. An architecture whose planning is worth *something* but less than the four executor steps it costs will look like a failure here and might succeed under an additive budget. Section 6.1.2 reads the budget accounting with this in mind, and reports how much of each architecture's allocation is actually consumed, which for several of them turns out to be the substantive part of the story.

4.11 The heterogeneous supervision track

Everything defined so far is *homogeneous*: in all fourteen architectures every component (executor, explorer, planner, monitor, judge, strategist) is the same underlying model. This last section introduces a track that breaks that assumption. A *strong* model occupies the supervisory role while a *weaker* model continues to do all the work in the environment. It is the only place in the study where a configuration differs in *who* runs a role rather than in *which* roles exist.

4.11.1 Why separate the roles

Chapter 6 will show that architectural scaffolding helps weak models and burdens strong ones. That result invites an obvious question, which the homogeneous grid cannot answer: when a scaffold helps a

weak model, *what* is it supplying? Two hypotheses are compatible with the homogeneous data.

1. **Structure.** The scaffold imposes an organisation (explore first, then plan, then execute, then verify), and the benefit comes from the organisation itself, independent of who produces it. Under this hypothesis, a weak model producing its own mediocre plan still gains, because having *a* plan beats having none.
2. **Capability.** The benefit comes from the *quality* of the supervisory output (the plan, the critique, the routing decision). Under this hypothesis a weak model supervising itself gains little, because it produces a weak plan, and the gain should grow substantially if a stronger model supplies that output.

Separating the supervisor from the worker discriminates between them. If capability at the coordination layer is what matters, replacing only the supervisor should improve results while leaving execution capability untouched. This is the thesis's answer to RQ5, and it is the axis on which the study is most clearly original: the literature evaluates homogeneous multi-agent systems almost exclusively.

4.11.2 Design

The harness routes model calls per *role* rather than per run. A configuration names a worker model and, optionally, a supervisor model; every component that performs work in the environment uses the worker, and every component that only reasons *about* the work uses the supervisor. When no supervisor is named, the two are the same object and the configuration is exactly the homogeneous architecture of Section 4.4, so the homogeneous grid is the strict special case, not a separate code path. Figure 4.11.1 shows the split.

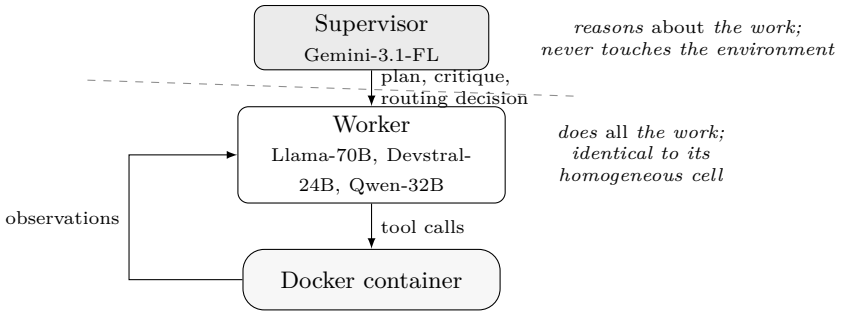


Figure 4.11.1: The heterogeneous supervision split. Routing is per *role*, not per run: every component that acts in the environment uses the worker model, every component that only reasons about the work uses the supervisor. Everything below the dashed line (model, budget, tools, prompt, task set) is identical to the corresponding homogeneous cell, so the only variable is which model answers the supervisory call.

Table 4.4: Role assignment in the heterogeneous track. The worker always executes; the supervisor never touches the environment.

Architecture	Supervisor role	What the supervisor contributes
G2-Explorer	Explorer	Reads the task directory and writes the Data Summary.
G2-Planner	Planner	Decomposes the task into subtasks (think-only; no side effects).
G2-Analyst	Analyst	Produces Data Summary and grounded plan in one read-only pass.
G2-Monitor	Monitor	Observes the run every 10 steps and decides whether to intervene.
G3-Judge	Judge	Evaluates the submitted work; approves or writes corrective feedback.
G4-Orchestrator	Strategist	Routes at runtime to specialist sub-agents and instructs them.

Table 4.4 names the role the supervisor fills in each case. Six of the fourteen architectures have a supervisory role that is separable in this way. The other eight either have no distinct supervisor (G1-

BASE, G1-PROMPT, G1-TEMP), or their second agent is an *executor* rather than a supervisor and so must remain the worker (G4-COOPERATION, G4-DEBATE), or their verification is structurally a continuation of the worker itself (G3-INSTRUCTION, G3-REFLEXION, G3-GATE).

The supervisor is Gemini-3.1-FL throughout, and the workers are the three models that span the capability range of the open grid: Llama-70B (weak), Devstral-24B (mid), and Qwen-32B (strongest open coder). This yields $3 \times 6 = 18$ heterogeneous cells.

What is held constant

The worker model, its budget, its tools, its prompt, and the task set are identical to the corresponding homogeneous cell. The *only* change is which model answers the supervisory call. Consequently the comparison

$$\underbrace{\text{supervisor} = Gemini - 3.1 - FL,\ \text{worker} = M}_{\text{heterogeneous}} \quad \text{versus} \quad \underbrace{\text{supervisor} = M,\ \text{worker} = M}_{\text{homogeneous}}$$

isolates supervisory capability with everything else fixed.

What this design cannot show

Two limits should be stated before the results. First, the supervisor is a *hosted* model while the workers are locally served and quantized, so supervisory capability is confounded with serving stack; the confound is unavoidable given the available hardware and is stated as such. Second, the design measures whether a *stronger* supervisor helps, not whether supervision *per se* helps, the latter being what the homogeneous G2/G3/G4 comparison already answers.

4.11.3 Interpreting the heterogeneous cells

A note on comparison: every heterogeneous cell is read against its identical homogeneous counterpart, so the two differ only in which model plays the supervisory role. The comparison is paired and same-task (Section 6.6); with the completed grid both tracks cover all 64 tasks in every cell, so the paired contrast and a raw one coincide, and the pairing is kept as a safeguard rather than because coverage diverges.

Section 6.6 then answers the three substantive questions, namely whether a strong supervisor beats the homogeneous agent and on which architectures, what it contributes in each of the six roles as read from the recorded artifacts, and what the heterogeneity costs in tokens relative to what it buys.

Chapter 5

Experimental Methodology

5.1 The experimental grid

The study is a full factorial of 7 models × 14 architectures over 64 tasks, plus a heterogeneous supervision track of 3 workers × 6 architectures over the same tasks. The grid contains 116 model–architecture cells (the $7 \times 14 = 98$ homogeneous cells plus the $3 \times 6 = 18$ supervision cells) and 7424 model–task rows, of which 7424 were executed (100.0%; see Section 5.4 for the remainder). Two derived counts recur later and are fixed here so they are never confused: the paired significance tests of Section 5.7 compare each non-baseline architecture against its own model's baseline, which is $7 \times 13 = 91$ homogeneous comparisons, while the 116 figure counts every cell including baselines and supervision.

5.2 Models

Seven models span the capability axis along two dimensions. The *within-family scale* axis compares Qwen2.5-Coder [82] at 7B, 14B, and 32B plus the 72B general model [83], holding training recipe approximately constant so that scale is the varying factor. The *cross-family* axis adds a Mistral code specialist tuned for coding-agent use [84], [85], a Meta general model [86], and a hosted closed-model reference, so that no conclusion rests on the idiosyncrasies of one family. Open code models form a well-populated lineage, Code Llama [87], StarCoder 2 [88], DeepSeek-Coder [89], from which this lineup selects the strongest member of each family that the available hardware can serve. Table 5.1 lists the seven, ordered by measured baseline capability rather than by parameter count, an ordering Section 6.2.1 shows the two do not share.

Table 5.1: The 7 models, ordered by measured baseline capability rather than by parameter count. Baseline is G1-Base solves out of 64.

Model	Family	Tuning	Role in the study	Base
Qwen2.5-Coder-7B	Qwen2.5	coder	scale ladder, floor	4
Llama-3.1-70B	Llama	general	cross-family, large but weak here	5
Qwen2.5-Coder-14B	Qwen2.5	coder	scale ladder	9
Devstral-Small-2-24B	Mistral	coder	cross-family code specialist	12
Qwen2.5-72B	Qwen2.5	general	scale ladder top; general, no coder variant	16
Qwen2.5-Coder-32B	Qwen2.5	coder	strongest open coder; anchor	19
Gemini-3.1-Flash-Lite	Gemini	general	hosted closed reference, grid ceiling	34

Two properties of this table are themselves findings and are developed in Section 6.2.1: capability does not track parameter count (the largest open models are not the most capable here), and the highest open baseline belongs to a 32B coder.

5.2.1 What is not in the lineup, and why

A lineup is defined as much by its exclusions, and the binding constraint throughout was that a model must serve on a single available GPU under vLLM with working prompt-mode tool calls. Four exclusions

are recorded here, because each names a different kind of limit.

- **Very large mixture-of-experts models** (DeepSeek-V3 class, 600B+ parameters) do not fit on the available hardware at any quantization the serving stack supports. This is a hard resource limit, and it means the study's open-weight ceiling is set by what a single accessible GPU can hold, which is the deployment setting the thesis is about, but does bound the capability range on the strong side. The hosted model is included partly to cover that end of the range.
- **Newer-generation frontier open models** available at the time were excluded for serving-stack reasons rather than capability ones: tool-calling support in the serving framework was unstable for them at the decision date, and an agent study whose tool parser intermittently fails measures the parser, not the model.
- **Base models without an instruction-tuned variant** are excluded by construction. The agent loop requires instruction following and structured tool emission; a base checkpoint cannot be evaluated on the same terms, and including one would confound tuning with family.
- **Reasoning-distilled models** were considered and dropped for scope. They would introduce a third axis, inference-time reasoning, which interacts with the step budget in a way the design does not control, and Section 9.4 names them as the most interesting single extension for that reason.

The general principle is that every open model here is the strongest member of its family that fits the constraint, and each represents a distinct training paradigm: code-specialised dense at three scales, a large general dense model, a mid-size cross-family code specialist, and a general model at the top of the scale ladder. That is a defensible spanning set for the capability axis, and it is not a claim to have sampled the model space uniformly.

5.2.2 Serving and quantization

All six open models are AWQ 4-bit quantized [90], one of the two standard post-training schemes, alongside GPTQ [91], and served with vLLM [92]; the hosted model is reached through its API using the same prompt-mode tool interface as the local models, so that tool-calling mechanics do not differ across the grid.

Quantization is a *uniform constraint* rather than a per-model choice, and the reason is hardware: the larger models in this study do not fit on the available GPUs at full precision, so the alternative to quantizing all of them is not quantizing none but running a smaller lineup. A uniform constraint is also the more defensible option, because it makes quantization a constant of the design rather than a variable confounded with model identity: had some models run quantized and others not, no comparison between them would be interpretable.

The consequence is that this study measures *deployable quantized artifacts*, not full-precision weights. That is the right object for its research question: these are the models one can actually run on accessible hardware, but it means absolute solve rates are not comparable with published full-precision numbers.

The concern this raises is documented rather than hypothetical: 4-bit quantization is close to lossless on many tasks but carries real accuracy risk on multi-step reasoning, where small perturbations accumulate along a trajectory [93], the regime an agent operates in. Whether it is instead responsible for the capability ordering that Chapter 6 reads everything against is an empirical question rather than a methodological one, and it is answered with data in Section 6.2.5.

The exact served artifacts are listed in Table 5.2 so the grid is reproducible. The six open models run under vLLM (image `vllm/vllm-openai:latest`) on an A100-80GB GPU at `gpu-memory-utilization 0.90`; the hosted model is reached through an OpenAI-compatible API with the same prompt-mode tool interface. Context windows are set per model to what fits on the available hardware, which is why Qwen-7B runs at half the window of every other local model.

5.2.3 Sampling

Temperature is 0 for every model and every architecture except G1-Temp, which exists to ablate that choice. Greedy decoding removes the controllable component of run-to-run variance; Section 5.5 discusses the component that remains.

Table 5.2: Served model artifacts: exact checkpoint, serving path, and context window, for reproducibility. All open models are AWQ 4-bit under vLLM; the hosted model is served over an OpenAI-compatible API.

Model	Checkpoint	Serving	Context
Qwen-7B	`Qwen/Qwen2.5-Coder-7B-Instruct-AWQ`	vLLM	14,000
Qwen-14B	`Qwen/Qwen2.5-Coder-14B-Instruct-AWQ`	vLLM	28,000
Qwen-32B	`Qwen/Qwen2.5-Coder-32B-Instruct-AWQ`	vLLM	28,000
Qwen-72B	`Qwen/Qwen2.5-72B-Instruct-AWQ`	vLLM	28,000
Devstral-24B	`cyankiwi/Devstral-Small-2-24B-Instruct-2512-AWQ-4bit`	vLLM	28,000
Llama-70B	`hugging-quants/Meta-Llama-3.1-70B-Instruct-AWQ-INT4`	vLLM	28,000
Gemini-3.1-FL	`gemini-3.1-flash-lite`	API	128,000

5.3 Dataset

The canonical task set is `core_64`: every TerminalBench task carrying the benchmark's own "easy" label, with nothing removed. Using the benchmark's external, reproducible label rather than a self-selected subset removes an obvious avenue for selection bias, and because the set is exhaustive the claim is literal: "we run all easy tasks", not "we run the tasks we chose". Nothing was dropped for being hard, for being unrepresentative, or for how any model performed on it.

5.3.1 Why only the easy band, and what that costs

Restricting to one difficulty band is the study's most consequential scoping decision, and it deserves a fuller defence than an assertion.

The argument for it. An architecture's effect can only be observed on tasks whose outcome can change. A task no model ever solves and a task every model solves both return a constant whatever architecture is applied, and Section 5.3.3 shows how much of even the easy band is already in that condition: 42 of the 64 tasks are solved by at most one of the 7 models. Adding a harder band would add tasks in which *no* model succeeds, enlarging the denominator of every cell without informing a single comparison.

What the evidence for it actually is, now measured. That these models fail almost entirely once the easy band is left was, in an earlier form of this study, an inference. It is now a measurement. We ran a pre-specified boundary probe: 4 models spanning the capability range (Llama-3.1-70B, Devstral, Qwen2.5-Coder-32B, and the hosted model), each at baseline only, on a random sample of 20 *medium* and 20 *hard* tasks drawn reproducibly from the benchmark's own difficulty labels, 160 runs in total. The sampling is described in Section 5.3.2, and the probe is kept outside `core_64` so it can never enter a grid solve rate.

The signal thins sharply the moment the easy band is left. Of 80 medium attempts, 8 succeed (10.0%); of 80 hard attempts, 1 succeeds (1.2%), the lone hard solve being `cancel-async-tasks`, by Devstral. Even the strongest model solves 5 of 20 medium tasks and 0 of the hard. This confirms the restriction rather than overturning it: adding a harder band would enlarge every cell's denominator with tasks almost nothing solves, exactly the constant-outcome inflation the easy restriction is meant to avoid. The indirect support points the same way: on the easy band our models solve between 6.2% and 53.1% of tasks, and the benchmark's own successor release reports frontier models and agents scoring below 65% on its harder set [77].

What it costs. Two things, and the second is the more serious.

The narrow one is external validity: nothing here speaks directly to how architecture behaves on harder work, and the results describe, as stated, the easy band of one benchmark.

The deeper cost is that the study's central mechanism predicts the restriction matters, and the probe brackets that prediction without settling it. If architecture pays in proportion to the gap between a model's first attempt and what it could reach with revision (Chapter 7), then strong models on harder tasks should recover a gap they do not have here, and the monotone decline reported in Section 6.2.2 would become a curve with an interior optimum. The boundary probe does not resolve this, because it is baseline-only and its harder tasks are mostly too hard for even the strongest model to gain traction (0 of 20 hard solved), so it measures where signal runs out, not the band where a strong model would have a large revisability gap to exploit. Distinguishing the two shapes would need a difficulty band tuned to

leave a capable model room to revise, run under the full architecture grid rather than at baseline; that is stated as a scope condition in Chapter 8 and as the most valuable extension in Chapter 9, and it remains a falsifiable prediction rather than a hedge.

5.3.2 The boundary probe

The probe answers one question (is there architectural signal past the easy band?) and is sampled for that question rather than for the grid's. Where the grid needs full coverage of a fixed task set to ask *which* architecture helps, the probe needs only a representative slice of harder work to ask *whether* anything could. Its design is fixed in advance so that a null result cannot be explained away after the fact.

Four models span the capability range (Llama-3.1-70B, Devstral-24B, Qwen2.5-Coder-32B, and the hosted model). The tasks are 20 drawn from the benchmark's *medium* pool and 20 from its *hard* pool, selected reproducibly by `random.Random(20260807).sample(·)` over the tasks in each label whose feasibility risk is not flagged high, giving 160 runs. Every run uses the G1-BASE configuration under the identical 60-step budget; no architecture is applied, because the question is whether the underlying models reach these tasks at all, not how to organise their attempts. The probe is stored separately from `core_64` and is excluded from every solve rate the grid reports; its results appear only where they answer their own question, in Section 5.3.1. Because the sample is random rather than curated, a solve is reported wherever it occurs: `git-leak-recovery` is solved by 3 of the 4 models, so the complementarity of the grid does not vanish immediately past easy even as the overall rate collapses.

5.3.3 Effective discriminative power

Not all 64 tasks carry information about architecture. A task that every configuration solves and a task that none solves both contribute a constant to every cell, inflating the denominator without informing the comparison.

Measured on the completed grid, **42 of the 64 tasks are solved by at most one of the 7 models** at baseline and 4 are solved by six or seven; only 18 sit in the mid-band (two to five models solving). Those 18 yield 22 flip events each against 5 for a task at the floor, so on the binary outcome most of the evidence behind any architectural comparison comes from a small part of the corpus (59% of all flips) even though 50 tasks move at least once. Section 6.5.3 develops the point, because it determines what a category mean can mean.

That accounting changes substantially under partial credit, which is one reason the study reports both measures (Section 6.5). Counted by the graders' individual checks rather than by whole-task success, 58 of the 64 tasks change under some architecture, against the 50 that move on the binary outcome. The apparent inertness of the floor band on the binary measure is in significant part a property of that metric rather than of the runs.

5.4 Coverage

All rates in this thesis are computed over the full 64 tasks, and every cell was run on every task: of the 7424 rows in the grid, 7424 were executed (100.0%). No outcome is inferred, no cell is partially covered, and no imputation policy is needed, a property stated explicitly, because a grid this size is more often reported with gaps quietly filled in.

One gap does exist, and it is not a coverage gap: a small number of executed runs lost their behavioural logs to a storage fault, so their pass/fail outcome is known but their trace is not. These runs contribute to all solve rates and are excluded (explicitly, and with the count reported) from behavioural aggregates only. The frozen dataset flags them per row, so no aggregate can silently mix the two populations.

5.5 Single-run validity

Every configuration is a single run per task at temperature 0, prioritising breadth (every architecture on every model) over repetition. This matches standard practice in agentic benchmarking: the SWE-bench leaderboard [35] reports single-run results, agentic code benchmarks report single-attempt `pass@1`, and most directly, **TerminalBench's own leaderboard [33] is single-run**. The benchmark this study measures on measures itself the same way.

The validity of the conclusions rests on four properties, none of which is a claim that individual cells are precise:

1. **The headline is a direction across a gradient**, spanning 13 architecture cells per model and 7 models simultaneously. A single noisy baseline enters both the capability axis and each delta, so part of the correlation is coupling rather than effect; the direction survives removing that coupling (see below), while only its magnitude is noise-inflated.
2. **Mechanism is verified in the traces.** Claims about *why* an architecture helps or hurts are checked against per-run behavioural evidence, not inferred from the solve count alone.
3. **Complementarity is read against noise.** Which tasks an architecture uniquely solves is a set-level property, but the oracle union pools many single runs and rises even with no real complementarity, so it is held to a `pass@k` noise null and claimed only where it clears it (see below).
4. **Greedy decoding removes controllable variance.** The residual is non-determinism in batched GPU inference, which is acknowledged rather than claimed absent.

Controlling for the single baseline. The baseline is one run, used on both the capability axis and inside each delta, so some of the gradient is coupling, not effect: under a *reshuffling null* that relabels which of a model's fourteen configurations is treated as the baseline (re-deriving both the capability axis and every delta from the relabelled cell), the expected rank correlation is already -0.21, not zero. The observed -0.886 still lies beyond that null ($p = 0.037$), and it holds under two checks that do not reuse the baseline at all. Taking capability as the mean of the thirteen non-baseline cells, so the baseline leaves the capability axis and the residual coupling now works *against* the result, still gives $\rho = -0.829$; a split-half that measures capability and effect on disjoint halves of the task set gives a median $\rho = -0.62$ (66% of splits at or below -0.5). What the single run inflates is the magnitude, not the direction, which is why the text argues the sign of the gradient rather than a precise slope. The oracle bound of Section 6.7.1 is held to the same standard: against the `pass@k` noise null its excess is real on the weaker models (mean +3.2 tasks) and within noise on the strongest (-0.2). A direct scale for all of this is the G1-Temp cell, a second baseline at $T = 0.2$: re-rolling it alone moves -5 to +4 tasks, the single-run noise floor that the small per-cell deltas are reported against.

The fourth point deserves one note of mechanism, since it is easy to assume temperature 0 buys reproducibility outright. At temperature 0 the sampling RNG is never consulted, so the seed is inert and the next-token choice is a deterministic function of the model's inputs; the residual enters below that level, where batched inference over quantized weights performs floating-point reductions whose order depends on how requests happen to be grouped, perturbing logits enough to change an arg max that is nearly tied. In a single completion this is invisible. In an agent loop it is not, because one different token early produces a different command, a different observation, and from there a different trajectory, which is why a pipeline of deterministic components is still not bitwise reproducible.

Accordingly this thesis never argues that "architecture X beats the baseline by k tasks". It argues the gradient, the interaction structure, and the mechanism, and reports small per-cell deltas as being within noise.

5.6 Reading failures from the trace

Two of the study's questions ask not whether a run failed but *how*. Answering them needs a rule that reads a failure from its recorded trace the same way every time, so that the failure profiles in Chapter 6 are measurements rather than impressions. This section fixes that rule; the chapter applies it. It is the precise counterpart of the coarse failure taxonomy the design chapter worked from (Section 4.4): that taxonomy named the recurring kinds well enough to motivate the architectures; this one gives the dominant kind, a wrong submission, a mechanical definition that a program can apply to every trace without judgement.

The signals. Every traced run carries four fields the harness records directly, with no post-hoc judgement: the **termination class** (how the episode ended: submitted, or against one of the budget caps); the **number of steps** taken; the **longest streak of identical consecutive shell commands**, the repeat counter used throughout Chapter 6; and the **fraction of the task's grader checks the final state passes**, a partial-credit signal available even when the run scores zero overall. The last is what

lets a wrong answer be placed on a scale from "nothing right" to "one check short" rather than collapsed to a single failed bit.

The four modes of a wrong submission. The dominant failure across every model is a run that submits a wrong answer, and that class alone is uninformative. The termination class picks out those runs; the other three fields then sort each of them into exactly one of four shapes, applied in this priority order:

1. **Looped** — the repeat counter reaches 5 or more: the run spent itself re-issuing one command. Behaviour takes precedence, because a run that ground to a halt is characterised by that whatever else is true of its final answer.
2. **Near-miss** — otherwise, the final state passes at least 50% of the grader checks: the run engaged the task and came close. Outcome takes precedence over length, so a short run that is nearly right is a near-miss, not a quick exit.
3. **Bailed** — otherwise, the run stopped within 10 steps below the near-miss bar (under half the checks passing): it quit early, having barely started.
4. **Off-track** — otherwise: a full-length attempt that worked at the task but finished on the wrong path.

The sort is deterministic and uses no language-model judge, so it is reproducible from the frozen dataset and carries no grader cost. An earlier version separated a run that "never inspected a file" using the count of read-tool calls; that was dropped, because agents inspect files through ordinary shell commands and the read-tool count is zero for about half of all runs regardless. Run length is the honest signal for a premature exit, so *bailed* is defined by steps alone.

One instrument limit, stated where it bites. The repeat counter is computed over a single agent's command stream and does not survive a hand-off between two agents. The cooperation and debate architectures split their trace across two agents, so their repeat counter reads zero for every run; looping cannot be measured for them, and Chapter 6 excludes those two from any per-family failure-shape comparison rather than recording a false zero.

5.7 Statistical treatment

One protocol is fixed here and applied everywhere; it is implemented once so that no two chapters can use different tests, pairings, or correction families.

Everything is paired within a model. An architecture is compared only against *that model's own* baseline, task by task. Pairing removes task-difficulty variance, which is by far the largest source of spread in a task set where some tasks are solved by everything and some by nothing. Comparisons *across* models are never made on raw rates.

Exact McNemar is the primary test. The data are paired binary outcomes, which is precisely what McNemar's test [94] addresses (only the discordant pairs carry information about a difference) and it is the test recommended for this data shape in the NLP evaluation literature [95]. The exact binomial form is used rather than the chi-square approximation because discordant counts here are small (a median of 7 per cell, never above 14) which is the regime where the approximation is unreliable.

The bootstrap supplies the interval. A paired percentile bootstrap [96] over *tasks* (10,000 resamples) gives a confidence interval on the difference in solve counts. Resampling tasks rather than runs is what makes the interval a statement about generalisation to other tasks of this kind, which is the inference the study actually wants. Where a test and an interval disagree in emphasis, the interval is the more honest object: at one run per configuration most cells are not individually distinguishable, and the interval says so directly.

Holm–Bonferroni within a family. The step-down correction of Holm [97] is applied within a family defined as the 13 architectures compared against baseline *for one model*. It is deliberately not applied across models: those are separate questions, and pooling them would impose a larger family than any

claim requires. The heterogeneous-supervision track and the task-shape groupings sit outside this pre-specified family and are read as *exploratory*: they are supported by direction and mechanism rather than by a significance claim, and each says so where it is used (Sections 6.6 and 6.5.4).

Relationships are tested at the level they are claimed. Because the study's claims are about the *relationship* between capability and architectural benefit rather than about individual cells, the corresponding tests operate on that level: Spearman rank correlation [98] with an exact permutation p-value (feasible at $n = 7$ models), two-sample permutation tests on pooled effects, and exact binomial sign tests on the direction of many cells.

No claim rests on a single cell. Section 6.2.2 reports what this protocol yields, including the fact that no individual cell survives correction. The protocol exists to put honest intervals on the numbers, not to manufacture significance.

5.8 Budget uniformity

For a clean comparison, no configuration may receive more agency than any other. That property is not asserted here but measured on the completed grid: **no run consumed more than 60 steps**, the maximum observed across all 7404 traced runs being exactly 60.

The guarantee survives an untidy history. Earlier phases of the study used tighter caps, and a minority of runs still carry one in their recorded configuration; every run that was actually cut short by such a cap was re-executed at the uniform budget, and 0 runs terminate on a non-standard cap in the final grid. For the remainder the cap was never reached, since those runs ended by submitting or by exhausting context, at a median well below any cap they were given, so they are runs that happened to be configured differently, not runs that were treated differently.

5.9 What the study consumed

A grid this size is a resource commitment, and reporting it is part of describing the method. Over the 7404 runs with recorded traces the study consumed **580 hours of wall-clock** (about 24 days of continuous execution) and **3.7 billion tokens**, of which 3.6 billion are input and 37 million output. The asymmetry is characteristic of agentic work rather than incidental: each step resends a conversation that has grown by the previous observation, so input dominates output by two orders of magnitude, and it is why context length rather than generation speed is the binding constraint in Section 6.3. The median run takes 153 seconds.

The split between the two serving paths is 473 hours and 2.7 billion tokens on the six locally served open models, against 107 hours and 1.0 billion through the hosted API, the latter covering both the frontier reference cell and the entire supervision track. Local inference ran on a single GPU throughout, which is the constraint that fixed the model lineup (Section 5.2.1) and the reason the study describes what one accessible machine can serve.

Two consequences are worth naming. Repeating the full grid to obtain per-cell error bars would cost another 24 days, which is the concrete form of the trade-off between breadth and repetition that Section 5.5 resolves in favour of breadth, and it is why Chapter 9 proposes repetition on a targeted subset instead. And the models, benchmark and serving stack are used under their respective public licences; the open weights are distributed under their publishers' community licences and the task set under the benchmark's own, with no redistribution of either in this work.

5.10 Task metadata

Every task is accompanied by a small set of *objective* descriptors, all extracted mechanically from the task files rather than judged: the benchmark's own category and tags, the authors' expert and junior effort estimates, the length of the reference solution, the number of grader checks, and whether the task declares a named output artifact. These are used throughout Chapter 6: the oracle length and check count in the discriminative-power accounting, the declared artifact in Section 6.5.4, the category labels in the task-type analysis of Section 6.5.

The category labels in particular are the benchmark authors' own. Using them rather than a self-generated grouping is deliberate: the authors' labels are data that a reader can check against the public task set, whereas any grouping we invented would be interpretation, and the task-type analysis is where this thesis makes its most contestable claims. Table 5.3 records, for orientation, what each category's tasks concretely involve, their deliverable and setting, read from the task statements rather than judged. This is objective description, not a claim about what a task "demands": the one place this thesis groups tasks by an interpretive judgement of their demand is the task-shape taxonomy of Section 6.5.4, which is flagged there as post-hoc rather than validated.

Table 5.3: What each task category concretely involves — the deliverable and setting, read from the task statements rather than judged. This orients the benchmark's topic labels used throughout the task-type analysis (Section 6.5); it is description, not a judgement of difficulty or required skill.

Category	n	What its tasks involve
`software-engineering`	13	modify, build, or extend existing code: repair a broken build or repository, refactor or optimise a routine, complete a formal proof
`data-science`	8	load and transform tabular data or stand up a data service: convert a format, run an ETL, write a query, start a notebook or model-tracking server
`security`	7	recover, inspect, or protect system state and secrets: decode obfuscated files, exploit a known CVE, read a secret from a running process, extract an archive safely
`file-operations`	6	produce an exactly specified file from data already present: aggregate records, join CSVs, create a bucket, render images
`system-administration`	6	stand up or repair services and infrastructure: summarise logs, fix a script, monitor a cluster, deploy a filesystem
`mathematics`	5	compute a single numeric or symbolic answer, usually by selecting the right library: integrate, solve an integer program, work a number puzzle
`data-processing`	4	clean and reshape structured records into a specified format: interpolate gaps, filter and count, transform JSON, port a data-loader
`debugging`	4	diagnose and repair a failing program or environment: a version conflict, a compile error, a layout warning
`games`	4	solve a rule-based puzzle by proposing candidates and checking them: a maze, a tile-rules engine, a self-referential string, a board read from an image
`machine-learning`	3	diagnose or set up a training run: a diverging loss, a net that will not learn, an RL library to install and run
`algorithms`	2	solve a constraint or logic puzzle that also requires decoding or helper scripts: seating under constraints, scheduling from provided data
`model-training`	1	a full pipeline to a target metric: read data, train, and clear an accuracy bar
`scientific-computing`	1	build and run a heavyweight scientific framework from source

Chapter 6

Results

This chapter answers the five research questions of Section 2.4, in the order given there.

Section 6.2 (RQ1) establishes that the effect of an architecture changes sign with the capability of the model beneath it, and that parameter count cannot stand in for that capability. Section 6.3 (RQ2) reads how runs actually end, how that profile differs from model to model, and what each family repairs or introduces. Section 6.4 (RQ3) asks which architecture suits which model, and whether the answer can be predicted from a single baseline run instead of a full sweep. Section 6.5 (RQ4) asks which kinds of task respond to which family, and whether an architecture helps the kind of task it was designed for. Section 6.6 (RQ5) asks whether a strong supervisor driving a weak worker beats the weak model working alone, and through which roles. Section 6.7 draws the five together and adds two properties of the grid that no single question asks about.

Two conventions hold throughout. Solve counts are out of 64 tasks, and every delta is measured against *that model's own* G1-BASE on the same task, so all comparisons are within-model and paired.

6.1 The grid

The homogeneous track produces one master table, and the first four questions all read from it. Table 6.1 gives the tasks solved by each of the 7 models under each of the 14 architectures, one run per cell over the same 64 tasks; the full per-model breakdown, which keeps the passed-some-checks distinction the solve counts collapse, is in Appendix A.6. The tables that follow cut these same runs a different way, by failure mode, by task, by architecture family; RQ5 is the exception, adding a separate track in which the supervisor is a different model from the worker.

Three subsections then settle what can be done with it: what the table says if only its column averages are taken, whether the budget behind it held, and what counts as evidence once it is disaggregated.

Table 6.1: The master grid: tasks solved out of 64 by each model under each architecture. Green marks an improvement over that model's own baseline (bold), red a regression. Models are ordered by baseline capability.

	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
Qwen2.5-Coder-7B	**4**	5	2	6	6	4	8	5	8	6	6	6	6	5
Llama-3.1-70B	**5**	7	9	9	7	8	10	9	9	6	6	9	9	8
Qwen2.5-Coder-14B	**9**	11	8	5	6	11	8	7	9	12	7	8	9	9
Devstral-24B	**12**	13	11	11	10	11	11	12	12	17	14	18	18	14
Qwen2.5-72B	**16**	18	12	13	14	14	17	15	15	16	15	13	14	14
Qwen2.5-Coder-32B	**19**	15	14	13	14	14	14	15	16	17	15	14	13	12
Gemini-3.1-FL	**34**	32	33	32	32	34	32	32	38	36	37	35	34	32

Two things are needed to read it. Its rows are ordered by *measured baseline solve rate*, running from 4/64 on the weakest model to 34/64 on the hosted reference, and that ordering is deliberately not parameter

count. The two do not coincide here, and Section 6.2.1 establishes why capability rather than size has to be the axis. Throughout this thesis, "weak" and "strong" refer to the measured rate and never to model size.

Its rows are also not one population. Six of the seven are AWQ-quantized open-weight models served locally through the same vLLM stack, while Gemini-3.1-FL is a hosted general model with roughly 4.5× the context budget. Wherever that distinction matters, results are reported for the homogeneous six and for all seven separately rather than pooled silently.

6.1.1 What the average hides

Before the grid is disaggregated, read it once the way the literature usually reports such a table. That reading produces a null result, and the null result is what organises everything after it.

Table 6.2: Mean effect of each architecture across all seven models. No architecture moves the aggregate by more than a few points: the main effect is near zero while the per-model effects of Table 6.1 are large.

Group	Architecture	Mean solve rate	Δ vs baseline
G1	Base	22.1%	+0.0 pp
	Prompt	22.5%	+0.4 pp
	Temp	19.9%	−2.2 pp
G2	Explor	19.9%	−2.2 pp
	Plan	19.9%	−2.2 pp
	Analyst	21.4%	−0.7 pp
	Monitor	22.3%	+0.2 pp
G3	Instr	21.2%	−0.9 pp
	Reflex	23.9%	+1.8 pp
	Judge	24.6%	+2.5 pp
	Gate	22.3%	+0.2 pp
G4	Coop	23.0%	+0.9 pp
	Debate	23.0%	+0.9 pp
	Orch	21.0%	−1.1 pp

Averaged over all 7 models, *no* architecture moves the aggregate solve rate by more than a few points in either direction. A reader who saw only this table would conclude that agent architecture does essentially nothing.

That conclusion would be wrong, and the reason it is wrong is what organises the rest of the chapter: the main effect is near zero *because* the per-model effects are large and of opposite sign, and they cancel. The intervention is strong, but its direction depends on the model it is applied to, so reporting only the aggregate, as a single-model study implicitly does, hides the structure that matters.

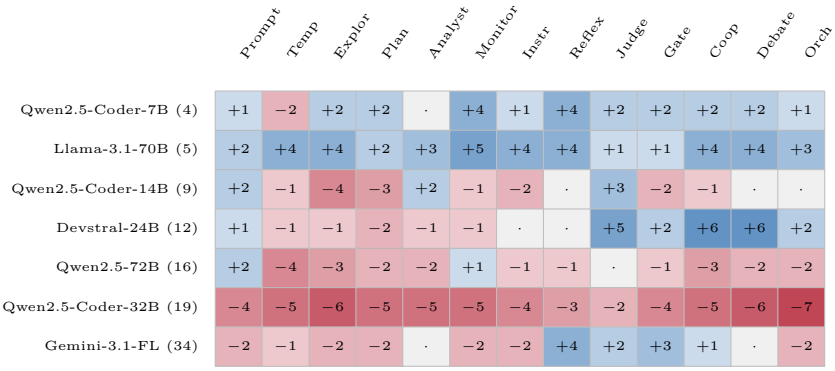


Figure 6.1.1: The model×architecture interaction. Each cell is the change in tasks solved relative to that model's own baseline (parenthesised after the model name); blue is a gain, red a loss, saturation is magnitude, and the signed number is exact. Rows run from the weakest model (top) to the strongest (bottom). Cell deltas span −7 to +6 tasks, yet every column mean is within a few points of zero (Table 6.2): this is the null main effect and the large interaction of Table 6.1 in a single view.

Figure 6.1.1 puts both facts in one frame. Reading down each column (one architecture across the capability range) the cells shift from blue at the top to red at the bottom, so a column whose mean is within a point or two of zero has been cancelled, not left untouched. Reading across each row (one model under every architecture) shows the same intervention helping a weak model and hurting a strong one. The near-zero aggregate of Table 6.2 and the large signed cells of Table 6.1 are the same data seen at two resolutions.

6.1.2 Budget accounting: did the design hold?

Two properties have to hold before any of the disaggregation that follows means anything: that no architecture was handed more room to work in than the plain agent, and that where an architecture did use more room, the extra work is not what produced the result. Neither can be settled by the design alone.

Table 6.3: Resource consumption of each architecture relative to the same model's own baseline, averaged over the seven models. The 60-step *cap* is identical for every configuration; what varies is how much of it is consumed. Values above 1 mean the architecture used more of the budget it already had, not that it was given more.

Architecture	Steps	Input tok.	Output tok.	Wall-clock
Base	1.00	1.00	1.00	1.00
Prompt	1.20	1.27	1.37	1.04
Temp	1.20	1.24	1.19	0.97
Explor	1.12	1.07	1.35	1.00
Plan	0.93	0.95	1.34	0.76
Analyst	1.16	1.14	1.42	1.16
Monitor	1.52	1.51	2.03	1.44
Instr	1.40	1.38	1.62	1.30
Reflex	1.40	1.36	1.81	1.43
Judge	1.37	1.40	1.44	1.37
Gate	1.62	1.54	1.88	1.60
Coop	1.62	1.29	1.32	1.37
Debate	1.57	1.27	1.65	1.37
Orch	1.67	1.46	2.05	1.42

The cap held; consumption did not equalise

Section 4.3 argued by construction that the 60-step budget is shared among an architecture's components rather than added to. The runs bear it out: no run in the grid exceeded the cap.

What the cap did not do is equalise consumption. Averaged over the seven models, Table 6.3 puts 12 of the 13 architectures above their baseline on steps, from 0.93× (Plan) to 1.67× (Orch), and output tokens rise further still because the added components all think in prose. Individual model–architecture cells spread much wider than the averaged rows suggest, from 0.62× (Plan on Devstral-24B) to 2.91× (Monitor on Qwen2.5-Coder-7B).

Every architecture had the *same budget of environment actions available* and they differed in how many they used. The gap exists because the baseline does not exhaust its own budget: mean baseline consumption runs from 12.6 steps on Llama-3.1-70B to 47.8 on Devstral-24B, so the plain agent leaves 47 steps unused on the one and 12 on the other. An architecture that keeps an agent working rather than letting it stop early consumes more of a budget it always had.

The naive fairness objection, that the heavier architectures win because they are given more compute, is therefore answered by the cap alone. What the cap cannot answer is the version underneath it: the baseline left that headroom unused, and the architectures spent it, so *is the benefit simply the extra work that the scaffold unlocked?*

Extra steps are a symptom of difficulty, not a mechanism of success

Table 6.4 answers it. Pooled over all 14 architectures, runs that *succeed* consume roughly half the steps of runs that *fail*, and the pattern holds in every one of the 7 models without exception: ratios range from

Table 6.4: Mean steps consumed by solved versus failed runs, pooled over all 14 architectures. In every model, runs that succeed finish in roughly half the steps of runs that fail.

Model	Solved	Failed	Ratio	n solved
Qwen2.5-Coder-7B	12.9	40.2	3.11	77
Llama-3.1-70B	9.4	22.2	2.37	111
Qwen2.5-Coder-14B	18.8	44.7	2.38	119
Devstral-24B	28.9	51.5	1.78	184
Qwen2.5-72B	14.1	37.6	2.67	206
Qwen2.5-Coder-32B	18.9	44.9	2.37	204
Gemini-3.1-FL	19.9	37.6	1.89	473
All pooled	18.7	39.8	2.12	1374

1.8× to 3.1×, and the pooled difference of 21.1 steps has a permutation $p < 0.0001$. A run still going at step 40 is usually one that has lost the thread, not one working harder toward a solution.

The architecture-by-architecture picture says the same thing. On Llama-70B, some of the *least* step-hungry architectures already help: G1-Prompt consumes 0.82× the baseline's steps and gains +2, and G2-Planner (the leanest on this model at 0.70×) gains +2. Meanwhile G3-Gate consumes 2.37× and gains only +1. At the other end of the capability range, G4-Orchestrator is the heaviest architecture by every measure in Table 6.3 and is −7 on Qwen-32B, the largest single regression in the grid. Within a model, step consumption simply does not track benefit.[1]

6.1.3 What counts as evidence

The rest of the chapter disaggregates the grid, so the standard of proof it uses should be clear in advance, and it is not statistical significance.

Each cell of the grid compares one architecture against the baseline over the same 64 tasks. A task can come out four ways, and two of them separate nothing: if the architecture and the baseline both solve it, or both fail it, the task says nothing about which of the two is better. Only a task that one solves and the other does not bears on the difference, and a cell has a median of 7 such tasks, never more than 14. The effective sample behind a cell is therefore not 64 but a handful, which is too thin for a test of that cell on its own: under the protocol of Section 5.7, 0 of the 91 cells reach significance.

The chapter is written accordingly. No claim rests on a single cell, and where one is quoted it illustrates a mechanism established elsewhere. What carries the arguments instead is the direction taken by many cells at once, together with what the runs themselves show: what the agent did, where it stopped, whether the scaffold's components functioned, which named tasks changed hands. Each test is reported at the claim it supports rather than gathered into a section of its own.

6.2 Capability, not scale, conditions the effect

This section reads the grid by its first axis, the *model*. It answers RQ1 in two moves: does scaffolding improve agentic coding uniformly, or does its effect (including its sign) depend on the capability of the model underneath (and is capability substitutable by parameter count)? And then, beneath that aggregate, is each model's response to architecture readable from its own baseline run, before any scaffold is applied?

6.2.1 The axis: capability, not parameter count

The ordering of the grid cuts across parameter count. Llama-70B, at 70B, solves 5 at baseline, fewer than the 14B coder's 9, so the largest open models here are not the most capable. A single baseline is a noisy point, though (Section 5.5), so the claim does not rest on any one-task gap between two models: it rests on the within-family ladder below and on the cross-model gradient. What orders a model on this axis is its measured capability, not parameter count, and within a shared training recipe that capability does track scale.

[1] Correlating each architecture's step ratio against its Δ across the 13 architectures gives $\rho = -0.06$ pooled over the two weakest models and $\rho = +0.23$ pooled over the two strongest open models ($n = 26$ each), neither distinguishable from zero.

Scale is not irrelevant so much as confounded with training recipe. Hold the recipe fixed and scale does order capability, which turns one corner of the grid into a controlled experiment: the three Qwen coders, built the same way and differing only in size. They share training data, tokenizer, tool interface, quantization method and serving stack, so the only thing that varies across them is scale, and with it capability. This is the most controlled version of the gradient the study contains, and the plainest:

Qwen2.5 coder	Baseline solves	Architectures that improve on it
7B	4	11 of 13
14B	9	3 of 13
32B	19	0 of 13

With everything but size held fixed, the benefit of architecture falls to zero as scale rises.

This is also the study's answer to the most obvious objection to the headline, which is that the architectures might simply be badly implemented. A badly implemented architecture should hurt *everyone*, or help at random. Instead the *identical* code lifts Qwen-7B in 11 of 13 cells and fails to lift Qwen-32B in any: the same family, the same implementation and the same budget produce opposite outcomes, so whatever is varying is not the quality of the scaffold.

The 72B general model of the same family is not part of this ladder, because it is not a coder: it solves 16 with 2 architectures improving on it, more than the 32B despite being larger. On the parameter axis that is an anomaly; on the measured capability axis it falls where its baseline puts it.

6.2.2 The gradient across all seven models

The controlled ladder is three models. The same relationship holds across all 7, and it is established here three times over, by three measurements whose weaknesses do not overlap, so that no one objection reaches all of them: a contrast between groups that assumes no ordering at all, a rank correlation that assumes one, and, in Section 6.2.3, a count of discrete events that uses no statistic whatever. Those three establish the direction. A fourth instrument, the bootstrap, establishes nothing about direction and is used only to say how far the magnitude may be pushed.

Table 6.5: The capability gradient. As a model's baseline rises, the number of architectures that beat it falls, and the best and worst available architectures move down with it.

Model	Baseline	# archs > base	Best	Worst
Qwen2.5-Coder-7B	4/64	11/13	Monitor (+**4**)	Temp (−2)
Llama-3.1-70B	5/64	13/13	Monitor (+**5**)	Judge (+1)
Qwen2.5-Coder-14B	9/64	3/13	Judge (+**3**)	Explor (−4)
Devstral-24B	12/64	6/13	Coop (+**6**)	Plan (−2)
Qwen2.5-72B	16/64	2/13	Prompt (+**2**)	Temp (−4)
Qwen2.5-Coder-32B	19/64	0/13	Judge (−**2**)	Orch (−7)
Gemini-3.1-FL	34/64	4/13	Reflex (+**4**)	Prompt (−2)

The central result of this thesis is visible in the third column of Table 6.5, and plotted in Figure 6.2.1: the number of architectures that beat a model's own baseline *falls* as that baseline *rises*. On Llama-70B, all 13 of the 13 non-baseline architectures improve on the baseline. On Qwen-7B, 11 do. By Devstral-24B it is 6, by Qwen-72B 2, and on Qwen-32B (the strongest open coder) **0** architectures beat the baseline. Gemini-3.1-FL does not continue the sequence, returning to 4. It is the one model that sits off this line, and also the only one that is not a locally served open-weight coder but a hosted, general model reached over an API; a trend measured on the local coders is not expected to extend to it, so the gradient below is stated over those six and the correlation reported both with and without Gemini-3.1-FL.

The same architecture therefore changes *sign* across the capability range. G2-Monitor, the best architecture for Llama-70B at +5, is −5 on Qwen-32B. G2-Explorer moves from +4 to −6. G4-Orchestrator moves from +3 to −7. This inversion is what no single-model study can observe, and it is the quantitative form of the claim that architecture is not a universal improvement.

The best and worst columns bracket that inversion. On Llama-70B even the *worst* of the thirteen architectures improves on its baseline; on Qwen-32B even the *best* of them does not. No architecture in

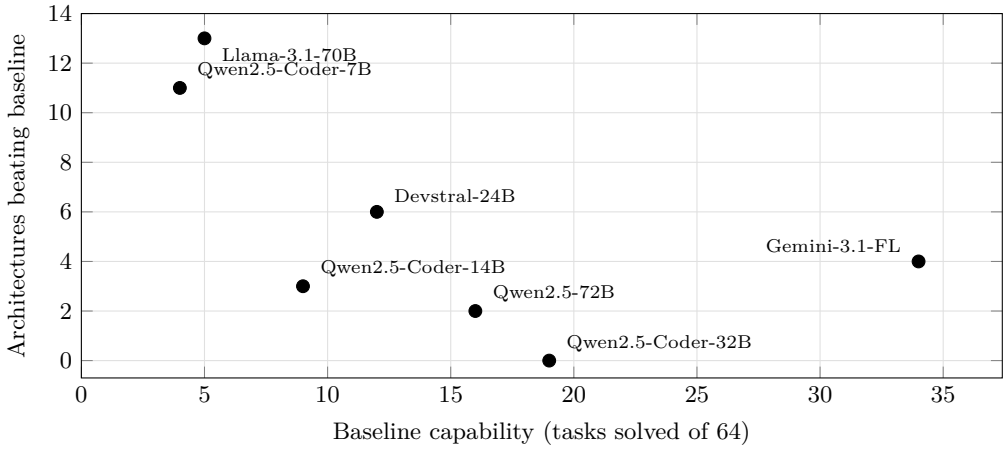


Figure 6.2.1: The capability gradient. Each point is one model: its baseline solve count against how many of the 13 non-baseline architectures beat that baseline. The relationship is negative and steep.

the study harms the weak model and none helps the strong one, and G3-Judge sits at both boundaries: tied for the least helpful on Llama-70B (with G3-Gate) and the least harmful on Qwen-32B.

The first of the three measurements is **the contrast between the two ends**, and it carries the claim because it assumes no ordering at all. On the two weakest models, 24 of 26 architecture cells improve on baseline and 1 regresses; on the remaining five the direction reverses, with 42 of 65 regressing. The difference in mean effect is +3.51 tasks (reshuffling null, $p \approx 0.033$). Identical architectures under an identical budget systematically help weak models and burden strong ones.

The correlation says what the contrast cannot. Splitting the models in two establishes that the direction reverses, but not that the effect is *graded* rather than a step between two regimes. A rank correlation answers that, and the protocol of Section 5.7 is applied at the level the claim is made. The two quantities correlated are per model: its *capability*, operationalised as tasks solved with no architecture at all, and the *benefit* it draws from them, its mean Δ across the 13 non-baseline cells. Over the six locally served open-weight models the first rises from 4 to 19 while the second falls, and the rank correlation is Spearman $\rho = -0.886$ (exact permutation $p = 0.033$, $n = 6$), close to the -1 of a perfectly inverse relationship, and the same value whether benefit is measured as mean Δ or as the count of architectures beating baseline. **The more capable the model, the fewer architectures help it.** Including Gemini-3.1-FL weakens the coefficient to $\rho = -0.679$ ($p = 0.110$), and both are reported.

Six points is few, and two sensitivity checks disagree in an instructive way. Dropping any single model leaves the coefficient between -0.90 and -0.80, so the *relationship* is not the creation of one model. But 6 of those 6 subsets individually fall short of significance, which is what five points buys: at that size the permutation test is badly underpowered, so only a coefficient very close to -1 would clear 0.05, and no leave-one-out subset is that extreme. Reordering two adjacent models loses significance in 3 of 5 cases. The estimate is stable and the test is underpowered, which is why the correlation is reported here as support and the group contrast above, which needs no ordering, carries the claim.

What the intervals say that the tests do not. A test asks whether an effect could be zero. It says nothing about how large the effect is, and size is what a reader wants when a cell is reported as +4. Table 6.6 supplies it for the two extreme cells of each model, the architecture that helped it most and the one that harmed it most, as a 95% interval on the difference in solves obtained by resampling the 64 tasks.

The widths are the finding. A median interval of 12 tasks, roughly a fifth of the corpus, is the direct consequence of one run per configuration on 64 tasks: a cell reported as +4 is compatible with anything between no effect at all and +9. This is why no per-cell magnitude is quoted anywhere in this chapter.

1 of the 14 rows excludes zero, and two cautions attach to it. These are each model's extremes, selected after seeing the data, and the intervals carry no correction for the 91 comparisons the grid contains, which is why this row is not a significant cell under the protocol of Section 6.1.3, where there are none. What it marks is where the strongest per-cell signal in the study lies and which way it points: Qwen2.5-Coder-32B

Table 6.6: Paired percentile bootstrap intervals (10,000 resamples over *tasks*) on the difference in solves against each model's own baseline, for the architecture that helps it most and the one that harms it most.

Model	Architecture	Δ solves	95% CI
Qwen2.5-Coder-7B	best: Monitor	+4	[+0, +9]
	worst: Temp	−2	[−5, +0]
Llama-3.1-70B	best: Monitor	+5	[−1, +12]
	worst: Judge	+1	[−3, +6]
Qwen2.5-Coder-14B	best: Judge	+3	[−2, +8]
	worst: Explorer	−4	[−9, +0]
Devstral-24B	best: Cooperation	+6	[+0, +13]
	worst: Planner	−2	[−8, +4]
Qwen2.5-72B	best: Prompt	+2	[−4, +8]
	worst: Temp	−4	[−10, +2]
Qwen2.5-Coder-32B	best: Judge	−2	[−7, +3]
	worst: Orchestra	−7	[−13, −2]
Gemini-3.1-FL	best: Reflexion	+4	[−3, +11]
	worst: Prompt	−2	[−9, +5]

under ORCHESTRA, at −7 tasks with an interval of [−13, −2]. The cell that comes closest to standing on its own is a loss rather than a gain.

The pattern is therefore consistent in direction, bounded in magnitude, and unexplained. The explanation is what the next three sections supply, in that order: Section 6.3 shows that the models at the two ends of this range fail in different ways, Section 6.4 shows that the architectures which help are the ones matched to those failures, and Section 6.5 identifies the specific tasks that change hands.

6.2.3 The same result without averaging

Everything said about the gradient so far has summarised before it compared. The headline counts how many architectures beat each baseline; the correlation and the group contrast both run on each model's mean delta. Either summary invites the objection that the shape belongs to the summary rather than to the runs, so the result is stated once more with nothing summarised at all, by descending to the atomic comparison.

That comparison is one model, one architecture, one task, against the same model's baseline on the same task: a *win* where the baseline failed and the architecture solved, a *loss* where it went the other way, and no information at all where neither changed (Section 6.1.3).

Table 6.7 classifies all 5824 comparisons in the homogeneous grid, and two things come out of it.

The gradient is there in the raw counts. Read down the models, the net column traces the same inverse relationship without a statistic to do it: strongly positive on the two weakest models, strongly negative on the strongest open coder, monotone apart from Devstral-24B. The column is a count of how often an architecture converted a failure minus how often it destroyed a solve, and nothing is averaged anywhere in it. The gradient is therefore not an artefact of the summary chosen to express it. The family ordering survives the same way: G3 is the most positive balance and G4 mildly positive, against G1 and G2 both negative, G2 the most. That is the ranking Section 6.4 reaches by an unrelated route.

And the null average is the same column, added up. Summed over models it gives 333 wins against 344 losses, a net of -11 tasks. That is the near-zero main effect of Section 6.1.1: large positive and large negative entries destroying one another. The gradient and the null average are one measurement, seen with and without summation.

One more number sets the scale. **5147 of the 5824 comparisons change nothing at all**, so architecture moves roughly one comparison in nine. Whatever it does in this study, it does in a small minority of cases.

Table 6.7: Every model–architecture–task comparison against that model's own baseline. A *win* is a task the baseline failed and the architecture solved, a *loss* the reverse, and *net* is wins minus losses; *moved* is the share of that model's or family's comparisons that were either. The remaining comparisons left the outcome unchanged and carry no information about a difference.

	Baseline	Win	Loss	Net	Moved
By model, ordered by baseline capability					
Qwen2.5-Coder-7B	4	35	14	**+21**	6%
Llama-3.1-70B	5	66	25	**+41**	11%
Qwen2.5-Coder-14B	9	34	41	**−7**	9%
Devstral-24B	12	66	50	**+16**	14%
Qwen2.5-72B	16	37	55	**−18**	11%
Qwen2.5-Coder-32B	19	28	89	**−61**	14%
Gemini-3.1-FL	34	67	70	**−3**	16%
By architecture family					
G1	—	44	52	−8	11%
G2	—	102	124	−22	13%
G3	—	102	86	+16	10%
G4	—	85	82	+3	12%
All	—	*333*	*344*	−11	*12%*

6.2.4 Why each model responds as it does

The gradient placed the models on an axis and the wins-and-losses view resolved it to single tasks; this section reads the same grid one model at a time, to explain why each responds to architecture as it does. The explanation is a characteristic behaviour each brings to the same scaffold, legible from its bare baseline run before any architecture is applied. Table 6.8 is that profile, a compact reading of each model that the paragraphs below take apart one at a time. Because every property is read from the baseline, two later results can lean on it: the Monitor's effect is ordered by the budget a model leaves unused (Section 6.4.2), and the selection rule of Section 6.7.2 assembles these same properties to choose an architecture for a new model from a single baseline run.

Table 6.8: What predicts a model's response to architecture. None of the three middle columns is a solve rate: they are how long the model persists before failing, what its dominant failure is, and whether the scaffold's own machinery functions on it (share of G2-Planner runs producing a schema-valid plan). Models are ordered by baseline capability.

Model	Base	Steps to fail	Dominant failure	Valid plans	Archs helping
Qwen2.5-Coder-7B	4	17	65% submits wrong, 32% context	**0%**	11/13
Llama-3.1-70B	5	13	100% submits wrong	95%	13/13
Qwen2.5-Coder-14B	9	40	85% submits wrong	**5%**	3/13
Devstral-24B	12	54	81% submits wrong	98%	6/13
Qwen2.5-72B	16	29	69% submits wrong	97%	2/13
Qwen2.5-Coder-32B	19	41	47% submits wrong, 33% context	59%	0/13
Gemini-3.1-FL	34	42	93% submits wrong	84%	4/13

Qwen-7B — helped broadly, by a scaffold that barely runs. Baseline 4/64. Its failures split 65% premature submission and **32% context exhaustion**: at half the capacity of every other local model, its window is the one small context in the study, though the overflow *terminations* that expose it are a pre-fix harness artefact (Section 8.2). `ancient-puzzle` shows how that bound bit before the fix: the baseline retries one malformed `awk`/`jq` pipeline unchanged, the transcript growing with each identical attempt until it overflows the window and the run ends, a bout of trial and error a full-size context would have absorbed. Its machinery largely does not function: **0% valid plans**, 31% valid summaries, and G4-Orchestrator hits the cycle cap in 61% of runs, the worst in the study. 11 architectures nonetheless improve on that baseline, which says more about the floor than about the scaffolds, and this is the model where the floor-effect caveat genuinely applies.

Llama-70B — the early quitter, helped by anything that continues it. Baseline 5/64, one above Qwen-7B and its mechanistic opposite: *every* one of the 13 architectures helps. **100% of its traced baseline failures end in a deliberate submission**: not one of them exhausts the step budget, hits the context wall, or times out. Its failed runs stop at a mean of 13 steps (the fastest-stopping model in the corpus) and consumes the fewest tokens of any model, so it is never resource-limited. It is limited entirely by stopping too early with a wrong answer, which is the largest possible revisability gap. Anything that keeps it working helps: G2-Monitor doubles its run length and gains +5. Its machinery is sound (95% valid plans, among the higher rates in the study) so its failures are not a scaffold that could not run; the model simply stops, and needs only to be made to continue. `predict-customer-churn` shows the gap in miniature: the baseline stops at eleven steps having written the model's *source code* into the `.pkl` file it was asked to pickle and a sentence of prose into the checksum file, then submits "the task is complete". The two architectures that keep it working, G3-Reflexion and G4-Debate, both solve it: the missing work was one revision away, and nothing about the baseline went back to take it.

Qwen-14B — the least responsive model, and the most revealing. Relative to its position on the capability axis it is the outlier: only 3 architectures help. Both mechanisms that help its neighbours are unavailable to it. It *does not stop early*, taking 40 steps on failure, three times Llama-70B's, so there is no idle budget to put to work; and its machinery *does not function*, at 5% valid plans. On G2-Planner, 61 of its 64 plans fail schema validation, so the executor is handed nothing and runs unscaffolded; its whole G2 result measures an architecture that never engaged. It is simultaneously too persistent to benefit from being pushed onward and too unreliable at structured output to benefit from being organised. What clearly helps are the designs that ask nothing structured of it: G1-Prompt (+2) and G3-Judge (+3); G2-Analyst also nets +2, small enough to sit within the single-run noise but noted here since it is a G2 design. What stays firmly negative are the heavy pre-execution pipelines that depend on a valid plan. **This is the clearest evidence that structured-output compliance is a separate axis from capability**: Qwen-14B solves nearly twice what Llama-70B does and produces valid plans one twentieth as often.

Devstral-24B — never stops, helped by a restart rather than more steps. The opposite of Llama-70B. It fails after a mean of **54 steps**, the longest of any model, with 15% of failures hitting the *step* budget outright (a little more once the handful of wall-clock timeouts are added, Table 6.10), and makes 2691 `bash` calls across its 64 baseline runs, more than twice as many as any other model. It does not give up; it keeps working, and it works *blind*: **82% of its steps chain several shell commands into one** without inspecting the intermediate output, the highest rate in the study. `security-vulhub-minio` shows the habit at its most extreme: the baseline issues the *same* `find /` sweep fifty-four times unchanged, sixty steps of it, until the budget ends, never once pausing to read what the sweep returned. Acting far more than it looks is the behavioural signature that predicts its whole profile. Extending the run cannot help, and G2-Monitor is indeed negative. Restructuring the effort does: the multi-agent family is its best, with G4-Debate at +6 and G4-Cooperation at +6, because a fresh agent with a clean context breaks a loop that more steps would only prolong. Verification largely cannot fire, since only **30% of its intercepted submissions leave the verifier any budget**, the lowest in the study; yet G3-Judge still nets +5, nearly its best. On the minority of runs where budget remains for the verifier to fire, a rejection injects a targeted critique into the agent's continuing memory, which on a grinding worker is enough to break it off the repeated command; with one round available this is a bounded effect, offered as the likely mechanism rather than established trace by trace.

How far Devstral-24B stands apart is clearest across all seven (Table 6.9). The chaining rate does not track capability ($\rho = +0.32$, $p = 0.50$): the strongest model, Gemini-3.1-FL, chains at 27%, below Qwen-14B's 43%, while Devstral-24B is the outlier by a wide margin.

Qwen-72B — the all-rounder, helped only by the cheapest nudge. The largest open model, and the one where the cheapest intervention wins: G1-Prompt is its best architecture at +2, with G2-Monitor the only other positive. Its machinery is fine, it does not stop early, and it has no context problem: there is simply no specific deficit for an architecture to address, so every scaffold that spends budget on structure loses more than it gains. One number distinguishes it: its self-judge approves **57%** of its own work, the most lenient of any model (in thirteen runs it signed off on output that had already failed the grader) and a model satisfied with its own output gains nothing from a family that depends on rejection to act. G3-Judge is exactly +0.

Table 6.9: Command chaining per model, from baseline runs: total `bash` calls over the 64 baseline runs and the share of steps that pack several commands into one without inspecting the intermediate output. The rate does not track baseline capability; Devstral-24B is the outlier.

Model	Baseline	`bash` calls	Chained
Qwen-7B	4	244	22%
Llama-70B	5	341	25%
Qwen-14B	9	701	43%
Devstral-24B	12	2691	**82%**
Qwen-72B	16	770	38%
Qwen-32B	19	1099	31%
Gemini-3.1-FL	34	950	27%

Qwen-32B — the strongest open coder, its failures too diffuse to target. At the top of the open range, and the only model where nothing helps at all: not one of the 13 architectures clears baseline, the best still losing -2 tasks and the average costing it −4.69, the only model in the study without a single neutral-or-better option. There are three candidate explanations, two of which the data rejects. *Context pressure* is rejected: it overflows in 33% of baseline failures, a pre-fix harness artefact (Section 8.2), and on post-fix runs alone its failures still split most evenly of any model, yet the architectures that most relieve that pressure are its worst performers, the G4 family driving overflow to near zero while scoring −6 and −7. Relieving the constraint does not help, so the constraint does not bind on outcomes. *Broken machinery* is rejected too: at 59% valid plans it is mid-pack, far above Qwen-14B. What remains is that its failures are the most evenly spread of any model, at 47% submission, 33% context and 18% budget, so it presents no single dominant failure for an architecture to target, and every architecture spends budget on a problem it does not have.

Gemini-3.1-FL — the ceiling, where only verification survives. Baseline 34/64, and the architectures that help are verification, bar one: G3-Reflexion +4, G3-Gate +3, G3-Judge +2, with a lone single-task gain from G4-Cooperation +1. Its failures are 93% deliberate submission after 42 steps: it works hard and then submits something wrong. `jupyter-notebook-server` is typical: forty-eight steps of competent setup (install, TLS certificate, password hash, config file) ending in a confident "the server is running on port 8888" that fails a check it never went back to test. There is no context problem and its machinery works. With no resource constraint and no machinery failure, the only remaining deficit is *the correctness of a completed attempt*, and post-execution verification is the only family addressing exactly that. Every G2 architecture is negative or zero, as is every G4 bar G4-Cooperation's marginal +1.

Across the seven, one pattern recurs: the architecture that helps is the one that addresses the model's characteristic deficit, extension for the model that stops early, restructuring for the one that never stops, verification for the one whose only fault is a finished answer that is wrong. Where a model has no single dominant deficit (Qwen-32B), or where its scaffold cannot reliably run (Qwen-7B, Qwen-14B), nothing helps. This is the principle Section 6.4 develops architecture by architecture, and that the selection rule of Section 6.7.2 turns into a procedure driven by the same baseline properties.

6.2.5 Three objections

The gradient and the per-model mechanism behind it are now on the table. Three readings would still undo the result, and each must be cleared before it becomes an answer. The first attacks the axis itself, holding that its ordering is an artifact of uneven quantization rather than a real difference in capability. The other two grant the ordering and attack its ends: that the weak models improve only because they start near zero, and that the strong models regress only because the shared budget starved them. Each makes a prediction the data can check.

Is the ordering quantization damage rather than capability?

The axis is built from measurements taken on compressed models. All six open models are AWQ 4-bit (Section 5.2.2), compression costs accuracy, and it does not cost every model the same amount. If it cost these six unevenly, what is read here as capability could be compression damage wearing capability's clothes.

Only one version of the objection matters. Uneven damage that leaves the models in the same order threatens nothing here: it stretches the axis without rearranging it, and the headline statistic is a rank correlation, unchanged by any transformation that preserves order. Every claim in this chapter is about which model sits above which. The objection bites only if quantization *reordered* the six.

Reordering requires the damage to fall hardest on the models this study ranks highest, and the two placements it would have to account for both run the wrong way for that. What is surprising in the ordering is that Llama-70B sits near the bottom and that Qwen-72B sits below the smaller Qwen-32B, and both are large models, which are the more robust to low-bit quantization [90] rather than the less. The objection needs compression to have behaved opposite to its documented direction. Inside the Qwen coders the comparison separates nothing, and that is worth admitting: family, recipe and quantizer are fixed there, so damage falling harder on the 7B and capability rising with size predict the same ordering.

The damage signature is also missing in the one place the objection most needs it. Compression damage shows as degraded output fidelity, measurable here as structured-output compliance (Section 6.4.2). Llama-70B, the model whose low position the objection would have to explain, emits a schema-valid plan in 95% of runs, among the highest in the study. Whatever puts it near the bottom of the capability axis, it is not an inability to produce well-formed output.

What survives is the order-preserving case, and it is stated rather than answered. If quantization depressed the weaker models more than the stronger ones, which is the direction the literature predicts, then the measured capability range is wider than the true one. That distorts distances and not order, so the rank-based claims stand while claims about absolute solve rates would not; none are made.

Is it a floor effect?

The obvious objection to "every architecture helps Llama-70B" is that its baseline is so low that almost any perturbation looks like an improvement. The study contains a natural control for this: Qwen-7B has an almost identical baseline (4 against 5), so a pure floor effect should produce similar behaviour in both. It does not.

	Llama-70B	Qwen-7B
Baseline solves	5	4
Architectures that help	**13**/13	11/13
Mean Δ across architectures	+3.15	+1.62
Union of solves	**20**	14
Steps used before failing, of 60	13	17
Grader checks passed at baseline	**34%**	25%

Two models one task apart at baseline differ by more than a factor of two in mean architectural gain, and the last two rows are where the difference sits. Llama-70B gives up after 13 of its 60 steps with 34% of the graders' checks already passing: it stops on work that is part-way done and has three quarters of its budget untouched. Qwen-7B runs longer, to 17 steps, and arrives at less, 25%. An architecture that keeps an agent working finds partly finished work on the first model and less of it on the second, which is also why the union of solves opens up so much further on Llama-70B.

So a floor effect is present and is not the explanation. Being at the floor is necessary for large gains, since there has to be headroom, and it is not sufficient: the headroom has to contain work the model was close to completing. Qwen-7B shows what a low baseline looks like on a model near its ceiling, Llama-70B what one looks like on a model stopping short of its own.

Are the strong models simply being starved of steps?

The most troubling reading of the gradient is that it is an artifact of the shared budget rather than a fact about architecture. An architecture with an explorer and a planner spends part of the 60 steps before the executor even begins, so a strong model might regress for a dull reason: it was cut off before it finished. If that were the story, the gradient would measure the budget split and not the scaffold.

The objection makes a prediction that can be checked instead of argued. If a lost run failed because it was starved of steps, it should be *shorter* than the run the bare model needed to solve the same task. The test population is every task one of the three strongest models solves alone but fails under some architecture, kept to runs with a surviving trace: **211 such pairs**, each differing from its own baseline in one variable, the architecture.

If the budget were being starved	What the 211 pairs show
the lost run would be *shorter* than the baseline needed	it is *longer*: a median of 28 steps against 18 for the baseline that solved the same task, +6 per pair
most losses would stop early	63% run *longer* than the baseline needed, 33% shorter, 5% the same
many would end by running out of budget	20% do; the other 70% stopped on their own and submitted something, and 10% overflowed context

The prediction fails on every row. The strong models are not cut off: they keep the budget they had, work past the point where the bare model was already done, and hand in something worse. The one row that could have supported starvation, the 20% that hit a wall, is a minority, and even those are not mostly the executor running dry: of the 42 such runs, 36 exhausted steps and 6 hit the multi-agent coordination-cycle cap. Within G2, the family that carves the most from the executor and where starvation should show first, only 36% of losses end before the baseline's own solve point.

How far off the different work lands. Ruling out starvation says the lost runs do *different* work, not *truncated* work, and that raises the obvious next question: how different? A near-miss that lands just short of the solution implicates the architecture far less than a run that ends up nowhere near it. The graders separate the two. Of the 143 wrong submissions graded in detail, 39% pass no check at all; that figure needs one correction first, since a task with a *single* check scores zero the instant it fails, and 55% of the zero-credit losses are such tasks. On the 112 losses with two or more checks, where the score can actually tell a near-miss from a collapse, **22% still pass nothing**. Much of the different work is not a near-miss: the run ended up somewhere else entirely.

What the two results say together. A capable model's first attempt is usually its good one, so an architecture that adds structure *substitutes* its own approach for one that was already working rather than filling a gap (Section 7.1): the strong models keep their budget, work past the point the bare model was already done, and arrive somewhere worse, often with nothing right in it.

`recover-accuracy-log` shows the substitution in a single pair. The task asks for six log files to be reconstructed and an accuracy score computed, and Gemini-3.1-FL's bare agent solves it in **22 steps**, writing its script across three passes and correcting it between runs. Under G2-Planner the same model is handed a plan that fixes one monolithic script at the outset; the executor then runs that script **nineteen times without ever rewriting it**, reaching the budget at **42 steps** with none of the three checks passing. The model that had just solved the task by revising its own code was given a procedure that ruled the revision out.

The shape of the penalty follows directly. The families that *direct* the agent, G2's explorer and planner, impose the most substitution and do the most damage, while G3-Judge and G3-Gate, which leave the agent's own approach in place and only ask it to look again, do the least.

6.2.6 The answer to RQ1

Architectural benefit depends on capability, and inversely. As a bare model's own solve count rises, the number of architectures that improve on it falls, reaching zero on the strongest open coder, where 0 of the 13 beat the baseline and the balance of discrete conversions is negative by a wide margin (Table 6.7). The same scaffold, byte for byte and on the same budget, changes sign along that range. That is the first half of the question answered, and it is why an architecture cannot be recommended without naming the model it will run on.

The second half is answered in the negative and had to be settled first: the axis is measured capability, never parameter count, since the least-helped model is the strongest open coder, not the largest by parameters (Section 6.2.1). Four readings that would have explained the gradient away were tested and rejected: uneven quantization damage, badly built architectures, a floor effect under the weak models, and step starvation under the strong ones. What the strong models lose is not resources but their own approach, replaced by the scaffold's.

The conditioning is not only aggregate. Each model's response is legible from three properties read off its baseline, how early it stops, what it fails at, and whether its scaffold's machinery runs (Section 6.2.4),

none of them a solve rate, and all varying nearly independently of capability. That is what makes the model, not the architecture, the thing to measure first: the same baseline run that places a model on the gradient also forecasts which kind of scaffold will help it.

Why the sign changes is taken up by the questions that follow: how models fail (Section 6.3), what makes an architecture help (Section 6.4), and which tasks change hands (Section 6.5); and, combining a model's profile with what each architecture does, the selection rule of Section 6.7.2.

6.3 How models fail, and what each architecture repairs

Section 6.2 established that architectural benefit falls as capability rises without taking apart why. The mechanism has to start here, because an architecture can only repair a failure that occurs: this section answers RQ2 by asking how runs actually end, how each model characteristically fails, and what each architecture fixes and introduces. The architectures were designed against the failure taxonomy of Section 4.4; here the question is what actually happens.

6.3.1 How runs end, and how each model fails

Every run records why its episode terminated. The harness emits nine distinct termination strings, which we map once, in the dataset build rather than per analysis so that no two sections can classify differently, onto six behavioural classes: the agent *submitted*, it exhausted the *step* budget, it hit the orchestra *cycle* cap, it hit the *wall-clock* limit, it overflowed its *context*, or a multi-agent scheme failed to reach *coordination*. A seventh class captures infrastructure errors, which are excluded from behavioural claims.

Table 6.10: How each model's *failed* baseline runs ended, as a share of that model's failures, with the number of failed runs and the median steps before failing. The *other* column is mostly context overflow, a harness-epoch artifact rather than a model property (Section 8.2).

Model	Failed runs	Wrong submission	Ran out of budget	Other	Median steps
Llama-3.1-70B	58	100%	0%	0%	7
Gemini-3.1-FL	30	93%	7%	0%	46
Qwen2.5-Coder-14B	55	85%	0%	15%	50
Devstral-24B	52	81%	19%	0%	60
Qwen2.5-72B	48	69%	15%	17%	27
Qwen2.5-Coder-7B	60	65%	0%	35%	12
Qwen2.5-Coder-32B	45	47%	18%	36%	38

Table 6.10 restricts attention to *failed* baseline runs, which is what makes it a statement about failure personality rather than about capability. Three findings follow, with one worked example between them.

The dominant failure is not running out of budget. Across every model, the largest category is a run that terminated because the agent *decided it was finished* and submitted an answer that failed the tests. Budget exhaustion (the failure mode that "give the agent more steps" would address) is a minority everywhere, with one qualification visible per model: the "submitted" class pools runs that chose to submit with runs forced to submit at the step cap, and for Devstral-24B, which grinds to the ceiling, a large share of its wrong submissions are of the second kind. The claim is about the *weak* models, which submit wrong early and of their own accord; for the mid model the two readings partly coincide. This is what makes post-execution verification the architecturally well-targeted family (Section 6.4.3): it is the only group that intervenes at the moment the dominant failure occurs.

What that dominant failure hides. A single termination class, "submitted", can be reached from opposite situations, and one task shows all of them at once. `heterogeneous-dates` asks for the mean daily difference between two temperature series given as separate CSVs; the trap is that the two files record dates in different formats, so a join on the date column fails unless the formats are normalised first. Three models submit, from three different places. Qwen-7B, the weakest, solves it in 5 steps by *never parsing a date at all*: the rows line up one-to-one, so the mean of the differences equals the difference of the means and the date column is decorative, and it submits a correct answer without ever seeing the difficulty. Gemini-3.1-FL does see it, writes a parser that tries several formats, runs it, checks the output,

and submits a correct answer in 9 steps. Qwen-32B also sees it: its fourth step records "*the date formats in the two CSV files are different; I will need to parse the dates*", and then issues **42 consecutive edits to the same script without running it once**, until the budget expires and it submits whatever it has. Not seeing the difficulty and stepping past it, seeing it and dispatching it, seeing it and being able neither to solve nor to abandon it: three trajectories that the termination class records identically as "submitted". Reading how a run *ended* is not the same as reading how it *failed*.

The three trajectories sit at different points on the capability axis, and on this task the worst outcome falls between the other two. Qwen-32B, the strongest open model but weaker than Gemini-3.1-FL, is the one stuck editing: it has enough competence to recognise the complication and not enough to dispatch it, and it has lost the naivety that let the 7B step past the difficulty without ever seeing it. The same trap, a model competent enough to see a complication it cannot resolve, reappears in Section 6.5, where it explains why no architecture rescues Qwen-32B on this task: every scaffold that supplies more information about the data only deepens the commitment that is destroying the run.

Models fail in characteristically different ways. The median-steps column is where the difference shows. Llama-70B submits a wrong answer in essentially every failed run after the fewest steps of any model, giving up early and confidently. Devstral-24B sits at the opposite extreme, spending almost its whole budget before failing and hitting the step ceiling in a substantial minority of runs: it keeps working long past the point of progress. Stopping too early against working too long is the axis that separates the models, and Section 6.4 shows it is the one that predicts which architecture helps each of them.

6.3.2 Inside the dominant failure: four shapes, and they track capability

Wrong submission dominates every model's failures, but on its own that is a shallow finding: the termination class says the agent stopped and handed in something wrong, not *why* the answer was wrong. The trace can say more. The deterministic rule of Section 5.6 reads each submitted-wrong baseline run from three mechanical signals: how long it ran, whether it kept repeating one command, and how many of the graders' checks its answer eventually passed. It sorts the 268 of them into four shapes, with no language-model judge.

Table 6.11: What a wrong submission actually was, read mechanically from each model's failed baseline runs that ended in one (Section 5.6): *bailed* if it quit within ten steps below the near-miss bar, *looped* if it spent itself repeating one command, *off-track* if a full-length attempt still passed under half, *near-miss* if it passed at least half the graders' checks. Ordered by baseline capability.

Model	Wrong subs.	Bailed	Looped	Off-track	Near-miss
Qwen2.5-Coder-7B	39	49%	5%	23%	23%
Llama-3.1-70B	58	34%	9%	21%	36%
Qwen2.5-Coder-14B	47	15%	23%	40%	21%
Devstral-24B	42	0%	71%	14%	14%
Qwen2.5-72B	33	21%	15%	24%	39%
Qwen2.5-Coder-32B	21	5%	48%	29%	19%
Gemini-3.1-FL	28	4%	21%	29%	46%

By name, the three `heterogeneous-dates` trajectories of Section 6.3.1 trace the same axis: the 7B's fast step-past has the shape of a *bail* but lands correct, the 32B *looped* its budget away, and Gemini-3.1-FL returns a solve that would have read as a *near-miss* had it failed. Only the 32B is a wrong submission the taxonomy classifies; the other two are correct submissions, and the point is that one termination class spans all three.

Pooled, the four are close to even (21, 26, 25 and 28 percent). Split by model they are not (Figure 6.3.1), and the pattern is the finding: **the shape of a model's typical failure shifts with its capability**. The weakest models *bail* (nearly half of Qwen-7B's wrong submissions are quit within ten steps) while the strongest fail by *near-miss*, nearly half of Gemini-3.1-FL's already most of the way to a solve. Across the seven models the bailed share falls steeply with baseline capability ($\rho = -0.71$); the near-miss share moves the opposite way at the endpoints, but its rank trend across all seven is weak ($\rho = +0.21$), blunted by the non-monotone middle. The one notable exception is Devstral-24B, which never bails: it fails by the opposite pathology, *grinding* its whole budget in a loop. Looping is the single largest share of both its wrong submissions and Qwen-32B's, so a weak model can fail by quitting too early or by never quitting at all.

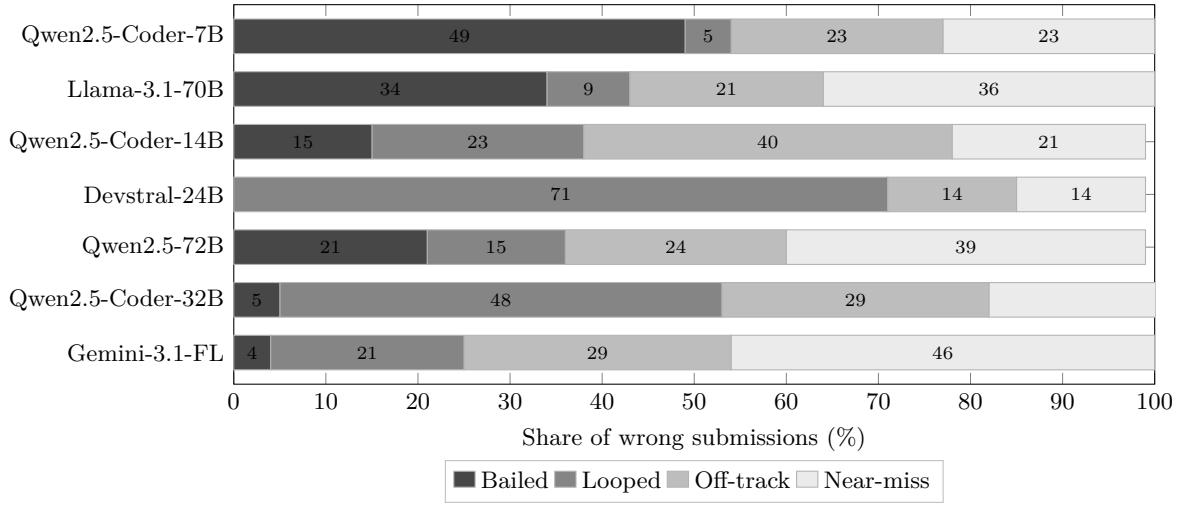


Figure 6.3.1: Failure-mode composition of each model's wrong submissions, weakest model on top. Reading down is reading rising capability: the bailed band gives way to near-miss.

This is the revisability gap, read straight off the failures. This shift is the mechanism behind the capability gradient of Section 6.2.2, stated in the language of failure. A bailed failure leaves the *largest* distance between the answer submitted and a correct one, because the model quit within a handful of steps having barely started, so a second attempt has almost everything still to do. A near-miss leaves the *least*, because the answer is already most of the way there. As capability rises, a model's characteristic failure slides from the first toward the second, and the room any architecture has to work slides with it. That is why the same scaffold helps the weak models and burdens the strong ones: not because weak models are easier to improve in the abstract, but because their typical failure *leaves more of the task unattempted*, and almost any intervention that keeps the agent working recovers some of it. The gradient of Section 6.2.2 counted how many architectures beat each baseline; this says what, underneath, they were beating.

A closer look at the looped sub-mode. Looping is worth isolating, because unlike the other three it is a behaviour rather than an outcome, and it is a property of the *model*. Counted across *all* of a model's baseline runs (not only the wrong submissions of Table 6.11) its incidence still varies widely under the identical baseline configuration (Table 6.12), and the order is not the capability order.

Table 6.12: Runs containing a stuck action loop — five or more identical consecutive shell commands — among each model's *baseline* runs. One configuration for every model, so the spread is a property of the model; looping runs almost never recover, and the order is not the capability order.

Model	Runs with a loop	Share	Of those, failed
Llama-3.1-70B	5	8%	100%
Qwen2.5-Coder-7B	6	9%	100%
Qwen2.5-72B	12	19%	100%
Gemini-3.1-FL	12	19%	67%
Qwen2.5-Coder-14B	16	25%	94%
Qwen2.5-Coder-32B	34	53%	82%
Devstral-24B	41	64%	93%

Devstral-24B loops in nearly two-thirds of its baseline runs and Qwen-32B in half, where Llama-70B and Qwen-7B loop in under one in ten, and a looping run almost never recovers, failing four times in five or more. The termination class hides all of this: a majority of looping runs, 51%, end as an ordinary *submission*, recorded as the agent deciding it was finished rather than as the spent motion it was. Whether any architecture can break these loops is a question for Section 6.3.4, once the design intent is on the table.

6.3.3 How the architectures fail

Sections 6.3.1 and 6.3.2 took the baseline's failures apart from the outside and the inside: how its runs end, and what its wrong submissions are underneath. This section applies the same two instruments once a scaffold sits on top of the model, pooled by architecture family and set against the bare baseline: how runs end (Table 6.13) and what the wrong submissions become (Table 6.14). The families fail about equally often (the null main effect of Section 6.1.1 from another angle), so what follows is not *how much* they change the outcome but *how* they change the failure.

How runs end, by family

Table 6.13: How runs end, per architecture family, as a share of that family's *failed* runs, with the bare baseline as the reference (the same failures-only basis as Table 6.10). *Ran out* is the step and wall-clock budgets; *other* is mostly context overflow, a harness-epoch artifact (Section 8.2); *new failure modes* are the cycle cap and coordination failure, which only a multi-agent scheme can reach.

Family	Runs	Wrong submission	Ran out	Other	New failure modes
Bare baseline	348	77%	8%	13%	—
G1 prompt / temperature	699	87%	11%	2%	—
G2 pre-execution	1414	85%	13%	2%	—
G3 verification	1376	38%	59%	3%	—
G4 multi-agent	1041	62%	24%	0%	13%

Table 6.13 pools the same termination classes by family, over each family's *failed* runs, with the bare baseline for reference. G1 and G2 leave the ending where the baseline left it (wrong submission dominates, running out stays a minority) because neither acts at the moment of submission: G1 only asks the model, in words, to check, and G2 targets a possible *cause* of a wrong answer rather than the ending itself. **G3 is the family that moves the distribution: wrong submission falls by more than thirty points and "ran out" rises to meet it.** The gate intercepts a submission and spends the remaining budget re-checking it, so a run that would have ended in a confident wrong answer ends instead against the budget, still correcting: the failure is not removed but *converted*, from submitting too early to running out while fixing. Section 6.4.3 measures how often that second pass actually lands.

G4 adds what no single agent can reach: a *new* way to fail, in the last column of the table. G4-Orchestrator ends against the *cycle cap* when its routing does not converge (about a third of its failed runs) and G4-Cooperation and G4-Debate lose a smaller share, two to three percent, to *coordination failure*, one agent waiting on a partner that never confirms. The heaviest architectures do not merely fail to help; they open a failure mode, under the same budget, that a single agent cannot, though the cycle cap is a matter of routing competence rather than something intrinsic to orchestration (Section 6.6.2).

What a wrong submission becomes

The termination class stops at "submitted wrong". Reading those wrong submissions with the four mechanical modes of Section 6.3.2 (bailed, looped, off-track, near-miss) shows what a scaffold does to the *shape* of a failure, not just its rate. Table 6.14 carries the composition down the families and Figure 6.3.2 draws it.

Table 6.14: The four sub-modes of Table 6.11, now read over each family's wrong submissions rather than each model's. Cooperation and debate are omitted: their repeat counter does not survive the two-agent hand-off, so looped cannot be measured (Figure 6.3.3).

Family	Wrong subs.	Bailed	Looped	Off-track	Near-miss
Baseline	268	21%	26%	25%	28%
G1 prompt/temp	609	17%	28%	31%	24%
G2 pre-execution	1198	13%	29%	32%	27%
G3 verification	520	22%	12%	27%	39%
G4-Orchestrator	105	4%	15%	36%	45%

The chart makes the pattern legible before any number is read. The top three bars (baseline, G1, G2) are almost the same bar: the dark bands on the left (bailed and looped, the cheap failures) and the light

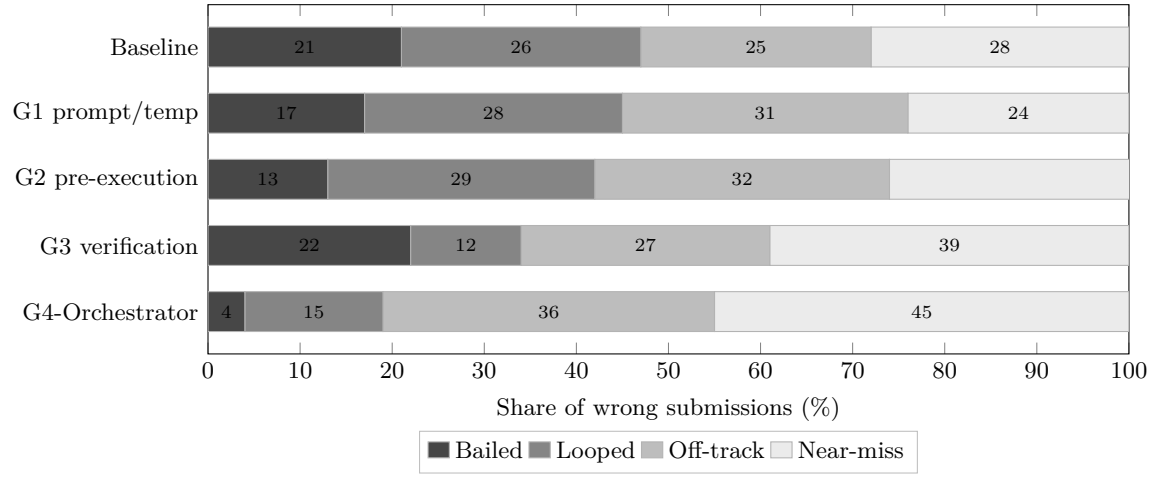


Figure 6.3.2: Failure-mode composition by family, over each family's wrong submissions. Reading top to bottom is reading increasing intervention: the bailed and looped bands shrink and the near-miss band grows, but no family closes the gap to a solve.

near-miss band on the right sit at the same widths. Only at G3 and, more sharply, at orchestra do the dark bands contract and the near-miss band stretch to the right. Two groups emerge: families that leave the failure where they found it, and families that shift it toward the solve line without reaching it.

The single-agent and pre-execution families leave the composition almost exactly where the baseline left it: bailed, looped, off-track and near-miss shift by at most a few points from G1 to G2. This is the mechanical form of the termination reading just above: these families left the *ending* where they found it, and they leave its *shape* there too, since the shape of a model's failure is a property of the model, and a prompt or a grounding phase changes how often the opening appears, not what the failure is once it does.

Verification and orchestration are the two that move it, and they move it the same direction. Under G3 the looped share of *submissions* halves, from 26% at baseline to 12%, but this is reclassification, not repair. Intercepting a submission sends the agent back to grind against the budget, so the run keeps looping: the loop chart below (Figure 6.3.3) shows the overall loop rate in fact *rises* under G3, but it no longer ends by submitting, so it drops out of this population; that is the same "ran out" rise the termination table recorded. What still submits lands closer to correct, with near-miss climbing from 28% to 39%. The orchestrator pushes it furthest, to 45% near-miss, with its bailed share falling to 4%.

This shift toward near-miss is the same move capability makes one level down: in Section 6.3.2 a *stronger model* failed nearer to a solve; here a *heavier scaffold* pushes the failure nearer. And both stop at the same wall. Pushing a failure from bailed to near-miss is what "help" looks like mechanically, but in neither case does it carry the run across the last checks into a solve, which is why the near-miss band grows without the solve rate following it. Cooperation and debate are absent from the table for the reason the caption gives: the repeat counter that defines looped does not survive their two-agent hand-off, so their wrong submissions cannot be shaped by this instrument, only counted.

6.3.4 Did the architectures repair what they were built for?

Each architecture was built against one of the coarse failure modes of Section 4.4. Two of those modes have a mechanical signature that a wrong submission can carry, and it is on that signature the design intent can be tested directly:

Coarse design mode (Section 4.4)	Mechanical signature (Section 6.3.2)
Wrong submission	*bailed* in its sharpest form, and its fuller shapes off-track and near-miss
Weak grounding	*bailed*: acting before understanding produces the same premature wrong submit
Loops / stagnation	*looped*
Context exhaustion	no submission signature: a run that overflowed never submitted

Context exhaustion is the one target with no measurable counterpart, and the closing paragraph returns to why no scaffold repairs it. For the rest the design can be tested by a directed question asked architecture for architecture: did the one built to suppress a failure actually suppress it? Table 6.15 reads, for all fourteen in the order of the design table, the incidence of each architecture's target mode against the baseline rate.

Table 6.15: Did each architecture suppress the failure it was built for? Each row takes the coarse target the design chapter assigned it (Table 4.3), maps it to the mechanical mode that is its sharpest measurable form (a wrong submission → bailed, loops → looped), and reads that mode's incidence among the architecture's runs against the baseline rate. Ordered by family. The Δ cell is shaded by outcome: green reduced its target by at least four points, amber by less, red increased it. Cooperation and debate carry no measurable counter — the repeat counter does not survive their two-agent hand-off — so their loop target cannot be scored.

Architecture	Grp	Built to	Target	Base	Arch	Δ
Prompt	G1	ground and verify before submit	Bailed	13%	12%	−1
Temp	G1	perturb to escape a stuck path	Looped	28%	27%	−1
Explorer	G2	inspect the input first	Bailed	13%	10%	−3
Planner	G2	decompose a complex task	Looped	28%	30%	+2
Analyst	G2	ground with summary and plan	Bailed	13%	8%	−5
Monitor	G2	watch and break stagnation	Looped	28%	26%	−2
Instruction	G3	re-verify from the instruction	Bailed	13%	8%	−5
Reflexion	G3	reflect, then re-verify	Bailed	13%	8%	−5
Judge	G3	advise against the unchecked submission	Bailed	13%	9%	−4
Gate	G3	block the unchecked submission	Bailed	13%	1%	−12
Cooperation	G4	relay between two agents	Looped	—	—	—
Debate	G4	critique across two agents	Looped	—	—	—
Orchestra	G4	route a complex task to specialists	Looped	28%	31%	+3

Of the eleven architectures whose target can be measured, one moves it *substantially*, and the reason is a design choice the other ten either weaken or lack. Four more (G2-Analyst, G3-Instruction, G3-Reflexion, G3-Judge) reduce their target by four to five points, but the gate is in another class: it cuts bailing from 13% to 1% (−12 points), where the next-largest drop in the table is only 5 points. The gate's distinction is that it *blocks*: a run that tries to submit early is stopped and made to continue. Its neighbour the judge was built for the same failure with the same information and moves it a third as far (−4), because it only *advises*: it appends a warning the agent is free to ignore, and mostly does. The gap is specific to the premature submission: measured on the *total* wrong-submission rate the two are indistinguishable, each converting about thirty points of it into runs that end against the budget (−30 for the gate, −31 for the judge), so what separates them is not how much they intercept but whether they stop the agent from bailing early. The two sit adjacent with the same intent and a factor-of-three gap on that sub-mode, and the only difference between them is whether the mechanism can override the agent or only talk to it. That is the chapter's clearest statement that, for these models, a scaffold that blocks acts and a scaffold that advises does not.

Read down the target column, the table tells a second story the design did not intend. **The architectures split cleanly by which failure they aim at, and the two halves fare oppositely.** Every architecture aimed at premature submission (the bailed-targeters) at least moves it in the right direction, on a ladder that tracks how hard the mechanism pushes: a prompt directive barely dents it (G1-Prompt, −1), a grounding or re-check pass takes a few points (G2-Explorer −3, G2-Analyst and G3-Instruction and G3-Reflexion around −5), and only the block clears it. The other half (the four architectures built against the loop) fails outright, and the loop deserves the closer look, because it is the failure a reader most expects a scaffold to fix.

It is both frequent and hard to escape: **2065 of 7404 traced runs (27.9%) spend themselves re-issuing a single command, and 89% of them fail.** The repeat policy is byte-identical across every configuration (a nudge at the third identical action, and the loop is never force-terminated) so any difference in Figure 6.3.3 is the architecture, not the rule.

No architecture repairs it. G2-Monitor, whose entire purpose is to notice stagnation and interrupt it, manages -2.2 points, and 9 of the 11 measurable architectures raise the rate instead, the gate most (+8.7), because intercepting a submission sends the agent back to re-issue commands it has already

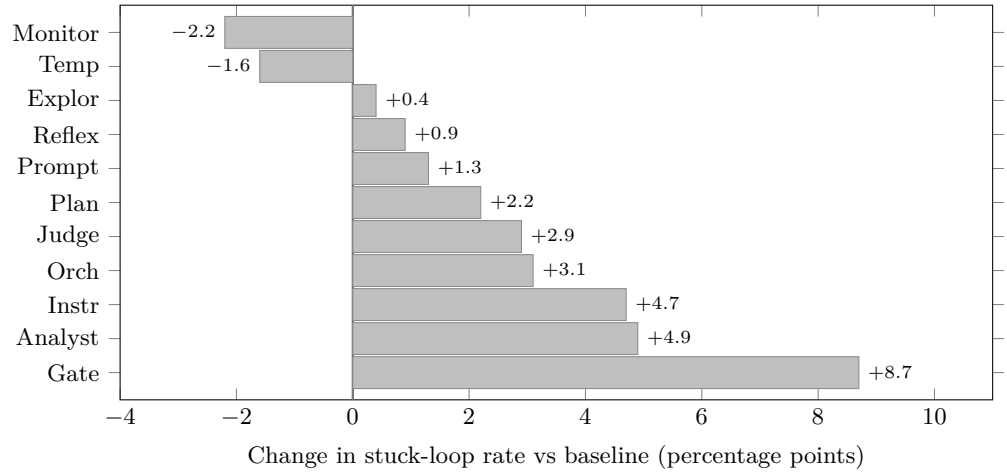


Figure 6.3.3: Change in each architecture's stuck-loop rate against the baseline, under a repeat policy byte-identical across all configurations, so any difference is the architecture. Left of zero is fewer loops. Only the monitor — the one design built against stagnation — and the temperature ablation reduce looping, and only barely; the rest raise it, the gate most. Cooperation and debate are unmeasured: their repeat counter does not survive the two-agent hand-off.

issued. That is also where the gate's suppressed bailing goes: into looping rather than into solves. This is the chapter's starkest case of detection and repair coming apart, since Section 6.6.3 shows the monitor *identifying* the stagnation correctly while the executor continues regardless. The intercept is real; the cure is not, and the one failure most visible in the traces is the one the fourteen architectures collectively leave untouched.

Looping does not, finally, explain which models architecture helps. If it did, the models that loop most would gain most, and they do not: Llama-70B loops least yet gains the most from architecture (+3.15 tasks on average), while Devstral-24B loops most yet gains less (+1.23). Looping is a mechanism of individual failures, not the reason architecture helps some models and not others.

The same conclusion appears from the solve side, and this is where the four sub-modes finally meet the outcome. For every model–task pair that flips from failure to solve under some architecture, the sub-mode of the baseline failure it started from tracks the overall failure mix (Table 6.16): **each sub-mode is rescued in rough proportion to how often it appears**, and not even the near-misses that sit closest to a solve are rescued more than their share. Architecture does not specialise by which failure it converts; it differs by *how often* a rescuable failure appears, set by the model's profile, the same reading the directed scorecard gave, now from the solve side.

Table 6.16: For every model–task pair that flips from failure to solve, the sub-mode of the baseline failure it started from, against how often that sub-mode occurs among all baseline failures. The two columns track each other: no sub-mode is rescued out of proportion to how often it appears — except context exhaustion, rescued least.

Baseline failure	Of the 96 flips	Of all baseline failures
Bailed	16%	16%
Looped	24%	20%
Off-track	20%	20%
Near-miss	23%	22%
Ran out	8%	10%
Context	9%	13%

The one exception bounds the whole family of methods. **Context exhaustion is rescued least** (9% of flips against 13% of failures) because a run that has already exceeded its window has lost the information a second pass would need. The remedy for context exhaustion is a larger window, not a scaffold.

6.3.5 The answer to RQ2

Runs do not mostly fail by running out of budget; they fail because the agent stops and submits a wrong answer, which is the largest single category in every model. That is why post-execution verification, the one family that acts at the moment of submission, is the best-targeted design, and why simply granting more steps would not have helped. Underneath that shared dominant mode the models differ sharply, from Llama-70B, which abandons a task after a handful of steps, to Devstral-24B, which grinds its whole budget away, and the difference is measured independently of how much each model solves. Read one level deeper, the wrong submissions themselves fall into four mechanical shapes, and which one dominates tracks capability: bailed failures, where the model quits within a handful of steps having barely started, fall away as capability rises ($\rho = -0.71$), and near-misses, one check short of a solve, become the characteristic shape at the capable end ($\rho = +0.21$, weaker by rank). That is the revisability gap of RQ1 restated as a failure mode: a weak model fails with more of the task still unattempted, so almost any scaffold that keeps it working recovers some of it, while a strong model's near-miss leaves only a narrow gap that a broad intervention overshoots, so the same scaffold that helps the weak burdens the strong, and only a targeted check at the point of submission (the verification family, which alone helps Gemini-3.1-FL) fits a gap that small. That independence is what lets the next section treat the match between a model's failures and the architectures that help it as a finding rather than a restatement.

The design intent, though, was only half met, and testing each architecture against the failure it was built for says how. Only one, the gate, suppressed its target substantially, and only because it can *block* the agent rather than advise it: the neighbouring judge, same intent and same information but advisory, moved the same failure a third as far. The architectures aimed at the loop, the failure most visible in the traces, repaired it nowhere. And where a scaffold does convert a failure into a solve it does not specialise: because premature submission dominates every model's failures, every family converts mostly that same mode, differing in *how often* they get the chance, set by the model's profile, not in *which* failure they fix. The one mode none of them converts is context exhaustion, because a run that has lost its window has lost the information a second pass would need.

6.4 What makes an architecture help

Section 6.2 read the grid by model and Section 6.3 by failure. This section reads it by its third axis, the *architecture* itself: setting aside which model ran and which task it faced, what makes one configuration help where another hurts, beyond its family and its apparent complexity? Each of the 14 is taken in the order it was introduced (Section 4.5) and given a verdict against every model it met, asking the same three things each time: what it was designed to do, whether it beat each model's *own* baseline, and where and by what mechanism it helped or did not. Read as a set (Section 6.4.5) the fourteen arrange along a single organizing axis, and it is neither complexity nor family.

6.4.1 G1: the single-agent ablations

The two G1 configurations add no machinery, changing only the agent's instructions or its sampling temperature, and they set the floor the elaborate architectures must clear.

G1-Prompt: how far three sentences go

G1-Prompt adds three directives to the system prompt (inspect the input before writing code, write solutions to files rather than inline commands, verify before submitting), with no model call and no budget spent (Section 4.6). It measures how much of the benefit attributed to elaborate scaffolding is already available from instruction alone.

On balance it helps, and cheaply. G1-Prompt beats its own baseline on **five of the seven models**: Qwen-7B +1, Qwen-14B +2, Qwen-72B +2, Llama-70B +2 and Devstral-24B +1. On Qwen-72B it is the best configuration in the entire grid, matching or beating every G2, G3 and G4 scaffold at no extra cost; on Qwen-14B only G3-Judge edges past it. That the cheapest possible intervention is the best on a model at all is itself the result: it sets how much an elaborate scaffold has to add before its budget is justified, since part of what the heavier designs supply is already reachable from three sentences.

The wins concentrate in data-transformation tasks, the `data-science`, `file-operations` and `data-processing` categories whose work is exactly what the first directive names, inspecting the input before writing code: `pandas-etl` (flipped on three models), `create-bucket`, `analyze-access-logs`,

`log-summary`, where reading the actual columns before writing code turns a near miss into a solve. The losses are just as structured. Two tasks go repeatedly, `extract-safely` and `recover-obfuscated-files`, each lost on three models, both short enough that a model already solves them at baseline and the added deliberation only displaces a four-command answer. The two models it hurts are the two with the least room: Qwen-32B, the strongest open model, at −4, and Gemini-3.1-FL at −2. This is the headroom account of Section 6.5.3 seen through one architecture: instruction helps a model that was falling short and taxes one that was already there.

The directives change behaviour measurably. Averaged over runs, G1-PROMPT inspects the input about a third more often than the baseline (read-file calls rise from 1.8 to 2.4), writes to files somewhat more, and deliberates longer, adding roughly five steps at the median. That last figure qualifies the word "free": the prompt costs nothing to add, but the behaviour it induces raises median token consumption by about 47%. On a task with headroom the extra inspection buys a solve; on a short or already-solved task it is the same behaviour, now pure overhead. `log-summary` is the extreme: its demand (read the log files, count, write the result) is almost exactly what the three directives call for, and G1-PROMPT solves it on all seven models where the far heavier G2-ANALYST manages only two.

In sum, the cheapest intervention in the study is among the most reliable on the weak and mid-tier models and on tasks that reward looking before acting. It is not universally good: the behaviour it induces costs real budget, and on strong models or short tasks that cost is not repaid. That the study's cheapest intervention is also the single best architecture on one model (Qwen-72B) and within one task of the best on another (Qwen-14B) is the finding to carry forward: elaboration earns its budget only where three free sentences fall short.

G1-Temp: stochasticity as an isolated control

G1-TEMP changes one thing from the baseline: it samples at temperature 0.2 rather than the baseline's temperature of zero (Section 4.6). The intent is to break *fixation*, to stop a model that re-issues the same action turn after turn and let a little randomness push it onto a different path. It is the only architecture that attempts this without touching the agent's context: every other design changes what the model sees, whereas G1-TEMP changes only which action it samples from an unchanged state. That makes it a clean test of one question: how many failures are *deterministic dead ends*, where the agent is stuck not for want of information but because deterministic decoding keeps producing the identical wrong step?

For most models the dead ends are few, so the noise mostly hurts: G1-TEMP is negative on **six of the seven**, because on a task with one correct action a random perturbation is likelier to spoil it than to improve it. The seventh earns the architecture its place. On Llama-70B it gains +4, and not from one lucky run: it flips five tasks to solved (`analyze-access-logs`, `catch-me-if-you-can`, `fix-permissions`, `heterogeneous-dates`, `log-summary`) against a single regression, and does so without lengthening the run at all, its mean step count sitting within a fraction of a step of the baseline's. No information was added and no extra work done; only the sampled action changed, and five tasks the deterministic baseline could not escape were solved.

One task shows the dead end concretely, on a different model. Devstral-24B at baseline fails `modernize-fortran-build` (switch a Makefile to `gfortran` and build) by hitting `missing separator` **41 times in 60 steps** and never escaping: `make` requires a literal tab before each recipe line, and Devstral-24B's `write_file` deterministically emits spaces, so the identical greedy write reproduces the identical error every turn. Its own tool cannot express the character the build demands, and the failure is recorded as the model's. The task is solved under exactly four of Devstral-24B's fourteen configurations (G1-TEMP, G3-JUDGE, G4-COOPERATION, G4-DEBATE) which are the ones that break determinism: G1-TEMP re-samples the write, and the other three hand a fresh or critiqued attempt a chance to write the file a different way. This makes visible what the closing paragraph claims: the heavier designs escape a deterministic dead end incidentally, by changing the state that G1-TEMP simply re-samples, rather than by their own logic.

Temperature does break fixation, but only where fixation, not incapability, was the problem, which is one model out of seven. Its value is diagnostic rather than practical: it shows that a measurable slice of Llama-70B's failures are deterministic dead ends a re-roll can escape, needing no information added at all. That both bounds what the heavier, information-adding architectures can be credited with (these particular escapes were free) and names a failure mode they address only by accident, since they all work by changing the state rather than resampling from it.

6.4.2 G2 context augmentation: it works only when it runs

Before asking whether G2 helps, one must ask whether it runs. Each G2 architecture produces a structured artifact (a Data Summary, a plan) that either validates or does not, and an artifact that does not validate is a scaffold that never ran.

Table 6.17: Whether the G2 sub-agent produced a *usable* artifact. Percentages are of runs in which the sub-agent ran at all. Models are ordered by baseline capability, which is visibly not the ordering of this table.

Model	Baseline	Explorer summary	Planner plan	Analyst summary
Qwen2.5-Coder-7B	4	31%	**0%**	27%
Llama-3.1-70B	5	86%	**95%**	81%
Qwen2.5-Coder-14B	9	41%	**5%**	41%
Devstral-24B	12	50%	98%	6%
Qwen2.5-72B	16	61%	97%	58%
Qwen2.5-Coder-32B	19	56%	59%	59%
Gemini-3.1-FL	34	83%	84%	45%

The answer is frequently no, and it does not track capability (Table 6.17): where a model sits on the capability axis barely predicts how often it complies, the two ranks correlating at only $\rho \approx +0.2$ to $+0.4$. Inside the Qwen coders the two smaller models almost never emit a usable plan (0% and 5%) while the two larger ones usually do, and Llama-70B, near the bottom of the range by solve rate, complies in 95%. Two different failures hide behind a low rate. On the weak models the sub-agent genuinely cannot produce well-formed output, since instruction-following is a capability benchmarked apart from task-solving precisely because it does not follow from it [25], [26], and the harness's extraction parser is hardened so that a low rate is the model's own, not an artifact of a fragile reader (Appendix A.3). A capable model fails otherwise: Gemini-3.1-FL's analyst validates in only 45% of runs, but the phase logs show this is not an inability to format: its invalid runs share one signature, in which the model inspects the files, reports it has "completed the analysis", then emits a structured artifact that fails to parse and, never noticing, spends the rest of the phase believing it has finished. Either way the executor is left ungrounded, which is why *the architecture never ran as designed* is a large part of what Table 6.1 scores as a G2 penalty: on Qwen-14B under G2-Planner an invalid plan even degrades to a single subtask that copies the task instruction back verbatim (*"Complete the task: ..."*), handing the executor its own prompt.

When the artifact does validate, it changes the outcome, and the comparison is built to rule out the obvious objection, that valid artifacts merely come from the stronger models, which would have solved more anyway. Rather than compare one model with another, it compares each model against *itself*: its runs where the sub-agent produced a valid artifact against its runs where it did not. Qwen-32B under G2-Planner solves 37% of its tasks when the plan is well-formed and **none at all** when it is not; pooled across the grid, valid-artifact runs solve 25% against 13%. The effect is consistent, not the average of a few lucky cases: of the eighteen model-and-architecture pairings that had enough runs of each kind to compare, the runs with a valid artifact solved more often than the runs without in fifteen.

This still does not license the tempting stronger claim, that G2 would have beaten baseline had its artifacts always validated. Validity is not handed out at random: a clean artifact comes more readily on a simple task, which was more likely to be solved regardless. The claim that does hold is narrower and still substantial: *when the scaffold actually runs, it helps, and much of the measured G2 penalty is not the idea failing but the scaffold never executing.*

G2-Explorer: grounding as a phase, and its two gates

G2-Explorer promotes ReAct's observation-before-action discipline into a separate read-only phase: an Explorer restricted to `bash`, `read_file` and `think` inspects the task directory and emits a structured JSON summary that is injected into the executor's context (Section 4.7), making grounding an architectural guarantee rather than something the model may skip. One self-imposed limit bounds it: to protect the budget the Explorer never reads the middle of a file longer than 80 lines, so on any real input its summary can be incomplete.

On balance it does not. It is negative on **five of the seven models**, and more sharply than G1-

PROMPT. The two it helps are the two weakest, Qwen-7B +2 and Llama-70B +4; every stronger model loses, Qwen-32B most at −6.

`log-summary` tells the whole story in one task. Under the Explorer it flips to solved on Llama-70B and Devstral-24B and away from solved on Qwen-14B, Qwen-32B and Qwen-72B: the same task and architecture moving in opposite directions by model. A separate read-only pass over the log rescues a weak model that was not inspecting on its own and costs a strong model, already solving, a phase it did not need. The other wins are the familiar data-transformation tasks (`csv-to-parquet`, `pandas-etl`, `create-bucket`) on weaker models, and the repeated losses are again `extract-safely` and `recover-obfuscated-files`, short tasks where a whole grounding phase is overhead. It is G1-PROMPT's split, helping the weak and taxing the strong (Section 6.5.3), but harsher, because a full read-only phase costs far more budget than three sentences: it helps only two models where G1-PROMPT helped five.

Two gates must both open for the Explorer to help: it must produce a valid summary, and the executor must be able to act on it. Both bind. On the weakest models the grounding phase often emits nothing usable, with summary validity falling as low as 31% on Qwen-7B (Section 6.4.2); where it does land it roughly doubles the solve rate, as valid artifacts do across G2. But even a valid summary is limited. It is a lossy compression (a header, a few sample rows and a row count) so on a data-heavy task it cannot stand in for the file: on Gemini-3.1-FL's `predict-customer-churn` it reduces a training set of several thousand rows to two examples, and the executor reads the file itself regardless, leaving the grounding phase redundant rather than additive. And where the obstacle is not in a file at all (an uninstalled solver, the state of a git repository) a file inspector grounds in the wrong thing: on `fix-git` the Explorer catalogues the visible files, reports no action is needed, and the executor, pointed there, never runs the `git reflog` the baseline used to recover the lost commit. There is also a looser tendency the traces suggest but do not settle: about half of the tasks the Explorer loses are submitted in fewer steps than the bare model needed to solve them, as though a confident-looking summary invites the executor to commit sooner and stop short, while the other half ground and then grind to the budget, so it reads as a pull rather than a rule.

The mechanism is real but aimed at the wrong average: the grounding phase runs on *every* task, those with no headroom, the short ones it only slows, the strong models already inspecting, so its cost is paid everywhere while its benefit lands only where a weak model meets an inspectable task. Its per-instance mechanism is sound and its aggregate verdict negative, and the 80-line reading ceiling is one concrete, fixable reason part of even that benefit never arrives.

G2-Planner: the decomposition that adds nothing to grounding

G2-PLANNER adds a decomposition phase on top of the Explorer (Section 4.7): after grounding, a think-only Planner (it cannot run commands, write, or submit) emits a JSON plan of one to four sequential subtasks, each with a success criterion, which a per-subtask executor then works through. An invalid plan falls back to single-subtask mode, so the *plan's structure* can never be worse than the Explorer's ungrounded pass, but the fallback still spends the planning phase's steps first (a median of three), so the *outcome* can regress below the Explorer even when the plan does not. Since G2-PLANNER is G2-EXPLORER plus that phase, the comparison between them isolates what the plan itself is worth.

Against baseline it does not, staying negative on **five of the seven models** on the same split as the rest of G2: it helps the two weakest (Qwen-7B +2, Llama-70B +2) and taxes the five stronger, Qwen-32B worst at −5. The sharper comparison is against its predecessor, since it isolates the plan: **G2-Planner and G2-Explorer net to zero across the seven models**, differing by more than a single task on only one (Llama-70B). Adding an explicit planning phase on top of grounding changes essentially nothing.

The plans themselves are not bad, and that is what makes the null result informative. Table 6.18 reads the decomposition off the models that can plan (the two smallest Qwen coders barely emit a valid plan and are set aside). Three things stand out. First, **the plans track difficulty**: every model splits the hardest tasks into about twice as many subtasks as the easiest (roughly two against one), so the decomposition is aligned with where work is actually needed rather than arbitrary. Second, **it still does not pay**: the change against the Explorer sits between −2 and +1 for all five, so however sensibly a task is divided, dividing it converts almost nothing. The plans are shallow (a median of one to two subtasks against a cap of four) and describe work the grounded executor would have done anyway, so a valid plan adds no information the Explorer had not already supplied. Third, **the division is nominal, not real**: in runs with two or more subtasks the busiest sub-executor takes *over half* the steps (52–61% across models), so the "equal sequential parts" collapse in practice onto one part doing most of the work while the others

Table 6.18: G2-Planner decomposition, over valid plans only. *Hard → easy*: mean subtask count on the hardest tasks (at most one baseline solver) against the easiest (at least five); a gap means harder tasks are split into more parts. Δ columns compare solves against baseline and Explorer on the valid-plan tasks only. *Busiest share*: fraction of executor steps taken by the most active sub-executor when a plan has two or more subtasks. The two smallest Qwen coders, which rarely emit a valid plan, are omitted.

Model	Valid plans	Mean subtasks	Hard → easy	Δ vs base	Δ vs Expl.	Busiest share
Llama-3.1-70B	60	1.9	2.1 → 1.1	+2	−2	59%
Devstral-24B	63	1.8	2.0 → 1.0	−3	−2	57%
Qwen2.5-72B	62	1.9	2.2 → 1.0	−4	+0	56%
Qwen2.5-Coder-32B	38	2.2	2.6 → 1.1	−4	+1	52%
Gemini-3.1-FL	54	1.6	1.8 → 1.1	−2	+0	61%

contribute little.

That last point exposes a structural cost the design shares with the multi-agent family. The subtasks run *sequentially*, each executor handed only a compressed summary of the previous one's result rather than its full trace. This linear hand-off both fragments a task the model could hold in one context and loses a little detail at every boundary, since each executor sees a summary rather than what actually happened. It is a cost G4-Cooperation pays too (Section 6.4.4). Whether a different split (parallel sub-executors, or an executor allowed to revise the plan mid-run) would fare better this grid cannot say; it tests only the fixed sequential decomposition.

A model that plans its own work gains nothing from writing the plan down: the plan repeats what the grounded executor would have done anyway, so the whole G2 story reduces to the Explorer's. The little that pre-execution augmentation buys comes from grounding, and the planning layer on top is overhead on the strong models and a no-op on the weak ones. Planning can still help when it comes from elsewhere: Section 6.6.3 shows that a plan written by a stronger model lifts a weaker worker, and the value lies in the plan's content, which a self-plan cannot exceed but a better planner can supply. The homogeneous and heterogeneous results agree, then, that a plan is worth the capability of whoever writes it.

G2-Analyst: the best pre-execution design still cannot clear baseline

G2-Analyst does in one phase what G2-Explorer and G2-Planner do in two (Section 4.7): a single read-only pass emits both a data summary and a grounded execution plan, so a plan written by the agent which inspected the files is grounded by construction. It asks the sub-agent for more in a single artifact than any other G2 design, which is the source of both its gains and its cost.

It is the most successful of the three grounding-phase designs by a clear margin: it **helps or leaves unchanged four of the seven models** and is negative on only **3**. It helps Qwen-14B +2 and Llama-70B +3, is flat on Qwen-7B and Gemini-3.1-FL, and still taxes the strong open models, Qwen-32B most at −5. Against its own predecessors the gain is clear: **G2-Analyst beats both G2-Explorer and G2-Planner by seven solves across the seven models**. Merging grounding and planning into one grounded pass recovers what the two-phase versions leave on the table.

Its cost, and why it still loses, follows from the same design. Demanding two structured artifacts from one pass makes the schema harder to satisfy: a valid summary is produced in **45%** of G2-Analyst runs against **58%** for the single-artifact G2-Explorer, and on Devstral-24B the rate collapses to 6%, a model that cannot emit two artifacts in one structured turn. The interface is the binding constraint, not the content: Section 6.6.3 shows that analysis is the one role a stronger supervisor cannot rescue, for the same reason: Gemini writes more into the combined artifact and so validates *less* often than the worker it replaces, its thoroughness turning into schema failures exactly where the interface is tightest. A better planner upstream does not help when the bottleneck is fitting more into one structured turn.

The best pre-execution architecture in the study is still net-negative against baseline. The one-pass grounded plan improves on separate grounding and planning phases, worth seven solves over each, yet even the family's strongest member cannot clear the floor, because the G2 premise is to spend a phase before execution and that phase is only repaid where grounding was the missing ingredient. G2-Analyst marks the ceiling of the idea: better engineering inside the pre-execution paradigm buys ground against the other pre-execution designs and none against doing less.

G2-Monitor: the one G2 that helps, and only where off-track is visible

G2-Monitor is the only G2 configuration that intervenes *during* execution rather than before it (Section 4.7). Every ten steps a tool-less Monitor receives the full action history; if the run is on track it returns a sentinel and nothing is injected, so the mechanism is free when it is not needed. Off track, guidance is injected as the executor's own `think` call rather than as an outside instruction (inception prompting), and the Monitor escalates its authority when advice is repeatedly ignored.

It is the **only G2 architecture with a positive mean effect**, +0.14 tasks, over a wide spread. It is negative on four of seven models but strongly positive on the weak ones, gaining +5 on Llama-70B and +4 on Qwen-7B, its largest single-architecture wins for either model, and +1 on Qwen-72B. It still taxes the strong narrow models, Qwen-32B worst at −5.

Table 6.19: How the G2-Monitor behaves per model. *Base steps* is the mean steps the bare baseline takes before failing, which caps how many of the ten-step check-ins the Monitor gets (a model that stops at 13 steps allows at most one). *Injections* is the median guidance injections per run; *obeyed* is the share after which the executor's next command differs from the one before it — the rest reissue it verbatim. Δ vs base is the change in solves against the baseline.

Model	Base steps	Injections/run	% injected	Obeyed	Δ vs base
Qwen2.5-Coder-7B	17	2	56%	60%	+4
Llama-3.1-70B	13	1	54%	74%	+5
Qwen2.5-Coder-14B	40	4	73%	61%	−1
Devstral-24B	54	5	88%	42%	−1
Qwen2.5-72B	29	3	66%	65%	+1
Qwen2.5-Coder-32B	41	4	86%	65%	−5
Gemini-3.1-FL	42	1	61%	90%	−2

Which models it helps is predictable in advance. The Monitor's advice is to change approach, and that can only pay if the model has budget left to try another one. So the models it helps should be the ones that *stop early* and leave budget unused, and the ones it hurts the ones that spend their whole budget before it can intervene. The mechanism is visible in the first two columns of Table 6.19: how early a model stops at baseline caps how many times the Monitor can even check in, so Llama-70B, failing after a mean of thirteen steps, allows barely one injection, while Devstral-24B at fifty-four allows five. A single number captures the effect, a model's *spare budget*, the 60 steps minus how many it typically uses before failing at baseline, read off the bare agent, so it cannot be circular. Ranked by it, the Monitor's effect follows almost exactly ($\rho = 0.81$ across the 7 models), turning negative once spare budget drops below about twenty steps. Llama-70B has the most to spare and gains the most; Devstral-24B, which grinds to the budget with only 6 steps to spare, has nowhere to redirect and is the model the rule fits worst, though even it only loses −1 (excluding it, $\rho = 0.94$). This is why the selection rule of Section 6.7.2 can reach for the Monitor on a model it has never scaffolded, from that model's baseline alone.

The wins concentrate where a run can be recognised as off track at a definite point. `flood-monitoring-basic`, a multi-stage pipeline, flips to solved on **four** models, the largest floor rescue in the study, and the other gains are `data-processing` and `data-science` tasks with the same staged structure (`csv-to-parquet`, `jq-data-processing`, `create-bucket`). Such tasks have a legible checkpoint: `flood-monitoring-basic` first interpolates the gaps in the sensor data and then counts the readings above a threshold, so a monitor checking in every ten steps can see that the first stage is done and the second has not started, and say so. A search task offers no such recognisable midpoint (there is nothing an observer can point to as "you are here") which is why the Monitor has nothing useful to inject on one. The losses are the familiar short tasks (`extract-safely`, `log-summary`) with no long trajectory to correct.

Table 6.19 exposes the ceiling on an advice-only intervention. The Monitor does mechanically what it was built to do: on the 78 runs where the bare baseline was looping and failing, its interruption breaks the loop, sharply cutting the repeat count (on Devstral-24B, from fifty-odd identical commands to a handful). But breaking a loop is not fixing the task, and **only 4 of those 78 (5%) became solves**: it can stop the agent repeating itself but cannot supply the action that works, so the run escapes one dead end and fails another way. The reason is that it advises, it cannot enforce: after an injection the executor changes its next command only **62%** of the time, the other 38% reissuing it verbatim. Obedience splits by model (Table 6.19): Gemini-3.1-FL acts on 90% of injections, Devstral-24B (which loops most and is injected most) on only **42%**, so the model that most needs the redirect is the one that ignores it. That injected

runs solve *less* often than un-injected ones (14% against 42%) is selection, not harm: the Monitor injects into runs that are already in trouble. Phrasing matters as much as timing: Section 6.6.3 shows Monitors addressing the agent as "you" succeed where those writing about "the agent" in the third person do not.

It is the one member of its family to clear zero, and only by being conditional: it needs a task with a legible off-track state and a model with budget left to change course. Meet both (a weak model on a staged pipeline) and it produces the study's largest floor rescues; meet neither and its interruptions are a net drag. G2-Monitor is the pre-execution family's one success, and it succeeds by not being pre-execution at all.

6.4.3 G3 verification: the closest thing to a robust improver

Verification is the only family that is not systematically harmful at the top of the capability range. On Gemini-3.1-FL, the three verification variants supply the only improvements over baseline worth the name (G3-Reflexion +4, G3-Gate +3, G3-Judge +2), with every other architecture negative or zero but for a single-task gain from G4-Cooperation. On Qwen-32B, where nothing improves, G3-Judge is the least harmful at −2. Across all models it is the best-performing family on average (Table 6.2).

Does the verifier get to run?

Verification is *conditional* by construction: the verifier can only act if the phase-1 agent submits (leaving something to check) and if budget then remains for it to run. Both conditions bind across all three verifier variants, and the gap between them is the structural limit of the family. (The three differ only in what the second phase does: Instruction and Reflexion hand the work to a fresh agent that closes whatever gap it judges is left, with or without a diagnosis carried over; Judge issues an explicit approve/needs-fix verdict. G3-Gate is a separate mechanism, in its own verdict.)

Table 6.20: Whether the verifier gets to run, and what the Judge decides. *Submitted* and *engaged* pool the three verifier variants; the gap between them is submissions the budget swallowed before the verifier could act. *Approve/needs-fix* are the Judge's verdicts only (the retry variants issue none).

	Executor	Verifier	Judge verdict	
Model	submitted	engaged	Approve	Needs-fix
Qwen2.5-Coder-7B	92%	44%	23%	77%
Llama-3.1-70B	91%	83%	15%	85%
Qwen2.5-Coder-14B	100%	41%	19%	81%
Devstral-24B	83%	24%	33%	67%
Qwen2.5-72B	80%	65%	57%	43%
Qwen2.5-Coder-32B	70%	43%	24%	76%
Gemini-3.1-FL	96%	84%	51%	49%
Pooled	87%	**55%**	34%	66%

Both shares are of all runs: agents submit in **87%**, but the verifier engages in only **55%**. The 33-point gap is runs where the agent submitted and the shared budget was then already gone: something to check, but nothing left to check it with. Put as a conditional probability, of the submissions the verifier intercepts only **63%** leave it budget to act (the two figures, 55% of all runs and 63% of intercepted ones, are the same fact against two denominators). This is the same spare-budget axis that ordered the Monitor, and the verifier engages on Gemini-3.1-FL and Llama-70B, which submit with budget still unspent, far more than on Devstral-24B, which spends its whole budget before submitting and leaves nothing behind for a check. It also explains the termination signature of Section 6.3.3, where G3 has by far the highest rate of runs ending against the step budget. The consequence for reading the G3 deltas is direct: they are not the effect of verification but of *verification that acts on only about two thirds of the submissions it intercepts*, so the family's true ceiling is higher than measured and the binding constraint is budget scheduling, not verdict quality.

When the judge does engage, its verdict leans toward rejection: **66%** ask for a fix and 34% approve. Whether that lean is worth having is the next question.

What its verdict is worth, and for whom

That reject-lean (from 43% to 85% needs-fix across models, partly a biased parser) is useful for a specific reason: *spotting a fault is easier than fixing it.* A weak model judging its own weak work still rejects it in the large majority of cases, since it can see that something is wrong even when it could not have produced the right answer unaided, so the rejection carries real information. That gap between what a model can recognise and what it can generate is what leaves room for revision, the *revisability gap* on which Chapter 7 (Section 7.1) builds its account of the whole grid.

LLM judges carry well-documented biases, toward the first option shown, toward longer answers, and toward output in their own style (*self-enhancement*) [65], [66]. Self-enhancement would have a model wave its own work through; here the self-judge does the opposite, rejecting two-thirds of the time. Part of that lean is built in: the judge replies in free text, and a rule turns that reply into a verdict, reading it as NEEDS-FIX whenever it does not clearly begin with APPROVED, a deliberate safety bias that prefers one more check to a premature pass. Those biases were all characterised for *offline* scoring, where a wrong verdict only mis-ranks an entry; used *in the loop* a wrong verdict instead redirects the run, and its two directions (wrongly approving, wrongly rejecting) are not equally recoverable (pooled over all models the judge issued 246 verdicts, 83 approvals and 163 rejections).

Two in five approvals are wrong. 32 (39%) of the 83 approvals were of work that did not pass the grader when the run was finally scored: the judge looked at broken work and approved it. Because the parser only lets the clearest approvals through, this is a floor: a judge that approved more freely would wave through *more* borderline work, not less. And a false approval is unrecoverable, since the approval ends the run. **Rejections have the opposite problem.** Fewer than a third are acted on: 52 of 163 (32%) ended in a solve, the rest naming a problem the agent could not fix, and their accuracy is not even measurable the way the approvals' is: an approval ends the run, so whether it was right is directly checkable, but a rejection triggers a retry and the benchmark grades only the final state, never the submission the judge saw. An unrescued rejection is therefore ambiguous: it is either a correct diagnosis the agent could not act on, or a false alarm on work that was already right, and the two cannot be told apart from the data.

The split by model gives the family its shape, along two rates that Table 6.20 does not carry: how often a model's *approvals* turned out wrong, and how often its *rejections* were rescued. Llama-70B, one of the weakest, is worst on both: it approves broken work (75% of its approvals failed) and cannot repair what it rejects (only 9% of them rescued), while the strong coders Qwen-32B and Devstral-24B convert 58% and 70% of their rejections into solves. (The per-model counts are small, a handful of approvals each, so these read as direction rather than precise rates.) **A model's value as its own verifier appears to track its capability more closely than its value as an executor does**, which is consistent with verification surviving at the top of the range and doing least at the bottom.

What a judge can and cannot check

What a verdict is worth turns on the task, not the judge: on whether correctness is something the judge can *see*. Where it is a single unambiguous value, recognition beats production and the verdict pays. `mixed-integer-programming` is the strongest case: it wants one number in a file, hard only operationally (a solver that will not terminate in budget, so **87 of 116 runs write no answer at all**); on Qwen-14B the phase-1 agent wrote `14`, the judge, the same model, replied "*the integer part should be 15, not 14*", and the run fixed it in three steps. Where correctness admits of taste the same accuracy becomes a liability: on `multistep-definite-integral` the judge rejected a *passing* answer on stylistic grounds, forcing a needless revision that cost budget the run could have kept.

Between the two sits the failure mode behind most false approvals: a judge confirming an artifact is *present* without confirming it is *correct*. `create-bucket` is the clean case: the task wants a bucket that is *also* public-read; the agent creates it, the judge sees a bucket and approves, and the run fails the public-read check neither tested. Its mirror image is that the most useful verdicts are often the cheapest: on `bank-trans-filter`, a single-check task Qwen-32B solves under G3-JUDGE alone, the judge's whole contribution is "*the file … does not exist*", a check a one-line shell test (`test -s /path`) would have made for free. All of this is the named-artifact effect of Section 6.5.4 from the verifier's side: a judge pays where correctness is a fact it can check, and costs where it is a matter it can only opine on.

The four verdicts below isolate what verification is worth, one component at a time. Three add information in increasing measure: a fresh retry (G3-INSTRUCTION), a fresh retry carrying a diagnosis

(G3-Reflexion), and full continuity with feedback (G3-Judge); the fourth, G3-Gate, strips the information away to isolate the bare obligation to look again. Taken in that order (as they were introduced in Section 4.5), their mean effects (G3-Instruction -0.57, G3-Reflexion +1.14, G3-Judge +1.57 and G3-Gate +0.14) tell one story: the value is in the information, and continuity beats a clean slate.

G3-Instruction: the cost of throwing the context away

G3-Instruction implements Self-Refine's iterative refinement stripped of memory (Section 4.8): on a needs-fix a fresh agent receives only the task instruction and the current environment state, with no phase-1 trace. The hypothesis is that a second attempt unbiased by the first trajectory sees the gap the first missed.

It does not, and its failure is instructive. G3-Instruction is the only G3 configuration with a negative mean, -0.57, negative on the same capability band as everything else and worse than G3-Gate, which adds nothing: throwing the phase-1 trace away costs more than the fresh perspective recovers. Gemini-3.1-FL shows it most clearly, losing −2 while every other G3 variant on Gemini-3.1-FL is positive, because the strongest model has the most useful trace to discard.

Gemini-3.1-FL's `processing-pipeline` run shows what the discarded trace costs. The phase-1 agent corrupted its own files with a misused `mv` (`mv file.tmp /app` rather than `mv file.tmp file`) and failed. G3-Instruction sent a fresh agent back in with only the task and the (now corrupted) environment, no record of that slip, and it failed again; G3-Reflexion, same model and task, sent the same kind of fresh agent but with a note naming the error, *"the agent accidentally corrupted files by using `mv` incorrectly"*, and it solved the task. That one diagnosis is the whole gap between the two rungs, worth +1.14 against G3-Instruction's -0.57 across the grid. The Self-Refine premise (that an unbiased retry beats a biased continuation) fails here: the first attempt's trace is an asset, not baggage, which is why G3-Instruction lands below even G3-Gate, the rung that adds nothing at all.

G3-Reflexion: the diagnosis is what a fresh start needs

G3-Reflexion is Reflexion (Section 4.8): before the fresh retry, an LLM writes a note on what phase 1 did and where it likely fell short, and the fresh agent receives the instruction, that note, and the environment state. It adds the diagnosis of the first attempt without its full trace.

It helps, and broadly: mean +1.14, the second-best G3 variant and one of the two strongest architectures on Gemini-3.1-FL at +4. It is the biggest single G3 winner on the weak models (Qwen-7B +4, Llama-70B +4) and the best architecture in the study on `create-bucket`, which it flips on **five** models: a compressed note like "you created the bucket but did not set public-read" is the kind of targeted correction a reflexion can carry and a from-scratch retry cannot reconstruct.

A single addition over G3-Instruction (the diagnosis) swings the mean from the family's worst to nearly its best, which pins down what verification is actually buying: not the retry, since both variants retry, but the *information* carried across it. G3-Reflexion stops short of G3-Judge because a compressed note is lossier than continuing the original agent's own memory, but it captures most of the benefit at lower risk, since a fresh agent cannot be dragged back into the first attempt's dead end. It is the architecture to recommend when full continuity is unavailable.

G3-Judge: continuity plus feedback, the family's best

G3-Judge places an LLM-as-judge in the loop (Section 4.8): on needs-fix the judge's feedback is injected into a clone of the original agent's memory and that same agent continues. It is the only G3 variant preserving full context continuity, so the agent that resumes knows everything it already tried plus a targeted critique.

It helps most of the family, and most reliably: mean +1.57, the best G3 variant, strictly positive on five of seven models and non-negative on six, the widest reach of any G3 configuration. It is the least-harmful architecture on Qwen-32B, where nothing clears baseline, at −2, positive on Gemini-3.1-FL +2, and the outright best architecture in the entire grid for Qwen-14B at +3. Its wins spread wider across task types than any other G3 variant (`processing-pipeline`, `simple-web-scraper`, `jq-data-processing`), because continuity lets it repair multi-step work rather than restart it.

G3-Judge is the strongest of the four because it combines the two things the others add separately: the diagnosis G3-Reflexion carries and the full working memory G3-Instruction discards. The G3 intro

(Section 6.4.3) gives the deeper mechanism, that rejection is informative even from a weak self-judge because recognition is easier than production, and the homogeneous ranking confirms the consequence: the more of the first attempt the verifier can see and build on, the more it helps. One caution carries over from the heterogeneous track (Section 6.6.4): a stronger judge's rejections rescue *less*, not more, so the value of the role lies in the feedback's content, not the severity of the standard. Where judge and worker are the same model that tension is muted, and G3-JUDGE is the study's most dependable improver, bounded only by the budget it competes for (Section 6.4.3).

G3-Gate: obligation without information

G3-GATE is the family's control, taken last: the one variant we introduced ourselves rather than adapting from the literature, a minimal ablation that strips away the information the other three add and leaves only the obligation to look again (Section 4.8). Mechanically it is the simplest intervention in the study, an "are you sure?" and nothing more. When the agent calls `submit`, the gate holds it ("*your answer has NOT been submitted yet; verify your solution*") and lets the *same* agent, with its full context, keep working; it does this at most twice before the next `submit` goes through. There is no fresh instance, no external judge and no new information, only the demand to check again.

It is almost exactly neutral, mean +0.14, positive on four of seven models by small margins. It gains on the weak models where a second look has room to work (Qwen-7B +2, Llama-70B +1) and on Gemini-3.1-FL +3, and it taxes the narrow strong coder, Qwen-32B −4. Its wins are the legible-progress tasks the monitor also rescued, `flood-monitoring-basic` flipped on four models and `csv-to-parquet`, where a forced self-check catches a run that stopped one step short. Llama-70B/`csv-to-parquet` shows exactly which step: the agent submits at step 5, "successfully converted", with `pyarrow` never installed and no valid Parquet on disk; the gate refuses the submission, and the forced re-check sends the agent back to install the missing engine and actually write the file. The baseline, with no gate, makes the same premature claim and stops there.

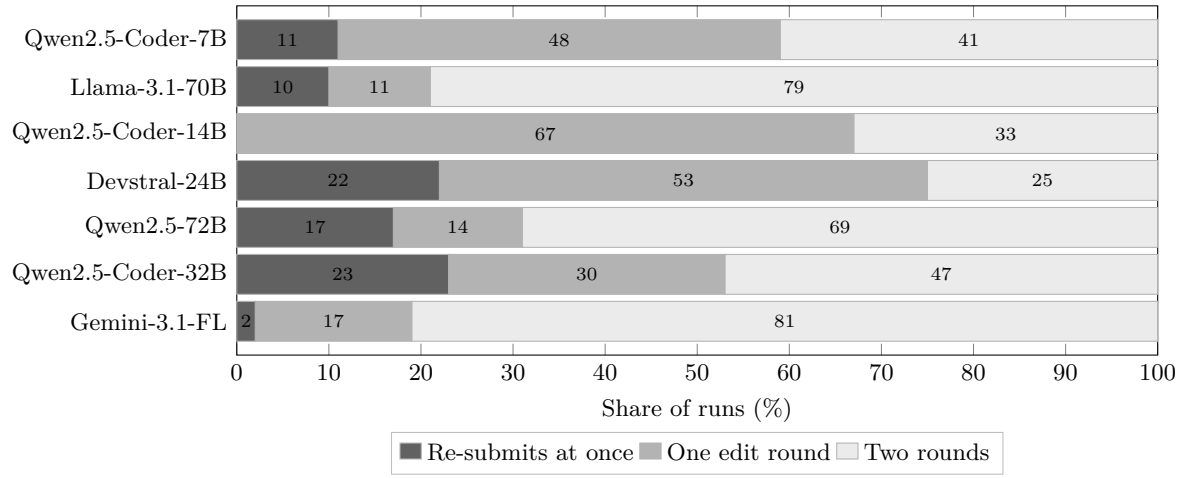


Figure 6.4.1: How each model answers the G3-GATE's "are you sure?", which holds a submission at most twice. *Re-submits at once* is finalising with no further edits; *one* and *two rounds* went back to edit before submitting again. Models that re-verify are doubting their own work; those that re-submit unchanged are confident or cannot see the fault.

Because the gate always fires, its own measure is not *whether* it fires but how the agent answers the "are you sure?": how often it goes back to edit versus simply re-submitting the same work (Figure 6.4.1). Overall the challenge changes the ending: **88%** of runs edit before finalising and only 12% re-submit unchanged. But the split by model is the more telling part. Gemini-3.1-FL and Llama-70B take the full two rounds of re-checking in 81% and 79% of runs (they doubt their own work when asked) while Devstral-24B and Qwen-32B re-submit unchanged far more often (22% and 23%), confident in a first answer or unable to see its fault, which is of a piece with Devstral-24B's low obedience to the Monitor.

The obligation alone changes the ending in most runs, yet moves the mean only +0.14: G3-GATE measures what looking again is worth when nothing new is added, and the answer is a little, on models with headroom: self-verification without new information mostly re-runs the reasoning that produced the submission. It is the control the other three rungs are read against: whatever G3-REFLEXION and G3-JUDGE achieve beyond +0.14 is the value of the information they add, not of the obligation to check.

6.4.4 G4 multi-agent: coordination cost

The G4 family puts more than one agent on the task, on the intuition that a second agent (or a coordinator directing several) can do what one cannot. They are the study's heaviest architectures by far, a median of 46–52 steps and near half a million tokens per run. That budget buys almost nothing against the baseline: a mean of +0.57 tasks per model for either dialogue design and -0.71 for the orchestrator. So what separates them is not how much they achieve but where the extra cost goes and what breaks when agents have to coordinate, the cost the family is named for. The two dialogue designs come first, differing only in whether the second agent continues the first's work or critiques it, then the orchestrator, which differs in kind.

The two dialogue designs share a termination rule that shapes what their turn counts mean. A single `submit` does not end the run: it is a *proposal*, and the turn passes to the partner, who may confirm it (by submitting the same answer, a dual-submit consensus that ends the run) or reject it by simply carrying on working, which leaves the proposal standing and hands control back for another round. Turns alternate seven steps each within the same 60-step budget; G4-Debate stops after eight turns (56 steps) while G4-Cooperation continues to the full 60, after which the last proposal is accepted unconditionally. Table 6.21 shows how often each model actually reaches consensus before that cap, and it reaches it rarely. Only Gemini-3.1-FL and Llama-70B agree early in most runs (69% and 70% under G4-Cooperation); every other model runs to the budget more often than not, its partner declining to confirm again and again. The adversarial stance of G4-Debate, designed to make the second agent disagree, barely moves this: convergence is almost identical to G4-Cooperation's column for every model (42% against 43% pooled), so on this task set forced disagreement changes how the turns are spent, not whether the two agents come together.

Table 6.21: How often the two dialogue architectures reach consensus before exhausting the budget. A run *converges* when both agents submit the same answer before the eight-turn cap; at the cap they never agreed. *Median turns* is over all runs. Cooperation and Debate side by side isolate what the adversarial stance does to convergence — which, as the columns show, is almost nothing.

	Cooperation		Debate	
Model	Converged	Median turns	Converged	Median turns
Qwen2.5-Coder-7B	30%	8	25%	8
Llama-3.1-70B	70%	5	78%	4
Qwen2.5-Coder-14B	28%	8	25%	8
Devstral-24B	23%	8	23%	8
Qwen2.5-72B	47%	8	45%	8
Qwen2.5-Coder-32B	28%	8	36%	8
Gemini-3.1-FL	69%	6	69%	6

G4-Cooperation: fresh context as the whole hypothesis

G4-Cooperation is a relay (Section 4.9): two equal agents take turns on the same terminal, seven steps each, handing off with a compressed summary rather than the full trace. Agent B continues A's work. The hypothesis is minimal, that fresh context alone breaks stagnation, since an agent inheriting a summary is not anchored to the approach that was failing.

It is a joint-best G4 configuration, mean +0.57, and it produces the family's clearest single win: G4-Cooperation is **the best architecture in the grid for Devstral-24B**, at +6, matched only by its own G4-Debate variant, and strongly positive on Llama-70B +4 and Qwen-7B +2. It damages Qwen-32B most, at −5, the same narrow strong coder every heavy scaffold hurts. Its wins cluster on tasks with a discrete retry structure (`broken-python`, `countdown-game`, `create-bucket`, each flipped on three models), where a second agent arriving with a clean context tries what the first was too committed to abandon.

One task isolates the mechanism, and it works for a reason more specific than the hypothesis stated. `hydra-debug-slurm-mode` requires installing a Hydra launcher plugin and configuring it with parameters the schema accepts only in a particular place. Gemini-3.1-FL reaches 29–86% of its checks under thirteen of the fourteen architectures and solves it under exactly one: G4-Cooperation. Both agents form a false belief early (that the container has no network and the plugin therefore cannot be installed) and spend three turns searching the filesystem for it; this run records nine such statements, where no other

Gemini-3.1-FL configuration records more than two. In turn seven agent A abandons the reasoning and simply tries the install. **It succeeds, and agent A does not notice**: it reissues the identical command and its turn ends. Agent B opens the next turn with "*wait, the last step shows* `pip install` *actually worked*", and from there resolves the remaining schema problem by introspecting the launcher class, in five mentions of the offending key, against 13 to 24 for the configurations that never got past it.

The relay supplied not a better idea but **a reader for an observation the first agent had generated and failed to absorb**, the same defect as the worker in Section 6.6.3 that downloaded the wrong architecture and never read the error it printed. Here the handoff summary, elsewhere a cost, is what pays: compressing the state forced a restatement in which the buried success became visible.

The turn counts state the mechanism plainly: G4-Cooperation's *successful* runs use fewer hand-offs than its failed ones (4.3 against 6.9 turns), so the relay helps by supplying *one* fresh restart at the point of stagnation rather than by accumulating collaboration, and runs that need many hand-offs are already lost. This makes G4-Cooperation a multi-agent analogue of G1-Temp and G3-Reflexion at once, breaking a deterministic dead end like temperature and doing so by handing a fresh agent a compressed state like reflexion. On Devstral-24B, a model that grinds its own budget away, the forced hand-off to a clean-slate partner is worth more than any single-agent intervention.

Set against that is a real cost, and it is why the mean stays small. Every hand-off fragments the task across two contexts: the incoming agent inherits only a compressed summary, not the full trace, so a little detail is lost at each hand-off (the same lossy hand-off as G2-Planner's sequential subtasks) and the agent spends its first steps re-orienting in an environment it did not build. That the same compression both hides work across the boundary and, occasionally, restates a buried success is the whole tension of the design: the restart is worth its cost only when the model was stuck, which is why the benefit concentrates on the models that stagnate and vanishes into overhead on the rest. The family's cheapest hypothesis, fresh context and nothing more, is its most effective; the G4-Debate and G4-Orchestrator elaborations examined next add cost without clearly adding to it.

G4-Debate: the critique shows up on subtests, not solves

G4-Debate is G4-Cooperation with one change (Section 4.9): agent B is told to critique A's approach and then try a deliberately different strategy, testing whether forced disagreement improves strategy selection over a plain continuation. Since G4-Cooperation and G4-Debate are identical but for the critique, their difference isolates what the adversarial stance is worth.

It helps marginally, and only on the finer measure. G4-Debate and G4-Cooperation are indistinguishable on solves (both mean +0.57, both 103 aggregate) so the critique completes no extra tasks; on the graders' partial credit it edges ahead, **46.4%** of subtests against G4-Cooperation's **42.9%**. It advances work without finishing more of it, the subtest-without-solve pattern of Section 6.5 isolated to a single design choice.

`jq-data-processing` on Qwen-32B shows the critique in action against the relay on the same task and model. Both inherit Agent A's split `jq` pipeline, but where G4-Cooperation's partner *continues* it ("*the next steps are to ensure it is correctly formatted*") and stalls one check short of a solve, G4-Debate's partner *rejects* it ("*split into multiple jq invocations, inefficient and incorrect*") and switches to a single command that solves it. The disagreement works, but it is not the binding constraint: for every task it rescues by discarding Agent A's approach, another fails because that approach should have been kept, and the two net to the same solve count. G4-Debate also asks the most of the format: its prompt must carry the task and an adversarial stance and still elicit well-formed tool calls, which is part of why it does not pay on the weaker models. The effect is real but narrow, on the propose–check tasks of Section 6.5.4, and its added complexity does not earn back on solve rate.

G4-Orchestrator: routing whose cost is its own competence

G4-Orchestrator replaces the peer relay with a tool-less Strategist that decides, each cycle, which specialist to deploy (`explorer`, `coder`, or `verifier`) and writes it a detailed instruction (Section 4.9). It is the study's most elaborate architecture, and the only one whose order of work is not fixed in advance: every other design runs a hard-coded sequence of phases (explore then execute, submit then verify), whereas here an LLM (the Strategist) chooses at each cycle what happens next. The control flow is decided by the model at runtime, not by the harness.

It does not. It is the worst G4 variant and one of the worst architectures overall, mean -0.71, negative

on three of seven models and, unlike the dialogue designs, negative even on Gemini-3.1-FL −2. It keeps small wins on the weak models (Llama-70B +3, Qwen-7B +1) but loses the best result G4-Cooperation reaches on Devstral-24B, and inflicts the grid's worst single delta on Qwen-32B, −7.

The Strategist's one job is to split cycles among explorer, coder and verifier, and how it splits them tracks the worker's capability (Figure 6.4.2). The weak coders route most calls to exploration and hardly any to verification: Qwen-7B sends 74% to the explorer and 3% to the verifier, Qwen-14B 69% and 8%, so their cycles drain into inspection that never turns into code. Gemini-3.1-FL is the only model to route a balanced mix and the only one to give the verifier a real share (25%). A model that routes badly spends the extra layer on itself.

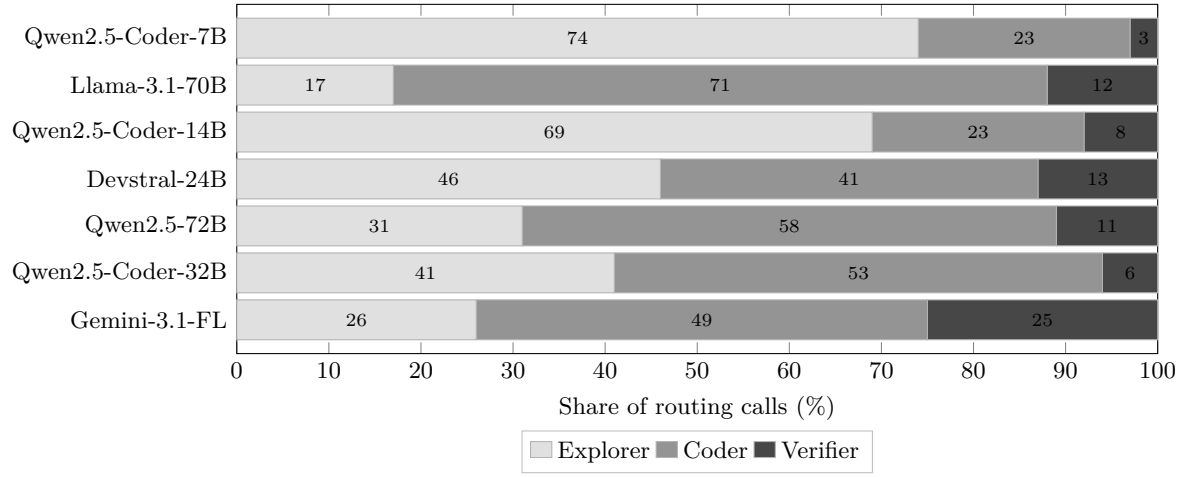


Figure 6.4.2: Share of the G4-Orchestrator Strategist's routing calls sent to each specialist, per model; the three shares sum to 100. Models are ordered by baseline capability.

The result is a failure signature no dialogue design can produce. Only **41% of G4-Orchestrator runs reach a submission at all**, against 82–84% of the relays' runs: the Strategist so often uses up the run on routing that no answer is ever proposed. And 29% of runs end in a way the relays never can, hitting the cycle cap: the Strategist keeps dispatching until the cycle budget is spent. `assign-seats`, which Gemini-3.1-FL's bare agent solves directly, shows the tax on even the strongest model: the Strategist spends all twenty cycles dispatching (twice sending an explorer to list files before any code is written) and the run ends at the cap without a single submission. What fails is the routing, not the idea of routing, and the heterogeneous track separates the two (Section 6.6.2): a stronger model in the Strategist seat cuts the cycle-cap rate sharply and makes G4-Orchestrator the supervised track's most improved architecture. A model routing itself pays a coordination tax it cannot afford; the same router directed by a better model is worth it.

G4-Orchestrator is where multi-agency's coordination cost becomes a distinct failure mode rather than a diffuse overhead. Homogeneously it is a net loss: the strategist's cycles come out of the same budget the specialists need, and a self-routed run stalls before it solves. Its value is conditional, appearing only when the routing role is filled by a model better than the worker, which makes it a finding about supervision more than about orchestration alone.

6.4.5 The answer to RQ3

Read as a set, the fourteen configurations line up along one axis, and it is neither complexity nor family. Ranked by mean effect (Table 6.2), the top is the two verification variants that give the agent new information while keeping what it already had, G3-Judge (+1.57 tasks per model) and G3-Reflexion (+1.14); the bottom is the pre-execution designs that spend a phase adding nothing the model could not reach itself, G2-Planner, G2-Explorer and G1-Temp, at -1.43. The heavy multi-agent schemes sit in the middle, and the single most cost-effective intervention in the study is three sentences of prompt. Sophistication, then, does not predict benefit. A simpler condition does: **an architecture helps when it supplies something the model lacks, in a form the model can use, at the moment it is needed, and costs the budget it spends whenever one of those three fails to hold.**

Each clause is visible in the within-family orderings, which recur across the grid. *Something the model lacks:* a grounding pass or a plan is not, since a capable model inspects and plans on its own (G2-

PLANNER's plan over G2-EXPLORER's grounding nets exactly zero) whereas a verifier's rejection is, which is why verification tops the ranking and pre-execution sits at the bottom. Some of the pre-execution losses are not grounding failing but scaffolds that never ran at all: the sub-agent's artifact failed to validate, so the phase was skipped and the run fell back to plain baseline behaviour (Section 6.4.2). Even when those runs are set aside and only the ones where the scaffold did execute are counted, pre-execution still does not help, for the same reason: grounding is the one deficit a capable model meets for itself. *A form the model can use:* the information must survive to the executor, so G3-JUDGE, which keeps the agent's full context, beats G3-INSTRUCTION, which discards it, with G3-REFLEXION recovering most of the gap by carrying a diagnosis across the break. *At the moment it is needed:* a single well-placed restart (G4-COOPERATION's relay, G2-MONITOR's redirect) rescues a model that has stalled and interrupts one that was converging.

The second finding the per-model columns force, and no aggregate can show: **no architecture is best for every model**. Table 6.22 lists the winner for each. Five different architectures take the top place across the seven models, spanning all four families (G3 supplying two of them); the same scaffold, byte for byte, changes sign along the capability range, so an architecture picked for its aggregate rank is the wrong choice for most individual models. On Qwen-32B, the strongest open coder, nothing clears baseline at all, so the plain baseline is the right choice there. The near-zero main effect of Table 6.2 is therefore not a null result but the average of large, systematically opposite per-model effects.

Table 6.22: The architecture that helps each model most, and its gain over that model's own baseline. Models are ordered by baseline capability.

Model	Best architecture	Δ vs baseline
Qwen-7B	G2-MONITOR / G3-REFLEXION	+4
Llama-70B	G2-MONITOR	+5
Qwen-14B	G3-JUDGE	+3
Devstral-24B	G4-COOPERATION / G4-DEBATE	+6
Qwen-72B	G1-PROMPT	+2
Qwen-32B	G3-JUDGE	−2
Gemini-3.1-FL	G3-REFLEXION	+4

Why each winner fits its model is the substance of the per-model profiles (Section 6.2.4); the table records only the fact. That leaves the question this chapter cannot settle alone: if the right architecture depends on the model, can it be chosen without running all fourteen? The profiles make that answerable from a single baseline run, and Section 6.7.2 turns it into a rule.

6.5 What makes a task respond to architecture

This chapter has so far asked which *model* and which *architecture* an effect belongs to; this section asks which properties of the *task* predict how it responds to architecture: whether it is helped, and by which family. Three properties earn a place, and the section takes them in turn.

The benchmark's own **category** turns out to carry a great deal of matching information. The kind of task a category contains calls, by design, for a particular family, and more often than not that is the family that helps it: grounding architectures improve the data-handling categories, verification improves those with an exact deliverable, and routing improves the multi-stage ones (Section 6.5.2). **Difficulty** predicts too, but only when it is measured on the models themselves rather than read from the benchmark's author estimates: the a-priori estimates order nothing, while the same tasks binned by how many models already solve them order the effect cleanly (Section 6.5.3). And the recurring **shape** of a task yields a usable rule for which family to reach for, one derived from the results after the fact rather than validated in advance (Section 6.5.4).

One note on the outcome measure. The solve rate is binary (all of a task's 1–22 grader checks must pass) so a great deal of work sits below it: **2887 failed runs passed at least one check**, more than the 1612 that solved outright. The graders' individual checks recover that movement [49] and are reported wherever they add something. But *movement on checks is not a solve*, and the two can diverge completely: `machine-learning` is the second most positive category on subtests yet solves none of its tasks in 294 runs, so the subtest fraction is used only as corroboration, never as a measure on its own; the graders resolve failure unevenly (Appendix A.4), and any claim resting on subtests alone is flagged.

6.5.1 The category grid

Read as a grid, the categories are not interchangeable. Table 6.23 gives the grid on solves, as a percentage-point change from each category's own baseline pooled over the seven models. A cell is a *rate*, not a count of tasks: a category holds only (tasks × seven) runs, so in a six-task category (42 runs) a +7 is about three more solved cells than baseline, not seven. What kind of work each category asks for is summarised in Table 5.3 of the methodology.

Table 6.23: Solve-rate change over baseline, in percentage points, by task category and architecture (pooled over the seven models). Categories are TerminalBench's own labels; cells are small (1–13 tasks × 7 models) and no significance is claimed.

Category	n	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
software-engineering	13	11%	−1	+3	−1	−3	−1	−2	−1	−1	+3	−1	+1	−1	−1
data-science	8	34%	+4	−2	+0	−2	+0	+0	+4	+4	+9	+2	−2	+4	+0
security	7	31%	−16	−6	−8	−4	−4	−6	−4	+2	−4	+0	−4	−4	−14
file-operations	6	36%	+7	+7	+7	+5	+10	+12	+2	+17	+7	+7	+10	+12	+2
system-administration	6	29%	+5	−10	−2	−2	−7	−5	−7	+2	+2	−5	+0	+0	+2
mathematics	5	34%	+3	−6	+0	−6	+6	−3	+3	+3	+0	+0	+3	+0	+6
data-processing	4	14%	+11	−4	+4	+7	+7	+18	+4	−4	+7	+14	+4	+0	+4
debugging	4	14%	+0	−11	−4	−7	−7	+4	−4	−4	−4	−7	+0	+0	+0
games	4	7%	+4	+0	−7	−4	−4	+0	−4	+0	+4	+0	+11	+4	+0
machine-learning	3	0%	+0	+0	+0	+0	+0	+0	+0	+0	+0	+0	+0	+0	+0
algorithms	2	21%	−7	−14	−14	−7	+0	−7	−7	+0	−7	+0	−7	−7	−21
model-training	1	29%	+0	+0	−29	+0	−29	−14	+14	+0	−14	−29	−14	+14	+0
scientific-computing	1	14%	+0	+0	−14	−14	+0	+0	−14	−14	+14	+0	−14	−14	+14

The categories are the benchmark's own labels, taken from each task's `task.yaml` rather than assigned by us, for the same reason the study uses the authors' `easy` label: they are data a reader can check. The only edit merges two single-task near-duplicates (`math` into `mathematics`, `game` into `games`), leaving 13 categories over the 64 tasks.

Read down the columns and a pattern shows: the shaded gains and losses cluster by family rather than scattering. Verification (`Judge`, `Reflex`, `Gate`) is positive or neutral in almost every row, while the pre-execution columns (`Explor`, `Plan`, `Analyst`) are negative in most, except in `data-processing` and `file-operations`, where they turn strongly positive. Different task categories favour different families, and the favouring is systematic.

That clustering is the real content of the grid, and it is what the design predicts: each architecture family was built for a particular kind of failure, hence a particular kind of task. Whether the predicted match actually holds (and what the traces say about why) is the next subsection. One caution carries into it: the grid should not be read *across* rows, because the categories hold very different numbers of tasks (3 to 13) at very different baselines, so a row's mean reflects its composition as much as the architecture. Every comparison that follows stays *within* a row (one family against another on the same tasks) never one category against another.

6.5.2 Did each family help the category its design implies?

The grid of Section 6.5.1 hints that particular families suit particular kinds of task. That is a claim the design makes in advance and the data can check, in the same way Section 6.3.4 checked whether each architecture repaired the failure it was built against.

The targets are not chosen after the fact. Each family was built for a failure mode (Section 4.5), and each failure mode is provoked by a recognisable kind of task, so the design already implies which task-kind each family should suit. **Grounding** (G2) was built against acting before inspecting, so it should help where the answer must be read from the data or the environment first. **Verification** (G3, and the G4 relay) was built against premature wrong submission, so it should help where a task has an exact deliverable a run can wrongly call finished. The **loop-breakers** (the monitor and the dialogue pair) were built against stalling and repetition, so they should help where progress comes only by trying and checking. **Decomposition and routing** (G2-Planner, G4-Orchestrator) were built against tasks too large

for one flat attempt, so they should help where the work is genuinely multi-stage. These are the *Target* entries in Table 6.24; *Best* is the family that in fact helped most.

Table 6.24: Solve-rate change over baseline by architecture family and category (percentage points, pooled over the seven models). *Target* is the family the design implies for that category (Section 4.5), assigned from the category's description alone; *Best* is the observed best family (bold in the row), shaded green when it matches the target and red when it does not. *None* marks a category where no family improves on baseline, so there is no winner to match.

Category	n	G1	G2	G3	G4	Target	Best
`software-engineering`	13	**+1.1**	−1.9	+0.0	−0.4	G3	G1
`data-science`	8	+0.9	−0.4	**+4.5**	+0.6	G2	G3
`security`	7	−11.2	−5.6	−1.5	−7.5	G2	*none*
`file-operations`	6	+7.1	**+8.3**	+8.3	+7.9	G2	G2
`system-administration`	6	−2.4	−4.2	−1.8	**+0.8**	G4	G4
`mathematics`	5	−1.4	−0.7	+1.4	**+2.9**	G3	G4
`data-processing`	4	+3.6	**+8.9**	+5.4	+2.4	G2	G2
`debugging`	4	−5.4	−3.6	−4.5	+0.0	G3	*none*
`games`	4	+1.8	−3.6	−0.0	**+4.8**	G4	G4
`machine-learning`	3	+0.0	+0.0	+0.0	+0.0	G2	*none*

Comparing *Target* with *Best* scores that prediction. It lands squarely for two families, fails for the third, and every miss turns out to be a mixed or intractable category rather than a wrong design.

Grounding, where the answer is in the data. The design points grounding (G2) at five categories, those whose obstacle should be the data or the environment itself. It is right on the two that cleanly are that. `data-processing` (pipelines that interpolate gaps then count) is where G2 leads by the widest margin: `flood-monitoring-basic`, unsolved by any model at baseline, reaches 4/7 under G2-Monitor, whose periodic check catches a run that has interpolated but not yet counted. `file-operations` is the same on solves: `heterogeneous-dates` joins two series whose dates use different formats, and the join fails unless they are read and reconciled first, moving 2/7 to 4/7. The other three miss, and each miss indicts the category, not the design. `data-science` hides two opposite kinds under one label: `csv-to-parquet` is pure grounding (an uninstalled library, fixed by reading the environment, 5/7 to 7/7) while `mlflow-register` is a stateful sequence a run ends too early, the same explorer stopping it after nine steps against a fifty-step baseline, so verification, harming neither, comes out ahead. `security`'s real obstacle is domain knowledge no scaffold supplies, with five of its tasks at the floor, so grounding cannot help and no family clearly does. `machine-learning` is a diagnosis-and-setup mix that no model solves at all.

Routing and dialogue, where the work is multi-stage or a search. The design's cleanest calls are its two G4 targets, and both land. `system-administration` holds the only genuinely multi-stage jobs in the corpus with separately verifiable stages (bring up a Hadoop cluster, poll and log a cluster, rank profiler stacks), and routing wins on the one metric these never-fully-solved tasks expose, subtest progress, stage by stage, rather than a binary solve (the category's solve lead belongs to the bare prompt). G4-Orchestrator posts its single largest subtest gain in the study here, lifting the cluster-monitoring task from 54% of its checks to 77% by getting a run stage by stage further than the flat agent, which stalls early. `games` is propose–check–revise search: `sha-puzzle` asks for a sentence whose initials spell the length of its own SHA-1 hash, where every edit invalidates the target, so a plan written before the first observation is wasted and only a check afterwards helps: G4 is the sole positive family, holding `sha-puzzle` at 14% of subtests where pre-execution drives it below baseline. The same G4-Orchestrator, incidentally, is the *worst* architecture in `security`, burning 55% of its runs against a budget on tasks soluble in a handful of commands: whether routing helps is a fact about the task's structure, not the architecture.

Verification: the family with no home. Verification (G3) is the telling one. The design points it at the three categories whose deliverable is checkable code or a single answer (`software-engineering`, `mathematics`, `debugging`) and it tops none of them: `software-engineering` is too hard for any family (no family clears baseline, and the bare prompt is least-bad), `mathematics` is taken jointly by G2-Analyst and G4-Orchestrator rather than any verifier, and `debugging` turns on a diagnosis wrong at the root that no checker catches: on `fix-pandas-version`, 23 traced runs read "unexpected keyword

argument" as reason to *downgrade* the library whose newer version they in fact need, a misreading every architecture faithfully executes, so no family helps there at all. Where G3 *does* lead is `data-science`, a category targeted at grounding, not verification. Verification owns no category because its domain, a task with an exact deliverable a run declares finished too early, is scattered across every category rather than concentrated in one; it is the family with the best aggregate balance (Section 6.4) and yet the top family in only one category, and its real signature surfaces not as a row here but as the named-artifact effect of Section 6.5.4.

The pattern is consistent, and it is the section's point. The prediction holds wherever a category is a single task-kind (`data-processing`, `file-operations`, `system-administration`, `games`) and fails wherever the label mixes kinds (`data-science`, `software-engineering`, `machine-learning`) or the obstacle lies outside any family (`security`, `debugging`). The interaction between architecture and task is real, but the topic label is the wrong unit to read it through, because a label routinely mixes task-kinds. The sharper cut is by task *shape*, developed next, after the variable that fixes the *sign* of every one of these effects.

6.5.3 Difficulty, read and measured

Two variables lie behind the sign of the architectural effect. This section names the first (difficulty) and the next names the second, task shape. Difficulty can be gauged two ways: read from the task before running anything, or measured on the models after a single baseline run. Only the second predicts.

Read from the task: nothing. The benchmark supplies two cheap a-priori measures (the authors' estimate of expert-minutes and the length of the reference solution) and neither orders the effect (Table 6.25). Under both the relationship is non-monotonic: the middle expert-minute band fares worst while the longest tasks turn positive, and reference length wanders the same way, with other bin edges giving the same non-result. The reason is shared. How hard a task is *for a human* is not how much room a *model* has left on it: a ten-minute task can be trivial or impossible for a model depending only on whether it needs knowledge the model already has.

Table 6.25: Architectural effect binned by two difficulty measures readable before running anything: the benchmark's expert-minute estimate and the length of its reference solution. Neither orders the effect. The Δ column is the mean solve-rate gain of the thirteen architectures over baseline within the bin. Tasks lacking the relevant metadata are omitted, so the bins total 50 and 58 of the 64 tasks respectively.

Measure	Bin	Tasks	Baseline solves	Mean Δ
Expert-minutes	≤ 5	10	27%	−0.2
	6–15	16	27%	+0.3
	16–30	17	13%	−1.2
	> 30	7	12%	−0.6
Reference LOC	≤ 15	15	34%	−2.8
	16–40	20	16%	+0.5
	41–80	12	20%	−1.0
	> 80	11	4%	+2.9

Measured on the models: headroom. The difficulty that does predict is measured on the models themselves. Call it *headroom*: how many of the 7 models already solve a task at baseline, and so how much room is left to gain. Table 6.26 sorts the 64 tasks into four bands by it, and the distribution is lopsided: most sit at or near the floor and few are contested. Because a category averages over many tasks that never move, its mean is diluted, which is the arithmetic reason a category mean is not a finding.

Grouped by headroom (Table 6.26) the picture looks symmetric (architecture loses on already-solved tasks and gains on floor tasks) but only one half is evidence. **The already-solved half is a finding.** Even the best architecture nets +0 solves there and the worst -12; decomposed, over the 910 comparisons in this band architecture wins 85 that the baseline failed and loses 148 it had solved, so 0 of the thirteen finish positive. On a task a model can already do, architecture only trades, at roughly seven losses for every four gains. **The floor half is arithmetic**: a band whose baseline is zero cannot go negative, so those gains are forced, and the fair reading is the magnitude: +2 to +7 rescues out of 175 opportunities, 1.1% to 4.0%. The two middle bands span zero (worst negative, best positive) and support no claim. The

Table 6.26: Architectural effect by task headroom, in solves. Tasks are grouped by how many of the 7 models already solve them at baseline; each band offers tasks × 7 solves. *Net* is the mean, over the thirteen architectures, of solves gained minus solves lost across the whole band; *Worst* and *Best* are the least and most any single architecture nets.

Task headroom	Tasks	Opportunities	Net	Worst	Best
Already solved (≥4/7)	10	70	−4.8	−12	+0
Contested (2–3/7)	12	84	+1.8	−3	+5
Near-floor (1/7)	17	119	−2.6	−7	+6
Floor (0/7)	25	175	+4.8	+2	+7

one-sided result is the point: **the absence of headroom reliably predicts harm, while its presence predicts only a small and uncertain benefit.** This is the model-level gradient of Section 6.2.2 seen from the task's side: an architecture pays only where a gap remains between what the first attempt reaches and what a second look could, and on an already-solved task that gap is zero, so structure is pure subtraction.

Half the set is out of reach. Headroom presumes a solve to build on, and for the open models much of the set offers none. Partitioning the 64 tasks by which models ever solve them gives three tiers: 41 are solved by at least one model other than Gemini-3.1-FL, 11 only by Gemini-3.1-FL, and 12 by nothing at all (Table 6.27). From the six open models' point of view the second tier is *indistinguishable* from the third: their mean subtest fraction is 30% on Gemini-3.1-FL-only tasks and 22% on never-solved ones, against 44% on the reachable. "Only Gemini-3.1-FL solves it" and "no one does" are, for every model architecture can improve, the same wall.

Table 6.27: The 64 tasks partitioned by which models ever solve them, across both tracks. For the six open-weight models the last two tiers are indistinguishable: what separates them is entirely Gemini-3.1-FL.

Tier	Tasks	Open-model subtests
Solved by some open or supervised model	41	44.0%
Solved only by Gemini-3.1-FL	11	29.8%
Solved by nothing	12	22.5%

Architecture converts a near-miss into a solve; it cannot manufacture the near-miss. For an open model 11 plus 12 tasks (over a third of the set) sit at a subtest floor with no near-miss to convert, which is the structural reason the per-model effects of Section 6.2.2 stay small even for the weakest models: much of their nominal room is not headroom but wall. What sits behind that wall is insight, not effort: `amuse-install`, solved in only 11 of 116 runs, spreads its few solvers across Devstral-24B, Qwen-32B, Qwen-72B and Gemini-3.1-FL rather than concentrating on the hosted model, because its difficulty is a long compile-and-install sequence rather than a missing idea. Hardness that is a matter of stamina leaves open models on the reachable side; hardness that is a matter of insight, as in `mahjong`, does not.

6.5.4 Task shape predicts the family

Section 6.5.2 showed the architecture–task interaction is real but that the topic label is too coarse a unit to read it through, because a category routinely mixes task-kinds. The natural correction is to group the tasks by the kind itself. This grouping is a distillation of the previous subsection, not a fresh analysis: the obstacle that keeps a task unsolved falls into a handful of recurring *shapes* (what the model is missing, independent of topic) and tasks that share a shape respond alike whatever their benchmark label. `csv-to-parquet` (`data-science`) and `create-bucket` (`system-administration`) both hinge on the same gap: the model knows what code to write but not what its environment provides (which library is installed, how an unfamiliar command behaves) and both are helped by the same family.

Table 6.28 groups the 25 classifiable tasks this way. Its two family columns split the thirteen architectures by *when* they act: pre-execution (G1–G2: prompting, exploration, planning, monitoring) work before or alongside the first attempt, and post-execution (G3–G4: verification and the multi-agent relays) act on an attempt that already exists. G4 sits in that second group less comfortably than G3 (a dialogue is not

a verifier) but it is placed there because its decisive move, challenging a candidate the first agent has produced, likewise comes after that first attempt.

Table 6.28: Architectural response by *task shape* rather than topic. *Pre-exec* is G1–G2 and *Post-exec* G3–G4; Δ is each family's mean subtest gain over the shape's baseline, and *Best* names the single strongest architecture with its gain.

Task shape	n	Base	Pre-exec	Post-exec	Best
Already solved, short path	7	76%	−12.3	−9.4	Debate −4
Specified transform, data present	6	52%	+8.6	+6.3	Analyst +15
Environment / tool discovery	5	46%	+0.4	+8.5	Reflex +19
Propose–check–revise search	5	35%	−5.1	−1.0	Debate +8
Multi-stage setup	2	40%	−8.4	+0.4	Judge +4

Most shapes are defined by a criterion readable from the task statement; the exception is *Already solved, short path*, which is read from baseline behaviour rather than the wording. The criteria themselves, moreover, were fixed after the outcomes were seen, so this is a post-hoc reading rather than a pre-specified one (the caveat is spelled out below, and the full task list per shape is in Appendix A.5):

- **Already solved, short path** — the reference solution is short and most models already solve it unaided. Every architecture only loses here; the row merely restates the headroom penalty and adds nothing.
- **Specified transform over present data** — the input is supplied and the task asks for a fixed transformation of it into an exactly specified output, with nothing to search for or discover. This is the one shape where *pre-execution* wins, because the plan can be written correctly before anything runs, and it is identifiable from the statement in advance.
- **Environment or tool discovery** — the algorithm is clear but success turns on an environmental unknown: which library or version is installed, how an unfamiliar tool behaves. Post-execution wins, because the deciding fact surfaces only once the task is run.
- **Propose–check–revise search** — the answer can be found only by producing a candidate and testing it (a fixed point, an unseen maze, a rules engine). Both families lose, pre-execution most, because no plan survives the first observation.
- **Multi-stage setup** — the work splits into several sequential, independently checkable stages (bring up services, run a pipeline). Post-execution wins.

Beneath the five shapes is one robust axis: timing. The five-way grouping is a fine reading, and much of it is speculative, as the closing paragraph says. But it rests on a coarser distinction that is not: whether an architecture acts *before* the first attempt (the G1–G2 families) or on a candidate that already exists (G3–G4). This is the same before-or-during timing that separates the shape rows, reduced to its two-way core, and its virtue is that it can be checked *without* any shape grouping at all, straight on the benchmark's own categories. Post-execution beats pre-execution in 9 of the 10 categories with at least three tasks (Table 6.29), on identical task sets, so a category's composition cancels and the result owes nothing to a partition of ours. `git-workflow-hack` shows the pre-execution side at its sharpest, because there the reading *is* the task: a sabotaged GitHub Actions workflow leaks a secret, discoverable only by surveying files the instructions never mention, so **runs that open the workflow files solve it 14 times in 50 and runs that do not none of 52**, and it is the pre-execution family that opens them, solving it 29% of the time against 5% for the non-pre-execution configurations (the plain baseline itself solves it on one of the seven models).

A more specific readable property: the named artifact. One property narrower than the shape is also readable straight from the statement: whether the task demands an *exact named output file*, which 50 of the 64 do. It predicts the family directly, and the prediction holds: architectures forcing a check before the submit should help specifically here. G3-Gate, whose entire design is to intercept the submit and make the agent verify its own work, gains **+6.0 subtest points** more on named-artifact tasks than on the rest; on solves G3-Judge is +12 on them against -1 elsewhere. The 3 architectures that do *not* do better locate the mechanism precisely: Temp, Plan, Coop. Two add no check before submission, as the prediction implies; the third, G4-Cooperation, does check, but hands the second agent a compressed

Table 6.29: Mean subtest gain over baseline by architecture family, per category (categories with at least three tasks). *Pre-execution* is G1–G2 and *post-execution* G3–G4; *positive* counts how many of each family's members beat baseline.

Category	Tasks	Pre-execution (G1–G2)		Post-execution (G3–G4)	
		positive	mean Δ	positive	mean Δ
`software-engineering`	13	1/6	−1.7	2/7	−1.1
`data-science`	8	1/6	−1.6	6/7	+2.7
`security`	7	1/6	−3.9	1/7	−2.6
`file-operations`	6	1/6	−1.1	5/7	+2.0
`system-administration`	6	4/6	−0.3	6/7	+2.9
`mathematics`	5	3/6	−0.5	5/7	+1.0
`data-processing`	4	5/6	+6.3	7/7	+9.3
`debugging`	4	2/6	−2.3	2/7	−3.6
`games`	4	0/6	−6.5	3/7	−1.8
`machine-learning`	3	6/6	+4.8	6/7	+5.0

summary rather than the artifact, so it cannot see the file whose exactness is graded. A check helps only if it can inspect the thing under test. The effect is modest, since dropping two already-solved no-artifact tasks takes G3-Gate from +6.0 to +2.0, but the direction is clear.

The five-way grouping, by contrast, is a hypothesis. Where the timing axis is measured, the finer partition into five shapes is not a validated predictor. It was not derived algorithmically: clustering the tasks by their response vectors does not reproduce, at a mean adjusted Rand index of 0.41 across ten initialisations of the same data, because with one run per cell and 42 tasks solved by at most one model the per-coordinate noise is of the same order as the signal. And while each shape's *criterion* is readable from the task statement, the *selection* of criteria was made after the outcomes were known. Nor do the five shapes cover the corpus: they hold 25 of the 64 tasks, and the 39 left out fall into two groups, those no model solves under any architecture, where no shape can be read from a task that never moves, and those whose obstacle is not a shape at all but domain knowledge (`security`'s CVEs) or a mixture of shapes under one label (the hard `software-engineering` set). The covered subset is not arbitrary, though: it carries 60% of the grid's movement and 13 of the 18 mid-band tasks (two to five models solving at baseline) from 39% of the corpus, so it is the movable, single-shape core rather than a convenient slice. The shapes are offered as a mechanism worth testing, with the timing axis beneath them already firm; Section 9.4 states what would validate the finer cut.

6.5.5 The answer to RQ4

RQ4 splits in two (*whether* architecture helps a task, and *which* family) and only the first resists being read from the statement.

Whether is fixed by headroom: its absence reliably predicts harm, its presence only a small and uncertain benefit. This is the capability gradient of Section 6.2.2 seen from the task's side: where a correct short path already exists, architecture has nothing to add and only something to replace. Neither the benchmark's topic label nor its a-priori difficulty estimate predicts the sign; headroom, which does, is measured on the models rather than read from the task, so the sign needs a baseline run. And for the open models over a third of the corpus lies behind a wall that no architecture crosses at all, headroom or not.

Which family helps, once one does, *is* readable in advance, and here the task's own description carries real information. Each family was built for a failure mode, hence a task-kind, and it helps the categories that are that kind (grounding where the answer is in the data, routing and dialogue where the work is multi-stage or a search) and fails only where a category mixes kinds. Verification is the exception that shows topic is the wrong unit: it tops no category, because its target (an exact deliverable a run declares finished too early) is scattered across every category, and its prediction lands not on a topic row but on the named-artifact tasks of Section 6.5.4. Reading the task by its *shape* rather than its topic makes this cleaner, and its robust core is a matter of timing: pre-execution designs win when the information a plan needs is present before the run, post-execution ones when the deciding fact appears only during it. The finer five-way shape taxonomy that refines this is a mechanism read off the outcomes, not yet a validated predictor.

So the task's own description tells you *which* family to reach for, but not *whether* to reach for one at all: that judgement needs the single baseline run that measures headroom.

6.6 Model cooperation: heterogeneous supervision

The four preceding sections describe a single arrangement: one model, wrapped in a scaffold, working alone. Within it architectural value is bounded by the model itself: a scaffold externalises a function the model already has some version of, and cannot conjure one it lacks entirely. RQ5 asks whether that bound can be crossed from *outside*: if a scaffold helps by supplying what a model lacks, could the supply instead come from a *different, stronger* model, placed in the coordination layer rather than the execution layer?

The heterogeneous-supervision track tests this directly. In each of the six architectures that carry a detachable supervisory role, that role is handed to Gemini-3.1-FL (the study's strongest model) while a weaker model continues to execute every action in the environment. The worker, its budget, its tools, and the task set are held identical to the matching homogeneous cell, so the *only* thing that changes is which model answers the supervisory call. Three workers spanning the capability range (Llama-70B, Devstral-24B, Qwen-32B) give eighteen supervised cells, each read against its identical homogeneous counterpart.

The finding developed below is that a strong supervisor helps, but that *how much*, and even in *which direction*, depends not on the supervisor's strength but on what its output *does* to the worker's process, the distinction stated in Section 6.6.1. As in the task-type analysis, these eighteen comparisons are read as *exploratory*: they were not among the pre-specified tests that Section 5.7 corrects for multiplicity, so nothing here is claimed on statistical significance. The argument rests instead on each effect pointing the same way across all three workers, and on the recorded artifacts (plans, injected messages, verdicts) explaining *why* it does. For calibration, 3 of the 18 cells exceed the single-run noise floor of 5 tasks (Section 5.5); the role distinction below is read off those, and the within-noise cells are not leaned on.

6.6.1 Supervision helps, and the architecture decides how much

Table 6.30: Effect of replacing the supervisory role with Gemini: each cell is the supervised solve count minus the worker's own homogeneous count for that architecture. Workers are ordered by baseline capability, with their solo baseline solves in parentheses.

Architecture	Llama-70B (5)	Devstral-24B (12)	Qwen-32B (19)
Explor	+0	+2	+4
Plan	+5	+3	+0
Analyst	+1	−2	+0
Monitor	−3	+9	+1
Judge	**+3**	**−3**	**−6**
Orch	**+4**	**+5**	**+8**
Total	+10	+14	+7

Aggregated over all six architectures, a Gemini supervisor is worth +10 solves on Llama-70B, +14 on Devstral-24B, and +7 on Qwen-32B. These sums hide the structure that matters, because they pool two opposing trends: **two architectures move in exactly opposite directions as the worker gets stronger**, with G4-Orchestrator rising (+4, +5, +8) while G3-Judge falls (+3, −3, −6).

The mechanism that decides how much: directive roles add, evaluative roles gate. The six roles split into two families by what the supervisor's output *does* to the worker's process, and that split, not how strong the supervisor is, fixes the direction of every effect below. A *directive* role (strategist, planner, monitor, explorer) *adds* to the process: an instruction, a plan, an injected message, a summary. A stronger model contributes a better version of that addition, so its capability transfers directly and nothing has to be tuned to the task. An *evaluative* role, the judge, instead imposes a *threshold*, accepting or rejecting the worker's output. There raw capability need not transfer: a gate helps only in so far as its bar is aligned with the criterion the task is actually scored against, so for this role the question is not how strong the supervisor is but how its judgments land on the worker, and, as Section 6.6.4 shows, a stronger judge does not simply do better.

The three subsections that follow are ordered by this mechanism, not by architecture family: how a role behaves under supervision follows from what its output adds or gates, not from which generation it belongs to, so the G4 strategist leads and the G2 roles follow. Orchestration (Section 6.6.2) is the clearest directive case, and the only role whose gain *grows* as the worker strengthens. The G2 context-augmentation roles (planner, monitor, explorer and analyst) come next (Section 6.6.3): three transfer, and the analyst does not, for a reason of *interface* rather than capability. The evaluative judge (Section 6.6.4) closes the section as the one role where a stronger supervisor makes matters worse.

6.6.2 Orchestration: the cost was the routing, not the coordination

In G4-Orchestrator the supervisor is the *strategist*. It holds no tools; it decides at runtime which specialist sub-agent to deploy and what to instruct it to do. Every action in the environment is still taken by the worker.

Table 6.31: G4-Orchestrator with the Strategist role given to Gemini, against the same worker strategising for itself. *Cycle-cap* is the share of runs that exhaust the routing budget without submitting.

Worker	Strategist	Solves	Cycles	Steps	Ktok	Cycle-cap
Llama-3.1-70B	self	8	9.2	35.7	222	8%
	Gemini	**12**	**7.6**	**30.0**	**190**	**5%**
Devstral-24B	self	14	15.0	45.3	681	48%
	Gemini	**19**	**11.9**	**44.3**	**648**	**25%**
Qwen2.5-Coder-32B	self	12	12.2	47.5	653	22%
	Gemini	**20**	**10.2**	**41.8**	**539**	**14%**

For *every* worker the swap is a strict improvement (Table 6.31): on Qwen-32B it converts 12 solves into 20 at fewer input tokens, more solves at strictly lower cost.

This reframes the coordination-cost result of the homogeneous grid. There, G4-Orchestrator appeared to be an architecture whose overhead grows with model capability. Separating the roles shows that the cost was never in the coordination itself: **it was in the routing decisions**. A worker strategising for its own specialists burns cycles it cannot spare, and pooled over the three workers, the self-strategists exhaust the cycle cap in 26% of runs, against 15% under a Gemini strategist.

That the gain *grows* with worker strength (+4, +5, +8) is the signature of a genuinely complementary pairing: a good routing decision is worth more when the executor can carry it out.

The reversal is sharpest on Qwen-32B. In the homogeneous grid G4-Orchestrator was its *single worst* architecture, at 12 solves against a 19-solve baseline, a deficit of −7, the deepest in its column. Handing the strategist role to Gemini lifts the same architecture to 20, which *clears* that baseline. On the strongest open worker, supervision does not merely cancel the coordination penalty; it turns the architecture that hurt the worker most into one that helps.

Why it is more efficient: a consistent routing policy. That efficiency has a visible cause in the routing decisions, since each call names a specialist. The self-strategists route idiosyncratically: Llama-70B under-explores badly (17% explorer, coding 71% of the time), while Devstral-24B and Qwen-32B over-explore (41–46%). Gemini applies *a far more uniform policy* (25–37% explore, 46–58% code, 15–17% verify), with under half the spread across workers (mean per-role standard deviation 3.9 against 9.4).

Crucially the correction runs in *opposite directions* for different workers: explorer share moves +20 points on Llama-70B and −21 and −14 points on Devstral-24B and Qwen-32B. That rules out a fixed recipe and shows the strategist correcting each worker's characteristic bias toward a common target. Verifier share is the clearest case: Gemini routes to the verifier 15–17% of the time for every worker, where the workers themselves range from 6% to 13%. Figure 6.6.1 shows the convergence directly: the three self-policies differ sharply, while under Gemini the same three workers are routed by nearly one shared policy.

The other half of the route: an instruction that adapts. A routing call names a specialist *and* hands it an instruction, written after the strategist has seen what the worker did on the previous cycle. This second channel, not the choice of specialist alone, is where the self-strategists lose the most, and not through weaker instructions: theirs are no shorter and no less grounded than Gemini's, they simply

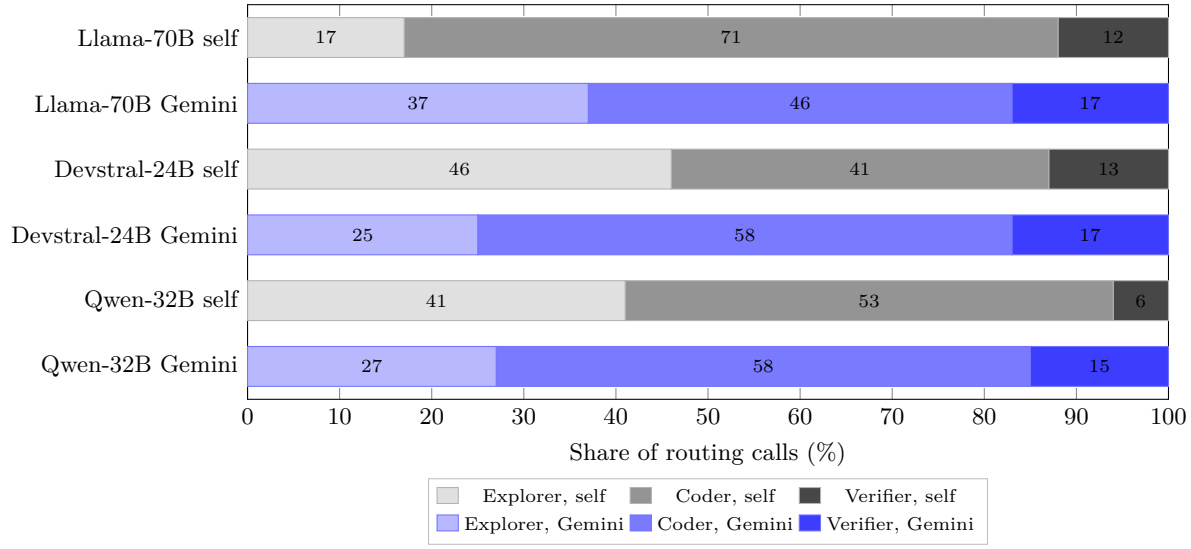


Figure 6.6.1: Routing share across the three specialists, each worker routing for itself (grey) versus Gemini as Strategist (blue); the three shares sum to 100.

fail to *change*. A self-strategist that cannot read its own lack of progress reissues a near-identical route (same specialist, same instruction) and stalls. Across the three workers, 47–69% of consecutive routing calls are such near-duplicates and 59–89% of runs contain at least one such loop; a Gemini strategist revises instead, cutting those to 9–23% and 20–53%. It is the identical-command loop of Section 6.3.4 raised to the coordination layer: the self-strategist loops on *its own* directives exactly as an unscaffolded worker loops on tool calls.

Table 6.32: The same task, `sql-injection-attack`, under Qwen-32B's own strategist and under a Gemini strategist. Both must locate the vulnerable query in `vulnerable_login.py` before an exploit can be written. Qwen-32B solves this task under none of its fourteen own architectures; under a Gemini strategist it does.

	Self-strategist	**Gemini strategist**
Route sequence	explorer ×10, then the step budget is spent	explorer ×7 → coder → explorer → coder ×2 → verifier
Instruction	*identical* every cycle: "Read the full content of `vulnerable_login.py` … identify the vulnerability"	*revised* every cycle: read → "use `cat`, capture the whole file" → "the previous reads were truncated; use `grep -n 'def authenticate_user'`" → write the exploit
Outcome	the query is never located; the run ends at the step budget, unsolved	the `grep` instruction breaks the loop, the function surfaces, and the exploit is written and verified: solved

6.6.3 Context augmentation under a strong supervisor

The four G2 roles all augment the worker's context before or during execution: the planner supplies a subtask list, the monitor an in-loop message, the explorer a data summary, and the analyst both at once. Each records a concrete artifact, so together they show not just *that* a directive role transfers but *what* the strong supervisor puts into it. Three transfer cleanly. The analyst is the instructive exception (the one directive role a stronger supervisor fails to improve) and it fails for a reason of *interface*, not capability. It is the first of the section's two ways that "more" stops helping; the evaluative judge of Section 6.6.4 is the second.

Planning: what a better plan actually contains

Gemini's planning is worth +5, +3 and +0 solves over the worker's own planning on Llama-70B, Devstral-24B and Qwen-32B. The planner is the one role whose whole output is a single artifact recorded verbatim

(the subtask list) so what earns that gain can be read off directly, by comparing Gemini's plan against the worker's own on the same task.

The gain is not better decomposition. Gemini's plans parse as valid about as often (89% for Gemini against 84% for the worker) and split the task into essentially the same number of subtasks (1.77 for Gemini, 1.79 for the worker). What differs is what each subtask *demands*: a better plan supplies two things the worker's does not, a **specification** and an **obligation**.

Specification: a task title turned into something executable. At the median the two plans are the same length (309 characters for Gemini against 317 for the worker), but Gemini's plans spell out far more concrete detail per subtask: 2.2 file paths against the worker's 1.5, and 2.0 explicit quantities against the worker's 0.8. Table 6.33 shows what that means on `jsonl-aggregator`, a task Llama-70B fails under its own plan and *solves* under Gemini's: the worker restates the task title, while Gemini spells out the glob pattern, the fields to aggregate, the two ranking operations, and (in the success criterion) the two-decimal rounding rule the task requires and the worker leaves out. The supervisor is not decomposing more finely; it is turning a task statement into a specification precise enough to execute without further interpretation.

Table 6.33: Plans for `jsonl-aggregator`: Llama-70B's own against Gemini's, both planning for Llama-70B. Gemini names the glob, the fields, the two rankings and the rounding rule; the worker restates the title.

	Llama-70B for itself	**Gemini for Llama-70B**
Plan	"Write a Python script to aggregate data from all JSONL files and create `/app/aggregates.json` with the required structure."	"Read all `/app/records_*.jsonl`, aggregate `amount` and `items` per user, count each tag, select the top 5 users by amount and top 5 tags by count, write to `/app/aggregates.json`."
Success	"`aggregates.json` exists and has the correct structure."	"... top 5 users and top 5 tags, **with amounts rounded to 2 decimal places**."

Obligation: a success criterion that keeps a give-up-prone worker working. `vimscript-vim-quine` asks for a Vim plugin that emits executable Vimscript reconstructing the editor's layout. Its defining property is that correctness is *not inspectable*: plausible Vimscript is easy to write, and only running the script establishes whether it rebuilds the layout, so even Gemini-3.1-FL's own baseline reaches just 75% of the checks. Four of 116 rows solve the task; three are Gemini-3.1-FL under second-attempt mechanisms (G1-Temp, G3-Instruction, G3-Reflexion), and the fourth is Llama-70B under Gemini's planning, a worker whose own baseline is three steps (`write_file`, `write_file`, `submit`) that never opens Vim. Eleven of its fourteen configurations leave it at 25% of the checks, because the post-execution families (judge, gate, debate, reflexion) all fire *on submission* and it submits before there is anything to evaluate.

Under Gemini's plan the same worker runs 20 steps and passes all eight checks. The plans differ in one respect (Table 6.34): Gemini's second subtask is an *empirical verification loop*. The executor trace is 20 steps of exactly that (invoke the command, fail, write a test layout, discover that `vim -c` will not source non-interactively, switch to `vim -S` with an explicit quit, wrap it in a shell script, run it), behaviour Llama-70B shows nowhere else on this task. It did not become more capable; it was given a reason to continue.

What the planner's gain shows. "Directive" is not one capability: the monitor of Section 6.6.3 supplies a *fact* the worker failed to read, while the planner here supplies an *obligation* it would not have imposed on itself. And this is the revisability gap from an unusual angle, since Llama-70B has a large gap between its first attempt and what revision could reach, but never opens it because it stops before there is anything to revise. That is why the post-execution family cannot help here and the pre-execution family can: verification arrives after a submission that came too early to be worth verifying; a plan arrives before it.

Does specification have a ceiling? Gemini's planning helps Llama-70B on six tasks and hurts it on one. That one, `recover-accuracy-log`, is the case where Gemini wrote its *longest* plan (nearly three

Table 6.34: Plans for `vimscript-vim-quine`: Llama-70B's own against Gemini's. Gemini's second subtask is an empirical verification loop; the worker's criterion ("works as expected") cannot be checked.

Llama-70B's own plan	**Gemini's plan**
1. "Create plugin file" *success:* "File exists"	1. "... uses `mksession!` or manual inspection to capture the current layout" *success:* "File exists and contains the implementation"
2. "Implement `:LayoutQuine` command" *success:* "Command works as expected"	2. "**Verify** the plugin: create a test layout in a temporary Vim session, run the command, source the generated script to recreate the layout. **Fix any issues found.**" *success:* "The generated script recreates the layout when sourced in a new Vim instance"

times its average, down to the error strings to exclude and a round-half-up rule) and the run still failed, in four steps against the five the worker's own vaguer plan took to *succeed* on the same task. It is the natural place to ask whether a plan can specify *too* much, handing the worker a rigid solution to transcribe that it cannot repair once it breaks. One loss against six gains cannot answer that; it is marked as the boundary worth probing, not a finding.

Monitoring: the same information, addressed differently

The monitor is a tool-less observer called every ten steps, whose guidance is injected as the executor's *own* `think` step. It is the axis's most erratic role (−3, +9, +1), and three things explain the spread in turn: what the monitor *says*, whether the worker *obeys* it, and whether there was a loop to break in the first place.

What differs in the text. Three differences hold across all three workers. First, *person*: the self-monitors narrate about the executor ("the agent" in 97% of their 568 messages) while Gemini addresses it directly, "you" in 92% of 484 against the self-monitors' 2%. This matters because of the injection mechanism: third-person commentary reads as an outside note the agent may disregard, whereas second-person reads as self-instruction. Second, *freshness*: the self-monitors repeat their previous message near-verbatim 29–50% of the time (regurgitating rather than re-reading the state) against Gemini's 0–12%. Third, *escalation*: Gemini raises an explicit CRITICAL in 9–54% of messages, the self-monitors in 0–4%.

What that buys: the worker actually obeys. These differences are not cosmetic; they change whether the advice is acted on. Measuring obedience as the executor's next command *differing* from the one before an injection, a Gemini monitor is obeyed more by every worker: 89% against 74% on Llama-70B, 80% against 42% on Devstral-24B, 80% against 65% on Qwen-32B. The largest jump is Devstral-24B, the worker that obeyed its *own* monitor least (Section 6.4.2) and loops most. And obedience shows up as loops broken where loops exist: Devstral-24B's mean longest identical-command streak falls from 24.6 to 12.0, while Llama-70B's and Qwen-32B's, whose longest streaks are already short, are unchanged (2.5→1.8, 4.5→4.5). Better advice earns more obedience, and obedience breaks loops, but only where there is a loop to break.

An illustrative success: the task solved by a single supervised configuration. `setup-custom-dev-env` builds a shell environment to a seven-check specification whose checks are serially dependent, so an early misstep caps the score. It is the hardest task in the study by one measure: across **116 rows (every model and architecture, both tracks) exactly one solves it**, and the average run passes only 36% of the checks. The one solve is Qwen-32B under a Gemini-3.1-FL monitor. Its failure mode is not reasoning but two facts the run must *read* from its own output: the `oh-my-zsh` installer refuses to run over an existing install (inviting a delete-and-retry loop), and the container is `arm64`, so the obvious Miniconda URL downloads the wrong architecture.

The comparison is clean because the winning configuration and its nearest control differ in one component: Qwen-32B under its *own* G2-MONITOR scores 57% in 60 steps; the same worker, architecture and budget, with Gemini supplying the monitor, scores 100% in 52.

	Self-monitored (57%)	**Gemini-monitored** (100%)
step 20	"*The agent* is repeatedly running ... without success"	"CRITICAL: *You* have spent 15 steps ... *without actually configuring*"
step 30	*identical paragraph*	"... you are writing to `/root/.zshrc` using `cat` inside a `bash` ..."
step 40	*identical paragraph*	"... downloading the **wrong architecture** (`x86_64` on an `arm64` machine)"

Both monitors *detected* the stagnation correctly. Only one *diagnosed* it, and the traces show the consequence step by step. After the step-20 message the worker's next `think` reads "let's stop re-running the installer and manually configure the `.zshrc`", the advice restated as the agent's own intention, which is exactly what the inception injection is for. After step 30 it stops reinstalling and inspects the directory. Most directly: **the step-40 message names the architecture mismatch, and step 41 downloads the `aarch64` build**. The self-monitored run, reading the same log, emitted "repeating similar commands" three times and found the architecture on its own only at step 59, six steps from exhaustion, too late to finish the conda chain.

What the supervisor contributed here was a *fact about the environment*, not a strategy, a fact the worker had produced on its own screen and not read. That is the most direct form of the directive role, and what it is not bears stating: one task, one run, and a configuration that solves nothing else uniquely. It is offered as the mechanism made visible, not as evidence that supervision solves environment-setup tasks.

This is why the sign follows the worker, not the supervisor. The monitor fires every ten steps and targets *stagnation*, so it helps exactly when there is a loop to break. Devstral-24B stagnates (47.8 mean baseline steps, grinding to the budget) giving a competent monitor four firings and a real loop, hence +9. Qwen-32B loops often but shallowly: its mean longest streak is only 4.5 (unmoved above), short of the sustained grinding a ten-step monitor is built to catch, so there is little for it to break and the role manages only +1. Llama-70B abandons tasks at 12.6 steps: the monitor fires about once, finds nothing to interrupt, and its injections only add noise, hence −3. **A better supervisor cannot repair a mismatch between the architecture's mechanism and the worker's failure mode.**

Exploration and analysis: reliability, and the cost of verbosity

Exploration: reliability, not better content. The explorer emits a single Data Summary, and Gemini produces a *valid* one 88% of the time for every worker, in roughly half the steps. The gain tracks the worker's own *validity deficit*, not its capability: Llama-70B's explorer is already 86% valid, so a reliable summary adds nothing (+0); Devstral-24B's is 50% and Qwen-32B's 56%, both far short of Gemini's, and both gain (+2 and +4). And it is reliability, not richer content: Gemini inspects no more of the environment than the worker (fewer files on Llama-70B, a comparable count elsewhere) and its summaries are no longer, so what the role supplies is an artifact that reliably conforms to the schema, not a better-informed one.

Analysis: where verbosity turns into a liability. G2-ANALYST is the one role a stronger supervisor does not improve (+1, −2, +0), and the reason is the interface, not the capability. It is the only role whose output is a *combined* artifact (a summary *and* a grounded plan in one structured pass) and Gemini writes 1.3–2.8× more into it than the workers do. That verbosity backfires: Gemini's artifact validates *less* often than the worker's own on two of the three workers (73% for Gemini against 81% for the worker on Llama-70B, 34% against 59% on Qwen-32B). **The more a model writes into a single structured artifact, the more likely it is to break the schema**, and the analyst asks for the most in one pass, so a stronger supervisor's fluency becomes a liability exactly where the format is tightest.

6.6.4 Judging: a stronger judge approves better and rescues less

G3-JUDGE is the one *evaluative* role: the supervisor does not add to the process but *evaluates* the worker's submitted work, approving it or injecting feedback into a clone of the worker's memory which then resumes. It is also the one role a stronger supervisor does not simply transfer. Its effect *reverses* with worker strength (+3 on Llama-70B, −3 on Devstral-24B, −6 on Qwen-32B), helping the weak worker and hurting the strong one. The verdicts show where that reversal lives; a final caveat marks where the data stops being able to explain it.

The loss is on the rejections, not the approvals. A judge does two separable things, approve and reject. Splitting Qwen-32B's cell by verdict, against its *own* self-judge so both columns share the same phase-1 runs and differ only in who evaluates them, puts the loss entirely in one branch.

	Qwen-32B self-judge		Gemini judge	
Verdict	Runs	Solves	Runs	Solves
Approved	6	5	12	**9**
Rejected	19	**11**	14	2
Never submitted	39	1	38	0
Total	64	**17**	64	11

The third row is runs that never reached a submission, so no verdict was issued; it is present so the columns account for all 64 tasks and reconcile to the totals.

On approvals Gemini is the better judge by volume, and no worse in precision: it passes 9 correct submissions against 5, at a comparable rate (9 of 12 against 5 of 6). The loss is not here. It is entirely on rejections: when Qwen-32B rejects its own work the retry succeeds 11 times in 19; when Gemini rejects, only 2 in 14. Mean steps are identical, so budget does not explain it.

It is not that the feedback is vaguer. The natural reading is that a stronger judge's critique is too abstract for a weaker worker to act on. Reading the feedback refutes it: on the 6 tasks both judges rejected, Gemini's critiques are the more specific and the more accurate.

- On `jsonl-aggregator`, Qwen-32B tells itself "the output should be truncated to only the top 5". Gemini identifies the actual bug: `total_items` is identical to `total_amount`, so the agent summed the amount into the items field, and names the script to fix.
- On `bank-trans-filter`, Qwen-32B claims the output path is wrong (it was not). Gemini observes that the file contains transactions for the wrong company entirely and that the JSON is a fragment rather than a valid array.

Across all rejections, Gemini's feedback cites *more* file paths and roughly twice as many code references as the self-judge. Vagueness is not the problem.

What a rejection is worth is what the worker can act on. The reversal follows from that. On the weak worker, whose submissions are poor, accurate rejections point at real, fixable faults, and a Gemini judge *helps* (+3). On the strong worker the same accuracy rescues far less. Qwen-32B's own self-judge rejects almost indiscriminately (19 times in 64), so much of what it flags was nearly right and gets fixed; Gemini rejects more selectively (14 times), and those rejections turn into a solve only 2 times. *What a rejection is worth is how much of it the worker can act on, not how accurate it is.* `multistep-definite-integral` is the purest case of a rejection with nothing to act on: Qwen-32B wrote `-2 + E`, the grader accepted it, and Gemini objected that the task asked for `e - 2` and that `-2 + E`, though "mathematically equivalent", was not clean. The worker spent 23 steps where its baseline had needed 13, and, that time, still landed on a passing answer. The cost was the wasted budget, not a lost solve.

What the traces cannot settle. Two limits, both inherited from Section 6.4.3, bound how far the last step can be pushed, from "its rejections rescue less" to "it rejects work the grader would have passed". First, a rejection is not directly checkable: the benchmark grades only the *final* state, never the submission the judge saw, so an unrescued rejection is ambiguous between a correct diagnosis the worker could not act on and a false alarm on work that was already right. Second, the judge often never fires at all: on Qwen-32B it issued no verdict on 38 of 64 runs, because the phase-1 agent ran out of budget before submitting, so a raw comparison of the judge cell to the plain baseline is mostly runs the judge never touched, and cannot be read as the judge's own doing. What survives both limits is the paired self-versus-Gemini contrast: a stronger judge approves at least as well and rescues markedly less, and on the strongest worker that trade is net negative. Which of the two readings of its rejections is correct, the traces cannot say.

6.6.5 The hardest test: does supervision add capability?

The gains so far mix two effects: turning a worker's occasional success into a reliable one, and producing a success on a task the worker never manages alone. Only the second supplies capability rather than *organising* what is already there, and it can be isolated, but with the right denominator. As a share of *output* it looks vanishing: only 14 of supervision's 237 solves (5.9%) clear a task the worker solves under none of its own fourteen architectures, so the solves are overwhelmingly consolidation. That figure is diluted by every easy task, though; the fair denominator is the *opportunity*, the 109 (worker, task) pairs no homogeneous configuration ever solves. Against it, a Gemini supervisor cracks **12, about one in nine (11%)**, genuine rescue from zero, on a minority of the tasks that need it, and fragile, since most are cracked in only one configuration. Supervision adds real capability; it just sharpens far more than it adds.

The additions, when they come, are all from one side of the mechanism of Section 6.6.1. Every rescue comes from a *directive* role: 14 of 545 directive runs over an unsolvable task end in a solve (2.6%), while the evaluative judge rescues **0 of 109**. A gate can only pass or block work the worker could already produce, never originate a solution it would not have reached; a directive supervisor can inject the missing step. `sql-injection-attack` is that in miniature: Qwen-32B never solves it alone, but under a Gemini-3.1-FL strategist the worker's thirty-step `cat vulnerable_login.py` loop (Section 6.3.4) is rerouted to `grep -n 'def authenticate_user'`, the function surfaces, and the worker writes and submits a working injection. The supervisor did not author the exploit; it broke the loop and redirected the search.

Does the supervisor just solve the task? The obvious objection is that a supervisor which *tells* the worker what to do is solving the task itself, making the "unlock" capability substitution in disguise. Three facts answer it. The ceiling is the *worker's*, not Gemini's: `simple-web-scraper` is the control, with Qwen-32B, Qwen-72B and Gemini-3.1-FL solving it under all fourteen of their own architectures and Llama-70B under none, and holding the supervisor at Gemini-3.1-FL while varying only the worker gives 6/6 on Qwen-32B, 4/6 on Devstral-24B, 0/6 on Llama-70B, tracking solo reach almost exactly. Gemini knows the answer and still cannot make the weak worker produce it. And execution never leaves the worker: Gemini holds no tools and directs blind from the worker's reports, so reading the code, writing the fix, running the tests and recovering from real errors are the worker's on every solved run. The supervisor supplies strategy; the worker supplies the execution the benchmark scores, and where it cannot execute, the strategy is inert.

6.6.6 The answer to RQ5

Capability *can* be injected at the coordination layer, but how it must be injected depends on what the role does to the worker's process. A **directive** role adds to that process (an instruction, a plan, an in-loop message, a summary) and a stronger supervisor contributes a better version of the same addition, so its capability transfers: the strategist, planner, monitor and explorer all help, and the strategist's gain even *grows* with worker strength. An **evaluative** role imposes a threshold instead, and there raw capability does not transfer the same way: the Gemini judge approves at least as well but its rejections rescue markedly less, net negative on the strongest worker.

Two roles gain nothing from a stronger supervisor, for structural reasons rather than any deficiency of the supervisor: the *analyst*, whose combined-artifact interface punishes the extra text a stronger model writes, and the *judge*, whose output is a threshold rather than an addition, so greater accuracy (above the worker's reach, or above the grader's tolerance) does not convert into solves.

Nor is the gain bought at a cost premium: per solved task the supervised cells are *cheaper* in tokens for all three workers (-18%, -18%, -12%), because they solve more for comparable total spend, though the figure pools worker and supervisor tokens and leaves the orchestrator's own routing calls unrecorded, so it flatters heterogeneity where the hosted model is dearer.

Above all, supervision **consolidates far more than it transfers**. Its solves are overwhelmingly re-solves of what the worker could already do; against the tasks it *cannot* do alone, a strong supervisor cracks only about one in nine, each in a single fragile configuration (Section 6.6.5). A stronger coordinator mostly makes a weak executor use what it has, and only now and then lends it something it lacks.

6.7 Synthesis

6.7.1 Complementarity and the oracle bound

No architecture dominates, but the architectures are not redundant either: they solve different tasks. The shape is familiar from repeated sampling, where coverage, the fraction of problems solved by any attempt, rises far above the success rate of a single one [31]. Here the diversity comes from architecture rather than sampling temperature, at identical budget, which implies a different remedy, routing rather than resampling.

Table 6.35: Three levels of architectural choice. Selecting the single best architecture for a model is worth between +0 and +6 tasks and collapses at the top of the capability range; selecting the right architecture *per task* raises the bound by a further 6 to 14 tasks, though against single-run noise this excess is real only on the models that still have headroom.

Model	Baseline	Best single	Oracle	Arch. gain	Routing gain
Qwen2.5-Coder-7B	4	8	14	+4	**+6**
Llama-3.1-70B	5	10	20	+5	**+10**
Qwen2.5-Coder-14B	9	12	21	+3	**+9**
Devstral-24B	12	18	32	+6	**+14**
Qwen2.5-72B	16	18	28	+2	**+10**
Qwen2.5-Coder-32B	19	19	31	+0	**+12**
Gemini-3.1-FL	34	38	49	+4	**+11**

Table 6.35 separates two decisions that are usually conflated. The first is *which architecture to adopt* for a model, and Section 6.2.2 has already shown that this decision buys little: the best single architecture gains between +0 and +6 tasks, and the gain shrinks as capability rises, so that on Qwen-32B the best non-baseline architecture is -2 against the plain agent, making the decision not merely worthless there but negative. The second decision is *which architecture to use for a given task*, and it is a different proposition entirely.

Where the routing headroom is real

The final column is the striking one, and it needs the noise null of Section 5.5 to read correctly. Choosing the right architecture per task raises the oracle bound by a further **6 to 14 tasks** (mean 10.3, standard deviation 2.3), but pooling fourteen single runs inflates any union, so the bound must be compared to a `pass@k` null. The excess over that null is +3.2 tasks on the weaker models (the three lowest-baseline open models, Qwen-7B, Llama-70B and Qwen-14B) and -0.2 on the strongest: real where the model still misses tasks, within noise where it does not. The raw routing gain on Qwen-32B is 12, but against the null it does not clear noise, while the largest in the grid belongs to Devstral-24B at 14.

On the strongest open coder no architecture helps on average, and its apparent per-task oracle gain is within single-run noise. The two statements agree: a model with little headroom leaves little for either per-model or per-task selection to recover. Where there is headroom, on the weaker models, averaging over tasks still destroys the signal, because the architectures' advantages are task-specific and cancel in aggregate, and there the per-task view recovers real solves the mean hides.

The reframing follows. "Does this architecture help?" is close to unanswerable, since the true answer is that it depends on the task: it gains where the model still has room and loses on already-solved tasks, and over the full set those two cancel (Section 6.5.3). "Which architecture for this task?" has a real payoff where the model still has headroom (clearing single-run noise on the weaker models, within noise on the strongest). This also gives the null main effect of Section 6.1.1 its interpretation: an aggregate near zero is what complementarity looks like measured with a mean.

Is the complementarity structured enough to be exploitable?

An oracle bound is only interesting if something could approach it, which requires the complementarity to be structured rather than random. Two pieces of evidence bear on this.

First, unique solves (tasks that exactly one architecture rescues) are concentrated rather than spread. On Qwen-32B, four of the six uniquely solved tasks are rescued by the verification family, even though

verification does not help that model on average. On Devstral-24B, the eight unique solves are spread across every non-baseline family: multi-agent, context-augmentation and verification alike. Architectures that look useless in aggregate are carrying specific tasks.

Second, the distribution of solve counts per task is bimodal rather than uniform: on Gemini-3.1-FL, 24 of the 49 ever-solved tasks are solved by at least twelve of the fourteen architectures while only two are solved by exactly one. The middle band (tasks where the architecture matters) is where all the routing headroom lives, and it is a minority of the task set.

Whether that middle band is *predictable* from task features determines whether the oracle bound is a design target or merely a documented ceiling. The evidence is mixed and we state it as such.

Against predictability: task difficulty (Section 6.5.3) does not separate the cases, so the most natural property knowable *before* running anything carries no information about which architecture will help. More decisively, the architecture that rescues a task is not even fixed across models. `png-generation` is the vivid case: render a PNG reading "Hello world!" and a JPEG reading "ScienceNews", solved at baseline only by Qwen-72B, and of the four failing models a scaffold rescues, no architecture rescues more than two: Qwen-14B recovers under G3-Reflexion, Devstral-24B under G1-Temp, G2-Monitor, G3-Reflexion, G3-Judge, G3-Gate or G4-Debate, Qwen-32B under G1-Prompt or G4-Debate, and Gemini-3.1-FL under G2-Explorer, G2-Planner, G4-Cooperation or G4-Orchestrator; the widest-reaching, G3-Reflexion and G4-Debate, each span only two of the four. The pattern generalises: over the 232 (architecture, task) pairs in which an architecture rescues at least one model, counted only on tasks where two or more models failed at baseline (so a win is a genuine choice among eligible models), its wins cluster by task far more than chance (the spread is 2.9× wider than a binomial) so certain architectures really do suit certain tasks, but the suitability does not survive a change of model. Two models that both fail a task are rescued by overlapping architecture sets at only a median $\kappa = 0.12$ over the 21 model pairs (a noisy statistic (few shared tasks per pair) but pointing the same way). A task-only router would therefore learn a signal that is real but mostly model-specific; it would have to condition on the model as well, returning the measurement cost the oracle bound was meant to avoid.

For predictability: the task-shape groupings of Section 6.5.4 do separate them, with a mechanism for each, and the named-artifact effect is a property readable directly from the task instruction. Task headroom separates them well but requires a baseline run first.

The position taken here is that a router is plausible but not demonstrated. What the study establishes is that the headroom exists and is large; what it does not establish is that it can be captured without first measuring the model on the task.

6.7.2 A selection rule

The oracle bound above would need the model measured on every task; a coarser question (which *family* to reach for) needs it measured only once. The three baseline properties of Section 6.2.4 (how early the model stops, what it fails at, and whether its scaffold's artifacts validate) compose into a rule that picks a family from a *single baseline run*, no sweep:

> If the model's valid-artifact rate is below 40%, use G1 only, since the scaffold will not run.
> Else if it stops early (under 20 steps), almost anything helps.
> Else if it grinds to the budget, use G4: restructure the effort rather than extend it.
> Else use G3: verify the completed attempt.

Scored against the family with the highest mean effect for each model, the rule names the right family for **6 of the 7**, but the qualification matters more than the figure. **The margins are thin:** "best family" is decided by under half a task in 3 of the 7 cases, separations that one run per configuration cannot tell from noise, so the rule points at the right region rather than scoring an accuracy. **Its one clear miss is instructive:** on Qwen-7B the rule says G1, because the model emits a valid plan only 0% of the time, yet the highest-mean family is in fact G3, a quarter of a task above a G2 that is itself carried entirely by G2-Monitor, the one G2 architecture that emits *no* structured artifact. That G3–G2 separation is within the same half-task band, so this model is also one of the 3 borderline cases. The rule was right that artifact-based scaffolds fail here, wrong only in treating a family as a block when the property belongs to individual architectures.

Its worth is not accuracy but economy and legibility: all three inputs come from one baseline run, a sixty-fourth of an architectural sweep (its cost is the subject of Section 6.7.3), and where it misses, the reason is a property it could be extended to read.

The same caveat as the task shapes. The rule is post-hoc in the sense the task shapes of Section 6.5.4 are: its thresholds and family assignments were fixed with the outcomes in view and scored on the same seven models, so it is read from the data rather than validated against held-out ones, and calling it a "predictor" on this evidence would be partly circular. What keeps it from being merely circular is that the mechanism it encodes (match the repair to how the model fails) was established independently in Sections 6.3 and 6.4; the rule reduces that mechanism to three baseline measurements rather than mining a fresh correlation from the outcomes, which is also why it has fewer free choices than the shape taxonomy and, unlike it, does not depend on a grouping that failed to reproduce. Confirming either as a genuine predictor would need models and tasks outside these seven, which Section 9.4 names as future work.

6.7.3 Cost–benefit

Steps are capped but tokens and wall-clock are not, so cost per solved task is the axis on which architectures differ most.

Table 6.36: Cost per solved task, pooled over all models (traced runs only, so a solve count can differ by one from Table 6.1), sorted by token efficiency. Steps are capped identically but tokens and time are not, so this is the axis on which architectures genuinely differ in cost. The comparison is against G1-Base, which is neither the cheapest nor the dearest.

Architecture	Mtok / solve	Seconds / solve	Solves
Plan	1.83	866	89
Debate	1.99	1337	103
Base	2.00	1064	99
Coop	2.03	1276	103
Analyst	2.06	1230	96
Judge	2.18	1310	110
Reflex	2.19	1379	107
Explor	2.22	1190	89
Prompt	2.27	1139	100
Orch	2.41	1395	94
Instr	2.53	1404	95
Temp	2.53	1162	89
Monitor	2.55	1476	100
Gate	2.64	1564	100

Table 6.36 reports that axis pooled over models, and G1-Base is *not* the cheapest way to solve a task. Two architectures, G2-Planner and G4-Debate, solve tasks at a lower token cost per solve than the plain baseline: G4-Debate spends more tokens in total but solves enough extra tasks that its *unit* cost dips just below the baseline's, while G2-Planner is the cheapest in the table outright. The spread from cheapest to dearest is roughly 1.5×: small enough that token cost should not be the deciding factor between architectures, and large enough that G1-Temp, among the most expensive and least effective, can be dismissed on both axes at once. Cost is rarely what decides between architectures here, because the accuracy differences reported in this chapter are larger than the cost differences and both point the same way for the architectures worth using.

Overhead measured where there is nothing to gain. Cost per solve pools tasks of every difficulty, which mixes overhead with benefit. One task separates them cleanly. `hello-world` (write a fixed string to a file) is solved by **all 116 runs**, so its headroom is exactly zero and every step beyond the minimum is pure architectural overhead. The median cost of the identical outcome ranges over a factor of five:

Architecture	Median steps	Input tokens
G2-Explorer, G3-Judge	2	8k
G1-Base and seven others	3	12–16k
G4-Orchestrator	6	36k
G3-Gate, G4-Cooperation, G4-Debate	7	33–39k

The split is by mechanism. G3-Judge is *cheaper than the baseline it wraps*: on work already correct the judge approves at once and the run ends a step early, so a conditional mechanism costs nothing

when it is not needed. G3-Gate is the most expensive architecture in the study on this task, because it intercepts *unconditionally* and forces a re-check of work that was right the first time. Here the conditional-versus-mandatory distinction separates the cheap architectures from the dear ones more sharply than any property of the models.

Per model, the pooled picture is contradicted. Reported *per model* rather than pooled, the cost picture is sharper, and in one respect reverses. Pooled across models, G1-Base sits mid-table on tokens per solved task. Per model, **the baseline is the cheapest option for only 2 of the seven**, and for the rest some architecture solves tasks more cheaply than the plain agent does:

Model	Baseline Mtok/solve	Cheapest architecture	Its Mtok/solve
Qwen-7B	2.27	G1-Base	2.27
Llama-70B	1.59	G2-Explorer	**0.66**
Qwen-14B	3.83	G2-Analyst	2.88
Devstral-24B	4.87	G4-Debate	2.53
Qwen-72B	1.39	G2-Planner	1.24
Qwen-32B	1.76	G1-Base	1.76
Gemini-3.1-FL	0.96	G3-Reflexion	0.58

On Llama-70B the best architecture solves tasks at *less than half* the baseline's token cost. The mechanism is arithmetic: cost per solve is total tokens over solves, so an architecture that raises the numerator modestly while raising the denominator substantially wins on this axis even when it looks expensive per run.

Qwen-72B is the telling case. G2-Planner is not its best architecture by solve rate, yet it is its cheapest per solve, because it is the leanest architecture in the study, consuming 70% of the baseline's steps. Optimising for cost and optimising for solves therefore pick different architectures, and this table separates the two.

6.7.4 There is no default architecture

The thesis's central claim can be made numerical. Taking each (model, task-category) cell with at least three tasks (70 cells in total) and asking which architecture is best in each (Table 6.37):

Table 6.37: Which architecture is best in each (model, task-category) cell, over the 70 cells with at least three tasks. Most cells have no winner, and no architecture wins many of those that do.

Winner	Cells	Share
No architecture beats the baseline	32	46%
G1-Prompt	10	14%
G2-Monitor	5	7%
G3-Reflexion	5	7%
G2-Planner	4	6%
G4-Cooperation	4	6%
G2-Analyst	3	4%
G3-Judge	3	4%
G4-Debate	2	3%
G2-Explorer	1	1%
G1-Temp	1	1%

Two numbers close the argument. In **46% of cells no architecture beats the plain baseline at all**. And **no architecture is best in more than 14% of cells**, that distinction belonging to the cheapest intervention in the study, three sentences of prompt.

This is the disaggregated form of the null main effect in Section 6.1.1, and it states the thesis's central claim in its most direct form: *there is no architecture one could adopt as a default.* Most architectures are the right answer somewhere and the wrong answer in most places (ten of the fourteen win at least one cell, while G3-Instruction, G3-Gate and G4-Orchestrator win none), and the aggregate that would justify a default does not exist.

Chapter 7

Discussion

Chapter 6 reported what happened, and interpreted each result where it stood. This chapter steps back from the individual findings to do what a single result cannot: argue for one account that fits the whole grid, set it against what other groups have found, convert what survives into guidance, and draw the methodological lesson the design is unusually placed to make.

7.1 One account of the whole grid

Every result in this study is consistent with one principle:

> *An architecture pays when it supplies something the model lacks, in a form the model can use, and costs the budget it consumes when it does not.*

Stated that plainly the principle sounds close to vacuous. It is not, because each of its three clauses is separately measurable, each fails in a different observed way, and the failures are distinguishable in the data. The first two clauses are the conditions under which an architecture *benefits* a run, the pair of separately measurable conditions the contribution rests on (Section 2.6), and the third fixes the *sign* of what happens when either fails.

7.1.1 The deficit must exist

The first clause is the revisability gap. The value an architecture adds is bounded by the distance between what the model achieves on its first attempt and what it could achieve given revision. Where that distance is zero, the architecture has room only to do harm.

The evidence is a gradient at two levels. Across models, the number of architectures beating the baseline falls from 13/13 on one of the weakest to 0/13 on the strongest open coder (Section 6.2.2). Across tasks, the *sign* of the binary effect is pinned at the extremes by how much room a task has left: every architecture loses solves on already-solved tasks and every architecture gains on floor tasks, while the contested middle bands span zero and settle nothing (Section 6.5.3). On the graders' partial credit the same bound holds without the same order: both families are negative or flat wherever headroom is scarce and positive only at the floor (+2.1 and +0.8 points), with the pre-execution family paying -4.4 on the tasks most models already solve.

The same principle explains a result that would otherwise look like an anomaly. Qwen-72B's best architecture is a three-sentence prompt, and its self-judge approves 57% of its own work, the most lenient in the study. A model satisfied with its own output has no deficit for a verification family to address, and indeed G3-Judge on that model is exactly +0.

Stating the clause positively sharpens it. **Scaffolding and capability are partial substitutes.** A scaffold performs *externally* (in a separate agent, a separate phase, an explicit artifact) the exploration, planning and self-checking that a stronger model has internalised and performs implicitly within a single forward pass. On that reading the inverse relationship with capability is not an empirical curiosity to be explained after the fact but a consequence of what the intervention *is*: two ways of buying the same function, so the one you supply externally is worth most where the internal version is weakest, and worth nothing where it is already present.

Two testable predictions follow from this, not merely rhetorical ones. The gain should track the *absence* of the function rather than the absence of capability in general, which is why Qwen-72B, a model whose self-judgement is lenient, gains from a prompt that supplies discipline but not from a judge that supplies scrutiny it will not act on. And an architecture cannot supply what has no internal counterpart to substitute for: no scaffold in this study injects knowledge or reasoning the base model lacks, which is why the floor tasks of Section 6.5.3 stay unsolved under all fourteen. Substitution has a ceiling, and it is the model's own capability.

7.1.2 The form must be usable

The second clause is the one the literature does not anticipate, and this study's clearest practical contribution. An architecture communicates through an artifact (a plan, a summary, a routing instruction, a verdict), and if the model cannot produce or cannot act on that artifact, the architecture does not run.

Three independent lines of evidence establish it. **Production:** Qwen-14B emits a schema-valid plan in 5% of runs and Qwen-7B in 0%, while Llama-70B (near the bottom of the range by solve rate) manages 95% (Section 6.4.2). Structured-output compliance is a separate axis from task capability, and where it fails, a reported G2 regression measures a scaffold that never executed rather than one that executed and failed. **Delivery:** the monitor's guidance is injected as the executor's own reasoning step, and self-monitors almost never write "you" while the effective supervisor almost always does (Section 6.6.3). **Interface capacity:** the analyst is the only role demanding a summary and a plan in one artifact, and it is the only supervisory role where a stronger model helps nothing at all, because it writes more into a fixed schema and the artifact validates less often (Section 6.6.3).

7.1.3 The unpaid cost is the budget

The third clause is why the effect is negative rather than merely absent when the first two fail. Because the step budget is carved rather than added (Section 4.3), every step spent on an architecture's machinery is a step not spent executing. On a model with a real deficit that trade is worth making; on one without, it is pure loss.

Section 6.1.2 closes the obvious objection to this account. If architectures helped by inducing more work, step consumption should predict benefit. It does not: within a model the correlation between an architecture's step ratio and its effect is approximately zero, the leanest architectures are among the most effective, and (decisively) runs that *succeed* consume roughly half the steps of runs that fail, in all 7 models without exception. Extra effort is a symptom of difficulty, not a mechanism of success.

7.2 Why the aggregate view misleads

Section 6.1.1 reported the near-zero main effect and Section 6.7.1 showed it to be what complementarity looks like when measured with a mean. What remains is to say what kind of error it invites, because the error is not specific to this grid.

The shape is familiar from the debate over emergent abilities, where an aggregate curve was found to depend on the choice of metric rather than on any discontinuity in the underlying model [49], [99]. Here the aggregate hides not a metric artifact but an interaction: averaging over models destroys the very quantity being measured. G2-Monitor is $+5$ on one model and -5 on another. G4-Orchestrator is the best architecture in one category and the worst in another, from the same code.

The interaction is not an artifact of how the averaging was done, the obvious rejoinder. Counted as discrete events, conversions cluster by task, yet two models that fail the same task barely agree on which architecture rescues them (Sections 6.2.3 and 6.7.1): the task-level structure is real but far smaller than the model-level one, and no choice of summary statistic recovers a main effect from that ratio.

This has a direct methodological implication. A study that evaluates a scaffold on one model and reports an improvement has measured one cell of an interaction and reported it as a main effect. Depending on which model it chose, the same architecture would have supported the opposite conclusion, and on this evidence the choice of model matters more than the choice of architecture. An architectural claim should therefore come with the capability profile of the models it was measured on, and ideally with at least one model on each side of the range.

7.3 Why each family behaves as it does

7.3.1 Pre-execution decomposition

G2 is the weakest family overall, and three distinct mechanisms contribute. It *fails to run* on models that cannot emit its artifacts. It *plans without information* on tasks whose structure only becomes apparent through execution: on the propose–check–revise tasks of Section 6.5.4, every pre-execution architecture is negative, because the first observation invalidates the plan. And it *spends budget on structure the model did not need*, which is the ordinary case on capable models.

A fourth mechanism is a matter of applicability rather than degree: **the family's premise does not always hold**. G2 assumes there is data in the environment to ground in, and where there is none the grounding phase wastes its own steps and sends the executor looking as well, for files that do not exist.

That case is a design fault rather than a model failure, and it is cheap to remedy: an architecture whose first phase discovers an empty environment should disable itself rather than proceed. It also sharpens the aggregate result, since part of what looks like "pre-execution decomposition underperforms" is that decomposition being applied to tasks with nothing to decompose or inspect. The family is not uniformly bad, and the exception locates the mechanism: on specified transformations over data that is present and inspectable, pre-execution architectures are the *best* family (Section 6.5.4). Where the information needed to plan is available before acting, planning pays.

7.3.2 Post-execution verification

This is the family the literature disagrees about most sharply. Intrinsic self-correction (a model revising its own reasoning unaided) has been found not to work [61], while tool-grounded critique does [62]. The G3 results fall on the second side, and the design explains why: the verifier here inspects the environment, so its critique is grounded in state rather than in the model's own recollection.

G3 is the most robust family, for a structural reason: it is *conditional*. It consumes budget only when the agent submits, so on runs where the first attempt is already correct it costs almost nothing, and it targets the dominant failure mode, a deliberately submitted wrong answer, the largest failure category for every model in the study.

Two findings qualify this. Verification is often *budget-constrained*: pooled across models, a submission the verifier intercepts leaves it enough budget to act only 63% of the time (Section 6.4.3), so the measured G3 effect is the effect of verification that acts on about two thirds of what it intercepts. The family's ceiling is higher than its measured value, and the binding constraint is budget scheduling rather than verification quality. And verification is *calibration-sensitive* in a way the other families are not, which Section 7.4 treats separately.

7.3.3 Multi-agent collaboration

This family's negative result is the one most corroborated elsewhere. Multi-agent gains shrink as the underlying model improves [1], single agents match or beat multi-agent systems once thinking budgets are equalised [3], and a substantial part of the reported benefit is attributable to additional sampling rather than to collaboration [74]. The contribution here is to show the same pattern under a budget that is carved rather than merely counted, across seven models of differing capability.

One qualification the supervision track adds: where a stronger model does help a weaker one, the contribution is often narrower than "better reasoning". In the one task solved by a single supervised configuration in the entire grid, what the supervisor supplied was one observation about the container that the worker had already printed and not read (Section 6.6.3). Coordination capability and environmental attention are not the same resource, and the second is cheaper to supply.

G4 introduces failure modes that cannot otherwise occur: coordination failure and cycle-cap exhaustion are termination classes exclusive to it (Section 6.3.3), and it is the family most exposed to the compute-fairness objection. Yet it is also the family that advances hard tasks furthest on partial credit, and on that measure G4-Orchestrator is the best architecture in the one category whose tasks are genuinely multi-stage and separably verifiable (the category's binary-solve lead belongs to the bare prompt).

What relaying actually buys. One G4 pattern the family's own rationale does not predict recurs across two architectures that share nothing structurally. G4-Cooperation, included to test whether

fresh context alone breaks stagnation, solves its one unique run not by supplying a better idea but by supplying **a reader for an observation the first agent had generated and not absorbed** (Section 6.4.4); the supervised monitor rescues `setup-custom-dev-env` by the opposite means, naming a wrong-architecture download the worker had printed and not read (Section 6.6.3). The common defect is neither a reasoning failure nor one any scaffold here was designed against: agents generate the information they need and then do not use it. That two unrelated architectures repair it cheaply (one by stationing an observer, one by rotating the seat) points to a deficit of attention to the environment rather than of capability, which Section 9.4 takes up as the intervention the results most directly argue for.

The heterogeneous track resolves the apparent contradiction. G4-Orchestrator looked in the homogeneous grid like an architecture whose overhead grows with capability; separating the roles shows the cost was never in coordination but in *routing competence* (Section 6.6.2). A weak model strategising for its own specialists burns cycles it cannot spare, exhausting the cycle cap in 26% of runs against 15% under a competent strategist. With a stronger strategist that cycle-cap rate falls from 26% to 15%, so the orchestrator's cost depends mainly on the quality of its routing rather than on orchestration itself.

7.4 Injecting capability at the coordination layer

The supervision track answers a question the homogeneous grid cannot: if scaffolding helps by supplying what a model lacks, can that supply come from a *different, stronger model*?

The answer distinguishes two kinds of role, and the distinction is the thesis's most transferable finding. **Directive roles** (strategist, planner, explorer, monitor) emit information that is *added* to the worker's process. More capability produces better information, so a better-informed worker does better, and the gain grows with the worker's ability to act on what it is given: G4-Orchestrator gains +4, +5, +8 across the three workers in increasing order of strength.

Evaluative roles (the judge) emit a *threshold* applied to the worker's output, and there raw capability does not transfer the same way. The Gemini judge approves at least as well as the worker's own, but its rejections rescue markedly less, and on the strongest worker the trade is net negative (Section 6.6.4). Whether its extra rejections sit above the grader's tolerance or are accurate but beyond the worker to repair cannot be separated from traces that grade only the final state; either way, a gate helps only in so far as its bar matches the criterion the task is scored against, and a stronger model does not guarantee that match.

> *In a directive role, maximise the supervisor. In an evaluative one, do not assume a stronger supervisor is a better gate: what a threshold is worth depends on its alignment with the grader, not on the capability behind it.*

The economics follow the same rule. Heterogeneity is cheaper per solved task for all three workers (Section 6.6.6): its benefit is the wasted work it prevents, so the denominator rises faster than the consumption does.

7.5 Design guidance

Each of the following is an imperative a practitioner could act on, supported by the evidence in Chapter 6 and bounded by the scope conditions of Chapter 8.

Measure the model before choosing the architecture. Baseline solve rate predicts architectural benefit; parameter count does not. A 70B model and a 7B model in this study are separated by one task, and the 32B coder outperforms the 72B general model of its own family.

Three measurements from one baseline run tell you which family to try. Section 6.7.2 derives a rule from how early the model stops, what its dominant failure is, and whether it can emit a valid structured artifact. It names the right family for 6 of 7 models, on margins thin enough that the figure should be read as pointing at a region rather than as an accuracy.

Check that the scaffold runs before concluding it failed. If a model emits valid plans 5% of the time, its planning results measure schema compliance and not planning. This is cheap to check and changes the interpretation entirely.

Establish the cheap floor first. A three-sentence prompt is the best architecture on one of the seven models and within a task of the best on another. Architectural complexity should be required to beat that floor, not merely the minimal baseline, and on `log-summary` the prompt reaches 7/7 where the analyst falls to 2/7, because what an already-solved task punishes is elaboration, not intervention.

Prefer conditional mechanisms. Verification costs almost nothing when unnecessary because it engages only on submission. That property, more than its accuracy, is why it is the most robust family, and it generalises: work on reducing agent cost without losing capability converges on the same principle, that machinery which fires only when needed dominates machinery that always runs [100].

Match the mechanism to the failure. Extending a run helps a model that stops early and does nothing for one that already exhausts its budget; restructuring helps the grinder. The monitor gains -1 on the model that stagnates and $+5$ on the model that abandons tasks early, with identical code.

Ask which task, not whether. The gain from choosing the best single architecture is $+0$ to $+6$ tasks and collapses with capability; the per-task oracle bound is 6 to 14 tasks, but against single-run noise its excess is real on the weaker models and within noise on the strongest (Section 6.7.1), so the per-task view pays off where a model still has tasks to gain.

7.6 Why failure analysis does not yield architecture

The last result is a methodological one, and the grid is unusually well placed to establish it because the grid was built on the inference it tests.

The architectures in this thesis were not chosen arbitrarily. Each was selected because trace analysis of the baseline runs identified a concrete, recurring failure and the architecture addressed that failure directly: the agent that never inspected its input files motivated the explorer, the agent that reissued the same broken one-liner twenty times motivated the monitor, the agent that submitted work failing an obvious check motivated the judge. Applying all fourteen across seven models therefore constitutes a controlled test of the reasoning that produced them.

The diagnoses hold. Chapter 6 confirms every one of them as a real and frequent behaviour: deliberate submission of wrong work is the largest failure category for every model, identical-command loops occupy 27.9% of runs, and the models that never inspect their inputs are identifiable from their tool-call profiles. What does not hold is the step from diagnosis to remedy. An intervention that removes a failure mode does not thereby improve the outcome, because it is paid for in the same budget the executor needed and because it introduces failure modes of its own, a general property of staged agent pipelines, in which a single early error propagates into every later decision [101]. The explorer's summary displaces the executor's own observation, the monitor's advice is ignored, the judge rejects work that was already correct.

The practical form of this is a warning that applies well beyond the present study: "this is why it fails" does not license "therefore fix this". The step breaks silently, precisely because the diagnosis it rests on is sound, and the only way to find out is to build the intervention and measure it against a budget-matched control.

What this account does *not* establish (that these are our own implementations, that the task band is narrow, that the budget is one point, that no cell is individually significant, and that the task-shape grouping is post-hoc) is set out in full in Chapter 8, and every claim in this chapter should be read against it.

Chapter 8

Threats to Validity

8.1 Statistical conclusion validity

Single-run point estimates. No configuration is repeated, so individual cell deltas are within noise. Section 5.5 sets out the four properties the conclusions rest on instead; the operative discipline is that no argument in this thesis depends on a single cell. A direct scale for the noise is the G1-Temp cell, a second baseline at $T = 0.2$: re-rolling it alone moves -5 to +4 tasks, so per-cell swings of a few tasks carry no weight.

Regression to the mean in the gradient. The headline relates a model's baseline to how much architecture helps it, and the baseline is a single run on both sides: it is the capability axis and it is subtracted inside every delta. That coupling pulls the correlation negative even with no real effect: reshuffling which configuration is labelled the baseline gives an expected ρ of -0.21, not zero. The gradient is therefore judged against that null rather than against zero: the observed -0.886 clears it at $p = 0.037$, and it survives two checks that do not reuse the baseline (capability from the thirteen non-baseline cells, $\rho = -0.829$; a split-half, median $\rho = -0.62$; Section 5.5). What the coupling inflates is the magnitude of the slope, so the claim is made as a direction, not a rate. The weak-versus-strong contrast is exposed the same way, because its weak group is itself selected on the noisy baseline; under the reshuffling null it is significant at $p \approx 0.033$, and that corrected figure is the one the thesis uses.

Residual non-determinism. Greedy decoding removes controllable sampling variance, but batched GPU inference is not bitwise deterministic, so repeated runs would not be identical even at temperature 0. Section 5.5 explains the mechanism; this is acknowledged rather than claimed absent.

Multiplicity. The grid contains many cells, and inspecting all of them invites over-interpretation of extremes. Section 5.7 fixes the correction procedure in advance of the comparisons.

8.2 Internal validity

Coverage. Every cell was run on every task (Section 5.4), so no rate in this thesis rests on an inferred outcome and coverage is not a threat to any comparison drawn from the grid.

One misconfigured cell. One run (Qwen-7B on `hello-world` under G3-Instruction) was launched with the G3-Reflexion verifier variant. It is recorded and counted as it ran. The task is solved by every model under every architecture (98 of 98 runs), so the misconfiguration cannot have changed an outcome, but it is reported rather than silently corrected.

Missing behavioural traces. A small number of executed runs lost their logs to a storage fault. Their outcomes are known and counted; their traces are absent and excluded from behavioural aggregates, with the exclusion recorded per row so the two populations can never be silently mixed.

A context-guard fix splits the grid in time. This is the most consequential internal threat in the study and it was found late, during a consistency check on the failure-mode tables. The mechanism that trims old messages when the context window fills was unreliable in the earliest runs. Every one of the 138 runs that terminated in context overflow was executed before a fix in May 2026; across the 4,525 failures recorded after it, the class does not occur once. The affected population is 585 of the 6,272 runs in the homogeneous grid (9.3%), concentrated in 8 of its 98 cells. The supervised track is untouched: it was executed entirely after the fix and contains no overflow at all.

The uncomfortable part is placement rather than size: four of those cells are *baselines*, those of Qwen-7B, Qwen-14B, Qwen-72B and Qwen-32B. An artificially depressed baseline inflates the measured architectural gain, and that cuts both ways here. For Qwen-7B, the weak model the architectures help, it exaggerates a gain the finding already expects, so the fault flatters the claim; but Qwen-14B, Qwen-72B and Qwen-32B sit where the finding is instead that architecture *hurts*, and an inflated gain understates that harm, so on three of the four cells the bias runs *against* this thesis's claim. Two things bound it. First, the direction is known and stated wherever those cells are quoted. Second, a deliberately extreme sensitivity check: assuming every one of the 138 overflowed runs would otherwise have solved its task (implausible on its face) lifts the affected baselines sharply (Qwen-7B's by nineteen solves) and so reshuffles the capability axis, yet the rank correlation it produces, $\rho = -0.886$, is still strongly negative, so the gradient is not an artefact of the fault. What the fault does reach are the per-cell magnitudes for those four models, and re-running the eight cells on the current harness is the cheapest outstanding item in this work. Section 6.2.4 corrects one claim that the fault had previously turned into a mystery.

The step budget truncates a small tail. 58 runs (3.6% of all solves) succeeded on step 60 exactly, meaning they finished on the last decision available to them, and a larger budget would have recovered a few more. All reported solve rates are therefore mild underestimates in absolute terms. The comparison is not biased, because the cap is identical across architectures and models, but two consequences should be noted: the G3 family is affected disproportionately, since its verification phase competes for this same tail of the budget (Section 6.4.3 shows that an intercepted submission leaves the verifier room only 63% of the time), and the absolute numbers would rise slightly under a larger cap.

The easy-band restriction is measured, not merely assumed. A pre-specified medium/hard boundary probe (160 baseline runs; Sections 5.3.2 and 5.3.1) replaces what was previously an inference: only 10.0% of medium and 1.2% of hard attempts solve, confirming that little architectural signal could survive past the easy band. What the probe does *not* settle is the study's own mechanistic prediction, that strong models on harder tasks would recover a revisability gap and benefit again; the probe is baseline-only and its tasks are mostly too hard for even the strongest model to gain traction, so it brackets that regime without populating it. This remains the limitation most likely to bound the generality of the headline result.

Architecture implementation quality. The architectures are our implementations of published designs, not the authors'. A poorly implemented architecture would understate that family's potential. The within-family evidence in Section 6.2.1 constrains this concern (the identical code helps some models and harms others) but does not eliminate it.

The dialogue budget split. The dialogue architectures run at seven steps per agent. G4-Debate's eight-turn cap therefore consumes 56 of the 60 steps every other configuration receives (G4-Cooperation continues to the full 60), a small budget disadvantage for G4-Debate, and one better set at six steps per agent (a clean ten turns of 60). It does not affect the family's verdict. The median dialogue run finishes in 51 steps, below even that cap, so the shortfall never binds on most runs; the runs that do reach the cap sit at a median of zero passed checks, far from a solution a further round could complete. If anything the slightly smaller effective budget makes the "coordination does not pay" result conservative rather than inflated.

8.3 Construct validity

Solve rate as the measure. Success is binary pass/fail on the benchmark's own test suite. This measures task completion, not solution quality, efficiency, or safety.

Capability as baseline solve rate. The capability axis is operationalised as G1-Base performance on this task set. It is a task-specific measure, not a general capability claim, and the ordering it induces differs from parameter count deliberately.

Named construct versus measured construct. A benchmark task names a capability; the harness decides which capability is actually exercised, and the two can diverge. The clearest instance is `gomoku-planner`, which the benchmark frames as visual game reasoning by supplying the board as an image. Because the agent framework has no image channel (detailed in the next paragraph), no model is shown the board; every model, the hosted one included, must write computer-vision code to recover it from pixels, so the task as run measures perception-by-code rather than perception. This does not bias any comparison in the study, since all models face the identical text-only channel, but it does mean a per-task result should be read as evidence about the capability the harness elicited, not necessarily the one the task title advertises.

What the agent actually observes. The observation returned after each shell command is not that command's standard output but the visible terminal screen. The agent's terminal wrapper brackets every command with a unique start marker and an end marker formed by appending a suffix to it, then extracts the text between them. Because the start marker is a strict prefix of the end marker, the two reverse searches locate the same position, the guard that requires the end to follow the start is satisfied, and extraction falls back to returning the captured pane in full. The between-markers path is therefore unreachable whenever a command completes, and the model receives the screen as a human at the terminal would see it: command echoes, shell prompts, the markers themselves, and any scrollback left by earlier commands. The information is usually present but embedded in noise; it is lost only when a long-running command (a package installation is the recurring case) fills the pane and pushes the relevant result off it. Traces show models reasoning about this directly, one remarking that the output "is still showing the installation logs, which is very strange" before working around it.

This is a defect in the agent implementation written for this study, not in the benchmark, which supplies both a pane-capture call that behaves as documented and an incremental-output call better suited to the purpose that the wrapper does not use. Its consequence for the comparisons is contained: the extraction routine is byte-identical across all eight system implementations, so every architecture and every model faced the same observation channel, and no contrast in this thesis is confounded by it. What it does change is the construct. The study measures agents operating on a noisy terminal view rather than on clean command output, which is one reason its absolute solve rates should not be read against published figures obtained with other agent frameworks. Individual failures can occasionally be traced to it: one G2-Analyst run issues the same existence check three times, is shown unrelated pane contents each time, and submits without the artifact, but the traces do not support any estimate of how many failures across the grid it accounts for, and none is offered.

A second and rarer fault in the same layer is worth recording because its consequence is total. The wrapper writes files by opening a shell heredoc, sending the content line by line through the terminal, and then sending the closing delimiter, with no check that the write took effect. If the delimiter is not registered, the shell stays in heredoc mode and silently consumes every subsequent command as content, so the session never recovers. A striking instance is a Gemini-3.1-FL run on `countdown-game`, which writes the correct answer (`11*(75-25-8)`, which evaluates to the required 462) as its very first command, receives a timeout, concludes across three further attempts that "the environment is completely unresponsive", and submits the right expression to a file that was never written. It is scored as a failure. This is bounded and small: 3.1% of traced runs contain at least one command timeout, **27 runs (0.4%) never recover**, and 25 of those 27 fail. An open heredoc is visible in only two of the 27, so the mechanism just described is not the general explanation for the rest, and no attempt is made to make it one.

8.4 External validity

Task difficulty band. The architecture grid uses only the benchmark's "easy" band. Conclusions may not transfer to harder tasks, where the revisability gap could behave differently, plausibly *larger*, since strong models would no longer be near their ceiling. The boundary probe (Section 5.3.2) measures the near side of this: at baseline the solve rate collapses to 10.0% on medium and 1.2% on hard tasks, so the easy restriction is well founded. What no data here can supply is the behaviour of the *full architecture grid* on harder tasks, which remains the most consequential scope limit and the direct route to future work.

Quantization and serving. All six open models are AWQ 4-bit, so results describe deployable quantized artifacts rather than full-precision weights, and absolute solve rates are not comparable with published full-precision figures. Section 6.2.5 tests whether quantization instead explains the *capability ordering* the study is built on, and rejects that on two independent grounds. What those tests cannot exclude is a uniform downward shift affecting all six models roughly equally, which would compress the measured range without reordering it, leaving every ordering claim intact but rendering absolute rates unreliable. Excluding it would require a full-precision run of at least one model, which the available hardware did not permit.

Heterogeneity confound. In the supervision track the supervisor is hosted while the workers are local and quantized, so supervisory capability is confounded with serving stack (Section 4.11.2).

Capability substitution. Because a directive supervisor issues instructions, a supervised solve could in principle reflect the *supervisor's* capability rather than the worker's, which would make the architectural "unlock" an artefact of substitution. The design and the data bound this. Everything but the supervisory call is held identical to the homogeneous cell (Section 4.11.2); the supervised solve rate is ceilinged by the worker's own reach, since only 5.9% of solves rescue a task it never solves alone (Section 6.6.5); and a task Gemini-3.1-FL solves standalone is not solved when a weaker worker must execute it. What the track measures is coordination-layer capability applied *through* the worker's execution, not the supervisor's capability substituted for it. The residual case is the 5.9% rescue, where the supervisor's contribution is most load-bearing; it is reported as such rather than folded into the aggregate.

Model generations. The study fixes a set of models at a point in time. The gradient is a claim about the relationship between capability and architectural benefit, not about the specific models occupying positions on it.

Chapter 9

Conclusion and Future Work

9.1 Summary

This thesis asked whether the value of an agent architecture depends on the capability of the model it wraps. To answer it we ran 14 architectures across 7 models on 64 TerminalBench tasks under a uniform, carved 60-step budget, plus a heterogeneous supervision track, totalling 7424 model–task rows.

The answer is that it does, in a consistent direction across the capability range. The number of architectures that beat a model's own baseline falls with that baseline, from 13 of 13 on one of the weakest models to 0 of 13 on the strongest open coder, and individual architectures change sign across the range. The claim's most robust form assumes no ordering at all: on the two weakest models 24 of 26 cells improve on baseline, on the remaining five the direction reverses, and the difference in mean effect is +3.51 tasks (reshuffling null, $p \approx 0.033$). The rank correlation across the open models, $\rho = -0.886$ ($p = 0.033$), agrees with it but rests on 6 points. Averaged across models the main effect of architecture is near zero. This does not mean architecture does little; large per-model effects of opposite sign cancel, so the average is the wrong lens on a complementary set. Different architectures solve different tasks, and choosing the one that fits each task recovers real headroom on the weaker models (+3.2 tasks beyond single-run noise) while leaving nothing beyond noise on the strongest.

We account for these results by where an architecture pays off: its value tracks the distance between a model's first-attempt quality and what it could reach with revision. Because the step budget is shared rather than added, the effect is not an artifact of a larger action budget. An architecture that adds no actions still helps weak models and burdens strong ones, as a structural consequence of spending shared steps on something other than execution.

9.2 Contributions revisited

The six contributions claimed in Section 2.6, assessed against what the evidence finally supported.

(1) A capability-conditional result: supported, with a stated limit. The count of architectures beating a model's own baseline falls from 13 of 13 to 0 of 13 across the capability range, and individual architectures change sign. Aggregated, the effect vanishes: in half of all (model, category) cells no architecture beats the baseline and none is best in more than 14% of them (Section 6.7), which is the null main effect a large interaction produces under a mean. The limit is that no single cell is statistically distinguishable; the claim rests on direction across many cells, and the thesis says so wherever it is quoted.

(2) Complementarity: supported, once read against noise. Different architectures solve different tasks; choosing the one that fits each task recovers +3.2 tasks beyond a single-run noise null on the weaker models and nothing beyond noise on the strongest. The per-task view therefore recovers real headroom where a model still misses tasks, a practical lever on the weaker models rather than a capability-independent law.

(3) A mechanistic account: supported, and one of its two conditions was not foreseen. The account survives, and the per-model failure profiles confirm it: an architecture pays off where a model's first attempt leaves room for revision. The second condition, that the architecture's output must be in a form the model can produce and act on, was not part of the original framing and emerged during the

analysis. Structured-output compliance turned out to vary by a factor of twenty across models and not to track capability, which reframes a substantial part of the G2 results as scaffolds that never ran.

(4) A per-model selection heuristic: a direction, not a validated rule. It reaches the best family for 6 of 7 models from a single baseline run. Two qualifications travel with that. It is derived from these seven models and evaluated on the same seven, so it is a decision procedure with a stated mechanism rather than a validated predictor; and 3 of the seven calls are decided by margins under half a task, which at one run per configuration is inside the noise. What survives is the shape of the rule (three cheap measurements identify the region to try) and the refinement its one clear miss names: distinguish artifact-emitting from artifact-free architectures within a family rather than treating the family as the unit.

(5) A heterogeneous supervision study: supported, and it produced the most transferable finding. The directive/evaluative distinction was not anticipated at the outset. It emerged from the observation that two architectures move in opposite directions as the worker strengthens, and it survived a refutation of the first explanation offered for it.

(6) A reproducible analysis substrate: supported. Every number in this thesis is generated from the frozen tables (Appendix A.1), which is what allowed several claims to be corrected when they proved wrong.

What independent work says. The headline result's weakness is that no single cell is statistically distinguishable, so it rests on direction across many cells. That weakness is substantially offset by evidence this study did not produce. Two independent groups report the same shape, each holding fixed what the other varied. Equalising *thinking budget* and varying the architecture, single agents match or beat multi-agent systems on multi-hop reasoning, the reported advantage being attributed to unaccounted computation [3]. Holding the architecture fixed and varying *capability*, the multi-agent advantage shrinks as the underlying model improves [1]. A third line shows that simply instantiating more agents and voting recovers much of the claimed benefit, which locates it in sampling rather than in collaboration [74].

This thesis varies both axes at once, a carved budget across seven models spanning the capability range, on a third task family, terminal work rather than reasoning benchmarks. The convergence matters for the objection this study is least able to answer from its own data. A reader may reasonably ask whether the multi-agent architectures here were simply implemented badly. Chapter 8 answers that internally: identical code lifts the weakest model and sinks the strongest. The external answer is stronger, since bad implementations in *this* harness cannot explain why two other groups, with their own implementations and benchmarks, arrive at the same relationship.

The convergence is not uniform. This study's most robust family is post-execution verification, which sits against a prominent finding that models largely cannot correct their own reasoning unaided [61]; Section 7.3 reconciles the two through the design (our verifier inspects the environment, not only its own recollection) but as an argument, not a measurement, which a reader may weigh as such.

Where the analysis weakened its own claims. The task-level analogue of the capability gradient is *not* statistically significant, and is reported as a non-result. And the task-shape grouping of Section 6.5.4 is post-hoc: the shapes were read off runs whose outcomes were already known, so they are offered as a hypothesis with an identified mechanism rather than as a validated predictor.

9.3 Limitations

The principal limits are single-run estimates, the easy-band task set, uniform quantization, and the post-hoc character of the task-shape grouping. Chapter 8 treats each in full.

9.4 Future work

Harder tasks. The most valuable extension, and the boundary probe of Section 5.3.2 takes its first step. That probe establishes the near side at baseline, where the solve rate collapses to 10.0% on medium and 1.2% on hard tasks, so signal thins sharply past easy. What it cannot reach is the prediction that matters: if the account is correct, that an architecture pays off where a model's first attempt leaves room to revise, strong models on harder tasks should recover that room and benefit from architecture again, turning a monotone gradient into a curve with a predictable optimum. Testing that needs the *full* grid

on a difficulty band tuned to leave a capable model with room to revise, not a baseline probe: a direct, falsifiable test of the mechanism rather than a broader sweep.

Repeated runs on the decisive cells. Repetition on a targeted subset (each model's baseline and its best and worst architecture) would convert the direction-based argument into one with error bars, at a fraction of the cost of repeating the full grid. It would also unlock a second result this data cannot reach, stated separately here because the attempt has been made and measured.

A task taxonomy the data can certify. The task shapes of Section 6.5.4 are the study's weakest construction: they are read off runs whose outcomes were already known. The principled replacement is to derive them by clustering tasks on their measured response to the architectures, and Section 6.5.4 reports what happens when that is attempted on the present data. The clustering does not reproduce, at an adjusted Rand index of 0.41 across random initialisations and 0.40 when a single model is withheld, and the only stable partition recovers the sign of the response, which headroom already supplies.

The obstacle is measurement noise rather than method. Each coordinate of a task's response vector is a difference between two single runs, so its uncertainty is of the same order as the effect being clustered; with 64 tasks in 13 dimensions there is no averaging to rescue it. Three repetitions per cell on the contested band would shrink that noise by roughly $\sqrt{3}$ and make the question answerable. This is a second, independent argument for repetition: not only to put intervals on the headline claim, but to convert the chapter's most contestable grouping from a hypothesis into a measurement.

Learned routing. If the complementarity of Section 6.7.1 proves structured, the per-task gain becomes a target: a router selecting an architecture per task from cheap task features.

Deeper heterogeneity. The supervision track varies one role. A full role-by-role capability assignment would ask which roles genuinely require capability and which can be served by a small model. The cheap version needs no hosted model at all: routing with the 72B general model over 32B specialists would test whether coordination quality can be bought within the open lineup, which is the deployable form of the finding.

Reasoning-distilled models. The lineup deliberately excludes models with a separate inference-time reasoning budget (Section 5.2.1), because that budget is a third axis the carved step budget does not control. Re-running the grid on such a model is the single extension most likely to test this account of where architecture pays off: a model that reasons at length before acting may already perform internally the deliberation a pre-execution scaffold supplies, which would predict the pre-execution family helping it even less.

9.4.1 Interventions the results argue for

The extensions above all vary the *study*. The following vary the *agent*, and each follows from a specific result rather than from a general wish for more work.

Hard interventions instead of advice. The clearest negative result in this thesis is that detection succeeds and recovery fails: the monitor identifies stagnation correctly and the executor continues regardless (Section 6.6.3), and in aggregate it reduces looping by -2.2 points while 9 of the 11 measurable architectures raise it (Section 6.3.4). The case study sharpens what would have to change. Two monitors reading the same log both detected the loop; the one that broke it did so by naming the *cause* (a wrong-architecture download visible in the command output) rather than the symptom. The deficiency is therefore not that advice is too soft but that it is too shallow: a repetition counter cannot produce that sentence, and a monitor that reads observations rather than counting commands might. Every intervention tested is *advisory*: it appends words to a context the model is free to ignore, and it does. The untested alternative is coercive: refuse to execute a command that repeats a recent one, require an explicit enumeration of alternatives before permitting another attempt, or force a tool switch after k identical failures. This converts the monitor from a commentator into a constraint. The risk is precisely stated and worth measuring rather than assuming: on tasks where persistence is the correct strategy, a repetition block is an intervention against a working approach, so the experiment must report what it costs on the tasks it does not help.

Closing the think–act gap. A recurring trace pattern is a correct diagnosis in `think` followed by an action that does not implement it. The failure is not reasoning but *transcription*, and it is invisible to every architecture here, all of which treat the reasoning step as free-form text. A scaffold could require each `think` to end in a concrete declared next action and then check the action actually taken against that declaration, turning an unenforced intention into a verifiable one. This is a cheap intervention that targets a failure mode none of the fourteen addresses.

A shared artifact store instead of a summary. Section 7.3 attributes much of G2's weakness to what survives the handoff: the executor receives a compressed summary rather than the observations that produced it. The architectures tested all pass *text*. An alternative passes *structure*: a store of file contents, discovered facts and error history that any phase can query, in the manner of the orchestrator pattern [71]. That would test whether pre-execution decomposition fails because the decomposition is wrong or because the channel between phases is too narrow, which the present design cannot separate.

Multi-attempt verification. Verification is capped at one round here. Given that the family is already budget-starved (Section 6.4.3), the interesting variant is not simply more rounds but *budget-aware* ones: a verifier that knows how much remains and adjusts between a full repair and a minimal correction. Pairing it with non-greedy sampling on the retry would test whether critique and stochastic exploration are complementary: G1-Temp and G3-Judge are the two architectures that help the models at opposite ends of the range, and they have never been combined.

Adaptive reading. The Explorer's fixed line cap (Section 4.7) means it never sees the middle of a long file, an avoidable instance of a documented weakness, since models attend least reliably to that region even when it is supplied [27]. Contemporary terminal-agent systems treat context curation as a first-class engineering concern for this reason [102]. Sampling by structure (headers, delimiter changes, type transitions) rather than by position would test whether G2's grounding deficit is architectural or merely an artifact of how its reading budget was spent.

Appendix A

Appendices

A.1 The frozen analysis dataset

The entire study is reduced to three tables from which the quantitative claims of this thesis are generated mechanically. Every number a claim rests on is a macro resolved at build time from these tables, so refreshing the data updates the prose and the two cannot drift apart; numeric literals that survive in the text are illustrative detail inside a single worked example, such as the step at which one run exhausted its context, the command a particular agent reissued, and no argument depends on them. The build additionally checks itself against the source, as Section A.1.3 describes.

A.1.1 Schema

`runs.csv` carries one row per (cell, task) pair: 7424 rows over 116 cells and 64 tasks, every one of them executed. The schema still carries an `executed` column, so a rate can always be recomputed and checked against the count of rows it was taken over. Table A.1 groups the columns.

Table A.1: Column groups in `runs.csv`.

Group	Columns
Identity	model, track, worker/supervisor model, family, parameters, tuning, serving, architecture, group, task, subset
Outcome	executed, resolved, has_trace, run directory, subtests run / passed / fraction
Behaviour	steps, elapsed seconds, termination reason and derived class, input and output tokens, per-tool call counts, guard counters, messages dropped
Explorer	summary validity, files inspected, steps, elapsed, tokens, termination
Planner	plan validity, subtask count, steps, elapsed
Verification	variant, phase-1 steps and elapsed, intercepted, verifier steps and elapsed, intercept count, judge verdict, feedback and reflexion presence
Orchestrator	variant, cycles, orchestrator calls, burst steps, knowledge-base entries
Dialogue	variant, total turns, steps per turn
Configuration	model id, provider, temperature, max steps, max wall-clock, context and token caps

`cells.csv` aggregates to the 116 model–architecture cells, and `tasks.csv` joins the 64 tasks with their metadata, separated explicitly into two kinds of column. The *objective* ones are extracted mechanically from the task files: category, tags, the authors' effort estimates, oracle length, number of grader checks, declared output artifacts. The *interpretive* ones record an earlier attempt to code each task by the capability it demands.

That separation is load-bearing and the reason the interpretive columns survive in the dataset without appearing in any result. The coding was done by one person (the author) so it carries no inter-rater agreement, and an earlier version of this study found that a taxonomy built on it did not reproduce reliably. Rather than report a grouping whose reliability could not be established, the analysis uses only the objective columns, which a reader can check against the public task set. No claim anywhere in this thesis rests on a judgement of what a task "demands" (Section 5.10); the columns are retained so that the decision is visible rather than silently reversed.

A.1.2 Two decisions that affect every figure

Ground truth for pass/fail is the per-cell aggregate, not the physical run directories. A storage fault removed the behavioural logs of a small number of executed runs; rebuilding outcomes from surviving directories would silently drop those results. The `has_trace` column records exactly which rows have behavioural data, so no aggregate can mix the two populations without saying so.

The termination class is derived once. The harness emits nine distinct termination strings; these are mapped to behavioural classes in the dataset build rather than per analysis, so two sections of the thesis cannot classify the same run differently. The related `submitted` field is deliberately *not* used as a behavioural signal: it is `true` for every run in the corpus, recording that the harness closed the episode rather than that the agent chose to finish.

A.1.3 Build and validation

```
scripts/writing/build_frozen.py        results-light/  -> the three CSVs
scripts/writing/build_numbers.py       CSVs -> LaTeX macros and tables
scripts/writing/build_stats_numbers.py CSVs -> inferential macros
thesis/writing/build.sh                all of the above, then the PDF
```

The build validates itself against the source: solve counts recomputed from the frozen tables agree with the per-cell result files in all 116 cells, and every cell contains exactly 64 rows.

A.2 Per-architecture implementation notes

Section 4.5 gives each architecture's mechanism, the failure mode it targets and how its budget is carved. What it omits, and what this appendix supplies, is the *contract*: the exact structured artifact each sub-agent must emit, and what happens when it does not. That second part matters more than it sounds. Section 6.4.2 shows that schema-valid emission ranges from 0% to 98% across the seven models, so for several cells the recorded result is the behaviour of the fallback path rather than of the architecture as designed.

The full prompt text lives in `systems/*/agent/prompts.py` (roughly 2,800 lines across the eight codebases) and is released with the code. Reproduced here is what a reimplementation actually needs to agree on.

The structured contracts

G2-Explorer — Data Summary. The Explorer's *final* `think` call must contain a JSON object and nothing else, with fields: `task_directory`; `files`, an array in which each entry carries `path`, `size_bytes`, `type`, and where applicable `columns`, `row_count`, `sample_head`, `sample_tail` and `notes`; `task_requirements`; and `observations`. The prompt names `columns` and `sample_head` as the two fields that must never be omitted, since they carry the grounding the executor cannot otherwise obtain.

G2-Planner — the plan. The Planner emits `subtasks`, an ordered array capped at four, each entry carrying `id`, `description` (what to do, which files, which approach), `success_criteria`, and a coarse `budget_hint` of `small | medium | large`. The Planner holds `think` and nothing else; the prompt states that calling any other tool is a fatal error.

G3-Judge — the verdict. The Judge must answer with exactly one of APPROVED, or NEEDS-FIX followed by specific feedback. The instruction is explicit that the feedback must name what is wrong rather than speculate, which is what makes the verdict parseable and the retry actionable.

G2-Monitor — the sentinel. The Monitor returns a fixed `ON_TRACK` sentinel when it sees no problem, and the harness then injects nothing at all. This is why the architecture costs nothing on runs that do not need it: the negative case is a string comparison, not a generation.

G4-Orchestrator — the routing call. Each routing decision is one JSON object: `next` (the specialist to deploy), `instruction` (what it should do, concretely), `reasoning`, and `context_refs` into the shared knowledge base. Because every decision is recorded in this form, the strategist's *policy* can be read directly from the traces, which is what Section 6.6.2 exploits.

What happens when the contract is broken

No schema failure is fatal, and this is a deliberate design choice rather than defensive coding: an architecture that crashed on malformed output would report an infrastructure failure where the honest result is a degraded run.

- **Explorer.** A malformed summary is passed through as raw text and the executor proceeds ungrounded, close to G1-Base behaviour rather than a crash.
- **Planner.** An unparseable plan falls back to a single synthesised subtask, which is exactly G2-Explorer behaviour. The architecture therefore cannot regress below its own predecessor by construction.
- **Both.** A second extraction pass re-reads the model's *raw* output for a well-formed object that never reached the `think` channel, a model that emits a correct plan in a fenced code block rather than through the tool interface is credited with it. This recovery path was added after observing that some models produce valid artifacts by the wrong route, and it is additive: the primary path is unchanged, so it cannot alter a result that already parsed.

The consequence for interpretation is stated in Chapter 6 and worth repeating here: where a model's valid-artifact rate is low, its G2 numbers measure the fallback, and the correct reading is that the architecture never ran as designed rather than that it ran and underperformed.

A.3 Harness engineering

A grid that compares model families only measures the models if the harness is equally fair to all of them. It is not fair by default. This appendix records the engineering that made it so, and one lesson that shaped how the whole study was read.

The lesson: apparent incapability is sometimes a parser bug

The single most consequential harness defect encountered during development was one character. The tool-call extractor scanned for balanced braces while tracking whether it was inside a string, and it treated the apostrophe as a string delimiter, but JSON has only one, the double quote. A model that wrote a sentence of prose containing an apostrophe before emitting its JSON therefore opened a string that never closed, and every subsequent brace was swallowed. The observable symptom was a run that made no tool calls at all.

Read naively, such a run is a model that cannot follow the tool protocol. It is not. It is a model whose output the harness discarded. The defect was found by capturing raw model output alongside the parsed result and comparing them, and the same comparison surfaced three further classes: a final object truncated by one closing brace, tool calls preceded by a fenced code block whose quotes derailed the scanner, and models that lead with the tool call and then continue generating.

This is why Section 6.4.2 treats structured-output compliance as a *measured axis* rather than an assumption, and why a low valid-artifact rate is reported as a property of the model–harness pair rather than of the model alone. Any agentic study that reports a model failing to use tools owes its reader the same check.

What the harness does

Tool-call extraction. A tolerant parser accepts the forms models actually emit: bare JSON, JSON in closed or unclosed code fences, JSON embedded in prose, and output preceded by reasoning tags. It uses a string-aware balanced-brace scan anchored on the tool signature, so quotes occurring earlier in the output cannot capture the real call. Clean bare JSON keeps a fast path that bypasses all of it, so the hardening cannot alter behaviour on well-formed output, verified as a no-change on the models that were already clean.

Parse failures are reported, not swallowed. When no tool call can be extracted, the model receives an explicit correction naming the problem rather than an empty turn. Silent failure was the original behaviour and it produced runs that appeared to stall for no reason.

Robust command transport. Commands that are fragile under shell quoting (multi-line scripts, very long commands, heavy escaping or backslashes) are base64-encoded and decoded inside the container, preserving working directory and environment. Simple commands take the ordinary path. This exists because early runs lost work to transport corruption: a command arrived mangled and the agent then debugged an error the harness had introduced.

Calibrated token accounting. Rather than estimating tokens from a fixed characters-per-token ratio, the harness calibrates the ratio per model from the server's own reported prompt-token count and updates it as the run proceeds. Context overflow is caught as a typed condition, the conversation is trimmed, and the call is retried, so an overflow is a recoverable event with a recorded cause rather than a crash.

Guards nudge; only the budget terminates. The harness never force-terminates a run. Repetition and parse failure produce feedback to the model, never a kill. The reasoning is stated in Section 4.1.4: a hard stop is an intervention by the experimenter, applied unevenly across models that repeat for different reasons, and the step and wall-clock budgets already terminate any loop. The cost is that a looping run consumes its full allocation; the benefit is that the failure modes in Chapter 6 are the models' own.

Applied uniformly. Every fix above was applied across all eight architecture codebases simultaneously, with a regression suite covering each. A fix that improved one model's parse rate but ran on only some architectures would have introduced the confound the grid exists to avoid.

A.4 Reading the subtest measure

Section 6.5 uses the graders' individual checks as a continuous companion to the binary solve rate, but only as corroboration, because the measure has two properties that make cross-task comparison of it unsafe. Both are recorded here so the main text can lean on the measure without re-deriving them.

Table A.2: Baseline performance measured two ways. Every model completes far more of the graders' checks than its pass/fail rate suggests; the gap is work done that the binary metric discards.

Model	Tasks solved	Subtests passed	Gap
Qwen2.5-Coder-7B	6.2%	24.6%	+18.4
Llama-3.1-70B	7.8%	33.9%	+26.1
Qwen2.5-Coder-14B	14.1%	32.2%	+18.1
Devstral-24B	18.8%	40.1%	+21.3
Qwen2.5-72B	25.0%	45.1%	+20.1
Qwen2.5-Coder-32B	29.7%	46.2%	+16.6
Gemini-3.1-FL	53.1%	73.5%	+20.3

Table A.2 shows the size of the discarded portion. Every model completes substantially more of the graders' checks than its solve rate suggests, by between 17 and 26 percentage points: Devstral-24B solves 19% of tasks but passes 40% of all subtests, Llama-70B 34% against 8%.

The measure does not resolve failure uniformly. The graders are written per task and differ in how far their checks decompose a failure. Restricting to the 62 tasks with at least five graded failures, **15 have graders on which every failing run scores exactly zero** (there the continuous measure is a copy of the binary one) while on **9 no failing run ever scores zero**, so the measure never reaches its floor. On `csv-to-parquet`, whose checks are serially dependent, all sixteen failures score zero; on `recover-obfuscated-files` not one of 32 does, because decoding filenames and decoding contents are separately creditable. A fourth task falls outside the scale: `broken-python` needs the system Python it is asked to repair for its own tests to run, so all 62 unsolved runs return no verdict at all. Across the corpus 189 of 7,424 runs (3%) are ungradeable in this way, and the frozen dataset counts them as unsolved, which is conservative but not measured.

The same fraction can mean different things. Reading the checks jointly rather than as a count shows that equal scores carry incompatible meanings. On `assign-seats` the two checks nest perfectly, so a half score means exactly *output produced, contents wrong* (51 of 116 runs produce no file, 35 a wrong

one, 30 correct). On `fix-git` the checks also nest but a half score means partial repair of lost work: 20 runs recover one of two changed files. And `flood-monitoring-basic` does not nest at all: ten of its 17 violating runs pass the final check while failing both earlier ones. Averaging the fraction over tasks, which is the only way to state a per-model figure, therefore mixes "how far a run got" with "how much of a multi-part repair it achieved". That average is the right summary for the claim it supports (that a great deal of work sits below the pass/fail threshold) and the wrong object for comparing one task's progress with another's, which is why the main text does not use it that way.

Some checks grade the method, not the answer. A grader can also withhold a solve from a correct output, because the task specified *how* it was to be reached. `jq-data-processing` is the clear case: of 116 runs, 43 produce content satisfying the schema, filtering and counting checks (correct by any ordinary reading) yet only 12 are scored solved. The gap is method, not correctness: the rest either compute the result in Python and fail `test_jq_was_used`, or produce the right content but fail a reproducibility check. Such tasks pull the subtest and solve measures apart for a reason unrelated to capability, and are flagged where they arise.

Two consequences, applied in the main text. Categories that look inert on solves are not necessarily so, since `machine-learning` contains no task whose pass/fail outcome ever changes, and `games` moves far less on solves than on subtests, yet both show some of the largest subtest gains in the study, and are reported on both measures. And wherever the two measures disagree both are given; where only one appears, it is the solve rate, the benchmark's own criterion.

A.5 Task-shape membership

The task-shape taxonomy of Section 6.5.4 is applied by hand, so its assignments are listed in full here for audit. Table A.3 gives the exact tasks in each shape and, for completeness, the unclassified remainder split into the tasks no model solves in the homogeneous grid and those whose obstacle is domain knowledge or a mixture of shapes.

Table A.3: The task list behind each shape of Table 6.28, and the unclassified remainder. The five shapes hold the 25 tasks in the top block; the 39 left out are either never solved in the homogeneous grid (no shape can be read from a task that does not move) or turn on domain knowledge or a mixture of shapes rather than one clean shape.

Shape	Tasks
Already solved, short path	`fix-git`, `extract-safely`, `recover-obfuscated-files`, `fix-pandas-version`, `assign-seats`, `predict-customer-churn`, `hello-world`
Specified transform, data present	`heterogeneous-dates`, `jsonl-aggregator`, `flood-monitoring-basic`, `recover-accuracy-log`, `pandas-etl`, `analyze-access-logs`
Environment / tool discovery	`csv-to-parquet`, `create-bucket`, `mixed-integer-programming`, `ilp-solver`, `amuse-install`
Propose–check–revise search	`sha-puzzle`, `blind-maze-explorer-5x5`, `mahjong-winninghand`, `gomoku-planner`, `countdown-game`
Multi-stage setup	`jupyter-notebook-server`, `processing-pipeline`
Unclassified: never solved	`build-stp`, `count-call-stack`, `cpp-compatibility`, `hdfs-deployment`, `logistic-regression-divergence`, `matlab-python-conversion`, `mnist-learning-fix`, `npm-conflict-resolution`, `overfull-hbox`, `security-vulhub-minio`, `setup-custom-dev-env`, `slurm-simple-node-monitoring`, `spinning-up-rl`
Unclassified: domain or mixed	`accelerate-maximal-square`, `aimo-airline-departures`, `ancient-puzzle`, `bank-trans-filter`, `broken-python`, `catch-me-if-you-can`, `cobol-modernization`, `cross-entropy-method`, `debug-long-program`, `fix-permissions`, `git-workflow-hack`, `grid-pattern-transform`, `hydra-debug-slurm-mode`, `jq-data-processing`, `log-summary`, `mlflow-register`, `modernize-fortran-build`, `multistep-definite-integral`, `new-encrypt-command`, `pandas-sql-query`, `png-generation`, `prove-plus-comm`, `schedule-vacation`, `simple-web-scraper`, `sql-injection-attack`, `vimscript-vim-quine`

A.6 Full per-model results

Every aggregate in this thesis reduces the same object: one outcome per model, architecture and task. That object is printed here in full so that any figure in Chapter 6 can be audited against it rather than taken on trust. The tables are generated from `runs.csv`; no cell is typed.

Three distinctions are preserved that the solve counts collapse. A run that failed while passing some of the grader's individual checks (○) is separated from one that passed none (·), because Section 6.5 shows the two behave differently under architecture.

One table follows per model, in the capability order used throughout: Qwen-7B (Table A.4), Llama-70B (Table A.5), Qwen-14B (Table A.6), Devstral-24B (Table A.7), Qwen-72B (Table A.8), Qwen-32B (Table A.9) and Gemini-3.1-FL (Table A.10).

Table A.4: Qwen2.5-Coder-7B: every task under every architecture. ● solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4).

Task	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
accelerate-maximal-square	·	·	·	·	·	·	·	·	·	·	·	·	·	·
aimo-airline-departures	○	○	○	○	○	○	○	○	○	○	○	○	○	○
amuse-install	○	·	○	○	○	○	○	·	○	○	○	○	○	○
analyze-access-logs	·	·	·	○	○	○	·	·	●	●	●	○	○	·
ancient-puzzle	·	·	·	·	·	·	·	○	·	·	·	·	·	·
assign-seats	·	·	·	·	·	·	·	·	·	·	○	○	·	·
bank-trans-filter	·	·	·	·	·	·	·	·	·	·	·	·	·	·
blind-maze-explorer-5x5	·	·	·	·	·	·	·	·	·	·	·	·	·	·
broken-python	·	·	·	·	·	·	·	·	·	·	·	·	·	·
build-stp	·	·	·	·	·	·	·	·	·	·	·	·	·	·
catch-me-if-you-can	·	·	·	·	·	·	·	·	·	·	·	·	·	·
cobol-modernization	○	○	○	○	○	○	○	○	○	○	○	○	○	○
count-call-stack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
countdown-game	○	·	○	·	○	○	·	○	○	·	○	○	○	○
cpp-compatibility	○	○	○	○	○	○	○	○	○	○	○	○	○	○
create-bucket	·	·	·	·	·	·	·	○	●	●	·	●	●	·
cross-entropy-method	○	○	○	○	○	○	○	○	○	○	○	·	○	○
csv-to-parquet	·	·	·	●	·	●	●	●	●	·	●	●	●	●
debug-long-program	○	○	○	○	○	○	○	○	○	○	○	○	○	○
extract-safely	○	○	○	○	●	○	○	○	○	○	○	○	○	○
fix-git	·	·	·	·	·	·	·	·	·	·	·	·	·	·
fix-pandas-version	·	·	·	·	○	·	●	·	·	·	·	·	○	●
fix-permissions	●	●	·	●	●	·	●	●	●	●	●	●	●	●
flood-monitoring-basic	·	·	·	○	○	○	●	·	·	○	○	○	○	·
git-workflow-hack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
gomoku-planner	○	·	·	○	·	·	○	○	·	○	·	·	·	·
grid-pattern-transform	·	·	·	·	·	·	·	·	·	·	·	·	·	·
hdfs-deployment	·	·	○	○	○	·	·	○	·	○	·	○	·	·
hello-world	●	●	●	●	●	●	●	●	●	●	●	●	●	●
heterogeneous-dates	●	●	●	●	●	●	●	●	●	●	●	●	○	·
hydra-debug-slurm-mode	○	○	○	○	○	○	○	○	·	·	·	·	·	·
ilp-solver	·	·	·	·	·	●	·	·	·	·	·	·	·	·
jq-data-processing	○	○	○	○	○	○	○	○	○	○	○	○	○	○
jsonl-aggregator	·	·	·	·	·	·	·	·	·	·	·	·	·	·
jupyter-notebook-server	·	·	·	·	·	·	·	·	·	·	·	·	·	·
log-summary	●	●	○	●	○	○	○	○	●	○	○	○	○	○
logistic-regression-divergence	·	○	·	·	○	○	·	○	○	○	○	○	○	○
mahjong-winninghand	○	·	·	·	·	○	·	·	○	·	○	○	○	○
matlab-python-conversion	○	○	·	·	○	·	·	·	·	·	·	·	·	·
mixed-integer-programming	·	·	·	·	·	·	○	·	·	○	·	○	·	·
mlflow-register	·	·	○	·	○	·	·	○	○	○	○	○	○	○
mnist-learning-fix	○	○	○	·	○	○	○	·	·	○	○	·	·	○
modernize-fortran-build	·	·	·	·	·	·	·	·	·	○	·	·	·	·
multistep-definite-integral	○	●	·	●	·	·	●	·	○	●	●	●	●	●
new-encrypt-command	·	·	·	·	·	·	·	·	·	·	·	·	·	·
npm-conflict-resolution	·	·	·	·	·	·	·	·	·	·	·	·	·	·
overfull-hbox	·	○	○	○	○	○	○	○	○	○	○	○	○	○
pandas-etl	·	·	·	·	·	·	·	·	·	·	·	·	○	○
pandas-sql-query	○	○	○	○	○	○	○	○	○	○	○	○	○	○
png-generation	○	·	·	○	○	○	○	○	○	○	○	·	·	○
predict-customer-churn	○	○	○	○	○	○	○	○	○	○	○	○	○	○
processing-pipeline	○	○	○	○	○	○	○	○	○	○	○	○	○	○
prove-plus-comm	○	○	○	○	○	○	○	○	○	○	○	○	○	○
recover-accuracy-log	·	·	·	·	·	·	·	·	○	○	·	·	·	·
recover-obfuscated-files	○	○	○	○	●	○	●	●	●	○	○	○	○	○
schedule-vacation	○	○	○	○	·	○	○	○	○	○	○	○	○	·
security-vulhub-minio	·	·	·	·	·	·	·	·	·	·	·	·	·	·
setup-custom-dev-env	○	○	○	○	·	·	○	·	○	·	·	·	○	·
sha-puzzle	·	·	·	·	·	·	·	·	·	·	·	·	·	·
simple-web-scraper	·	·	·	○	●	·	·	·	○	·	·	○	●	·
slurm-simple-node-monitoring	·	○	·	○	○	·	○	○	·	·	·	·	○	○
spinning-up-rl	·	·	·	·	·	·	·	·	·	·	·	·	·	·
sql-injection-attack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
vimscript-vim-quine	·	·	·	·	○	·	○	·	○	○	○	○	○	·

Table A.5: Llama-3.1-70B: every task under every architecture. ● solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4).

Task	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
accelerate-maximal-square	·	·	·	·	·	·	·	·	·	·	·	·	·	·
aimo-airline-departures	○	○	○	○	○	○	○	○	○	○	○	○	○	○
amuse-install	○	○	○	○	○	○	○	○	○	○	○	○	○	○
analyze-access-logs	·	●	●	○	○	○	●	●	·	○	·	○	●	○
ancient-puzzle	·	·	·	○	○	·	·	○	○	○	○	·	○	○
assign-seats	○	○	○	○	○	○	·	·	·	○	○	·	·	·
bank-trans-filter	·	·	·	·	·	·	·	·	·	·	·	·	·	·
blind-maze-explorer-5x5	○	○	○	○	○	○	○	·	○	○	○	○	○	○
broken-python	·	·	·	·	·	·	·	●	·	·	·	●	·	●
build-stp	·	·	·	·	·	·	·	·	·	·	·	·	·	·
catch-me-if-you-can	·	·	●	·	·	·	·	·	·	·	·	·	·	·
cobol-modernization	○	○	○	○	○	○	○	○	○	○	○	○	○	○
count-call-stack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
countdown-game	○	○	○	○	●	·	●	○	●	○	○	●	○	○
cpp-compatibility	○	○	○	○	○	○	○	○	○	○	○	○	○	○
create-bucket	○	·	·	·	·	·	●	●	●	○	○	○	○	○
cross-entropy-method	·	○	○	·	○	·	○	·	·	·	·	·	○	○
csv-to-parquet	·	·	·	●	·	●	●	●	●	●	●	●	·	●
debug-long-program	·	·	·	·	·	·	·	·	·	·	·	·	·	·
extract-safely	●	○	○	○	○	○	○	○	○	○	○	○	○	○
fix-git	●	●	●	●	○	○	·	○	○	○	○	●	○	○
fix-pandas-version	·	·	·	○	○	·	·	·	·	·	○	·	●	○
fix-permissions	·	●	●	●	●	●	●	●	●	●	●	●	●	●
flood-monitoring-basic	·	●	·	○	○	○	○	○	○	○	●	·	●	○
git-workflow-hack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
gomoku-planner	·	·	·	○	○	○	·	○	○	○	○	·	○	○
grid-pattern-transform	·	·	·	·	·	·	·	·	·	·	·	·	·	·
hdfs-deployment	·	·	·	○	○	○	○	○	○	○	○	○	○	○
hello-world	●	●	●	●	●	●	●	●	●	●	●	●	●	●
heterogeneous-dates	○	○	●	○	●	●	●	○	○	○	○	○	○	○
hydra-debug-slurm-mode	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ilp-solver	·	·	·	·	·	·	·	·	·	·	·	·	·	·
jq-data-processing	○	○	○	○	○	○	○	○	○	○	○	○	○	○
jsonl-aggregator	·	·	·	●	·	●	●	·	·	·	·	·	·	·
jupyter-notebook-server	·	·	·	○	○	·	○	○	○	○	○	○	○	○
log-summary	○	●	●	●	●	○	●	●	●	●	○	●	●	●
logistic-regression-divergence	○	·	○	·	○	·	○	·	○	○	○	○	·	·
mahjong-winninghand	○	○	○	○	○	○	○	○	○	○	○	○	○	○
matlab-python-conversion	○	○	○	○	○	○	○	○	○	○	○	○	○	○
mixed-integer-programming	○	○	·	○	○	○	·	·	○	·	·	·	○	○
mlflow-register	○	○	○	○	○	○	○	○	○	○	○	○	·	○
mnist-learning-fix	·	·	·	○	·	·	·	·	·	·	·	○	·	·
modernize-fortran-build	·	·	·	·	○	·	·	·	·	·	·	·	·	·
multistep-definite-integral	●	●	●	●	●	●	●	●	●	●	●	●	●	●
new-encrypt-command	·	·	·	·	·	·	·	·	·	·	·	·	·	·
npm-conflict-resolution	○	·	·	○	○	·	○	·	·	·	·	○	·	○
overfull-hbox	○	○	○	○	○	○	○	○	○	○	○	○	○	○
pandas-etl	○	·	·	·	○	○	○	○	○	○	○	○	○	○
pandas-sql-query	○	○	○	○	○	○	○	○	○	○	○	○	○	●
png-generation	○	○	○	○	○	·	○	○	○	○	○	○	○	○
predict-customer-churn	○	○	○	○	○	○	○	○	●	○	○	○	●	○
processing-pipeline	○	○	○	○	○	○	○	○	○	○	○	○	○	○
prove-plus-comm	○	○	○	○	○	○	○	○	○	○	○	○	○	○
recover-accuracy-log	○	·	·	●	●	●	·	○	○	○	○	○	○	○
recover-obfuscated-files	●	○	●	●	○	●	○	●	●	●	●	●	●	●
schedule-vacation	○	○	○	○	○	○	○	○	○	○	○	○	○	○
security-vulhub-minio	·	·	·	·	·	·	·	·	·	·	·	·	·	·
setup-custom-dev-env	○	○	○	○	○	○	○	○	○	○	○	○	○	○
sha-puzzle	·	·	·	·	·	·	·	·	·	·	·	·	·	·
simple-web-scraper	○	○	○	○	○	○	○	○	○	○	○	○	○	○
slurm-simple-node-monitoring	○	○	○	○	○	○	○	○	○	○	○	○	○	○
spinning-up-rl	·	·	·	·	·	·	·	·	·	·	·	·	·	·
sql-injection-attack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
vimscript-vim-quine	○	○	○	○	○	○	○	○	○	○	○	○	○	○

Table A.6: Qwen2.5-Coder-14B: every task under every architecture. ● solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4).

Task	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
accelerate-maximal-square	·	·	·	·	·	·	·	·	·	·	·	·	·	·
aimo-airline-departures	·	○	○	○	○	○	○	○	○	○	○	○	○	○
amuse-install	○	○	○	○	○	○	○	○	○	○	·	○	○	○
analyze-access-logs	●	○	●	○	●	●	○	●	·	●	○	·	·	●
ancient-puzzle	·	·	·	·	○	·	·	·	·	·	·	·	·	·
assign-seats	·	·	·	·	·	·	·	·	·	·	·	○	·	·
bank-trans-filter	·	·	·	·	·	·	·	·	·	·	·	·	·	·
blind-maze-explorer-5x5	·	○	·	·	·	·	·	·	·	·	·	·	·	·
broken-python	·	●	●	·	·	●	·	·	·	●	●	●	●	·
build-stp	·	·	·	·	·	·	·	·	·	·	·	·	·	·
catch-me-if-you-can	·	·	·	·	·	·	·	·	·	·	·	·	·	·
cobol-modernization	○	○	○	○	○	○	○	○	○	○	○	○	○	○
count-call-stack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
countdown-game	○	●	○	○	·	○	·	·	·	·	·	●	·	●
cpp-compatibility	○	○	○	○	○	○	○	○	○	○	○	○	○	○
create-bucket	○	●	●	●	●	●	●	●	●	●	·	●	●	·
cross-entropy-method	·	○	○	○	○	○	○	○	○	·	·	·	·	·
csv-to-parquet	●	●	●	●	●	●	●	●	●	●	●	●	●	●
debug-long-program	○	○	○	○	○	○	○	○	○	○	○	○	○	○
extract-safely	○	○	○	○	○	○	○	○	○	●	○	○	○	○
fix-git	·	·	·	·	○	·	·	·	·	·	·	·	·	●
fix-pandas-version	●	●	○	○	○	●	●	○	●	●	●	○	●	●
fix-permissions	●	●	●	●	·	●	●	●	●	●	●	●	●	●
flood-monitoring-basic	·	·	·	·	·	·	·	·	·	·	·	·	·	○
git-workflow-hack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
gomoku-planner	○	·	○	·	○	·	○	○	○	○	○	○	○	·
grid-pattern-transform	·	·	·	·	·	·	·	·	·	·	·	·	·	·
hdfs-deployment	○	○	○	○	○	○	○	○	○	○	○	·	○	○
hello-world	●	●	●	●	●	●	●	●	●	●	●	●	●	●
heterogeneous-dates	○	○	○	·	·	●	●	·	·	·	·	·	○	·
hydra-debug-slurm-mode	○	○	○	○	○	○	○	○	·	·	·	·	·	·
ilp-solver	·	·	·	·	·	●	·	·	·	·	·	·	·	·
jq-data-processing	○	○	○	○	○	○	○	○	○	○	○	○	○	○
jsonl-aggregator	·	·	·	·	·	·	·	·	·	·	·	·	·	·
jupyter-notebook-server	·	·	·	·	○	·	·	·	·	·	·	·	·	·
log-summary	●	●	●	○	●	○	○	○	○	○	○	○	○	●
logistic-regression-divergence	·	○	○	○	·	○	·	○	·	·	○	·	·	·
mahjong-winninghand	○	·	○	○	○	○	○	○	○	·	·	○	○	·
matlab-python-conversion	○	○	○	○	○	○	○	○	○	○	○	○	○	·
mixed-integer-programming	·	·	·	·	·	·	○	·	●	●	·	○	·	·
mlflow-register	●	○	○	○	○	○	○	○	○	○	○	○	○	○
mnist-learning-fix	·	·	·	·	○	○	○	○	○	·	·	·	·	·
modernize-fortran-build	·	·	·	·	·	·	·	·	·	·	·	·	·	·
multistep-definite-integral	●	●	●	●	●	●	●	●	●	●	●	●	●	●
new-encrypt-command	·	·	·	·	·	·	·	·	·	·	·	·	·	·
npm-conflict-resolution	○	·	○	○	○	○	○	·	·	·	·	·	·	·
overfull-hbox	○	○	○	○	○	○	○	○	○	○	○	○	○	○
pandas-etl	·	●	·	·	○	○	○	○	○	○	·	○	○	·
pandas-sql-query	·	○	○	○	○	○	○	○	○	○	○	○	○	·
png-generation	·	·	·	·	○	·	·	·	●	·	·	·	○	·
predict-customer-churn	○	●	○	○	○	○	○	●	○	○	○	○	●	○
processing-pipeline	○	○	○	○	○	○	○	○	○	○	○	○	○	○
prove-plus-comm	○	○	○	○	○	○	○	○	○	○	○	○	○	○
recover-accuracy-log	·	○	·	·	○	·	·	·	·	·	·	○	·	·
recover-obfuscated-files	●	○	○	○	○	●	●	○	●	●	●	●	●	○
schedule-vacation	○	○	·	○	○	○	○	○	○	○	○	○	○	○
security-vulhub-minio	·	·	·	·	·	·	·	·	·	·	·	·	·	·
setup-custom-dev-env	○	○	○	○	○	○	○	○	○	○	○	·	○	·
sha-puzzle	·	·	·	·	·	·	·	·	·	·	·	·	·	·
simple-web-scraper	○	○	○	·	·	·	·	·	·	●	·	○	·	·
slurm-simple-node-monitoring	○	○	○	○	○	○	○	○	○	○	○	○	○	○
spinning-up-rl	·	·	·	·	·	·	·	·	·	·	·	·	·	·
sql-injection-attack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
vimscript-vim-quine	○	○	○	○	○	○	○	○	○	○	○	○	○	·

Table A.7: Devstral-24B: every task under every architecture. ● solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4).

Task	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
accelerate-maximal-square	·	·	·	·	·	·	·	·	·	●	·	·	·	·
aimo-airline-departures	○	·	·	○	○	·	○	○	·	○	○	○	○	○
amuse-install	○	○	○	○	○	○	○	○	○	○	○	○	○	●
analyze-access-logs	○	●	●	·	·	●	·	·	·	·	·	○	●	○
ancient-puzzle	·	·	·	·	·	·	○	·	·	·	●	·	·	·
assign-seats	●	●	○	●	·	●	·	●	●	●	●	●	○	·
bank-trans-filter	·	·	·	·	·	·	·	·	·	·	·	·	·	·
blind-maze-explorer-5x5	·	○	·	·	○	·	·	·	·	·	·	●	·	·
broken-python	·	·	·	·	·	·	·	·	·	·	·	●	·	●
build-stp	·	·	·	·	·	·	·	·	·	·	·	·	·	·
catch-me-if-you-can	·	·	·	·	·	·	·	·	·	·	·	·	●	·
cobol-modernization	○	○	○	○	○	○	○	○	○	○	○	○	○	○
count-call-stack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
countdown-game	·	●	·	●	●	●	·	●	·	●	·	●	●	●
cpp-compatibility	○	○	○	○	○	○	○	○	○	○	○	○	○	○
create-bucket	○	○	○	·	○	○	○	○	●	○	○	●	●	○
cross-entropy-method	○	○	○	○	○	○	○	○	○	○	○	○	○	○
csv-to-parquet	●	●	●	●	●	●	●	●	●	●	●	●	●	●
debug-long-program	○	○	·	·	○	○	○	○	○	○	○	·	·	○
extract-safely	●	○	●	○	○	●	○	●	●	●	●	●	●	●
fix-git	●	·	·	·	·	·	·	·	·	·	·	·	●	·
fix-pandas-version	●	·	·	●	●	·	●	●	·	·	·	●	●	·
fix-permissions	●	●	●	●	●	●	●	·	●	●	●	●	●	●
flood-monitoring-basic	○	○	·	·	·	○	○	○	·	·	○	·	○	·
git-workflow-hack	·	·	·	·	●	·	·	·	·	·	·	·	·	·
gomoku-planner	·	·	·	·	○	·	·	·	·	·	·	·	·	○
grid-pattern-transform	·	·	·	·	·	·	·	·	·	·	·	○	·	○
hdfs-deployment	○	○	○	·	·	○	·	○	○	○	○	·	·	·
hello-world	●	●	●	●	●	●	●	●	●	●	●	●	●	●
heterogeneous-dates	○	○	○	●	○	○	○	○	○	○	○	○	○	○
hydra-debug-slurm-mode	·	·	○	○	○	·	○	○	○	○	○	○	○	○
ilp-solver	·	·	·	·	·	·	·	·	·	·	·	·	·	·
jq-data-processing	○	○	○	○	○	○	○	○	○	○	○	○	○	○
jsonl-aggregator	●	●	●	·	●	●	●	●	●	●	●	·	●	●
jupyter-notebook-server	·	·	·	·	·	·	·	·	·	·	·	·	○	·
log-summary	·	●	·	●	●	·	·	●	●	·	●	·	·	●
logistic-regression-divergence	○	○	○	○	○	○	○	○	○	○	○	○	○	○
mahjong-winninghand	·	○	·	·	·	·	○	·	○	·	·	·	○	·
matlab-python-conversion	·	·	·	·	·	·	·	○	·	·	·	·	○	·
mixed-integer-programming	·	·	·	·	·	·	·	·	·	·	·	·	·	·
mlflow-register	○	·	·	·	·	·	·	·	○	·	·	●	●	○
mnist-learning-fix	○	·	○	○	○	○	○	○	○	○	○	○	·	○
modernize-fortran-build	·	·	●	·	·	·	·	·	·	●	·	●	●	·
multistep-definite-integral	●	●	●	●	·	●	●	●	●	●	●	●	●	●
new-encrypt-command	·	·	·	●	·	·	·	·	·	·	·	●	·	·
npm-conflict-resolution	·	·	·	·	·	·	·	·	·	·	·	·	○	·
overfull-hbox	○	○	○	○	○	○	○	○	○	○	○	○	○	○
pandas-etl	○	●	○	○	●	○	●	○	○	●	●	●	●	●
pandas-sql-query	○	·	○	·	·	·	○	·	·	○	·	·	○	○
png-generation	○	○	●	○	○	○	●	○	●	●	●	·	●	○
predict-customer-churn	○	○	○	○	○	○	○	○	○	○	○	○	○	●
processing-pipeline	●	○	○	○	○	○	○	○	○	●	○	○	○	○
prove-plus-comm	○	○	○	○	○	○	○	○	○	●	○	●	○	○
recover-accuracy-log	●	●	●	○	●	●	●	●	●	●	●	●	○	●
recover-obfuscated-files	●	●	●	○	○	○	●	●	●	●	●	●	●	○
schedule-vacation	○	○	○	○	○	○	○	○	○	○	○	○	○	○
security-vulhub-minio	·	·	·	·	·	·	·	·	·	·	·	·	·	·
setup-custom-dev-env	·	·	○	·	○	·	·	·	·	·	·	·	·	○
sha-puzzle	·	·	·	·	·	·	·	·	·	·	·	·	·	·
simple-web-scraper	○	●	·	●	·	●	●	●	·	●	●	○	●	●
slurm-simple-node-monitoring	·	·	·	·	·	○	·	·	·	·	·	·	○	○
spinning-up-rl	·	·	·	·	·	·	·	·	·	·	·	·	·	·
sql-injection-attack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
vimscript-vim-quine	○	·	○	○	○	○	·	○	○	○	○	○	○	○

Table A.8: Qwen2.5-72B: every task under every architecture. ● solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4).

Task	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
accelerate-maximal-square	·	·	·	·	·	·	·	·	·	·	·	·	·	·
aimo-airline-departures	○	○	○	○	○	○	○	○	○	○	○	○	○	○
amuse-install	○	●	○	·	○	○	●	○	○	●	○	○	○	○
analyze-access-logs	●	●	●	●	●	●	●	●	●	●	●	●	●	·
ancient-puzzle	·	·	·	·	·	·	·	·	·	·	·	·	○	○
assign-seats	○	○	·	○	●	○	○	○	○	○	○	○	○	○
bank-trans-filter	·	·	·	·	·	·	·	·	·	·	·	·	·	·
blind-maze-explorer-5x5	·	·	·	·	·	·	○	·	○	·	·	·	·	·
broken-python	●	●	●	●	●	●	●	●	·	●	●	·	●	●
build-stp	·	·	·	·	·	·	·	·	·	·	·	·	·	·
catch-me-if-you-can	·	●	·	·	·	·	·	·	·	·	·	●	·	·
cobol-modernization	○	○	○	○	○	○	○	○	○	○	○	○	○	○
count-call-stack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
countdown-game	●	●	·	●	●	●	●	●	●	●	●	○	○	●
cpp-compatibility	○	○	○	○	○	○	○	○	○	○	○	○	○	○
create-bucket	○	●	●	●	○	●	●	●	●	○	●	○	●	●
cross-entropy-method	·	·	○	·	○	·	·	·	·	○	·	·	·	○
csv-to-parquet	●	·	●	·	·	●	●	●	●	●	●	●	●	●
debug-long-program	○	○	○	○	○	○	○	○	○	○	○	○	○	○
extract-safely	●	●	○	●	○	●	●	●	●	○	●	●	●	●
fix-git	·	○	○	○	●	·	●	○	○	○	○	·	○	○
fix-pandas-version	○	●	●	·	○	·	·	·	·	·	·	●	·	●
fix-permissions	●	●	●	●	●	●	●	●	●	●	●	●	●	●
flood-monitoring-basic	·	·	●	·	○	·	●	·	·	·	●	·	○	·
git-workflow-hack	·	·	·	●	●	●	·	·	·	·	·	·	·	·
gomoku-planner	·	○	·	·	·	○	·	○	○	○	○	○	○	○
grid-pattern-transform	·	·	·	·	·	·	·	·	·	·	·	·	·	·
hdfs-deployment	·	○	○	·	○	○	○	○	○	○	○	○	○	○
hello-world	●	●	●	●	●	●	●	●	●	●	●	●	●	●
heterogeneous-dates	○	●	○	○	○	○	○	○	○	○	○	○	○	○
hydra-debug-slurm-mode	○	○	·	·	·	○	○	○	○	○	○	○	○	○
ilp-solver	·	·	·	·	·	·	·	·	·	·	·	·	·	·
jq-data-processing	○	○	○	○	○	○	●	○	○	●	○	○	○	○
jsonl-aggregator	●	●	●	●	●	●	●	●	●	●	●	●	●	●
jupyter-notebook-server	○	·	·	·	·	·	○	○	○	·	○	○	·	·
log-summary	●	●	○	○	●	○	●	○	●	●	●	●	●	●
logistic-regression-divergence	·	·	·	·	·	○	·	·	·	○	·	·	·	○
mahjong-winninghand	○	○	○	·	○	○	○	○	·	○	○	○	○	·
matlab-python-conversion	○	○	○	·	○	○	○	○	○	○	○	○	○	○
mixed-integer-programming	●	○	·	·	·	·	·	●	●	·	○	·	●	·
mlflow-register	○	·	○	○	○	○	○	○	○	○	○	○	○	○
mnist-learning-fix	·	·	·	·	·	○	·	·	·	·	·	○	·	○
modernize-fortran-build	·	·	·	·	·	·	·	·	·	·	·	·	·	·
multistep-definite-integral	●	●	●	●	●	●	●	●	●	●	●	●	●	●
new-encrypt-command	·	·	·	·	·	·	·	·	·	·	·	·	·	·
npm-conflict-resolution	○	○	·	·	○	○	○	○	○	○	○	○	·	○
overfull-hbox	○	○	○	·	·	○	○	○	○	○	○	○	○	○
pandas-etl	○	●	○	●	○	○	○	○	○	●	○	○	○	○
pandas-sql-query	●	●	○	○	○	○	○	○	○	○	○	○	○	○
png-generation	●	○	○	○	●	·	○	○	●	●	○	●	●	●
predict-customer-churn	○	○	○	○	○	○	○	○	○	○	○	○	○	○
processing-pipeline	○	○	○	○	○	○	○	○	○	○	○	○	○	○
prove-plus-comm	○	○	○	○	○	○	○	○	○	○	○	○	○	○
recover-accuracy-log	●	●	·	●	·	●	○	●	○	●	○	●	○	●
recover-obfuscated-files	●	○	●	○	●	●	●	●	●	●	●	○	●	○
schedule-vacation	○	○	○	○	○	○	○	○	○	○	○	○	○	○
security-vulhub-minio	·	·	·	·	·	·	·	·	·	·	·	·	·	·
setup-custom-dev-env	○	○	○	○	○	○	○	○	○	○	○	○	○	○
sha-puzzle	·	·	·	·	·	·	·	·	·	·	·	·	·	·
simple-web-scraper	●	●	●	●	●	●	●	●	●	●	●	●	●	●
slurm-simple-node-monitoring	○	○	○	○	·	○	○	○	○	○	○	○	○	○
spinning-up-rl	·	·	·	·	·	·	·	·	·	·	·	·	·	·
sql-injection-attack	·	·	·	·	·	·	·	●	●	·	●	·	·	·
vimscript-vim-quine	·	○	○	·	○	○	○	○	○	○	○	○	○	○

Table A.9: Qwen2.5-Coder-32B: every task under every architecture. ● solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4).

Task	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
accelerate-maximal-square	·	·	·	·	·	·	·	·	·	·	·	·	·	·
aimo-airline-departures	○	○	○	○	○	○	○	○	○	·	○	○	○	○
amuse-install	●	·	●	○	○	○	○	○	○	●	○	○	○	○
analyze-access-logs	●	●	●	●	●	○	○	●	●	●	○	○	○	●
ancient-puzzle	·	○	·	·	○	·	·	·	·	·	○	·	·	·
assign-seats	●	·	●	○	·	·	●	·	·	·	●	·	○	○
bank-trans-filter	·	·	·	·	·	·	·	·	·	●	·	·	·	·
blind-maze-explorer-5x5	·	·	○	·	·	○	○	○	·	○	·	·	○	·
broken-python	●	●	●	●	●	●	●	●	●	●	●	●	●	●
build-stp	·	·	·	·	·	·	·	·	·	·	·	·	·	·
catch-me-if-you-can	·	·	·	·	·	·	·	●	●	·	·	·	·	·
cobol-modernization	○	○	○	○	○	○	○	○	○	○	○	○	○	○
count-call-stack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
countdown-game	●	○	●	●	·	●	○	●	○	○	●	○	●	●
cpp-compatibility	○	○	○	○	○	○	○	○	○	·	○	○	○	○
create-bucket	●	●	●	●	○	●	·	○	●	●	●	●	●	●
cross-entropy-method	·	·	·	·	·	○	·	○	·	○	·	·	○	○
csv-to-parquet	●	●	●	●	●	●	●	●	●	●	●	●	●	●
debug-long-program	○	○	○	○	○	○	○	○	○	○	○	○	○	○
extract-safely	●	○	●	○	○	○	●	○	●	●	●	○	○	○
fix-git	●	●	●	·	●	·	●	●	●	●	·	·	○	·
fix-pandas-version	●	●	·	●	○	○	●	●	●	●	○	●	○	·
fix-permissions	●	●	·	●	●	●	·	·	·	·	●	●	●	●
flood-monitoring-basic	○	·	○	○	●	●	●	○	○	○	●	○	○	○
git-workflow-hack	·	·	·	●	●	●	·	·	·	·	·	·	·	·
gomoku-planner	○	·	○	○	·	·	○	·	·	·	·	·	○	·
grid-pattern-transform	·	·	·	·	·	·	·	·	·	·	·	·	·	·
hdfs-deployment	○	○	○	○	○	○	○	○	○	·	○	○	○	○
hello-world	●	●	●	●	●	●	●	●	●	●	●	●	●	●
heterogeneous-dates	·	·	○	○	○	○	○	○	○	○	○	●	○	○
hydra-debug-slurm-mode	○	○	○	○	○	○	○	○	○	○	·	·	·	○
ilp-solver	·	·	·	·	·	·	·	·	·	·	·	·	·	·
jq-data-processing	●	·	○	○	○	○	○	○	○	○	○	○	●	○
jsonl-aggregator	·	·	·	·	·	·	·	·	·	·	●	·	·	·
jupyter-notebook-server	·	·	·	·	·	·	·	·	·	·	·	·	○	·
log-summary	●	●	○	○	○	●	●	●	●	●	○	●	●	●
logistic-regression-divergence	·	·	·	○	○	○	·	○	·	·	·	·	·	○
mahjong-winninghand	·	·	·	·	·	·	·	·	·	○	·	○	○	·
matlab-python-conversion	○	○	○	○	○	○	○	○	○	○	○	○	○	·
mixed-integer-programming	·	·	·	·	·	·	·	·	·	·	·	·	·	·
mlflow-register	○	·	○	○	●	●	○	○	●	○	○	○	○	○
mnist-learning-fix	·	·	·	·	·	·	·	·	·	·	·	·	·	·
modernize-fortran-build	○	·	○	·	·	○	·	·	·	·	·	·	·	·
multistep-definite-integral	●	●	●	●	●	●	●	●	●	●	●	●	●	●
new-encrypt-command	·	·	·	·	·	·	·	·	·	·	·	●	·	·
npm-conflict-resolution	·	·	·	○	○	·	·	·	○	○	·	·	○	○
overfull-hbox	○	○	○	○	○	○	○	○	○	○	○	○	○	○
pandas-etl	●	●	○	●	○	●	●	●	●	○	●	●	●	·
pandas-sql-query	○	○	○	○	○	○	○	○	●	○	○	○	○	○
png-generation	○	●	○	·	·	○	○	○	○	○	○	○	●	·
predict-customer-churn	●	·	●	○	●	○	○	●	○	●	○	○	○	○
processing-pipeline	○	○	○	○	○	○	○	○	○	●	○	○	○	○
prove-plus-comm	○	○	○	○	○	○	○	○	○	○	○	○	○	○
recover-accuracy-log	○	●	○	·	●	○	●	●	○	●	●	●	○	●
recover-obfuscated-files	●	●	●	●	●	●	●	●	●	●	●	●	●	●
schedule-vacation	○	○	○	○	○	○	○	○	○	○	○	○	○	○
security-vulhub-minio	·	·	·	·	·	·	·	·	·	·	·	·	·	·
setup-custom-dev-env	○	○	○	○	○	○	○	○	○	○	○	○	○	○
sha-puzzle	·	·	·	·	·	·	·	·	·	·	·	·	·	·
simple-web-scraper	●	●	●	●	●	●	●	●	●	●	●	●	●	●
slurm-simple-node-monitoring	○	·	○	○	○	○	○	○	○	○	○	○	○	○
spinning-up-rl	·	·	·	·	·	·	·	·	·	·	·	·	·	·
sql-injection-attack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
vimscript-vim-quine	○	○	○	○	○	○	○	○	○	○	○	○	○	○

Table A.10: Gemini-3.1-FL: every task under every architecture. ● solved; ○ failed but passed at least one grader check; · failed with no check passing. Every cell was executed (Section 5.4).

Task	Base	Prompt	Temp	Explor	Plan	Analyst	Monitor	Instr	Reflex	Judge	Gate	Coop	Debate	Orch
accelerate-maximal-square	·	·	·	·	·	·	·	·	·	·	·	·	·	·
aimo-airline-departures	●	●	●	○	○	●	○	●	○	●	●	●	○	●
amuse-install	○	○	○	○	○	●	○	○	○	○	●	○	○	●
analyze-access-logs	●	●	●	●	●	●	●	●	●	●	●	●	●	●
ancient-puzzle	○	●	·	·	·	·	·	·	·	●	·	·	·	·
assign-seats	●	●	○	○	●	●	●	○	●	●	●	●	●	·
bank-trans-filter	●	●	●	·	·	●	·	·	·	●	●	·	·	·
blind-maze-explorer-5x5	○	●	·	○	○	○	·	○	·	○	○	●	●	○
broken-python	●	·	●	●	·	●	●	●	●	●	●	·	·	●
build-stp	·	·	·	·	·	·	·	·	·	·	·	·	·	·
catch-me-if-you-can	●	·	●	●	●	●	●	·	●	·	●	·	●	●
cobol-modernization	○	○	○	○	●	●	○	○	●	○	●	○	○	○
count-call-stack	·	·	·	·	·	·	·	·	·	·	·	·	·	·
countdown-game	●	●	●	●	●	●	●	●	●	·	●	●	●	●
cpp-compatibility	○	○	○	○	○	○	○	○	○	○	○	○	○	○
create-bucket	●	○	●	●	●	●	●	●	●	○	●	●	●	●
cross-entropy-method	●	○	●	●	●	●	●	●	●	●	●	○	○	○
csv-to-parquet	●	●	●	●	●	·	●	●	●	●	●	●	●	●
debug-long-program	●	●	●	●	●	●	○	●	●	●	●	●	●	●
extract-safely	●	●	●	●	●	●	●	●	●	●	●	●	●	●
fix-git	●	●	●	●	·	●	●	●	●	●	●	●	●	●
fix-pandas-version	●	●	○	●	●	●	●	●	●	●	●	●	●	●
fix-permissions	●	●	●	●	●	●	●	●	●	●	●	●	●	●
flood-monitoring-basic	○	●	○	●	●	○	●	●	●	○	●	○	●	●
git-workflow-hack	●	·	·	·	●	·	·	·	·	●	●	·	·	·
gomoku-planner	○	●	·	○	·	○	●	·	●	●	·	●	●	●
grid-pattern-transform	○	●	●	●	○	●	○	○	○	●	○	○	●	○
hdfs-deployment	○	○	○	○	○	·	○	○	○	○	○	○	○	○
hello-world	●	●	●	●	●	●	●	●	●	●	●	●	●	●
heterogeneous-dates	●	●	●	●	●	●	●	●	●	●	●	●	●	●
hydra-debug-slurm-mode	○	○	○	○	○	○	○	○	○	○	○	●	○	○
ilp-solver	●	●	●	●	●	●	●	●	●	●	●	●	●	●
jq-data-processing	○	●	○	●	●	●	●	○	○	●	●	●	○	○
jsonl-aggregator	●	●	●	●	●	●	●	●	●	●	●	●	●	●
jupyter-notebook-server	○	·	·	●	●	·	○	●	●	●	○	○	○	○
log-summary	●	●	●	●	·	●	·	●	●	●	●	●	●	●
logistic-regression-divergence	○	○	○	○	○	○	○	○	○	○	○	○	○	○
mahjong-winninghand	●	●	●	○	●	●	●	●	●	●	●	●	○	●
matlab-python-conversion	○	○	○	○	○	○	○	○	○	○	○	○	○	○
mixed-integer-programming	·	○	·	·	·	·	○	·	●	·	·	·	·	·
mlflow-register	●	●	●	●	●	○	●	●	●	●	●	●	●	●
mnist-learning-fix	·	·	·	·	·	·	·	·	·	·	·	·	·	·
modernize-fortran-build	·	·	·	·	·	·	·	·	·	·	·	·	·	·
multistep-definite-integral	●	●	●	●	●	●	●	●	●	●	●	●	●	●
new-encrypt-command	●	·	●	●	●	●	●	●	●	●	●	●	●	·
npm-conflict-resolution	·	·	○	·	·	·	·	·	·	·	·	·	·	·
overfull-hbox	○	○	○	○	○	○	○	·	○	○	○	○	○	○
pandas-etl	●	●	●	●	●	●	●	●	●	●	●	●	●	●
pandas-sql-query	●	○	●	●	●	●	●	●	●	●	●	●	●	●
png-generation	○	○	○	●	●	○	○	○	○	○	○	●	○	●
predict-customer-churn	●	●	●	○	●	○	●	●	●	○	○	●	●	●
processing-pipeline	○	○	○	○	●	●	●	○	●	●	○	●	●	○
prove-plus-comm	●	●	●	●	○	○	●	·	●	●	●	●	●	●
recover-accuracy-log	●	●	●	●	·	●	●	●	●	●	●	●	●	●
recover-obfuscated-files	●	●	●	●	●	●	●	●	●	●	●	●	●	●
schedule-vacation	○	○	○	○	○	●	○	●	●	○	○	○	●	○
security-vulhub-minio	·	·	·	·	·	·	·	·	·	·	·	·	·	·
setup-custom-dev-env	○	○	·	○	○	○	○	○	○	○	○	○	○	○
sha-puzzle	●	·	●	·	·	·	·	·	·	●	●	●	●	·
simple-web-scraper	●	●	●	●	●	●	●	●	●	●	●	●	●	●
slurm-simple-node-monitoring	○	○	○	○	○	○	○	○	○	○	○	○	○	○
spinning-up-rl	·	·	·	·	·	·	·	·	·	·	·	·	·	·
sql-injection-attack	●	●	●	●	●	●	●	●	●	●	●	●	●	●
vimscript-vim-quine	○	○	●	○	○	○	○	●	●	○	○	○	○	○

Appendix B

Bibliography

[1] Gao, M., Li, Y., Liu, B., Yu, Y., Wang, P., Lin, C.-Y., and Lai, F., *Single-agent or multi-agent systems? why not both?* 2025. arXiv: 2505.18286.

[2] Kim, Y. et al., *Towards a science of scaling agent systems*, 2025. arXiv: 2512.08296 [cs.AI].

[3] Tran, D. and Kiela, D., *Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets*, 2026. arXiv: 2604.02460.

[4] Ndzomga, F., *Efficient benchmarking of AI agents*, 2026. arXiv: 2603.23749 [cs.AI].

[5] Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., and Molchanov, P., *Small language models are the future of agentic AI*, 2025. arXiv: 2506.02153.

[6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D., *Scaling laws for neural language models*, 2020. arXiv: 2001.08361.

[9] Ouyang, L., Wu, J., Jiang, X., Almeida, D., et al., *Training language models to follow instructions with human feedback*, 2022. arXiv: 2203.02155.

[10] OpenAI, *GPT-4 technical report*, 2023. arXiv: 2303.08774.

[11] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y., "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[12] Xi, Z., Chen, W., Guo, X., He, W., et al., *The rise and potential of large language model based agents: A survey*, 2023. arXiv: 2309.07864.

[13] Wang, L., Ma, C., Feng, X., Zhang, Z., et al., *A survey on large language model based autonomous agents*, 2023. arXiv: 2308.11432.

[14] Masterman, T., Besen, S., Sawtell, M., and Chao, A., *The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey*, 2024. arXiv: 2404.11584.

[15] Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L., *Cognitive architectures for language agents*, 2024. arXiv: 2309.02427.

[16] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A., *Voyager: An open-ended embodied agent with large language models*, 2023. arXiv: 2305.16291.

[17] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S., "Generative agents: Interactive simulacra of human behavior," in *ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv: 2304.03442.

[18] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S., "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[19] Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H., "Executable code actions elicit better LLM agents," in *International Conference on Machine Learning (ICML)*, 2024.

[20] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K., "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[21] Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., et al., *Graph of thoughts: Solving elaborate problems with large language models*, 2023. arXiv: 2308.09687.

[22] Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., and Wang, Y.-X., "LATS: Language agent tree search unifies reasoning, acting, and planning in language models," *arXiv preprint arXiv:2310.04406*, 2023, Published at ICML 2024.

[23] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T., "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv: 2302.04761.

[24] Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., and Neubig, G., "PAL: Program-aided language models," in *International Conference on Machine Learning (ICML)*, 2023. arXiv: 2211.10435.

[25] Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A., "Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2310.11324.

[26] Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L., *Instruction-following evaluation for large language models*, 2023. arXiv: 2311.07911.

[27] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P., "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, 2024. arXiv: 2307.03172.

[28] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E., *MemGPT: Towards LLMs as operating systems*, 2023. arXiv: 2310.08560.

[29] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025. arXiv: 2501.12948.

[30] Snell, C., Lee, J., Xu, K., and Kumar, A., *Scaling LLM test-time compute optimally can be more effective than scaling model parameters*, 2024. arXiv: 2408.03314.

[31] Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A., *Large language monkeys: Scaling inference compute with repeated sampling*, 2024. arXiv: 2407.21787.

[32] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D., "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR)*, 2023. arXiv: 2203.11171.

[33] TerminalBench Team, *TerminalBench: A benchmark for terminal-based coding agents*, 2025.

[34] Yang, J., Prabhakar, A., Narasimhan, K., and Yao, S., *InterCode: Standardizing and benchmarking interactive coding with execution feedback*, 2023. arXiv: 2306.14898 [cs.CL].

[35] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K., "SWE-bench: Can language models resolve real-world GitHub issues?" In *International Conference on Learning Representations (ICLR)*, 2024.

[36] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O., *SWE-agent: Agent-computer interfaces enable automated software engineering*, 2024. arXiv: 2405.15793 [cs.SE].

[37] Xu, F. F. et al., *TheAgentCompany: Benchmarking LLM agents on consequential real world tasks*, 2024. arXiv: 2412.14161 [cs.CL].

[38] Mateega, S., Yang, J., Costello, T., Jadhav, S., Tian, N., and Garcinuño, A., *IDE-Bench: Evaluating large language models as IDE agents on real-world software engineering tasks*, 2026. arXiv: 2601.20886 [cs.SE].

[39] Liu, X., Yu, H., Zhang, H., Xu, Y., et al., *AgentBench: Evaluating LLMs as agents*, 2024. arXiv: 2308.03688.

[40] Xie, T., Zhang, D., Chen, J., Li, X., et al., *OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments*, 2024. arXiv: 2404.07972.

[41] Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., and Scialom, T., *GAIA: A benchmark for general AI assistants*, 2023. arXiv: 2311.12983.

[42] Huang, Q., Vora, J., Liang, P., and Leskovec, J., *MLAgentBench: Evaluating language agents on machine learning experimentation*, 2024. arXiv: 2310.03302.

[43] Yao, S., Shinn, N., Razavi, P., and Narasimhan, K., *τ-bench: A benchmark for tool-agent-user interaction in real-world domains*, 2024. arXiv: 2406.12045.

[44] Wang, X., Ding, B., Chen, Z., et al., "OpenHands: An open platform for AI software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.

[45] Zhang, Y., Ruan, H., Fan, Z., and Roychoudhury, A., "AutoCodeRover: Autonomous program improvement," in *International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

[46] Feng, Y. et al., *LongCLI-Bench: A preliminary benchmark and study for long-horizon agentic programming in command-line interfaces*, 2026. arXiv: 2602.14337 [cs.SE].
[47] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., et al., *Evaluating large language models trained on code*, 2021. arXiv: 2107.03374.
[48] Jain, N., Han, K., Gu, A., et al., *LiveCodeBench: Holistic and contamination free evaluation of large language models for code*, 2024. arXiv: 2403.07974.
[49] Schaeffer, R., Miranda, B., and Koyejo, S., *Are emergent abilities of large language models a mirage?* 2023. arXiv: 2304.15004 [cs.AI].
[50] Song, Y., Wang, G., Li, S., and Lin, B. Y., *The good, the bad, and the greedy: Evaluation of LLMs should not ignore non-determinism*, 2024. arXiv: 2407.10457.
[51] Liang, P., Bommasani, R., Lee, T., Tsipras, D., et al., "Holistic evaluation of language models," *Transactions on Machine Learning Research*, 2023. arXiv: 2211.09110.
[52] Zhou, D. et al., "Least-to-most prompting enables complex reasoning in large language models," in *International Conference on Learning Representations (ICLR)*, 2023. arXiv: 2205.10625.
[53] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., and Lim, E.-P., "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv: 2305.04091.
[54] Hong, S. et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023, Published at ICLR 2024.
[55] Prasad, A., Koller, A., Hartmann, M., Clark, P., Sabharwal, A., Bansal, M., and Khot, T., "ADaPT: As-needed decomposition and planning with language models," in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.
[56] Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N. A., and Lewis, M., *Measuring and narrowing the compositionality gap in language models*, 2022. arXiv: 2210.03350.
[57] Xia, C. S., Deng, Y., Dunn, S., and Zhang, L., "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.
[58] Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., and Ghanem, B., "CAMEL: Communicative agents for "mind" exploration of large language model society," *arXiv preprint arXiv:2303.17760*, 2023, Published at NeurIPS 2023.
[59] Madaan, A. et al., "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[60] Chen, X., Lin, M., Schärli, N., and Zhou, D., "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2304.05128.
[61] Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D., "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2310.01798.
[62] Gou, Z., Shao, Z., Gong, Y., Shen, Y., Yang, Y., Duan, N., and Chen, W., "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2305.11738.
[63] Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K., *Let's verify step by step*, 2023. arXiv: 2305.20050.
[64] Zheng, C. et al., *A survey of process reward models: From outcome signals to process supervisions for large language models*, 2025. arXiv: 2510.08049.
[65] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., et al., "Judging LLM-as-a-judge with MT-Bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[66] Gu, J. et al., *A survey on LLM-as-a-judge*, 2024. arXiv: 2411.15594.
[67] Wu, Q. et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
[68] Qian, C. et al., "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
[69] Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Tu, Z., and Shi, S., "Encouraging divergent thinking in large language models through multi-agent debate," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
[70] Wang, J., Wang, J., Athiwaratkun, B., Zhang, C., and Zou, J., "Mixture-of-agents enhances large language model capabilities," *arXiv preprint arXiv:2406.04692*, 2024.
[71] Austin, D., *Orchestrator: Multi-agent coding system*, 2025.
[72] Chen, W., Su, Y., Zuo, J., Yang, C., et al., *AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors*, 2023. arXiv: 2308.10848.

[73] Yan, B., Zhou, Z., Zhang, L., et al., *Beyond self-talk: A communication-centric survey of LLM-based multi-agent systems*, 2025. arXiv: 2502.14321.
[74] Li, J., Zhang, Q., Yu, Y., Fu, Q., and Ye, D., *More agents is all you need*, 2024. arXiv: 2402.05120.
[75] Ge, C., Kryvosheieva, D., Fried, D., Girit, U., and Hariharan, K., *Agent psychometrics: Task-level performance prediction in agentic coding benchmarks*, 2026. arXiv: 2604.00594 [cs.AI].
[76] Cemri, M. et al., *Why do multi-agent LLM systems fail?* 2025. arXiv: 2503.13657 [cs.AI].
[77] Merrill, M. A. et al., *Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces*, Published at ICLR 2026, 2026. arXiv: 2601.11868 [cs.SE].
[78] SWE-agent Team, *Mini-SWE-agent: 100 lines of python for software engineering*, 2025.
[79] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D., "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
[80] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I., "Improving factuality and reasoning in language models through multiagent debate," in *International Conference on Machine Learning (ICML)*, 2024.
[81] Gao, Y., Xiong, Y., Gao, X., Jia, K., et al., *Retrieval-augmented generation for large language models: A survey*, 2023. arXiv: 2312.10997.
[82] Qwen Team, "Qwen2.5-coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.
[83] Yang, A., Yang, B., Zhang, B., et al., *Qwen2.5 technical report*, 2024. arXiv: 2412.15115.
[84] Rastogi, A., Yang, A., Jiang, A. Q., Liu, A. H., Sablayrolles, A., et al., *Devstral: Fine-tuning language models for coding agent applications*, 2025. arXiv: 2509.25193.
[85] Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., et al., *Mistral 7B*, 2023. arXiv: 2310.06825.
[86] Grattafiori, A. et al., *The Llama 3 herd of models*, 2024. arXiv: 2407.21783.
[87] Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., et al., *Code Llama: Open foundation models for code*, 2023. arXiv: 2308.12950.
[88] Lozhkov, A., Li, R., Ben Allal, L., Cassano, F., et al., *StarCoder 2 and The Stack v2: The next generation*, 2024. arXiv: 2402.19173.
[89] Guo, D., Zhu, Q., Yang, D., et al., *DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence*, 2024. arXiv: 2401.14196.
[90] Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S., "AWQ: Activation-aware weight quantization for LLM compression and acceleration," *Machine Learning and Systems (MLSys)*, 2024.
[91] Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D., "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations (ICLR)*, 2023. arXiv: 2210.17323.
[92] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I., "Efficient memory management for large language model serving with PagedAttention," *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
[93] Liu, R., Sun, Y., Zhang, M., Bai, H., Yu, X., Yu, T., Yuan, C., and Hou, L., *Quantization hurts reasoning? an empirical study on quantized reasoning models*, 2025. arXiv: 2504.04823.
[94] McNemar, Q., "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
[95] Dror, R. and Reichart, R., *Recommended statistical significance tests for NLP tasks*, 2018. arXiv: 1809.01448.
[96] Efron, B. and Tibshirani, R. J., *An Introduction to the Bootstrap* (Monographs on Statistics and Applied Probability). New York: Chapman & Hall, 1993.
[97] Holm, S., "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
[98] Spearman, C., "The proof and measurement of association between two things," *The American Journal of Psychology*, vol. 15, pp. 72–101, 1904.
[99] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., et al., "Emergent abilities of large language models," *Transactions on Machine Learning Research*, 2022. arXiv: 2206.07682.
[100] Wang, N. et al., *Efficient agents: Building effective agents while reducing cost*, 2025. arXiv: 2508.02694.
[101] Zhu, K. et al., *Where LLM agents fail and how they can learn from failures*, 2025. arXiv: 2509.25370.
[102] Bui, N. D. Q., *Building effective AI coding agents for the terminal: Scaffolding, harness, context engineering, and lessons learned*, 2026. arXiv: 2603.05344.